

目次

第 1 章	環境構築とワークフロー	1
1.1	キーワード	1
1.2	CLI 操作における注意	1
1.3	環境構築 (Windows)	2
1.4	環境構築 (Mac)	3
1.5	ターミナル操作とパス	4
1.6	GitHub と Git の初期設定	5
1.7	作業ディレクトリの作成とリポジトリの clone	5
1.8	Python 環境の構築	6
1.9	VS Code からこのリポジトリを開く	6
1.10	VS Code の実行・デバッグ機能で <code>.venv</code> を使う (任意)	7
1.11	最初の動作確認	8
1.12	教材 repo と作業 repo の使い分け	8
1.13	うまく進まないとき	9
1.14	公式資料	10
第 2 章	第 1 章: 基本操作	11
2.1	はじめに	11
2.2	GitHub	11
2.3	Git の用語	12
2.4	GitHub の用語	14
2.5	実践 GitHub	14
第 3 章	Python 入門	17
3.1	Python の特徴	17
3.2	Python の実行	17
3.3	変数	18
3.4	エスケープシーケンス	26
3.5	文字列メソッド	27
3.6	f 文字列	27
3.7	浮動小数点数がもつメソッド	28
3.8	複合データ型	28
3.9	Python プログラムのコマンドライン実行	34
3.10	制御構文	35
3.11	関数	41
3.12	クラス	46

第 4 章	NumPy, Matplotlib を用いた初等数学の復習	53
4.1	NumPy と Matplotlib を使う準備	53
4.2	多次元配列を定義する	53
4.3	多次元配列の要素を選択する	56
4.4	ndarray のデータ型	58
4.5	多次元配列を用いた計算	59
4.6	直線のグラフの描画	61
4.7	2 次関数のグラフの平行移動	62
4.8	三角関数	65
4.9	指数関数	67
4.10	対数関数	68
4.11	ブロードキャスト	71
4.12	基本的な統計量の求め方	77
4.13	注釈	79
第 5 章	微分の基礎	81
5.1	微分と関数最小化の関係	81
5.2	2 点間を通る直線の傾き	82
5.3	接線の傾き	83
5.4	微分の公式	85
5.5	線形性	86
5.6	合成関数の微分	86
5.7	偏微分	87
5.8	注釈	88
第 6 章	積分の基礎	89
6.1	この章の目的	89
6.2	不定積分	89
6.3	定積分	90
6.4	置換積分と部分積分の考え方	91
6.5	広義積分	91
6.6	多重積分	92
6.7	Python で数値積分を確かめる	92
6.8	この章で押さえるべき点	93
第 7 章	線形代数の基礎	95
7.1	スカラー・ベクトル・行列・テンソル	95
7.2	ベクトルを入力とする関数	105
7.3	ベクトル値関数	105
7.4	ベクトル値関数の微分	106
7.5	合成関数の微分（多変数バージョン）	106
7.6	注釈	109
第 8 章	確率・統計の基礎	111
8.1	確率・統計と機械学習の関係	111

8.2	確率変数と確率分布	111
8.3	同時分布・周辺確率	112
8.4	条件付き確率	114
8.5	ベイズの定理	117
8.6	尤度と最尤推定	118
8.7	事後確率最大化推定 (MAP 推定)	120
8.8	統計量	122
8.9	注釈	124
第 9 章	Pandas 入門	127
9.1	CSV ファイルの読み書き	127
9.2	DataFrame の表示	128
9.3	先頭の数件だけを表示	130
9.4	CSV ファイルの保存	131
9.5	データフレームの形	131
9.6	統計量の算出	131
9.7	並べ替え	134
9.8	データの選択	135
9.9	条件指定による要素の選択	137
9.10	欠損値の除去・補間	141
9.11	ndarray とデータフレームを相互に変換	142
9.12	グラフの描画	144
第 10 章	Matplotlib 応用	145
10.1	この章の目的	145
10.2	読み込みと保存先の準備	145
10.3	散布図	146
10.4	ヒストグラム	147
10.5	箱ひげ図	149
10.6	折れ線グラフ	151
10.7	図を読める形に整える	152
10.8	subplots で複数図を並べる	153
10.9	seaborn の使いどころ	153
10.10	この章で押さえるべき点	154
第 11 章	PySoundFile 入門	157
11.1	この章の目的	157
11.2	PySoundFile とは何か	157
11.3	インストール	158
11.4	音声を読む	158
11.5	モノラルとステレオの配列形状	158
11.6	dtype と振幅範囲	159
11.7	一部分だけ読む	159
11.8	音声を書き出す	159
11.9	Python で最小の読み書きと検証を行う	159

11.10	情報だけ見たいときは SoundFile オブジェクトを使う	160
11.11	この章で押さえるべき点	160
第 12 章	Pillow 入門	163
12.1	この章の目的	163
12.2	Pillow とは何か	163
12.3	インストール	163
12.4	画像を開いて属性を確認する	164
12.5	結果を保存する流れを最初にする	164
12.6	サイズ変更	164
12.7	回転と反転	164
12.8	切り出し	165
12.9	描画と注釈	165
12.10	フィルタ処理	165
12.11	色空間の変換	165
12.12	NumPy 配列との相互変換	166
12.13	特定画素の値を調べる	166
12.14	ヒストグラム	166
12.15	二値化	167
12.16	最小の前処理パイプライン	167
12.17	この章で押さえるべき点	167
第 13 章	信号処理の基礎	169
13.1	この章の目的	169
13.2	信号とは何か	169
13.3	サンプリング周期とサンプリング周波数	170
13.4	基本信号	170
13.5	基本操作	171
13.6	エネルギーと平均電力	171
13.7	Python で基本信号を確かめる	172
13.8	この章で押さえるべき点	172
第 14 章	スペクトル解析の基礎	173
14.1	なぜ周波数領域で考えるのか	173
14.2	正弦波は分解の基底になる	173
14.3	離散フーリエ変換	174
14.4	周波数ビンと実際の周波数	174
14.5	DC 成分とナイキスト周波数	174
14.6	スペクトルを見るときに注意	175
14.7	Python で DFT を確かめる	175
14.8	この章で押さえるべき点	176
第 15 章	LTI 系の基礎	177
15.1	システムとして信号処理を見る	177
15.2	線形性	177

15.3	時不変性	178
15.4	インパルス応答	178
15.5	畳み込み	178
15.6	差分方程式による表現	179
15.7	因果性と BIBO 安定性	179
15.8	周波数応答	179
15.9	Python で移動平均と IIR を比べる	180
15.10	この章で押さえるべき点	181
第 16 章	単回帰分析と重回帰分析	183
16.1	単回帰分析	183
16.2	重回帰分析	190
16.3	注釈	196
第 17 章	scikit-learn 入門	197
17.1	scikit-learn を用いた重回帰分析	197
17.2	各ステップの改善	202
第 18 章	ニューラルネットワークの基礎	205
18.1	事前に読んでおく必要がある章	205
18.2	ニューラルネットワークとは	205
18.3	ニューラルネットワークの計算	207
18.4	ニューラルネットワークの訓練	212
18.5	注釈	223

第 1 章

環境構築とワークフロー

本章は、環境構築の手順とその意味を、初めて取り組む学生が確認できるように説明する。授業用の handout と重なる操作も含め、OS 別の導入から Python スクリプトの実行までを本章内で扱う。Windows では「環境構築 (Windows)」, Apple Silicon の Mac では「環境構築 (Mac)」を行い、その後は共通手順へ進む。設定済みの項目は確認コマンドで状態を確かめ、成功していれば再設定しなくてよい。

1.1 キーワード

- **Operating System**

コンピュータ全体を管理するための基本ソフトウェア。代表例として、Windows、macOS、Linux がある。

- **ファイルシステム**

ファイルやフォルダを保存し、整理し、読み書きするための仕組み。

- **Command line interface (CLI)**

コマンドを入力してコンピュータを操作する仕組み。操作を自動化しやすいため、開発や研究でよく使われる。

- **ターミナル**

CLI でコンピュータを操作するためのソフトウェア

- **テキストエディタ**

文字情報を編集するためのソフトウェア。メモを書くための簡単なものから、プログラムを書くための高機能なものまである。代表例として、Visual Studio Code、Vim、Emacs などがある。

- **Python と仮想環境**

Python は、機械学習などで広く使われるプログラミング言語。仮想環境は、プロジェクトごとに Python の実行環境を分けて管理する仕組みである。

- **Git**

ソースコードや文書の変更履歴を管理するためのバージョン管理システム。いつ、誰が、どのような変更を加えたかを記録できる。Git で管理されたプロジェクトをオンラインで記録・共有するためのサービスとして GitHub がある。

1.2 CLI 操作における注意

- むやみにコマンドを実行しない
- コマンドの実行結果をよく確認する
- エラーメッセージを無視しない

1.3 環境構築 (Windows)

この資料では、Windows 上に Ubuntu 24.04 LTS の WSL2 環境を作成し、その Ubuntu 上で Python と Git 操作を行うものとする。そのため、まずは WSL2 環境を構築する。

まず PowerShell を管理者権限で開き、VS Code, Windows Terminal, WSL 本体をインストールする。

```
1 winget install --id Microsoft.VisualStudioCode --exact
2 winget install --id Microsoft.WindowsTerminal --exact
3 wsl --install --no-distribution
```

再起動を求められたら再起動し、PowerShell を開き直す。

```
1 wsl --update
2 wsl --version
```

- `wsl --install --no-distribution` 後は PC の再起動が必要になる場合がある。
- Ubuntu 24.04 LTS 以降は新しい WSL distro format で配布されるため、`wsl --version` で WSL 2.4.10 以上になっていることを確認する。
 - 古い場合は `wsl --update` を実行し、Windows Terminal を開き直す。

次に、インストール可能な distro 名を確認し、Ubuntu-24.04 を明示してインストールする。

```
1 wsl --list --online
2 wsl --install -d Ubuntu-24.04
```

Ubuntu 24.04 LTS が WSL2 として入っているか、PowerShell 側で確認する。

```
1 wsl -l -v
```

Ubuntu-24.04 の VERSION が 2 であれば WSL2 として動く。必要なら既定の distro に設定する。

```
1 wsl --set-default Ubuntu-24.04
2 wsl ~ -d Ubuntu-24.04
```

インストール後、Windows Terminal から Ubuntu 24.04 LTS を起動する。初回起動時には Ubuntu 側の user name と password を設定する。password 入力中は画面に文字が表示されないが、入力自体は受け付けられている。

Ubuntu 側に入ったら、Ubuntu の release 情報も確認する。

```
1 cat /etc/os-release
```

VERSION_ID="24.04" が確認できれば、この授業で使う Ubuntu 24.04 LTS 環境になっている。

Ubuntu を起動したら、Git, curl, SSH クライアントを入れる。Python 3.12 は後述の uv で管理する。以降の作業は、原則としてこの Ubuntu のターミナルで行う。

```
1 sudo apt update
2 sudo apt install git curl openssh-client
```

GitHub CLI (gh) は、GitHub への login や repository 操作をターミナルから行うために使う。Ubuntu 24.04 LTS では、GitHub CLI の公式 apt 配布元を登録してからインストールする。公開鍵ファイルはダウンロードしたパッケージの署名確認に使い、`signed-by` でそのファイルを指定する。以下の各コマンドの成功を確認してから次の行へ進む。


```

1 sudo mkdir -p -m 755 /etc/apt/keyrings
2 sudo curl -fsSL https://cli.github.com/packages/githubcli-archive-keyring.
   gpg \
3   -o /etc/apt/keyrings/githubcli-archive-keyring.gpg
4 sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg
5 echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/
   githubcli-archive-keyring.gpg] https://cli.github.com/packages stable
   main" \
6   | sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null
7 sudo apt update
8 sudo apt install gh
9 gh --version

```

WSL 上のディレクトリを VS Code で開くには、Windows 側の VS Code に WSL extension が入っている必要がある。インストールには、PowerShell で次を実行する。

```

1 code --install-extension ms-vscode-remote.remote-wsl

```

その後、Ubuntu 側のターミナルを開き、次を実行できることを確認する。

```

1 code --version
2 code .

```

初回の `code .` では、WSL 側に VS Code Server が自動 install される。`code: command not found` になる場合は、Windows Terminal と Ubuntu を開き直し、Windows 側の VS Code と WSL extension が入っているか確認する。

1.4 環境構築 (Mac)

この手順は Apple Silicon (M シリーズ) の Mac を対象にする。この repo で固定している PyTorch 2.7.1 には Intel Mac 用の wheel がいないため、Intel Mac ではこの手順のまま環境をそろえることはできない。Intel Mac を使う場合は、授業用に別の対応環境を用意する必要がある。

Mac では、まず、ソフトウェアをインストールするための package manager として、Homebrew を導入する。Terminal を開き、Homebrew 公式ページに掲載されている以下のコマンドを実行する。

```

1 /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/
   install/HEAD/install.sh)"

```

- install の途中で password の入力や Enter の入力を求められることがある。
- install 完了時に Next steps として PATH 設定用の コマンドが表示された場合は、それをそのまま実行する。

次のコマンドで Homebrew が使えることを確認する。

```

1 brew --version

```

Homebrew が使えるようになったら、VS Code, Git, GitHub CLI をそろえる。Python 3.12 は、後述する uv でインストールして管理する。Mac に Git が入っている場合でも、`git --version` で確認してから進める。

```

1 brew install --cask visual-studio-code
2 brew install git gh

```

Mac で terminal から VS Code を開くには、code command が PATH に入っている必要がある。まず次で確認する。

```
1 code --version
```

command not found: code などと表示される場合は、VS Code を開き、Command Palette から Shell Command: Install 'code' command in PATH を実行する。その後、Terminal を開き直して、もう一度 code --version を確認する。確認できたら、作業 directory で次を実行すると VS Code が開く。

```
1 code .
```

1.5 ターミナル操作とパス

ターミナルでの作業には、常に「現在地」となるディレクトリ（**カレントディレクトリ**）がある。相対パスで指定する操作はこの現在地を基準に行われるため、まずどこにいるかを把握することが重要である。

コマンド	役割
pwd	現在地のパスを表示する
ls	現在地にあるファイルやディレクトリの一覧を表示する

パス (path) は、ファイルやディレクトリ（フォルダ）の場所を表す文字列である。住所が分かれば建物にたどり着けるのと同じように、パスが分かれば目的のファイルにたどり着ける。パスは、ディレクトリ名を / (スラッシュ) でつないで書く。

記号	意味
/	ディレクトリの区切り。先頭の / はルート（最上位）
.	カレントディレクトリ（現在地）
..	親ディレクトリ（1つ上）
~	ホームディレクトリ（Ubuntu では /home/username, Mac では /Users/username）

絶対パス：ルートディレクトリ / から始まるパス

```
1 /home/yourname/workspace/data.csv
```

相対パス：現在地を基準として書くパス (/ で始めない)

```
1 data.csv           # 現在地にある data.csv
2 outputs/log.txt    # 現在地にある outputs ディレクトリの中にある log.txt
3 ../README.md       # つ上のディレクトリにある1 README.md
4 ../../shared/config.yml # つ上のディレクトリにある2 shared/ 内の config.yml
```

現在地を別のディレクトリへ移動するには cd を使う。

```
1 cd workspace      # workspace ディレクトリへ移動
2 cd ..             # つ上のディレクトリへ戻る (1.. は親ディレクトリを表す)
```

新しいディレクトリを作るには mkdir を使う。-p オプションを付けると、途中のディレクトリもまとめて作成される。

```
1 mkdir -p outputs/session01
```

1.6 GitHub と Git の初期設定

GitHub を使うには、GitHub account, GitHub CLI の認証が必要になる。この資料では GitHub との通信に SSH を使う前提にする。

```
1 gh auth login
```

`gh auth login` の質問に対して、接続先, Git 通信の方式, 使用する SSH 鍵, 認証方法を次のように選ぶ。表示される質問や順序は, `gh` のバージョンや既存の認証・SSH 鍵の設定によって変わる。

- What account do you want to log into?: GitHub.com
- What is your preferred protocol for Git operations?: SSH
- SSH 公開鍵の登録を求められたら, 使用する鍵を選ぶ。鍵がない場合は, 案内に従って作成する。
- browser を使う認証を選び, 表示された code を GitHub の画面に入力する

SSH を選ぶと, `gh` は既存の SSH 鍵を探す。適切な鍵がない場合は, 新しい鍵の作成と GitHub への登録を促す。このとき GitHub に登録されるのは公開鍵であり, 秘密鍵は自分の PC または WSL 環境の中に残る。秘密鍵を他人に見せたり, GitHub に貼り付けたりしてはいけない。

認証と SSH key の設定が終わったら, 次で状態を確認する。

```
1 gh auth status
2 ssh -T git@github.com
```

初回の SSH 接続では, 接続先のホスト鍵を信頼するか確認されることがある。表示された fingerprint (鍵の識別値) を [GitHub 公式の公開鍵一覧](#) と照合し, 一致した場合に `yes` を入力する。自分の GitHub ユーザー名を含む認証成功メッセージが出ればよい。GitHub は SSH でのシェル操作を提供しないため, `ssh -T git@github.com` は認証成功時にも終了コード 1 を返す。Permission denied (publickey) は成功ではない。使用する公開鍵が自分の GitHub アカウントに登録されているかを確認する。

Git の著者情報は次で設定する。

```
1 git config --global user.name "Your Name"
2 git config --global user.email "your-email@example.com"
3 git config --global --list
```

`user.name` と `user.email` は commit に記録される。自身の GitHub account と対応する情報にしておく。

1.7 作業ディレクトリの作成とリポジトリの clone

この教材では, 作業場所をホームディレクトリの下にまとめる。例として `~/workspace` を作り, その中にこのリポジトリや提出用リポジトリを置く。

```
1 mkdir -p ~/workspace
2 cd ~/workspace
3 git clone git@github.com:ykinolab-tokai/signal-ml-training.git
```

すでに同じ場所へ取得済みの場合は `git clone` を省略する。リポジトリを取得したら, まず次を確認する。

```
1 pwd
2 ls
3 cd signal-ml-training
```

```
4 ls
5 git status
```

`pwd` は現在地, `ls` は現在地のファイル, `git status` は Git が認識している変更状態を確認するために使う。

1.8 Python 環境の構築

この repo は Python 3.12 系を標準にする。Python 環境の管理には `uv` を用いる。

```
1 curl -LsSf https://astral.sh/uv/install.sh | sh
```

インストール末尾に表示される `PATH` の案内に従い、ターミナルを開き直すか、案内された `source` コマンドを実行する。次のコマンドでバージョンが表示されることを確認してから先へ進む。

```
1 uv --version
```

repo ルートで次を実行すると, `pyproject.toml` と `uv.lock` に従い Python 3.12 環境と `.venv` が用意される。`pyproject.toml` は必要なパッケージの条件, `uv.lock` は解決済みのバージョンを記録する。`--locked` は lockfile を書き換えず、不整合があれば停止する指定である。取得前の空のディレクトリで実行せず、教材 repo のルートへ移動してから実行する。

```
1 uv python install 3.12
2 uv sync --locked
3 source .venv/bin/activate
4 python --version
5 which python
```

`python --version` が 3.12 系であり, `which python` が `.venv` 配下を指していれば、授業用の環境に入っている。

1.9 VS Code からこのリポジトリを開く

この repo を VS Code で開くには、Ubuntu (WSL) または Mac のターミナルで次を実行する。

```
1 cd ~/workspace/signal-ml-training
2 code .
```

VS Code の Explorer に `pyproject.toml` と `uv.lock` が見えていることを確認する。

WSL の場合は、VS Code の左下やウィンドウ名に `WSL: Ubuntu-24.04` のように表示されていることを確認する。このウィンドウのターミナルも Ubuntu 側で動作する。

ターミナルからプログラムを実行する場合は、前節で準備した `uv` を使って repo の環境で実行できる。VS Code のターミナルでも、通常の Ubuntu や Mac のターミナルでも同じ操作でよい。次の例は、「最初の動作確認」のサンプルファイルを保存した後に実行する。

```
1 cd ~/workspace/signal-ml-training
2 uv run --locked python exercises/exc01_01.py
```

`uv run --locked` は lockfile を更新せずに repo の環境を確認・同期してから実行するため、事前に `source .venv/bin/activate` を実行する必要はない。一方, `python exercises/exc01_01.py` と直接実行する場合は、そのターミナルで `.venv` が有効になっていることを先に確認する。新しいターミナルで `python` を直接使うときは、その都度 `.venv` を有効にする。詳しくは [uv のプロジェクト操作](#) を参照する。

1.10 VS Code の実行・デバッグ機能で .venv を使う（任意）

以下は、VS Code の実行ボタンやデバッグ機能から、repo の .venv にある Python を使いたい場合の任意の設定である。repo を VS Code で開いて編集するだけの場合や、ターミナルで `uv run` を使う場合には、この設定は必要ない。「最初の動作確認」へ進んでよい。授業用の Python 環境は、どの実行方法でも「Python 環境の構築」で説明した `uv` で管理する。

この設定を行う場合は、まず repo のルートで次を実行し、`pyproject.toml` と `uv.lock` に従って .venv を準備する。

```
1 cd ~/workspace/signal-ml-training
2 uv sync --locked
```

VS Code の Python 実行・デバッグ機能を使うには、Microsoft の Python 拡張機能が必要である。入っていない場合は、VS Code の拡張機能画面で Python を検索し、Microsoft が提供しているものをインストールする。WSL の場合は、拡張機能画面で Ubuntu 側でも有効になっていることを確認し、Install in WSL: Ubuntu-24.04 と表示される場合はそのボタンでインストールする。

次に、この repo で使う Python を明示的に選ぶ。

1. Windows では `Ctrl+Shift+P`、Mac では `Cmd+Shift+P` で Command Palette を開く。
2. Python: Select Interpreter を実行する。
3. この repo 内の `.venv/bin/python` に対応する Python 3.12 を選ぶ。

選択するパスの例は次のとおりである。

- WSL : `/home/yourname/workspace/signal-ml-training/.venv/bin/python`
- Mac : `/Users/yourname/workspace/signal-ml-training/.venv/bin/python`

yourname は自分の user name に読み替える。候補に .venv が見つからない場合は、repo のルートで `uv sync --locked` が成功したことを確認し、Command Palette の Developer: Reload Window でウィンドウを再読み込みしてから選び直す。

選択後、既存のターミナルをゴミ箱ボタンで終了し、メニューの Terminal → New Terminal から新しいターミナルを開く。次を実行して、そのターミナルの現在地と Python を確認する。

```
1 cd ~/workspace/signal-ml-training
2 pwd
3 which python
4 python --version
```

`pwd` が repo のルート、`which python` がこの repo 内の `.venv/bin/python`、`python --version` が 3.12 系を示すことを確認する。新しいターミナルで仮想環境が自動的に有効にならない場合は、repo のルートで `source .venv/bin/activate` を実行し、もう一度確認する。

VS Code の実行ボタンで使われる Python も確認する。

次の 2 行を repo 直下の `interpreter_check.py` として保存する。

```
1 import sys
2 print(sys.executable)
```

このファイルをエディタで開いた状態で、右上の Run Python File in Terminal を押す。同じ操作は、エディタ内を右クリックし、Run → Python File in Terminal から選べる。表示されるパスがこのリポ

ジトリ内の `.venv/bin/python` であることを確認する。異なる場合は `Python: Select Interpreter` でこの repo の `.venv` を選び直す。

`Run Python File in Terminal` は、VS Code で選択した Python を使ってファイルを実行する。デバッグ機能も、実行環境を個別に指定していなければ、この選択を使う。教材を更新して依存パッケージが変わった場合は、実行前に repo のルートで `uv sync --locked` を行う。詳しくは [VS Code の Python 実行手順](#) を参照する。

1.11 最初の動作確認

環境構築後は、Python を実行できることまで確認する。まず repo のルートに移動し、スクリプトの保存先を作る。

```
1 cd ~/workspace/signal-ml-training
2 mkdir -p exercises
```

次の Python スクリプトを repo 直下の `exercises/exc01_01.py` として保存する。

```
1 from pathlib import Path
2
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 output_dir = Path("outputs/setup_check")
7 output_dir.mkdir(parents=True, exist_ok=True)
8
9 x = np.linspace(0, 2 * np.pi, 400)
10 y = np.sin(x)
11
12 plt.plot(x, y)
13 plt.xlabel("x")
14 plt.ylabel("sin(x)")
15 plt.tight_layout()
16 plt.savefig(output_dir / "sin.png", dpi=150)
17 plt.close()
18
19 print("setup check completed")
```

repo のルートで仮想環境を有効にして、保存したスクリプトを実行する。

```
1 cd ~/workspace/signal-ml-training
2 source .venv/bin/activate
3 python exercises/exc01_01.py
```

`setup check completed` と表示され、repo 直下に `outputs/setup_check/sin.png` が作られることを確認する。この画像を開いて正弦波が描かれていれば、動作確認は完了である。

`git status` も実行し、Git が認識している変更の有無を確認する。無視対象のファイルは通常の `git status` に表示されない。出力画像の有無は実際にファイルを開いて確認する。

1.12 教材 repo と作業 repo の使い分け

教材 repo は教材・見本・共通の依存関係を読む場所、第1回の動作確認もここで行う。第2回以降の解答は自分の作業 repo（提出 repo / submission repo）へ保存する。[第1回の作成手順](#)に従い、作業

用テンプレートから ykinolab-tokai 所有の非公開 signal-ml-work-**<GitHub ユーザー名>** を作成する。作成した repo の README に従って~/workspace/signal-ml-work に clone する。取得済みならそのフォルダを開き、二重に clone しない。教材 repo は~/workspace/signal-ml-training, 作業 repo は~/workspace/signal-ml-work に並ぶ。

第2回以降と補足の課題の相対パスは、特記がなければ作業 repo のルートを基準とする。

- コードの標準は exercise/excXX_YY.py (回番号・問題番号は2桁)。各回が scripts/ 以下などを指定した場合はその指定を優先する。フォルダなしのファイル名だけを指定した場合は作業 repo のルートに置く。
- 図・音声・数値結果の標準は outputs/ 以下。説明はコードコメントまたは既存の結果ファイルに残し、別レポートは必要な課題が明示した場合だけ作る。同じ内容を転記しない。
- data/cat.png や data/piano.wav を使う場合は、教材 repo の同名ファイルを作業 repo の data/ へコピーする。

環境の初回準備は本章の前半に従う。教材更新後の依存関係の同期は教材 repo で行う。

```
1 cd ~/workspace/signal-ml-training
2 uv sync --locked
```

演習時は作業 repo へ移動し、教材側の仮想環境を有効にする。

```
1 cd ~/workspace/signal-ml-work
2 source ../signal-ml-training/.venv/bin/activate
3 pwd
4 python --version
5 which python
```

pwd が作業 repo, which python が教材側の .venv/bin/python を指すことを確認する。例えば第2回第1問は python exercise/exc02_01.py で実行する。配置を変えた場合は実際のパスへ読み替える。venv は作業 repo へコピーせず、毎回作り直さない。保存前に Path(...).mkdir(parents=True, exist_ok=True) などで出力ディレクトリを作る。保存規則と PR 練習後の通常作業への戻り方は[共通手順](#)を参照する。

1.13 うまく進まないとき

- WSL のバージョンが 2.4.10 未満：PowerShell で wsl --update を実行し、Windows Terminal を開き直して wsl --version を再確認する。
- wsl -l -v で Ubuntu の VERSION が 1：PowerShell で wsl --set-version Ubuntu-24.04 2 を実行し、wsl -l -v で 2 になったことを確認する。
- code や uv が見つからない：インストール完了時の PATH 設定を確認してターミナルを開き直し、code --version または uv --version を再実行する。
- uv sync --locked がプロジェクトを見つけられない：pwd と ls で教材 repo の pyproject.toml と uv.lock がある場所か確認する。
- lockfile と設定の不整合で停止する：授業で指定された教材の版か確認し、担当者にエラーを伝える。自分だけ依存関係を変更しない。
- Python のパスが .venv ではない：教材 repo で source .venv/bin/activate を実行し、which python と python --version を再確認する。
- 画像が見つからない：実行時の現在地を pwd で確認する。サンプルはその現在地を基準に outputs/setup_check/sin.png を作る。

1.14 公式資料

- [Microsoft : WSL のインストール](#)
- [Ubuntu : WSL2 への導入と必要な WSL バージョン](#)
- [GitHub CLI : Linux への導入](#)
- [GitHub CLI : 認証](#)
- [GitHub : SSH 接続の確認](#)
- [Homebrew : 導入](#)
- [uv : 導入](#)
- [uv : lockfile と環境の同期](#)
- [VS Code : WSL](#)
- [VS Code : Python 環境](#)

第 2 章

第 1 章: 基本操作

2.1 はじめに

この資料は、木下研で研究をはじめると必要となる、信号処理や機械学習（ディープラーニングを含む）の仕組みやプログラムを理解するために書かれたものです。

基礎となる数学の知識から、Python というプログラミング言語を用いたコーディングの基本、機械学習・ディープラーニングの基礎的な理論、音や画像の信号処理の理論とコーディングに至るまで、幅広いトピックを解説しています。

研究には、ある程度、線形代数や確率統計といった数学の知識から、何らかのプログラミング言語が使えることなどが必要となってきます。

しかし、そういった数学やプログラミングの全てに精通していなければ研究をはじめられないかというと、必ずしもそうではありません。

本資料では、まず必要になる最低限の数学とプログラミングの知識から学び始められるように、資料を充実させています。

また、本資料に沿って作成する提出物は GitHub 上の repo で管理します。

GitHub は、現在のソフトウェア開発にはなくてはならないサービスです。

まずは、GitHub のことから勉強をはじめましょう。

2.2 GitHub

GitHub は、Git を使った**バージョン管理**システムをインターネット上で提供するサービスです。

さて、Git や**バージョン管理**という聞き慣れない用語が出てきました。

以下の図を見てください。

レポートを作成する際、図のような状況になったことはありませんか？

なにか大きな修正が必要となったときに、昔の状態に戻せなくなってしまうことが不安でファイルをコピーしておき、

どれが新しいバージョンなのかわかるように名前をつけておく。

ただ、修正したものよりも、修正前のほうがやっぱり良いと思って古いバージョンのファイルに修正を加えてそれを最新版とする…。

こういったことを繰り返すと、どれが最新のファイルなのかわからなくなってしまいます。

プログラミングでも同様の事が起こります。

つまり、プログラムを修正する際にバックアップのためプログラムをコピーすることを繰り返すと、最新のプログラムがどれかわからなくなってしまいます。

1 人の状況ならまだ良いですが、複数人で開発を行うとなると状況はさらに複雑になってしまいます。



図 2.1 バージョン管理の失敗例

バージョン管理システムは、このような混乱を避け、ファイルのバージョンをきれいに管理するためのシステムです。

具体的には、以下のような機能を持ちます。

- ファイルの変更や更新を自動で管理してくれる
- だれが、いつ、どこを変更したかを記録してくれる
- 過去の状態にすぐに戻ることができる

こうした機能を提供するシステムの1つがGitで、GitHubはgitを使って動いているサービスです。GitHubを使ってファイルを管理するとそれらファイルがインターネット上のサーバにもコピーされるため、卒業論文の締め切り前にPCが壊れてファイルがなくなってしまうと、すぐにダウンロードできます。

2.3 Git の用語

Git には専門的な用語が複数使われます。

これら用語を初心者にも馴染みのある言葉で説明します。

わかりやすさを重視のために正確性を多少犠牲にしています。

2.3.1 リポジトリ (repository)

リポジトリとは、バージョン管理を行っているフォルダのことを言います。

そのフォルダ (リポジトリ) の中に作られるすべてのフォルダやファイルがバージョン管理の対象となります。

リポジトリには、以下の2種類があります。

- ローカルリポジトリ

手元のPCに保存されているリポジトリです

- リモートリポジトリ

GitHub のサーバ上に保存されているリポジトリです

2.3.2 クローン (clone)

クローンは、一言でいうと**ダウンロード**です。

リモートリポジトリ (GitHub のサーバ上にあるリポジトリ) を
ローカルリポジトリ (手元の PC) にダウンロードすることを指します。

2.3.3 プル (pull)

プルも、一言でいうと**ダウンロード**です。

クローンと何が違うのかと思ったのではないのでしょうか。

クローンは、リモートリポジトリのデータをすべてダウンロードするのに対し、

プルは、**ローカルリポジトリにないデータだけをダウンロード**します。

2.3.4 アッド (add)

ローカルリポジトリのファイルを変更したとします。

このとき、Git は、「今回、リポジトリ内のどのファイルがどれだけ変更されたか」を記録します。

この変更されたファイルのリストを作成するための作業をアッドと言います。

2.3.5 コミット (commit)

コミットは、今回の変更を Git に保存することです。

アッドしたファイルを実際にリポジトリの変更履歴として保存します。

ライザップとは関係ありません。

コミットを行うと**その時点でのリポジトリの状態が保存**されます。

これにより、もし間違った修正などをしてしまったときには、過去にコミットした時点の状態にいつでも戻すことができます。

2.3.6 プッシュ (push)

一言でいうと**アップロード**です。

コミットして変更が保存されたローカルリポジトリをリモートリポジトリにアップロードすることです。

2.3.7 ブランチ (branch)

リポジトリの内容をコピーしたものを作成することです。

作成されたコピーのこともブランチと呼びます。

初期状態では、リポジトリには main ブランチと呼ばれるブランチが 1 つのみあります。

main ブランチから作成された新しいブランチのファイルは、元々のファイルと独立に変更することができます。

2.3.8 マージ (merge)

ブランチ同士を結合して一つのブランチにまとめることです。

個々のブランチで変更された箇所を集約します。

複数人で作業を行う際などには、一人ひとりが自分用のブランチを作成し、作業が完了したところでブランチを main ブランチにマージするといった手順が取られます。ちょっと面倒ですが、こういった手順を踏むことで、現在のファイルに誤った修正を加えてしまうリスクを抑え、安全にファイルを更新していくことができます。

2.4 GitHub の用語

Git をベースとして動いている GitHub ですが、GitHub ならではの機能もあります。

2.4.1 フォーク (fork)

フォークは、他の誰かが GitHub 上で作成したリモートリポジトリを、自分自身のリモートリポジトリとしてコピーするための機能です。

他の人のリポジトリで公開されているプログラムに、自分にとってより使いやすくなるよう修正したいとします。

しかし、他の人のリポジトリを勝手に編集することは、普通できません。

ここでフォークを利用すると、そのリポジトリの全てのコードを自分自身のリポジトリとしてコピーすることができます。

そして、コピーしたリポジトリに対しては、自由に変更を加えることが可能になります。

このゼミでは、B3 の通常提出で fork は使いません。

fork は「他人の公開 repo を自分側にコピーして改善したいときに使う機能」と理解すれば十分です。

2.4.2 プルリクエスト (pull request)

フォークしたリポジトリへの変更が他の人にとっても有用だと思う場合、元々 (フォーク元) のリポジトリに同じ変更を反映させたいと思うかもしれません。そのような場合に利用できる機能がプルリクエストです。

これは、フォークしたリポジトリから元々のリポジトリへ変更を反映させるリクエストを送る機能で、元々のリポジトリの管理者がその変更を承認することで変更が反映されます。

このゼミでは、pull request は Git/GitHub の学習項目として1回体験しますが、B3 の毎週の通常提出では必須にしません。

以上が Git および GitHub の大まかな説明ですが、やはり言葉だけの説明では理解しにくいと思います。実際にこれらの操作を行って、理解を深めましょう。

2.5 実践 GitHub

このゼミでは、教材 repo と自分の作業 repo (提出 repo / submission repo) を分けて使う。第1回の[自分用リポジトリの作成](#)で、ykinolab-tokai 所有の非公開 signal-ml-work-<GitHub ユーザー名> を作成し、作成した repo の README に従って~/workspace/signal-ml-work に clone する。教材 repo は~/workspace/signal-ml-training に置く。取得済みなら、もう一度 clone せず次の手順で既存の作業 repo を開く。未作成・未取得なら、先に第1回と作業 repo の README の手順を完了する。

2.5.1 通常作業を始める

以下のコマンドはターミナル（Windows では WSL の Ubuntu）で実行する。

```
1 cd ~/workspace/signal-ml-work
2 code .
3 git status
```

通常作業は main で行う。他のブランチで練習していた場合は、後述の「PR 練習後に通常作業へ戻る」を先に行う。Git の名前・メールアドレスをまだ設定していない場合は、自分の情報を設定する。

```
1 git config --global user.name ユーザ名"<>"
2 git config --global user.email メールアドレス"<>"
```

この章の操作練習では、作業 repo 内に exercise/git-practice.txt を作り、例えば「Git の操作を確認した」と書く。VS Code で作成してよい。ターミナルで作る場合は次を実行する。

```
1 mkdir -p exercise
2 vim exercise/git-practice.txt
```

Vim では i で入力を始め、入力後に Esc, :wq, Enter の順で保存して終了する。現在地は作業 repo のルートのままにする。演習コードと結果の保存規則は[共通手順](#)に従う。Python を実行するときは、教材側の環境を有効にする。

```
1 source ../signal-ml-training/.venv/bin/activate
```

作成・変更したファイルを確認して、記録・送信する。

```
1 git status
2 git add exercise/git-practice.txt
3 git commit -m "Record Git practice"
4 git push origin main
```

通常の演習でも、対象ファイル名をその回に作成・変更したものへ置き換えて edit -> commit -> push を行う。fork や毎回の PR 作成は不要である。

2.5.2 pull request の練習

main の作業を commit・push し、未 commit の変更がないことを git status で確認してから、練習ブランチを作る。

```
1 git switch -c pr-practice
```

exercise/git-practice.txt に「PR の差分を確認する」と追記し、保存してから次を実行する。

```
1 git add exercise/git-practice.txt
2 git commit -m "Record PR practice"
3 git push -u origin pr-practice
```

GitHub 上で、**自分の作業 repo の main を宛先 (base)**、pr-practice を比較元 (compare) にした PR を作成する。PR 本文に「変更点」「確認方法」「未確認事項」を書く。教材 repo や作業テンプレート repo へ送る PR ではない。練習の追記・レビューへの修正は同じ練習ブランチに commit して push する。

2.5.3 PR 練習後に通常作業へ戻る

次回の通常作業の前に、練習ブランチの変更を保存・commit・push してから次を実行する。

```
1 git status
2 # Continue only with a clean working tree
3 git switch main
4 git pull --ff-only origin main
5 git status
```

最後の表示が main であることを確認してから、次回のファイルを編集する。PR がマージ済みなら、同期後の main に練習の変更も含まれる。レビュー待ちなら、変更は練習ブランチに残り、まだ main には含まれない。ブランチは残し、修正するときは `git switch pr-practice` で戻る。次回の作業が未マージの変更を必要とする場合は、先にレビューとマージの扱いを担当者に確認する。切り替えや同期が失敗した場合はエラーを確認し、変更を削除して進めない。練習ブランチで commit した変更は、`git push origin main` では送信されないことにも注意する。

第 3 章

Python 入門

本章では、プログラミング言語 Python の基礎的な文法を学んでいきます。
次章以降に登場するコードを理解するにあたって必要となる最低限の知識について、最短で習得するのが目標です。

より正確かつ詳細な知識を確認したい場合には、[公式のチュートリアル](#)などを参照してください。

Python にはバージョンとして 2 系と 3 系の 2 つの系統があり、互換性のない部分もあります。

本研修では、3 系である **Python 3.12** 系を標準とします。上流の最新 stable は 2026 年 4 月時点で Python 3.14.4 ですが、この repo は共有環境との整合のため 3.12 系で揃えます。

3.1 Python の特徴

プログラミング言語には、Python 以外にも C 言語や Java、Ruby、R のように様々なものがあります。それぞれの言語がすべての用途に適しているわけではなく、しばしば用途によって得手不得手があります。

木下研では基本的に Python というプログラミング言語を扱います。

その理由は、Python はデータ解析・機械学習のためのライブラリが充実しており、データ解析や機械学習の分野で最もよく使われている言語だからです。

また、Web アプリケーションフレームワークの開発も活発で、データ解析だけでなく Web サービス開発まで同じ言語で統一して行える点も魅力です。

さらには、初学者にとっても学びやすい言語です。

初学者がプログラミングを学び始めるときにつまづきがちな難しい概念が他の言語と比べ多くなく、入門しやすい言語といえます。

まとめると、Python には

- データ解析や機械学習によく使われている
- Web アプリケーションの開発などでもよく使われている
- 初学者がはじめやすい言語

のような魅力があります。

3.2 Python の実行

みなさんは、[Google Colaboratory](#) を利用して Python を書いたことがあると思います。

Google Colab は、Python をさっと書いて実行結果を確認する目的では大変便利です。

一方、コードが長く複雑になってくると、Google Colab 上でプログラミングするのは難しくなってきます。

そこで、本研修では、みなさんの PC 上で Python を実行するやり方に慣れてもらおうと思います。

教材 repo の共有 Python 環境を使う。準備・更新は教材 repo でいい、演習は作業 repo のルートで教材側の `.venv` を有効にして実行する。手順と保存先は [共通手順](#) を参照する。新しい仮想環境は作らず、第1回で用意した環境を使う。

```
1 cd ~/workspace/signal-ml-work
2 source ../signal-ml-training/.venv/bin/activate
```

仮想環境が有効になった状態で `python` と入力すると、Python プログラムを1行ごとに入力・実行できる対話型のインタプリタが起動します。インタプリタを終了するには以下のように入力します。

```
1 >>> exit()
```

また、仮想環境を無効にするには以下のコマンドを入力します。

```
1 $ deactivate
```

3.2.1 文法とアルゴリズム

プログラミングによってある特定の処理をコンピュータで自動化したい場合、**文法**と**アルゴリズム**の2つを理解しておく必要があります。

プログラムでは、まずはじめにコンピュータに命令を伝えるためのルールとなる**文法**を覚える必要があります。

文法を無視した記述があるプログラムは、実行した際にエラーとなり処理が停止します。そのため、文法はしっかりと理解しておく必要があります。

ただし、文法さえ理解していれば十分かというとそうではありません。一般的に、初学者向けのプログラミングの参考書では、この文法だけを取り扱うことも多いのですが、コンピュータに処理を自動化させることが目的であれば、文法だけでなく**アルゴリズム**も理解する必要があります。アルゴリズムとは、どういう順番でどのような処理をしていくかの一連の手順をまとめたものです。

この章では、Python の文法について紹介し、機械学習やディープラーニングで必要となるアルゴリズムについてはこれ以降の章で紹介します。

ここでは以下4つの文法に主眼を置きながら説明していきます。

- 変数
- 制御構文
- 関数
- クラス

3.3 変数

変数 (variable) とは、様々な値を格納することができる、**名前がついた入れ物**です。

この変数に値を格納したり、更新したりすることで、計算結果を一時的に保持しておくことができます。

3.3.1 代入と値の確認

それでは、以下のセルのように、`a` という名前の変数に `1` を**代入**してみましょう。

```
1 a = 1
```


代入は `=` の記号を用います。

数学的には `=` は等しいという意味を持ちますが、Python では「**左辺の変数に、右辺の値を代入する**」という意味になります。

また、これ以降、Python のプログラムは上のようにコードセルを用いて記述します。

対話型インタプリタ上では、変数名だけを記述した行を入力すると、値を確認することができます。

```
1 a
```

```
1
```

このように、変数に格納されている値を確認することができました。

また、値を確認するための他の方法として、`print()` と呼ばれる**関数 (function)** を使用することもできます。

関数について詳しくは後述しますが、`print()` のように Python には予め多くの関数が定義されています。そのような関数を**組み込み関数 (built-in function)** といいます。

```
1 print(a)
```

```
1
```

変数名だけをセルに記入して実行する場合と `print()` を利用する場合の違いについては、後述します。

変数につける名前は、コードを書く人が自由に決めることができます。

ただし、わかりやすい名前をつけることがとても大切です。

例えば、人の名前を格納するための変数が `a` という変数名だと、それがどのような使われ方をするのかを容易に類推することができません。

`name` という名前であれば、ひと目で見て何のための変数かが分かるようになります。

これは、自分のコードを読む他人や、未来の自分にとってコードを理解するための大きな手がかりとなります。

3.3.2 コメント

Python では、`#` の後からその行の終わりまでに存在する全ての文字列は無視されます。

この `#` の後ろに続く部分を**コメント (comment)** と呼び、すでに書かれたコードをコメントにすることを**コメントアウト (comment out)** と言います。

コメントは、コード中に変数の意味や処理の意味をコードを読む人に伝えるためによく使われます。

Jupyter Notebook のコードセルに書かれたコードを行ごとコメントアウトしたい場合は、その行を選択した状態で `Ctrl + /` を入力することで自動的に行の先頭に `#` 記号を挿入することができます。複数行を選択していれば、選択された複数の行が同時にコメントアウトされます。また、コメントアウトされている行を選択した状態で同じキー入力を送ると、コメントアウトが解除されます。これを**アンコメント (uncomment)** と呼ぶこともあります。

下のセルを実行してみましょう。

```
1 # この行及び下の行はコメントアウトされているため実行時に無視されます
2 # print(a)
```

`print(a)` が書かれているにも関わらず、何も表示されませんでした。

これは、`print(a)` 関数が書かれた行がコメントアウトされていたためです。

3.3.3 変数の型

プログラミングで扱う値には種類があります。

Python では、**整数 (integer)**、**実数 (real number)**、**文字列 (string)** などが代表的な値の種類です。

それぞれの種類によって、コンピュータ内での取扱い方が異なり、この種類のことは一般に**型 (type)** と呼びます。

例えば、整数、実数、そして文字列をそれぞれ別々の変数に代入するコードは以下のとおりです。

```
1 a = 1

1 b = 1.2

1 c = 'Chainer'
```

コンピュータの中での取り扱いは異なりますが、Python では**変数の型を自動的に決定する**ため、初めのうちはあまり気にする必要はありません。

ただし、違う型同士の演算では場合によってエラーが発生するなどの問題が生じるため、簡単に型の特徴は把握しておく必要があります。

まずは、上記の a, b, c の型を確認する方法を紹介します。

型の確認は `type()` という組み込み関数を使用します。

```
1 type(a)

int

1 type(b)

float

1 type(c)

str
```

a は `int` という整数の型をもつ変数であり、b は `float` という実数の型をもつ変数です。

この `float` という型の名前は、コンピュータ内で実数を扱うときの形式である**浮動小数点数 (floating-point number)** に由来しています。

c は `str` という文字列の型をもつ変数であり、値を定義するにはシングルクォーテーション ' ' もしくはダブルクォーテーション " " で対象の文字列をくくる必要があります。

Python では、. を含まない連続した数字を `int`、直前・直後も含め . が含まれる連続した数字を `float` だと自動的に解釈します。

例えば、7 や 365 は `int` ですが、2.718、.25、10. などでは `float` になります。

実数の 0 は 0.0 とも .0 とも 0. とも書くことができます。

```
1 type(0)

int

1 type(0.)
```

```
float
1 type(.0)
float
```

例えば、実数の 5 は以下のように書くことができます。

```
1 type(5.0)
float
1 type(5.)
float
```

一方、.5 と書くと、これは 0.5 の略記と解釈されることに注意してください。

```
1 type(.5)
float
1 print(.5)
0.5
```

3.3.4 複数同時の代入

Python では複数の変数に対する代入を一度に行うことができ、**複数同時の代入 (multiple assignment)** と呼びます。

例えば、上記の `a = 1` と `b = 1.2` を同時に一行で記述すると以下ようになります。

```
1 a, b = 1, 1.2
```

3 つ以上の変数に対して、複数同時の代入を行うことも可能です。

```
1 a, b, c = 1, 1.2, 'Chainer'
```

複数同時の代入は後の章で頻出するため、覚えておきましょう。

3.3.5 算術演算子

様々な計算を意味する**演算子**と呼ばれる記号があります。

はじめに紹介するのは**算術演算子 (arithmetic operator)** と呼ばれるもので、2 つの変数または値を取り、1 つの演算結果を返します。

代表的な演算として**四則演算 (加算・減算・乗算・除算)** があります。

四則演算に対応する演算子として、Python では以下の記号が用いられます。

演算	記号
加算 (足し算)	+
減算 (引き算)	-
乗算 (掛け算)	*
除算 (割り算)	/

具体例を見ながら使い方を説明します。

```
1 # 整数と整数で加算 -> 結果は整数
2 1+1
```

```
2
```

このように、演算子を使う際には、**記号の両側に値を書きます**。

このとき、演算子の両側にひとつずつ空白を空けることが多いです。

文法的な意味はありませんが、コードが読みやすくなります。

この空白は Python のコーディング規約である [PEP8](#) でも推奨されています。

```
1 1 + 1
```

```
2
```

値が代入されている変数との演算も下記のように行うことができます。

```
1 a + 2
```

```
3
```

また、int と float は異なる型同士ですが、計算を行うことができます。

int と float の演算結果の型は float になります。

```
1 # 整数と実数で加算 -> 結果は実数
2 a + b
```

```
2.2
```

他の演算子の例を示します。

```
1 # 整数と整数で減算 -> 結果は整数
2 2 - 1
```

```
1
```

```
1 # 実数と整数で減算 -> 結果は実数
2 3.5 - 2
```

```
1.5
```

```
1 # 整数と整数で乗算 -> 結果は整数
2 3 * 5
```

```
15
```

```
1 # 実数と整数で乗算 -> 結果は実数
2 2.5 * 2
```

```
5.0
```

```

1 # 整数と整数で除算 -> 結果は実数
2 3 / 2

```

```
1.5
```

```

1 # 整数と整数で除算 -> 結果は実数
2 4 / 2

```

```
2.0
```

Python 3 では、/ 記号を用いて除算を行う場合、除数（割る数）と被除数（割られる数）が整数であっても、計算結果として実数が返ります。

計算結果として実数を返す除算のことを特に、**真の除算 (true division)** と言います。

一方、商（整数部分）を返すような除算演算子として、// 記号が用意されています。/ 記号を 2 回、間を空けずに繰り返します。計算結果として商を返す除算のことを、**切り捨て除算 (floor division)** と呼びます。

商を計算したい場合に便利な演算子であるため、こちらも覚えておきましょう。

※ Python 2 では、除数も被除数も整数であった場合、/ 記号を用いても切り捨て除算が行われるので注意してください。

```

1 # 整数と整数で切り捨て除算 -> 結果は整数
2 3 // 2

```

```
1
```

```

1 # 整数と整数で切り捨て除算 -> 結果は整数
2 4 // 2

```

```
2
```

また、ここで注意すべき点として、整数や実数と文字列の演算は基本的にエラーになります。

```

1 # error
2 a + c

```

```

-----
TypeError                                Traceback (most recent call last)
<ipython-input-32-e81e582b6fa9> in <module>()
----> 1 a + c

TypeError: unsupported operand type(s) for +: 'int' and 'str'

```

エラーメッセージを読みましょう。

↓ TypeError: unsupported operand type(s) for +: 'int' and 'str'

とされています。「+ にとって int と str はサポートされていない被作用子（+ が作用する対象のこと。operand）です」と書かれています。「int に str を足す」ということはできないというわけです。

int もしくは float と、str の間の加算、減算、除算では上記のエラーが生じます。

ただし、str と int の**乗算**は特別にサポートされており、計算を実行することができます。

```

1 # str と int で乗算
2 c * 3

```

```
'ChainerChainerChainer'
```

上のコードは、c という文字列を 3 回繰り返す、という意味になります。

str 同士は足し算を行うことができます。

```
1 name1 = 'Chainer'
2 name2 = 'チュートリアル'
3
4 name1 + name2
```

```
'チュートリアルChainer'
```

整数と文字列を連結したいこともあります。

例えば、1 という整数に、' 番目' という文字列を足して '1 番目' という文字列を作りたいような場合です。

その場合には、型を変換する**キャスト (cast)** という操作をする必要があります。

何かを int にキャストしたい場合は int() という組み込み関数を使い、str にキャストしたい場合は str() という組み込み関数を使います。では、1 という整数を str にキャストして、' 番目' という文字列と足し算を行ってみましょう。

```
1 1
```

```
1
```

```
1 type(1)
```

```
int
```

```
1 str(1)
```

```
'1'
```

```
1 type(str(1))
```

```
str
```

```
1 str(1) + ' 番目'
```

```
  番目
```

```
'1'
```

また、+= や -= もよく使います。

これは、演算と代入を合わせて行うもので、**累積代入文 (augmented assignment statement)** と呼ばれます。

下記に示すとおり、+= では左辺の変数に対して右辺の値を足した結果で、左辺の変数を更新します。

```
1 # 累積代入文を使わない場合
2 count = 0
3 count = count + 1
4 count
```

```
1
```

```
1 # 累積代入文を使う場合
2 count = 0
3 count += 1
4 count
```

```
1
```

四則演算の全てで累積代入文を利用することができます。

つまり、+=, -=, *=, /= がそれぞれ利用可能です。

Python には、他にも幾つかの算術演算子が用意されています。

例えば以下の演算子です。

演算	記号
累乗	**
剰余	%

** を使うと、 2^3 は以下のように記述することができます。

```
1 # 累乗
2 2 ** 3
```

```
8
```

% を使って、9 を 2 で割った余りを計算してみましょう。

```
1 # 剰余
2 9 % 2
```

```
1
```

3.3.6 比較演算子

比較演算子は、2 つの値の比較を行うための演算子です。

演算	記号
小なり	<
大なり	>
以下	<=
以上	>=
等しい	==
等しくない	!=

比較演算子は、その両側に与えられた値が決められた条件を満たしているかどうか計算し、満たしている場合は True を、満たしていない場合は False を返します。

True や False は、**ブール (bool) 型**と呼ばれる型を持った値です。

ブール型の値は True もしくは False の 2 つしか存在しません。

いくつかの比較演算子の計算例を示します。

```
1 1 < 2
```

```
True
```

```
1 # 型の確認
2 type(1 < 2)
```

```
bool
```

```
1 2 == 5
```

```
False
```

```
1 1 != 2
```

```
True
```

```
1 3 >= 3
```

```
True
```

```
1 'test' == 'test'
```

```
True
```

等しいかどうかを判定する比較演算子 `==` を使う際は、代入演算子 `=` と間違えないように気をつけてください。

3.4 エスケープシーケンス

通常の文字列では表せない特殊な文字を、規定された特別な文字の並びにより表したものを**エスケープシーケンス (escape sequence)** と呼びます。

よく使用するものとして、**改行**を意味する `\n` (もしくは `\n`)、**タブ**を意味する `\t` (もしくは `\t`) があります。

```
1 print('Hello\nWorld')
```

```
Hello
World
```

```
1 print('Hello\tWorld')
```

```
Hello    World
```

最初に Jupyter Notebook 上で変数の値を確認する際に、`print()` を使う場合と使わない場合の違いについて触れましたが、エスケープシーケンスを評価したい場合には、`print()` を使う必要があります。

```
1 d = 'Hello\nWorld'
```

```
1 # エスケープシーケンスが評価されない
2 d
```



```
'Hello\nWorld'
```

```
1 # エスケープシーケンスが評価される
2 print(d)
```

```
Hello
World
```

3.5 文字列メソッド

`str` 型の変数には、いくつか便利な機能がついています。

例えば、その変数を持つ全ての文字を小文字や大文字に変換する `lower()` や `upper()` といった機能があります。

このような型が持っている関数を**メソッド (method)** と呼びます。

```
1 name = 'Chainer'
2
3 name
```

```
'Chainer'
```

```
1 # すべてを小文字に変換
2 name.lower()
```

```
'chainer'
```

```
1 # すべてを大文字に変換
2 name.upper()
```

```
'CHAINER'
```

3.6 f 文字列

文字列の特殊な (かつ便利な) 書き方として **f 文字列** があります。

これは、ある文字列の一部分に、あとから別な文字列を埋め込むために使用します。

f 文字列は、`f` というキーワードではじまり、その後に通常の文字列と同じく `'` (または `"`) で文字列を囲みます。

このとき、を文字列中に記述すると、そこに別の文字列を埋め込むことができます。

```
1 name = 'Chainer'
2
3 f'{name}_チュートリアルへようこそ'
```

```
'Chainer チュートリアルへようこそ'
```

```
1 name1 = 'Chainer'
2 name2 = 'チュートリアル'
3
4 f'{name1}_{name2}へようこそ'
```

```
'Chainer チュートリアルへようこそ'
```

f 文字列を用いると int 型 や float 型 の変数を、str 型 へ明示的にキャストすることなく文字列に埋め込むことができます。

```
1 version = 3.7
2
3 f'Python_{version}'
```

```
'Python 3.7'
```

3.7 浮動小数点数がもつメソッド

「メソッド」は str 型 の変数だけが持つものではありません。

int 型 の変数や、float 型 の変数にも、その型の特徴に合わせた機能が、メソッドとして提供されています。

例えば、float 型 の変数には、as_integer_ratio() というメソッドがあり、比がその浮動小数点数の値となるような整数の組を返します。

例えば、0.5 という値は、分数で表すと $\frac{1}{2}$ です。

これは、以下のようにして調べることができます。

```
1 0.5.as_integer_ratio()
```

```
(1, 2)
```

0.25 であれば、 $\frac{1}{4}$ となります。

```
1 0.25.as_integer_ratio()
```

```
(1, 4)
```

このような、型に紐付いたメソッドなどについては、この章の最後にある「クラス」という概念の説明の際にもう少し詳しく解説します。

3.8 複合データ型

これまでは `a = 1` のように 1 つの変数に 1 つの値を代入する場合を扱ってきましたが、複数の値をまとめて取り扱いたい場面もあります。

Python では複数の変数や値をまとめて扱うのに便利な、以下の 3 つの複合データ型があります。

- リスト (list)
- タプル (tuple)
- 辞書 (dictionary)

3.8.1 リスト

複数の変数を、(カンマ) 区切りで並べ、それらの全体を `[ ]` で囲んだものを **リスト (list)** と言います。

リストに含まれる値を**要素**と呼び、整数の**インデックス** (要素番号) を使ってアクセスします。

```
1 # リスト型の変数を定義
2 numbers = [4, 5, 6, 7]
3
4 # 値の確認
5 print(numbers)
```

```
[4, 5, 6, 7]
```

```
1 # 型の確認
2 type(numbers)
```

```
list
```

`numbers` には 4 つの数値が入っており、**要素数** は 4 です。

リストの要素数は、リストの**長さ** (`length`) と呼ばれ、組み込み関数の `len()` を用いて取得することができます。

`len()` はよく使う関数であるため、覚えておきましょう。

```
1 # 要素数の確認
2 len(numbers)
```

```
4
```

リストの各要素へアクセスする方法はいくつかあります。

最も簡単な方法は `[]` を使ってアクセスしたい要素番号を指定して、リストから値を取り出したり、その位置の値を書き換えたりする方法です。

ここで、注意が必要な点として、Python では先頭の要素のインデックス番号が 0 である点があります。

インデックス番号 1 は 2 番目の要素を指します。

```
1 # 先頭の要素にアクセス
2 numbers[0]
```

```
4
```

```
1 # 先頭から番目の要素にアクセス3
2 numbers[2]
```

```
6
```

```
1 # 2 番目の要素を書き換え
2 numbers[1] = 10
```

```
1 # 値の確認
2 numbers
```

```
[4, 10, 6, 7]
```

また、インデックスに負の値を指定すると、末尾からの位置となります。

要素番号 `-1` で最後の要素を参照することができます。

```
1 # 末尾の要素にアクセス
2 numbers[-1]
```

```
7
```

```
1 # 末尾から番目の要素にアクセス3
2 numbers[-3]
```

```
10
```

次に、リストから一度に複数の要素を取り出す操作である**スライス (slice)** を紹介します。

開始位置: 終了位置 のようにコロン : を用いてインデックスを範囲指定し、複数の部分要素にアクセスします。

このスライスの処理は、この後の章でも多用するため、慣れておきましょう。

例えば、先頭から 2 つの要素を取り出したい場合、以下のように指定します。

```
1 numbers[0:2]
```

```
[4, 10]
```

開始位置: 終了位置 と指定することで、開始位置から**終了位置のひとつ手前**までの要素を抽出します。

終了位置に指定したインデックスの値は含まれないことに注意してください。

また、指定する開始番号が 0 である場合、以下のような略記がよく用いられます。

```
1 numbers[:2]
```

```
[4, 10]
```

このように、先頭のインデックスは省略することができます。

このような記法を使う場合は、終了位置を示す数字を**取り出したい要素の個数**と捉えて、**先頭から 2 つを取り出す**操作だと考えると分かりやすくなります。

同様に、ある位置からリストの末尾までを取り出す場合も、終了位置のインデックスを省略することができます。

例えば、2 個目の要素から最後までを取り出すには以下のようにします。

```
1 numbers[1:]
```

```
[10, 6, 7]
```

この場合は、取り出される要素の個数は `len(numbers) - 1` 個となることに注意してください。

以上から、`numbers[:2]` と `numbers[2:]` は、ちょうど 2 個目の要素を境に `numbers` の要素を 2 分割した前半部分と後半部分になっています。

ここで、インデックスが 2 の要素自体は**後半部に含まれる**ということに注意してください。

また、開始位置も終了位置も省略した場合は、すべての要素が選択されます。

```
1 numbers[:]
```

```
[4, 10, 6, 7]
```

現状では、`numbers[:]` と `numbers` の結果が同じであるため、どのように使用するか疑問に思われるかも知れません。

しかし、後の章では NumPy というライブラリを用いてリストの中にリストが入ったような**多次元配列 (multidimensional array)** を扱っていきます。

そして多次元配列を用いて行列を表す場合には、0 **列目のすべての値**を抽出するために `[:, 0]` のような記法を用いるケースが登場します。

これは Python 標準の機能ではありませんが、Python 標準のスライス表記を拡張したものになっています。

リストは数値以外に、文字列を扱うこともでき、また複数の型を同一のリスト内に混在させることもできます。

```
1 # 文字列を格納したリスト
2 array = ['hello', 'world']
3 array
```

```
['hello', 'world']
```

```
1 # 複数の型が混在したリスト
2 array = [1, 1.2, 'Chainer']
3 array
```

```
[1, 1.2, 'Chainer']
```

リストにリストを代入することもできます。

また、Python 標準のリストでは入れ子になったリスト内の要素数がバラバラでも問題ありません。

```
1 array = [[1, 1.2, 'Chainer', True], [3.2, 'Tutorial']]
2 array
```

```
[[1, 1.2, 'Chainer', True], [3.2, 'Tutorial']]
```

リストを使う際に頻出する操作として、**リストへの値の追加**があります。

リスト型には `append()` というメソッドが定義されており、これを用いてリストの末尾に新しい値を追加することができます。

上記の `array` に値を追加してみましょう。

```
1 # 末尾に 2.5 を追加
2 array.append(2.5)
```

```
1 # 値の確認
2 array
```

```
[[1, 1.2, 'Chainer', True], [3.2, 'Tutorial'], 2.5]
```

また、今後頻出する処理として、**空のリスト**を定義しておき、そこに後段の処理の中で適宜新たな要素を追加していくという使い方があります。

```
1 # 空のリストを定義
2 array = []
3
4 # 空のリストに要素を追加
5 array.append('Chainer')
6 array.append('チュートリアル')
```

```
7
8 array
```

```
['Chainer', チュートリアル']
```

3.8.2 タプル

タプル (tuple) はリストと同様に複数の要素をまとめた型ですが、リストとは異なる点として、定義した後に**中の要素を変更できない**という性質を持ちます。

タプルの定義には `( )` を用います。

```
1 # タプルを定義
2 array = (4, 5, 6, 7)
3 array
```

```
(4, 5, 6, 7)
```

```
1 # 型の確認
2 type(array)
```

```
tuple
```

タプルの定義する際に `( )` を使用したため、要素へのアクセスも `( )` を使うように感じるかもしれませんが、実際にはリストと同様 `[ ]` を使用します。

```
1 # 先頭の要素へアクセス
2 array[0]
```

```
4
```

```
1 # リストと同様、スライスも使用可能
2 array[:3]
```

```
(4, 5, 6)
```

先述の通り、タプルは各要素の値を変更することができません。

この性質は、定数項などプログラムの途中で書き換わってしまうことが望ましくないものをまとめて扱うのに便利です。

実際に、タプルの要素に値の書き換えを行うとエラーが発生します。

```
1 # error
2 array[0] = 10
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-86-a7859ae56fbc> in <module>()
----> 1 array[0] = 10

TypeError: 'tuple' object does not support item assignment
```

`tuple` のように中身が変更できない性質のことを**イミュータブル (immutable)** であると言います。反対に、`list` のように中身が変更できる性質のことを**ミュータブル (mutable)** であると言います

タプルも Chainer でデータセットを扱うときなどに頻出する型です。その性質と取り扱い方を覚えておきましょう。

3.8.3 辞書

リストやタプルでは、複数の値をまとめて扱うことができました。

そこで、定期テストの結果をまとめることを考えてみましょう。

例えば、数学 90 点、理科 75 点、英語 80 点だったという結果を `scores = [90, 75, 80]` とリストで表してみます。

しかし、これでは**何番目がどの教科の点数に対応するか**、一見して分かりにくいと思われれます。

Python の `dict` 型は、**キー (key)** とそれに対応する**値 (value)** をセットにして格納することができる型であり、このようなときに便利です。

リストやタプルでは、各要素にアクセスする際に整数のインデックスを用いていましたが、辞書ではキーでインデックス化されているため、整数や文字列など、色々なものを使って要素を指定することができます。

辞書は `{}` を用いて定義し、要素にアクセスする際には、リストやタプルと同様に `[]` を使用し、`[]` の中にキーを指定して対応する値を取り出します。

```
1 # 辞書を定義
2 scores = {'Math': 90, 'Science': 75, 'English': 80 }
3 scores
```

```
{'English': 80, 'Math': 90, 'Science': 75}
```

```
1 # key が Math の value にアクセス
2 scores['Math']
```

```
90
```

```
1 # key に日本語を使用することも可能
2 scores = {'数学': 90, '理科': 75, '英語': 80}
3 scores
```

```
数学
{'': 90, '理科': 75, '英語': 80}
```

```
1 scores['数学']
```

```
90
```

他の人が定義した辞書に、**どのようなキーが存在するか**を調べたいときがあります。

辞書には、そのような場合に使える便利なメソッドがいくつか存在します。

- `keys()`: キーのリストを取得。 `dict_keys` というリストと性質が似た型が返る
- `values()`: 値のリストを取得。 `dict_values` というリストと性質が似た型が返る
- `items()`: 各要素の (key, value) のタプルが並んだリストを取得。 `dict_items` というリストと性質が似た型が返る

```
1 # キーのリスト
2 scores.keys()
```

```
dict_keys(数学('', 理科'', 英語''))
```

```
1 # 値のリスト
2 scores.values()
```

```
dict_values([90, 75, 80])
```

```
1 # キー(, 値)というタプルを要素とするリスト
2 scores.items()
```

```
dict_items(数学([(' ', 90), 理科(' ', 75), 英語(' ', 80)])
```

`dict_keys`, `dict_values`, `dict_items` と新しい型が登場しましたが、これは辞書型特有の型であり厳密には標準のリストとは異なりますが、リストと性質の似た型であるという程度の認識で問題ありません。

辞書に要素を追加する場合は、新しいキーを指定して値を代入します。

```
1 scores['国語'] = 85
```

```
1 scores
```

```
国語
{' ': 85, 数学'': 90, 理科'': 75, 英語'': 80}
```

また、既に存在するキーを指定した場合には、値が上書きされます。

```
1 scores['数学'] = 95
```

```
1 scores
```

```
国語
{' ': 85, 数学'': 95, 理科'': 75, 英語'': 80}
```

3.9 Python プログラムのコマンドライン実行

だんだんとプログラムが長くなってくると、プログラムを1行1行インタプリタに入力していくのは大変です。

Python には、テキストファイルに記述された Python プログラムを一度に実行する機能があるので、これを利用しましょう。

一度、Python インタプリタを `exit()` により終了し、以下のコマンドを入力しましょう。

```
1 $ cd ~/workspace/signal-ml-work
2 $ mkdir -p exercise
```

その後、`exercise/sample.py` という名前のテキストファイルを作成して、以下のプログラムを書き込みましょう。

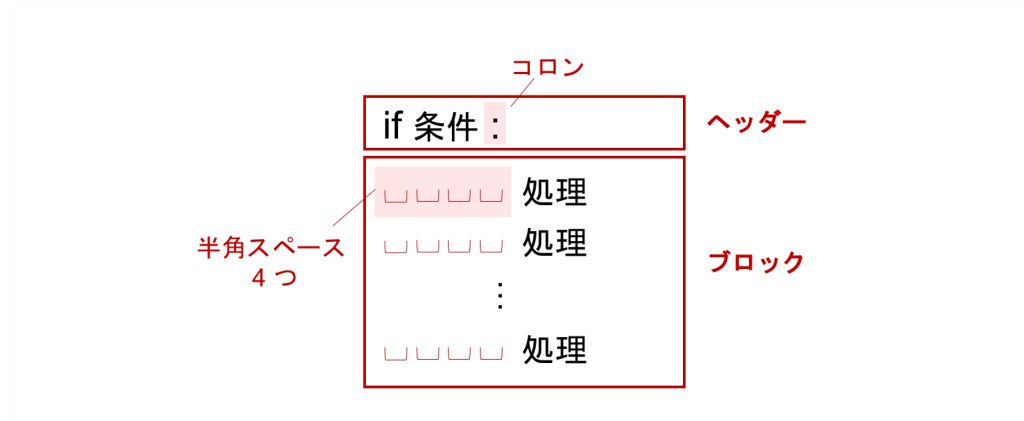


図 3.1 ヘッダーとブロック

```

1 array = [50, 80, 40]
2 s = 0
3 s += array[0]
4 s += array[1]
5 s += array[2]
6 s /= 3
7 print(s)

```

この Python プログラムでは、`array` というリストの要素の平均値を求めています。

このプログラムを実行するには、`python` コマンドの後ろに、プログラムを記述したファイルの名前を入力します。

すなわち、以下のコマンドを入力します。

```

1 $ python exercise/sample.py

```

すると、すべての行が実行され、`s` の値が画面に表示されます。

3.10 制御構文

複雑なプログラムを記述しようとする、繰り返しの処理や、条件によって動作を変える処理が必要となります。

これらは**制御構文**を用いて記述します。

ここでは最も基本的な制御構文を 2 つ紹介します。

- 繰り返し (`for`, `while`)
- 条件分岐 (`if`)

Python の制御構文は、**ヘッダ** (header) と **ブロック** (block) と呼ばれる 2 つの部分で構成されています。これらを合わせて **複合文** (compound statement) と呼びます。

上図に示すように、制御構文ではヘッダ行に `for` 文や `if-else` 句を記述し、行末に `:` 記号を書きます。次に、ヘッダ行の条件で実行したい一連の処理文を、ブロックとしてその次の行以降に記述していきます。その際、**インデント** (indent) と呼ばれる空白文字を先頭に挿入することで、ブロックを表現します。同じ数の空白でインデントされた文がブロックとみなされます。

Python では、インデントとして**スペース 4 つ**を用いることが推奨されています。

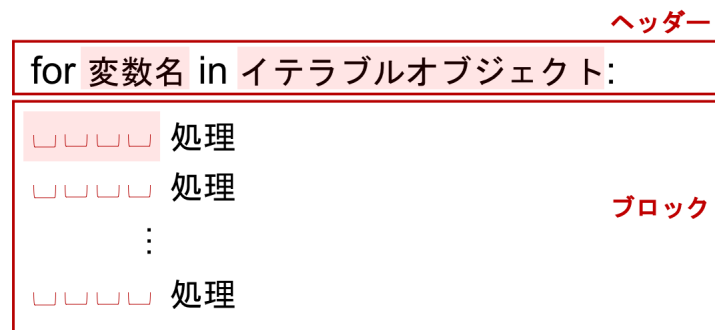


図 3.2 for 文

3.10.1 繰り返し (for 文)

同じ内容のメールを宛名だけ個別に変えて、1000 人に一斉送信したい場合など、繰り返す処理を記述する制御構文である `for` を使います。

`for` 文の文法は上図のとおりです。

イテラブルオブジェクト (iterable object) とは、反復可能オブジェクトのことであり、要素を一度に 1 つずつ返せるオブジェクトのことを指します。

`range()` という組み込み関数を使うと、引数に与えた整数の回数だけ順番に整数を返すイテラブルオブジェクトを作ることができます。

`range(5)` と書くと、0, 1, 2, 3, 4 という整数 5 つを順番に返すイテラブルオブジェクトになります。

後述しますが、このイテラブルオブジェクトとして、リストやタプルも指定することができます。

```

1 # 回繰り返す5
2 for i in range(5):
3     print(i)
  
```

```

0
1
2
3
4
  
```

上記の例では、イテラブルオブジェクトが 1 つずつ返す値を変数 `i` で受け取っています。

最初は `i = 0` から始まっていることに注意してください。

最後の値も、5 ではなく 4 となっています。

このように、`range()` に 1 つの整数を与えた場合は、その整数 - 1 まで 0 から 1 つずつ増えていく整数を順番に返します。

```

1 # 繰り返し処理が終わった後の値の確認
2 i
  
```

```

4
  
```

Jupyter Notebook では変数名をコードセルの最後の行に書いて実行するとその変数に代入されている値を確認できましたが、`for` 文の中のブロックでは明示的に `print()` を使う必要があります。

`print()` を用いないと、以下のように何も表示されません。

```
1 # 変数の値は表示されない
2 for i in range(5):
3     i
```

`for` 文を使って、0 から始まって 1 ずつ大きくなっていく整数順番に取得し、これをリストのインデックスに利用すれば、リストの各要素に順番にアクセスすることができます。

```
1 names = ['佐藤', '鈴木', '高橋']
2
3 for i in range(3):
4     print(names[i])
```

佐藤鈴木高橋

少し応用して、自動的に敬称をつけて表示してみましょう。

```
1 for i in range(3):
2     print('さん{}'.format(names[i]))
```

佐藤さん鈴木さん高橋さん

つぎに、さらに汎用性の高いプログラムを目指します。

上記のコードに関して、汎用性が低い点として、`range(3)` のように 3 という値を直接記述していることが挙げられます。

この 3 はリストの要素の数を意味していますが、リストの要素の数が変わると、このプログラムも書き換える必要があり、手間がかかったり、ミスが発生する原因となったりします。

リスト内の要素の数は、組み込み関数である `len()` を用いて取得できるため、これを使用した汎用性の高いプログラムに書き換えましょう。

```
1 len(names)
```

3

```
1 for i in range(len(names)):
2     print('さん{}'.format(names[i]))
```

佐藤さん鈴木さん高橋さん

これでリストの要素数に依存しないプログラムにすることができました。

また、リスト自体をイテラブルオブジェクトとして指定することにより、リスト要素数の取得も `[]` でのインデックス番号の指定もせずに、より可読性の高いプログラムを書くことができます。

```
1 # リストをイテラブルオブジェクトに指定
2 for name in names:
3     print(f'{name}さん')
```

佐藤さん鈴木さん高橋さん

最初のケースと比べていかがでしょうか。

動作としては変わりありませんが、可読性という観点も重要です。

リストをイテラブルオブジェクトとして指定した場合、要素番号を取得できませんが、状況によっては要素番号を使用したいことがあります。

そのような場合は、`enumerate()` という組み込み関数を使います。

これにイテラブルオブジェクトを渡すと、**(要素番号, 要素)** というタプルを 1 つずつ返すイテラブルオブジェクトになります。

```
1 for i, name in enumerate(names):
2     message = f'{i番目}: {nameさん}'
3     print(message)
```

番目

0: 佐藤さん番目

1: 鈴木さん番目

2: 高橋さん

`enumerate()` と同様、`for` 文と合わせてよく使う組み込み関数に `zip()` があります。

`zip()` は、複数のイテラブルオブジェクトを受け取り、その要素のペアを順番に返すイテラブルオブジェクトを作ります。

このイテラブルオブジェクトは、渡されたイテラブルオブジェクトそれぞれの先頭の要素から順番に、タプルに束ねて返します。

このイテラブルオブジェクトの長さは、渡されたイテラブルオブジェクトのうち最も短い長さと一致します。

```
1 names = ['Python', 'Chainer']
2 versions = ['3.7', '5.3.0']
3 suffixes = ['!!', '!!', '?']
4
5 for name, version, suffix in zip(names, versions, suffixes):
6     print(f'{name} {version} {suffix}')
```

Python 3.7 !!

Chainer 5.3.0 !!

`suffixes` の要素数は 3 ですが、より短いイテラブルオブジェクトと共に `zip` に渡されたため、先頭から 2 つ目までしか値が取り出されていません。

3.10.2 条件分岐 (if 文)

`if` は、指定した条件が `True` か `False` かによって、処理を変えるための制御構文です。

`elif` と `else` は任意であり、`elif` は 1 つだけでなく複数連ねることができます。

例えば、0 より大きいことを条件とした処理を書いてみます。

```
1 # if の条件を満たす場合
2 a = 1
3
4 if a > 0:
5     print('より大きいです0')
6 else:
7     print('以下です0')
```

より大きいです

0

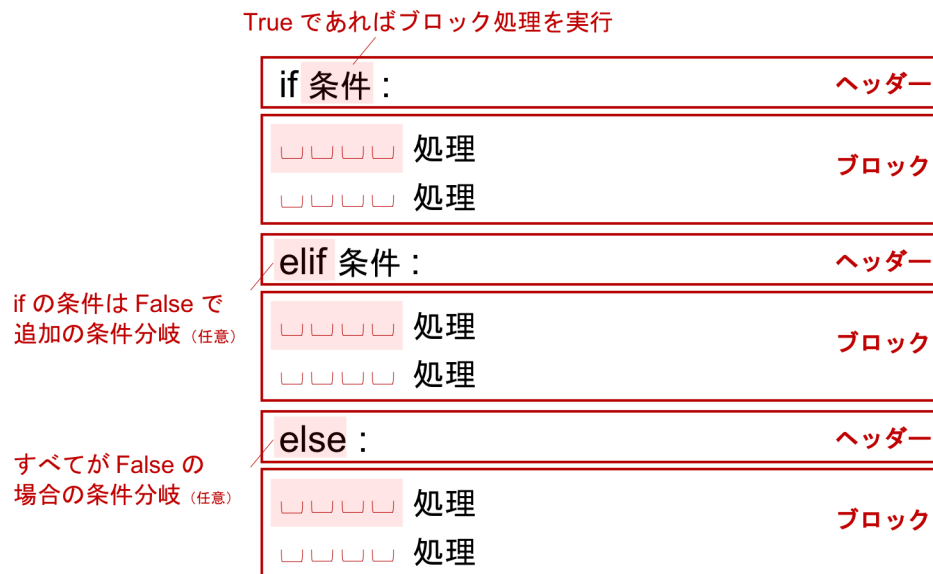


図 3.3 if 文

```

1 # if の条件を満たさない場合
2 a = -1
3
4 if a > 0:
5     print('より大きいです0')
6 else:
7     print('以下です0')

```

```

以下です
0

```

また、if に対する条件以外の条件分岐を追加する場合は、下記のように elif を使います。

```

1 a = 0
2
3 if a > 0:
4     print('より大きいです0')
5 elif a == 0:
6     print('0です')
7 else:
8     print('より小さいです0')

```

```

0です

```

3.10.3 繰り返し (while 文)

繰り返し処理は、for 以外にも while を用いて記述することもできます。

while 文では、以下のようにループを継続する条件を指定します。

指定された条件文が True である限り、ブロックの部分に記述された処理が繰り返し実行されます。

while 文を使用した 3 回繰り返すプログラムは下記のとおりです。

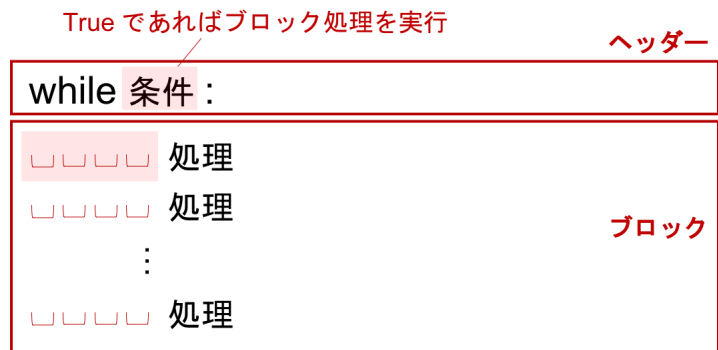


図 3.4 while 文

```

1 count = 0
2
3 while count < 3:
4     print(count)
5     count += 1

```

```

0
1
2

```

ここで使われている `count` という変数は、ループの中身が何回実行されたかを数えるために使われています。

まず 0 で初期化し、ループ内の処理が一度行われるたびに `count` の値に 1 を足しています。

この `count` を使った条件式を `while` 文に与えることで、ループを回したい回数を指定しています。

一方、`while True` と指定すると、`True` は変数ではなく値なので、変更されることはなく、ループは無限に回り続けます。

`while` 文自体は無限ループの状態にしておき、ループの中で `if` 文を使って、ある条件が満たされた場合はループを中断する、という使い方ができます。

これには `break` 文が用いられます。

以下は、`break` 文を使って上のコードと同様に 3 回ループを回すコードです。

```

1 count = 0
2
3 while True:
4     print(count)
5     count += 1
6
7     if count == 3:
8         break

```

```

0
1
2

```

`count` の値が 3 と等しいかどうか毎回チェックされ、等しくなっていれば `break` 文が実行されて `while` ループが終了します。

`while` 文を使って、指定された条件を満たして**いない**間ループを繰り返すという処理も書くことができます。`while` 文自体の使い方は同じですが、条件を反転して与えることで、与えた条件が `False` である間繰り返されるようにすることができます。

これには、ブール値を反転する `not` を用います。

`not True` は `False` を返し、`not False` は `True` を返します。

```
1 not True
```

```
False
```

```
1 not False
```

```
True
```

```
1 not 1 == 2
```

```
True
```

このように、`not` はあとに続くブール値を反転します。

これを用いて、`count` が 3 **ではない**限りループを繰り返すというコードを `while` 文を使って書いてみましょう。

```
1 count = 0
2
3 while not count == 3:
4     print(count)
5     count += 1
```

```
0
1
2
```

3.11 関数

何かひとまとまりの処理を書いた際には、その処理のためのコードをまとめて、プログラム全体の色々な箇所から再利用できるようにしておくと、便利な場合があります。

ここでは、処理をひとまとめにする方法の一つとして**関数 (function)** を定義する方法を紹介します。

3.11.1 関数を定義する

例えば、**受け取った値を 2 倍して表示する関数**を作ってみましょう。

関数を定義するには、まず名前を決める必要があります。

今回は `double()` という名前の関数を定義してみます。

関数も制御構文と同じく**ヘッダー**と**ブロック**を持っています。

```
1 # 関数 double() の定義
2 def double(x):
3     print(2 * x)
```

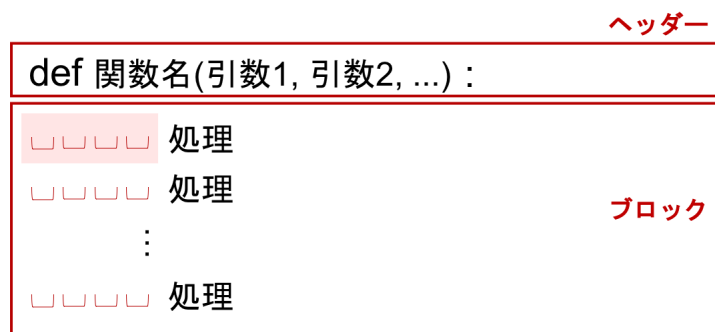


図 3.5 関数の定義

関数は定義されただけでは実行されません。

定義した関数を使用するためには、定義を行うコードとは別に、実行を行うコードが必要です。

```
1 # 関数の実行
2 double(3)
```

```
6
```

```
1 double(5)
```

```
10
```

```
1 double(1.5)
```

```
3.0
```

`double(x)` における `x` のように、関数に渡される変数や値のことを**引数 (argument)** といいます。

上の例は、名前が `double` で、1つの引数 `x` をとり、`2 * x` という計算を行い、その結果を表示しています。

3.11.2 複数の引数をとる関数

複数の引数をとる関数を定義する場合は、関数名に続く `()` の中に、カンマ , 区切りで引数名を並べます。

例えば、引数を 2 つとり、足し算を行う関数 `add()` を作ってみましょう。

```
1 # 関数の定義
2 def add(a, b):
3     print(a + b)
```

```
1 # 関数の実行
2 add(1, 2)
```

```
3
```

```
1 add(3, 2.5)
```

```
5.5
```



```
1 add(1, -5)
```

```
-4
```

今回の `double()` や `add()` は定義を行い自作した関数ですが、Python には予め多くの関数が定義されています。

そのような関数を**組み込み関数 (built-in function)** と呼びます。

すでに使用している `print()` や `len()`, `range()` などが、これに該当します。

組み込み関数の一覧は[こちら](#)で確認することができます。

3.11.3 引数をとらない関数

引数をとらない関数を定義する場合でも、関数名の後に `()` を加える必要があります。

例えば、実行するとメッセージを表示する関数を定義して、実行してみましょう。

```
1 # 引数のない関数の定義
2 def hello():
3     print('チュートリアルによこそChainer')
```

```
1 # 引数のない関数の実行
2 hello()
```

```
チュートリアルによこそ
Chainer
```

3.11.4 引数のデフォルト値

引数には、あらかじめ値を与えておくことができます。

これは、引数をとる関数を定義する際に、何も引数に値が渡されなかったときにどのような値がその引数に渡されたことにするかをあらかじめ決めておける機能で、その値のことを**デフォルト値**と呼びます。

例えば、上の `hello()` という関数に、`message` という引数をもたせ、そこにデフォルト値を設定しておきます。

```
1 def hello(message='チュートリアルによこそChainer'):
2     print(message)
```

この関数は引数に何も与えずに呼び出すと、「Chainer チュートリアルによこそ」というメッセージを表示し、引数に別な値が渡されると、その値を表示します。

```
1 hello()
```

```
チュートリアルによこそ
Chainer
```

```
1 hello('Welcome to Chainer tutorial')
```

```
Welcome to Chainer tutorial
```

デフォルト値が与えられていない引数は、関数呼び出しの際に必ず何らかの値が渡される必要がありますが、デフォルト値を持つ場合は、何も指定しなくても関数を呼び出すことができますようになります。

3.11.5 返り値のある関数

上で定義した足し算を行う関数 `add()` では、計算結果を表示するだけで、計算結果を呼び出し元に戻していませんでした。

そのため、このままでは計算結果を関数の外から利用することができません。

そこで、`add()` 関数の末尾に `return` 文を追加して、計算結果を呼び出し元に返すように変更してみましょう。

```
1 # 返り値のある関数の定義
2 def add(a, b):
3     return a + b
```

このように、呼び出し元に戻したい値を `return` に続いて書くと、その値が `add()` 関数を呼び出したところへ戻されます。

`return` で返される値のことを**返り値 (return value)** と言います。

以下に、計算結果を `result` という変数に格納し、表示する例を示します。

```
1 result = add(1, 3)
2
3 result
```

4

計算結果が `result` に格納されているので、この結果を用いてさらに別の処理を行うことができます。

```
1 result = add(1, 3)
2
3 result_doubled = result * 2
4
5 result_doubled
```

8

また、返り値は「呼び出し元」に返されると書きました。

この「呼び出し元」というのは、関数を呼び出す部分のことで、上のコードは `add(1, 3)` の部分が 4 という結果の値になり、それが左辺の `result` に代入されています。

これを用いると、例えば「2 と 3 を足した結果と、1 と 3 を足した結果を、掛け合わせる」という計算が、以下のように書けます。

```
1 add(2, 3) * add(1, 3)
```

20

3.11.6 変数のスコープ

関数の中で定義した変数は基本的には関数の外では利用できません。

例えば、以下の例を見てみましょう。

```
1 a = 1
2
3 # 関数の内部で a に 2 を代入
```

```

4 def change():
5     a = 2
6
7 change()
8
9 a

```

```
1
```

関数の外で `a = 1` と初期化した変数と同じ名前の変数に対して、`change()` 関数の内部で `a = 2` という代入を行っているにもかかわらず、`change()` 関数の実行後にも関数の外側では `a` の値は `1` のままになっています。

関数の外側で定義された変数 `a` に、関数内部での処理が影響していないことがわかります。

なぜこうなるかという、関数の中で変数に値が代入されるとき、その変数はその関数の**スコープ (scope)** でだけ有効な**ローカル変数**になり、関数の外にある同じ名前の変数とは別のものを指すようになるためです。

スコープとは、その変数が参照可能な範囲のことです。

上の例では、`a = 2` の代入を行った時点で `change()` 関数のスコープに `a` という変数が作られ、`change()` 関数の中からは `a` といえばこれを指すようになります。関数から抜けると、`a` は `1` を値に持つ外側の変数を指すようになります。

ただし、代入を行わずに、参照するだけであれば、関数の内側から外側で定義された変数を利用することができます。

```

1 a = 1
2
3 def change():
4     print('From inside:', a)
5
6 change()
7
8 print('From outside:', a)

```

```

From inside: 1
From outside: 1

```

この場合は、`change()` 関数のスコープには `a` という変数は作られないので、関数の中で `a` といえば外側で定義された変数を指します。

関数の外で定義された変数は**グローバル変数**と呼ばれます。

グローバル変数は、特に特別な記述を要せず参照することはできますが、関数の中で**代入**を行う場合は、`global` 文を使って、代入先をグローバル変数とする宣言を行う必要があります。

```

1 a = 1
2
3 def change():
4     global a    # a をグローバル変数である宣言
5     a = 2       # グローバル変数への代入
6
7 # 関数の実行
8 change()
9
10 # 結果の確認 <- a の値が上書きされている
11 a

```

2

`global a` という行を `change()` 関数内で `a` という変数を使用する前に追加すると、その行以降は `a` という変数への代入も関数の外側で定義されたグローバル変数の `a` に対して行われます。

3.12 クラス

オブジェクト指向プログラミング (object-oriented programming) の特徴の一つである **クラス (class)** は、**オブジェクト (object)** を生成するための設計図にあたるものです。

まず、クラスとは何か、オブジェクトとは何かについて説明します。

ここで、唐突に感じられるかもしれませんが、家を何軒も建てる時のことを考えましょう。

それぞれの家の形や大きさ、構造は同じですが、表札に書かれている名前は異なっているとします。

この場合、家の設計図は同じですが、表札に何と書くか、において多少の変更がそれぞれの家ごとに必要となります。

この**全ての家に共通した設計図の役割を果たすのがクラス**です。

そして、設計図は、家として現実に存在しているわけではありませんが、個別の家は、現実に家としての**実体**を持って存在しています。

よって、**設計図に基づいて個別の家を建てる**ということを抽象的に言うと、**クラスから実体を作成する**、となります。

クラスから作成された実体のことを**インスタンス (instance)** または**オブジェクト (object)** とも呼び、**クラスから実体を作成する**という操作のことを**インスタンス化 (instantiation)** と呼びます。

3.12.1 クラスの定義

それでは、家の設計図を表す `House` というクラスを定義してみましょう。

`House` クラスには、インスタンス化されたあとに、各インスタンス、すなわち誰か特定の人の家ごとに異なる値を持つ、`name_plate` という変数を持たせてみます。

`name_plate` という変数には、個別の家の表札に表示するための文字列が与えられますが、クラスを定義する際には「`name_plate` という変数を持つことができる」ようにしておくだけでよく、**実際にその変数に何か具体的な値を与える必要はありません**。

クラスは、**設計図**であればよく、具体的な値を持たせなくてもよいからです。

具体的な値は、個別の家を作成するとき、すなわちインスタンス化の際に与え、各インスタンスが `name_plate` という値に自分の家の表札の名前を保持するようにします。

このような、インスタンスに属している変数を**属性 (attribute)** と呼びます。同様に、インスタンスから呼び出すことができる関数のことを**メソッド (method)** と呼びます。

クラスは、以下のような構文を使って定義します。

具体的には、以下のようになります。

```
1 # クラスの定義
2 class House:
3
4     # __init__() メソッドの定義
5     def __init__(self, name):
6         self.name_plate = name
```

ここで、`__init__()` という名前のメソッドが `House` クラスの中に定義されています。

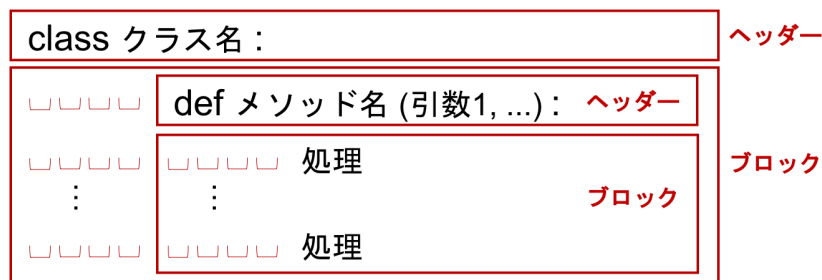


図 3.6 クラス

メソッドの名前は自由に名付けることができますが、いくつか特別な意味を持つメソッド名が予め決められています。

`__init__()` はそういったメソッドの一つで、**インスタンス化する際に自動的に呼ばれるメソッド**です。

`House` クラスの `__init__()` は、`name` という引数を取り、これを `self.name_plate` という変数に代入しています。

この `self` というのは、クラスがインスタンス化されたあと、作成されたインスタンス自身を参照するのに用いられます。

これを使って、`self.name_plate = name` とすることで、作成された個別のインスタンスに属する変数 `self.name_plate` へ、引数に渡された `name` が持つ値を代入することができます。

`self` が指すものは、各インスタンスから見た「自分自身」なので、各インスタンスごとに異なります。

これによって、`self.name_plate` は各インスタンスに紐付いた別々の値を持つものとなります。

メソッドは、インスタンスから呼び出されるとき自動的に第一引数にそのインスタンスへの参照を渡します。そのため、メソッドの第一引数は `self` とし、渡されてくる自分自身への参照を受け取るようにしています。

ただし、呼び出す際には**そのインスタンスを引数に指定する必要はありません**。

以下に具体例を示し、再度このことを確認します。

それでは、上で定義した `House` クラスのインスタンスを作成してみます。

クラスのインスタンス化には、クラス名のあとに `()` を追加して、クラスを呼び出すような記法を使います。この際、関数を呼び出すときと同様にして、`()` に引数を渡すことができます。

その引数は、`__init__()` メソッドに渡されます。

```
1 my_house = House('Chainer')
```

`House` というクラスの `__init__()` メソッドに、`'Chainer'` という文字列を渡しています。

`my_house` が、`House` クラスから作成されたインスタンスです。

ここで、クラス定義では `__init__()` メソッドは `self` と `name` という 2 つの引数をとっていましたが、呼び出しの際には `'Chainer'` という一つの引数しか与えていませんでした。

この `'Chainer'` という文字列は、1 つ目の引数であるにも関わらず、`__init__()` メソッドの定義では 2 つ目の引数であった `name` に渡されます。

前述のように、**メソッドは、インスタンスから呼び出されるとき自動的に第一引数にそのインスタンスへの参照を渡すため**です。

この自動的に渡される自身への参照は、呼び出しの際には明示的に指定しません。

また、かならず 1 つ目の引数に自動的に渡されるため、呼び出し時に明示的に与えられた引数は 2 つ目以降の引数に渡されたものとして取り扱われます。

それでは次に、このクラスに `hello()` というメソッドを追加し、呼び出すと誰の家であるかを表示するという機能を実装してみます。

```

1  # クラスの定義
2  class House:
3
4      # __init__() の定義
5      def __init__(self, name):
6          self.name_plate = name
7
8      # メソッドの定義
9      def hello(self):
10         print('の家です。{}'.format(self.name_plate))

```

それでは、2つのインスタンスを作成して、それぞれから `hello()` メソッドを呼び出してみます。

```

1  sato = House('佐藤')
2  suzuki = House('スズキ')
3
4  sato.hello()    # 実行の際には hello() の引数にある self は無視
5  suzuki.hello() # 実行の際には hello() の引数にある self は無視

```

佐藤の家です。スズキの家です。

`sato` というインスタンスの `name_plate` 属性には、'佐藤' という文字列が格納されています。

`suzuki` というインスタンスの `name_plate` 属性には、'スズキ' という文字列が格納されています。

それぞれのインスタンスから呼び出された `hello()` メソッドは、`self.name_plate` に格納された別々の値を `print()` を用いて表示しています。

このように、同じ機能を持つが、インスタンスによって保持するデータが異なったり、一部の動作が異なったりするようなケースを扱うのにクラスを利用します。

Python の `int` 型、`float` 型、`str` 型…などは、実際には `int` クラス、`float` クラス、`str` クラスであり、それらの中では個別の変数（インスタンス）がどのような値になるかには関係なく、同じ型であれば共通して持っている機能が定義されています。

5 や 0.3 や 'Chainer' などは、それぞれ `int` クラスのインスタンス、`float` クラスのインスタンス、`str` クラスのインスタンスです。

以上から、クラスを定義するというのは、**新しい型を作る**ということでもであると分かります。

3.12.2 継承

あるクラスを定義したら、その一部の機能を変更したり、新しい機能を付け足したりしたくなることがあります。

これを実現する機能が**継承 (inheritance)**です。

例えば、`Link` というクラスを定義し、そのクラスを継承した `Chain` という新しいクラスを作ってみましょう。

まず、`Link` クラスを定義します。

```

1  class Link:
2
3      def __init__(self):
4          self.a = 1
5          self.b = 2

```

この `Link` というクラスは、インスタンス化を行う際には 1 つも引数を取りませんが、属性として `a` と `b` の 2 つの変数を保持し、それぞれには `__init__()` メソッドで 1 と 2 という値が代入されます。

このクラスのインスタンスを作成してみます。

```
1 l = Link()
2
3 l.a
```

```
1
```

```
1 l.b
```

```
2
```

`l` という `Link` クラスのインスタンスが持つ 2 つの属性を表示しています。

インスタンス化を行った際に `__init__()` メソッドの中で代入していた値が、表示されています。

次に、このクラスを**継承**する、`Chain` というクラスを定義してみます。

継承を行う場合は、クラス定義の際にクラス名に続けて `()` を書き、その中にベースにしたいクラスの名前を書きます。

`()` の中に書かれたクラスのことを、定義されるクラスの**親クラス**といいます。

それに対し、`()` の中に書かれたクラスからみると、定義されるクラスは**子クラス**と呼ばれます。

親から子へ機能が受け継がれるためです。

```
1 class Chain(Link):
2
3     def sum(self):
4         return self.a + self.b
```

`Chain` クラスは `__init__()` メソッドの定義を持ちません。

`__init__()` メソッドが定義されていない場合、親クラスの `__init__()` メソッドが自動的に呼び出されます。

そのため、`Chain` クラスでは一見何も属性を定義していないように見えますが、インスタンス化を行うと親クラスである `Link` の `__init__()` メソッドが自動的に実行され、`a`、`b` という属性が定義されます。

以下のコードで確認してみましょう。

```
1 # Chain クラスをインスタンス化
2 c = Chain()
3
4 c.a
```

```
1
```

```
1 c.b
```

```
2
```

`Chain` クラスの `sum()` メソッドでは、この親クラスの `__init__()` メソッドで定義されている 2 つの属性を足し合わせて返しています。

今作成したインスタンスから、この `sum()` メソッドを呼び出してみます。

```
1 # sum メソッドを実行
2 c.sum()
```

3

このように、**親クラスを継承し、親クラスに無かった新しい機能が追加された、新しいクラスを定義することができます。**

それでは、この Chain というクラスにも `__init__()` メソッドを定義して、新しい属性 `c` を定義し、`sum()` メソッドでは親クラスの `a`、`b` という属性とこの新たな `c` という属性の 3 つの和を返すように変更してみます。

```

1 class Chain(Link):
2
3     def __init__(self):
4         self.c = 5 # self.c を新たに追加
5
6     def sum(self):
7         return self.a + self.b + self.c
8
9 # インスタンス化
10 C = Chain()
```

```

1 # error
2 C.sum()
```

```

-----
AttributeError                                Traceback (most recent call last)
<ipython-input-149-5d0c7cc9b080> in <module>()
----> 1 C.sum()

<ipython-input-148-1e932a4aed85> in sum(self)
      5
      6     def sum(self):
----> 7         return self.a + self.b + self.c
      8
      9 # インスタンス化

AttributeError: 'Chain' object has no attribute 'a'
```

エラーが出ました。

エラーメッセージを読みましょう。

```
> AttributeError: 'Chain' object has no attribute 'a'
```

‘Chain’ というオブジェクトは、‘a’ という名前の属性を持っていない、とされています。

`a` という属性は、Chain の親クラスである Link の `__init__()` メソッドで定義されています。

そのため、Chain クラスをインスタンス化する際に、親クラスである Link の `__init__()` メソッドが呼ばれているのであれば、このエラーは起こらないはずです。

なぜエラーとなってしまったのでしょうか。

それは、Chain クラスにも `__init__()` メソッドを定義したため、親クラスである Link の `__init__()` メソッドが上書きされてしまい、実行されなかったためです。

しかし、親クラスの `__init__()` メソッドを明示的に呼ぶことで、これは解決できます。

それには、`super()` という組み込み関数を用います。

これを用いると、子クラスから親クラスを参照することができます。

```

1 class Chain(Link):
```



```

2
3     def __init__(self):
4         # 親クラスの '__init__()' メソッドを呼び出す
5         super().__init__()
6
7         # self.c を新たに追加
8         self.c = 5
9
10    def sum(self):
11        return self.a + self.b + self.c
12
13    # インスタンス化
14    c = Chain()

```

```
1 c.sum()
```

```
8
```

今回はエラーが起きませんでした。

Link クラスの `__init__()` メソッドの冒頭で、まず親クラスの `__init__()` メソッドを実行し、`a`、`b` という属性を定義しているためです。

あるクラスを継承して作られたクラスを、さらに継承して別のクラスを定義することもできます。

```

1 class MyNetwork(Chain):
2
3     def mul(self):
4         return self.a * self.b * self.c

```

`MyNetwork` クラスは、`Link` クラスを継承した `Chain` クラスをさらに継承したクラスで、`a`、`b`、`c` という 3 つの属性を掛け合わせた結果を返す `mul()` というメソッドを持ちます。

このクラスのインスタンスを作成し、`mul()` を実行してみましょう。

```

1 net = MyNetwork()
2
3 net.mul()

```

```
10
```

$1 \times 2 \times 5 = 10$ が返ってきました。

以上で、Python の基本についての解説を終了します。

Python には他にもここでは紹介されていない多くの特徴や機能があります。

さらに詳しく学びたい方は、[Python チュートリアル](#)などを参照してください。

第 4 章

NumPy, Matplotlib を用いた初等数学の復習

本章では、数学の基礎的な事項を復習しながら、Python で数値計算を高速に行うためのライブラリ（[注釈 1](#)）である NumPy と、グラフの描画を行うためのライブラリである Matplotlib の使い方を学びます。

NumPy による**多次元配列**（multidimensional array）の扱い方を知ることは、他の様々なライブラリを利用する際に役立ちます。

例えば、様々な機械学習手法を統一的なインターフェースで利用できる **scikit-learn** や、ニューラルネットワークの記述・学習を行うためのフレームワークである **PyTorch** は、NumPy に慣れておくことでとても使いやすくなります。

それでは、まず NumPy の基礎的な使用方法を説明します。

4.1 NumPy と Matplotlib を使う準備

NumPy と Matplotlib は教材 repo の共有環境に含まれる。教材 repo の共有 Python 環境を使う。準備・更新は教材 repo で行い、演習は作業 repo のルートで教材側の `.venv` を有効にして実行する。手順と保存先は [共通手順](#)を参照する。

ライブラリの機能を利用するには、そのライブラリが提供するモジュールを読み込む必要があります。

例えば A というモジュールを読み込みたいとき、一番シンプルな記述方法は `import A` です。

ただ、もし A というモジュール名が長い場合は、`import A as B` のようにして別名を付けることができます。

`as` を使って別名が与えられると、以降そのモジュールはその別名を用いて利用することができます。

`import A as B` と書くと、A というモジュールは B という名前で利用することができます。

これは Python の機能なので NumPy 以外のモジュールを読み込みたい場合にも使用可能です。

慣習的に、`numpy` にはしばしば `np` という別名が与えられます。

コード中で頻繁に使用するモジュールには、短い別名をつけて定義することがよく行われます。

それでは、`python` コマンドを実行して対話型インタプリタを起動し、`numpy` を `np` という名前で `import` してみましょう。

```
1 import numpy as np
```

4.2 多次元配列を定義する

このあとの例で見ていくように、関数のグラフを描画したりする際には、入力変数を様々に変化させながら出力変数を計算する必要があります。

このようにたくさんの数を同時に扱う際、プログラミング上は多次元配列を用いることが便利です。

NumPy では `ndarray` というクラスで多次元配列を表現します（[注釈 2](#)）。

早速、これを用いて 1 次元配列を定義してみましょう。

```
1 # 次元配列の定義1
2 a = np.array([1, 2, 3])
3
4 a
```

```
array([1, 2, 3])
```

このように、Python リスト [1, 2, 3] を np.array() に渡すことで、[1, 2, 3] という配列を表す ndarray オブジェクトを作ることができます。

ndarray オブジェクトは shape という**属性 (attribute)** を持っており、その多次元配列の**形 (shape)** が保存されています。

上で定義した a という ndarray オブジェクトの形を調べてみましょう。

```
1 a.shape
```

```
(3,)
```

(3,) という要素数が 1 の Python のタプルが表示されています。

ndarray の形は、要素が整数のタプルで表され、要素数はその多次元配列の**次元数 (dimensionality, number of dimensions)** を表します。

形は、その多次元配列の各次元の大きさを順に並べた整数のタプルになっています。

次元数は、ndarray の ndim という属性に保存されています。

```
1 a.ndim
```

```
1
```

これは、len(a.shape) と同じ値になります。

今、a という ndarray は 1 次元配列なので、a.shape は要素数が 1 のタプルで、ndim の値は 1 でした(注釈 3)。

では次に、数を縦横方向に 3 つずつ並べた 2 次元配列を定義してみましょう。

```
1 # 次元配列の定義2
2 b = np.array(
3     [[1, 2, 3],
4      [4, 5, 6],
5      [7, 8, 9]]
6 )
7
8 print(b)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

形と次元数を調べます。

```
1 print('Shape:', b.shape)
2 print('Rank:', b.ndim)
```

```
Shape: (3, 3)
Rank: 2
```

ここで、`size` という属性も見てください。

```
1 b.size
```

```
9
```

これは、`b` という `ndarray` が持つ要素の数を表しています。

`b` は 3×3 行列なので、要素数は 9 です。

「形」「次元数」「サイズ」という言葉がそれぞれ意味するものの違いを確認してください。

NumPy の `ndarray` の作成方法には、`np.array()` を用いて Python のリストから多次元配列を作る方法以外にも、色々な方法があります。

以下に代表的な例をいくつか紹介します。

```
1 # 形を指定して、要素が全て 0 で埋められた ndarray を作る
2 a = np.zeros((3, 3))
3
4 a
```

```
array([[0., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.]])
```

```
1 # 形を指定して、要素が全て 1 で埋められた ndarray を作る
2 b = np.ones((2, 3))
3
4 b
```

```
array([[1., 1., 1.],
       [1., 1., 1.]])
```

```
1 # 形と値を指定して、要素が指定した値で埋められた ndarray を作る
2 c = np.full((3, 2), 9)
3
4 c
```

```
array([[9, 9],
       [9, 9],
       [9, 9]])
```

```
1 # 指定された大きさの単位行列を表す ndarray を作る
2 d = np.eye(5)
3
4 d
```

```
array([[1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 1.]])
```

```

1 # 形を指定して、0 ~ 1 の間の乱数で要素を埋めた ndarray を作る
2 e = np.random.random((4, 5))
3
4 e

```

```

array([[0.00641218, 0.3610797 , 0.62227974, 0.72312181, 0.62354201],
       [0.81754533, 0.61357192, 0.43806013, 0.74668941, 0.87153261],
       [0.67836525, 0.70502511, 0.46533526, 0.96933122, 0.16267139],
       [0.65303541, 0.7168003 , 0.78618327, 0.20075752, 0.23958439]])

```

```

1 # 3 から始まり 10 になるまで 1 ずつ増加する数列を作る (10 は含まない)
2 f = np.arange(3, 10, 1)
3
4 f

```

```
array([3, 4, 5, 6, 7, 8, 9])
```

4.3 多次元配列の要素を選択する

前節では NumPy を使って多次元配列を定義するいくつかの方法を紹介しました。

本節では、作成した ndarray のうち特定の要素を選択して、値を取り出す方法を紹介します。

最もよく行われる方法は [] を使った**添字表記 (subscription)** による要素の選択です。

4.3.1 整数による要素の選択

例えば、上で作成した e という 4 × 5 行列を表す多次元配列から、1 行 2 列目の値を取り出すには、以下のようになります。

```

1 val = e[0, 1]
2
3 val

```

```
0.3610797004614067
```

「1 行 2 列目」を指定するのに、インデックスは [0, 1] でした。

これは、NumPy の ndarray の要素は Python リストと同じく、添字が 0 から始まる**ゼロベースインデックス (zero-based index)** が採用されているためです。

つまり、この行列の i 行 j 列目の値は、[i - 1, j - 1] で取り出すことができます。

4.3.2 スライスによる要素の選択

NumPy の ndarray に対しても、Python のリストと同様に**スライス表記 (slicing)** を用いて選択したい要素を範囲指定することができます。

ndarray はさらに、カンマ区切りで複数の次元に対するスライスを指定できます。

```

1 # 4 x 5 行列 e の真ん中の 2 x 3 = 6 個の値を取り出す
2 center = e[1:3, 1:4]
3
4 center

```

```
array([[0.61357192, 0.43806013, 0.74668941],
       [0.70502511, 0.46533526, 0.96933122]])
```

前節最後にある `e` の出力を見返すと、ちょうど真ん中の部分の 2×3 個の数字が取り出せていることが分かります。

ここで、`e` の中から `[1, 1]` の要素を起点として 2 行 3 列を取り出して作られた `center` の形を、`e` の形と比較してみましょう。

```
1 print('Shape of e:', e.shape)
2 print('Shape of center:', center.shape)
```

```
Shape of e: (4, 5)
Shape of center: (2, 3)
```

また、インデックスを指定したり、スライスを用いて取り出した `ndarray` の一部に対し、値を代入することもできます。

```
1 # 先程の真ん中の 6 個の値を 0 にする
2 e[1:3, 1:4] = 0
3
4 e
```

```
array([[0.00641218, 0.3610797 , 0.62227974, 0.72312181, 0.62354201],
       [0.81754533, 0.          , 0.          , 0.          , 0.87153261],
       [0.67836525, 0.          , 0.          , 0.          , 0.16267139],
       [0.65303541, 0.7168003 , 0.78618327, 0.20075752, 0.23958439]])
```

4.3.3 整数配列による要素の選択

`ndarray` の `[]` には、整数やスライスの他に、整数配列を渡すこともできます。

整数配列とは、ここでは整数を要素とする Python リストまたは `ndarray` のことを指しています。

具体例を示します。

まず、 3×3 行列を表す `a` という `ndarray` を定義します。

```
1 a = np.array(
2     [[1, 2, 3],
3      [4, 5, 6],
4      [7, 8, 9]]
5 )
6
7 a
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

この `ndarray` から、

1. 1 行 2 列目: `a[0, 1]`
2. 3 行 2 列目: `a[2, 1]`
3. 2 行 1 列目: `a[1, 0]`

の3つの要素を選択して並べ、形が(3,)であるような ndarray を作りたいとします。

これは、以下のように、順に対象の要素を指定して並べて新しい ndarray にすることでももちろん実現できます。

```
1 np.array([a[0, 1], a[2, 1], a[1, 0]])
```

```
array([2, 8, 4])
```

しかし、同じことが**選択したい行、選択したい列を、順にそれぞれリストとして与える**ことでも行えます。

```
1 a[[0, 2, 1], [1, 1, 0]]
```

```
array([2, 8, 4])
```

選択したい3つの値がどの行にあるかだけに着目すると、それぞれ1行目、3行目、2行目にある要素です。ゼロベースインデックスでは、それぞれ0, 2, 1行目です。

これが a の [] に与えられた1つ目のリスト [0, 2, 1] の意味です。

同様に、**列に着目**すると、ゼロベースインデックスでそれぞれ1, 1, 0列目の要素です。

これが a の [] に与えられた2つ目のリスト [1, 1, 0] の意味です。

4.4 ndarray のデータ型

1つの ndarray の要素は、全て同じ型を持ちます。

NumPy では様々なデータ型を使うことができますが、ここでは一部だけを紹介します。

NumPy は Python リストを渡して ndarray を作る際などには、その値からデータ型を推測します。

ndarray のデータ型は、dtype という属性に保存されています。

```
1 # 整数 (Python の int 型) の要素をもつリストを与えた場合
2 x = np.array([1, 2, 3])
3
4 x.dtype
```

```
dtype('int64')
```

```
1 # 浮動小数点数 (Python の float 型) の要素をもつリストを与えた場合
2 x = np.array([1., 2., 3.])
3
4 x.dtype
```

```
dtype('float64')
```

以上のように、Python の int 型は自動的に NumPy の int64 型になりました。

また、Python の float 型は自動的に NumPy の float64 型になりました。

Python の int 型は NumPy の int_ 型に対応づけられており、Python の float 型は NumPy の float_ 型に対応づけられています。

この int_ 型はプラットフォームによって int64 型と同じ場合と int32 型と同じ場合があります。

float_ 型についても同様で、プラットフォームによって float64 型と同じ場合と float32 型と同じ場合があります。

特定の型を指定して ndarray を作成するには、以下のようにします。


```

1 x = np.array([1, 2, 3], dtype=np.float32)
2
3 x.dtype

```

```
dtype('float32')
```

このように、dtype という引数に NumPy の dtype オブジェクトを渡します。

これは 32 ビット浮動小数点数型を指定する例です。

同じことが、文字列で指定することによっても行えます。

```

1 x = np.array([1, 2, 3], dtype='float32')
2
3 x.dtype

```

```
dtype('float32')
```

これはさらに、以下のように短く書くこともできます。

```

1 x = np.array([1, 2, 3], dtype='f')
2
3 x.dtype

```

```
dtype('float32')
```

一度あるデータ型で定義した配列のデータ型を別のものに変更するには、astype を用いて変換を行います。

```

1 x = x.astype(np.float64)
2
3 x.dtype

```

```
dtype('float64')
```

4.5 多次元配列を用いた計算

ndarray を使って多次元配列を定義して、それらを用いていくつかの計算を行ってみましょう。

ndarray として定義された多次元配列の**要素ごとの加減乗除**は、Python の数値同士の四則演算に用いられる +、-、*、/ という記号を使って行えます。

それでは、同じ形の配列を 2 つ定義し、それらの**要素ごとの加減乗除**を実行してみましょう。

```

1 # 同じ形 (3 x 3) の配列を 2 つ定義する
2 a = np.array([
3     [0, 1, 2],
4     [3, 4, 5],
5     [6, 7, 8]
6 ])
7
8 b = np.array([
9     [1, 2, 3],
10    [4, 5, 6],
11    [7, 8, 9]
12 ])

```

```

1 # 足し算
2 c = a + b
3
4 c

```

```

array([[ 1,  3,  5],
       [ 7,  9, 11],
       [13, 15, 17]])

```

```

1 # 引き算
2 c = a - b
3
4 c

```

```

array([[ -1,  -1,  -1],
       [ -1,  -1,  -1],
       [ -1,  -1,  -1]])

```

```

1 # 掛け算
2 c = a * b
3
4 c

```

```

array([[ 0,  2,  6],
       [12, 20, 30],
       [42, 56, 72]])

```

```

1 # 割り算
2 c = a / b
3
4 c

```

```

array([[0.          , 0.5          , 0.66666667],
       [0.75         , 0.8          , 0.83333333],
       [0.85714286 , 0.875         , 0.88888889]])

```

NumPy では、与えられた多次元配列に対して要素ごとに計算を行う関数が色々と用意されています。

以下にいくつかの例を示します。

```

1 # 要素ごとに平方根を計算する
2 c = np.sqrt(b)
3
4 c

```

```

array([[1.          , 1.41421356, 1.73205081],
       [2.          , 2.23606798, 2.44948974],
       [2.64575131 , 2.82842712 , 3.          ]])

```

```

1 # 要素ごとに値を n 乗する
2 n = 2
3 c = np.power(b, n)
4
5 c

```

```
array([[ 1,  4,  9],
       [16, 25, 36],
       [49, 64, 81]])
```

要素ごとに値を n 乗する計算は、以下のようにしても書くことができます。

```
1 c ** n
```

```
array([[ 1, 16, 81],
       [256, 625, 1296],
       [2401, 4096, 6561]])
```

4.6 直線のグラフの描画

NumPy と Matplotlib を利用して、直線 $y = ax + b$ 上の点を散布図として描画してみましょう。

まず、 $x = -5, -4, \dots, 4, 5$ として、各 x に対応する y を計算します。

ただし、傾き a は 2 とし、切片 b は 1 としました。

```
1 import numpy as np
2
3 a = 2 # 傾き
4 b = 1 # 切片
5
6 x = np.arange(-5, 6) # -5 から 6 までは含まない (6) の整数を並べた次元配列
7 y = a * x + b
8
9 for i in range(len(x)):
10     print(f"x: {x[i]:3}, y: {y[i]:3}")
```

```
x:  -5, y:  -9
x:  -4, y:  -7
x:  -3, y:  -5
x:  -2, y:  -3
x:  -1, y:  -1
x:   0, y:   1
x:   1, y:   3
x:   2, y:   5
x:   3, y:   7
x:   4, y:   9
x:   5, y:  11
```

上で生成した x と y の組を散布図として表示します。

Matplotlib を利用してグラフを描画する際には、`matplotlib.pyplot` モジュールをインポートします。

慣習的に、`matplotlib.pyplot` には `plt` という別名が与えられます。

散布図を表示するためには、`plt.scatter` 関数を利用します。

`plt.show` 関数を実行するとグラフが表示されます。

```
1 import matplotlib.pyplot as plt
2
3 plt.scatter(x, y) # 散布図グラフを描画
4 plt.show()
```

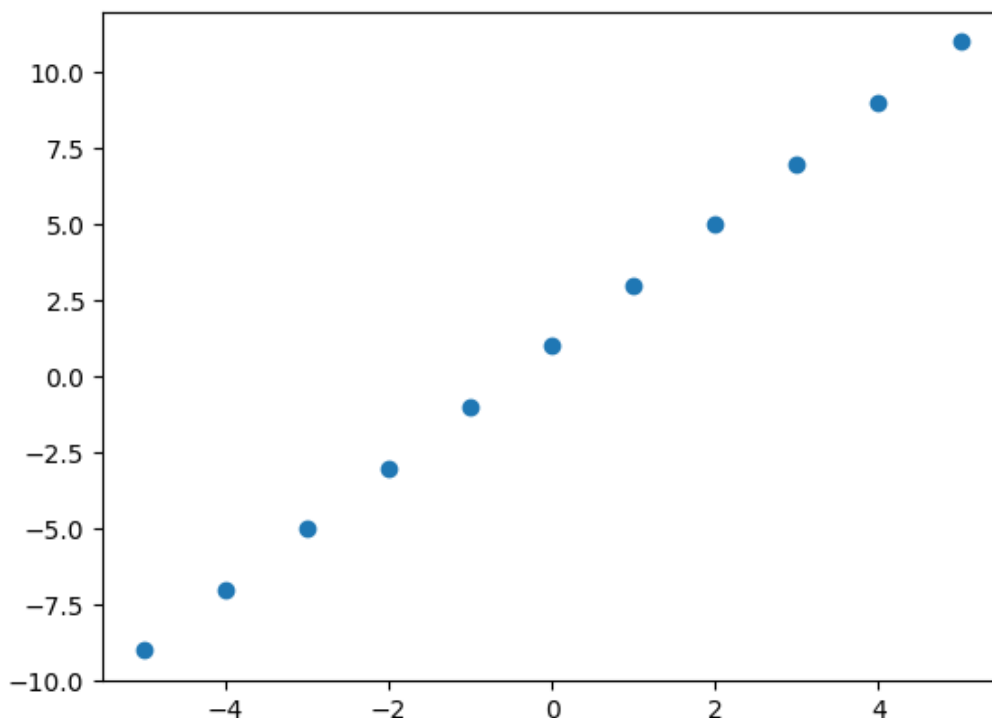


図 4.1 Notebook figure 03-01

WSL を利用している場合など、Graphical User Interface (GUI) でない環境においては、`plt.show` 関数によりグラフを画面に表示することができません。

その場合には、`plt.savefig` 関数を利用してグラフを画像ファイルとして保存することができます。

散布図を `scatter.png` という名前の PNG ファイルとして保存するには、以下のコードを実行します。

ただし、ファイルは作業ディレクトリに保存されます。別のディレクトリにファイルを作成したい場合には、そのファイルパスを `plt.savefig` 関数の引数として渡すこともできます。

```
1 import matplotlib.pyplot as plt
2
3 plt.scatter(x, y) # 散布図グラフを描画
4 plt.savefig("scatter.png")
```

散布図ではなく、折れ線グラフとして描画することもできます。

折れ線グラフを描画するには、`plt.scatter` のかわりに `plt.plot` を用います。

```
1 plt.plot(x, y)
2 plt.show()
```

4.7 2 次関数のグラフの平行移動

上の例では直線のグラフを描画しましたが、 y を計算する式を変更すれば他にも様々な関数のグラフを描くことができます。

2 次関数 $y = ax^2 + bx + c$ のグラフを描いてみましょう。

```
1 a = 1/2
2 b = -1
3 c = 1
```

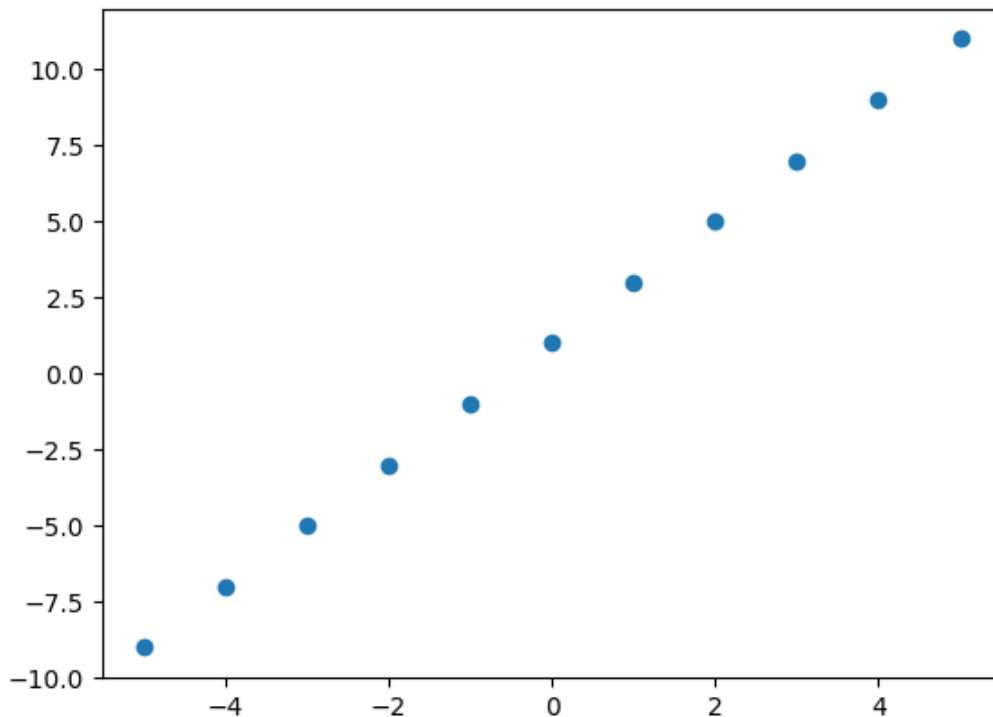


図 4.2 Notebook figure 03-02

```

4
5 x = np.arange(-5, 6)
6 y = a * x**2 + b * x + c
7
8 plt.plot(x, y)
9 plt.show()

```

2 次関数は曲線であるため、折れ線グラフとして描いたときに少しかくかくしてしまっています。

これは、 x を変化させる幅をより細かくすることによって改善することができます。

`np.arange` と似た `np.linspace` 関数を使うと、 -5 から 5 までの範囲を 100 等分した配列を作ることができます。

これを x として、再度同じ関数をプロットしてみましょう。

```

1 x = np.linspace(-5, 5, 100) # -5 から 5 までの範囲を 100 等分した次元配列 1
2 y = a * x**2 + b * x + c
3
4 plt.plot(x, y)
5 plt.show()

```

曲線がなめらかになりました。

関数 $y = ax^2 + b^2 + c$ の x を $x - \alpha$ に置き換えると、グラフを右に α だけ平行移動させることができます ($\alpha < 0$ の場合は左に平行移動します)。

`plt.plot` を連続で実行することで、平行移動前後の 2 つのグラフを 1 つの図にまとめて描画してみましょう。

```

1 alpha = -1
2
3 x = np.linspace(-5, 5, 100)

```

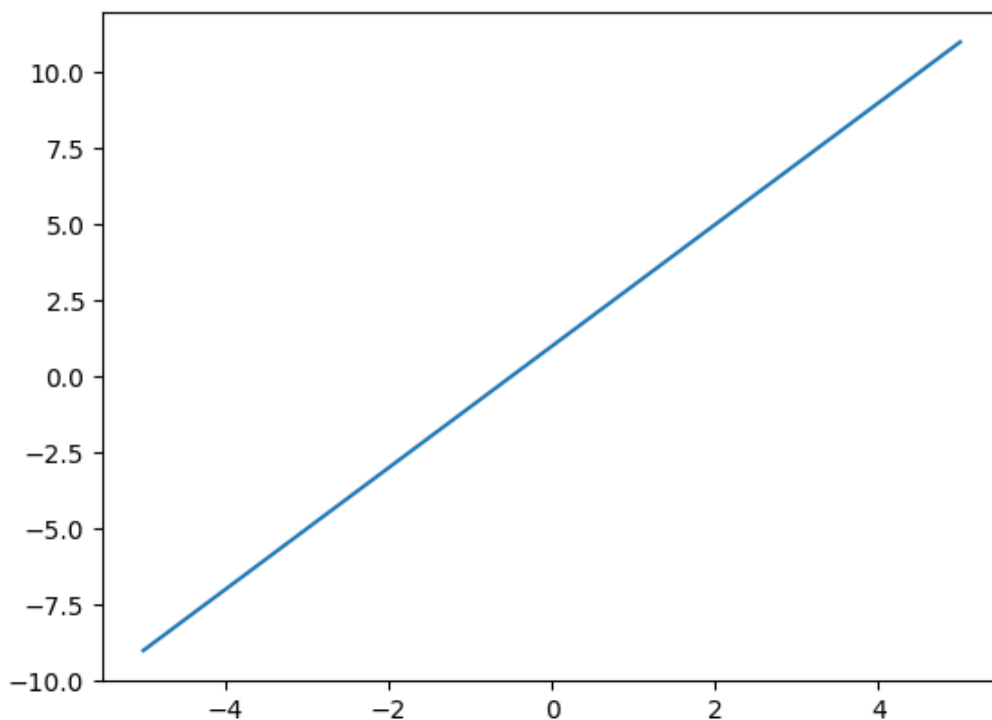


図 4.3 Notebook figure 03-03

```

4 y1 = a * x**2 + b * x + c
5 y2 = a * (x - alpha) **2 + b * (x - alpha) + c
6
7 plt.plot(x, y1)
8 plt.plot(x, y2)
9 plt.show()

```

関数の x を $x - \alpha$ に置き換えるとグラフを右に平行移動させることができましたが、次式のように y を $x - \beta$ に置き換えるとグラフを上 β だけ平行移動させることができます ($\beta < 0$ の場合は下に平行移動)。

$$y - \beta = ax^2 + bx + c$$

プログラミングのために β を右辺に移項して

$$y = ax^2 + bx + (c + \beta)$$

として、実際にプロットしてみましょう。

```

1 beta = 2
2
3 x = np.linspace(-5, 5, 100)
4 y1 = a * x**2 + b * x + c
5 y2 = a * x**2 + b * x + (c + beta)
6
7 plt.plot(x, y1)
8 plt.plot(x, y2)
9 plt.show()

```

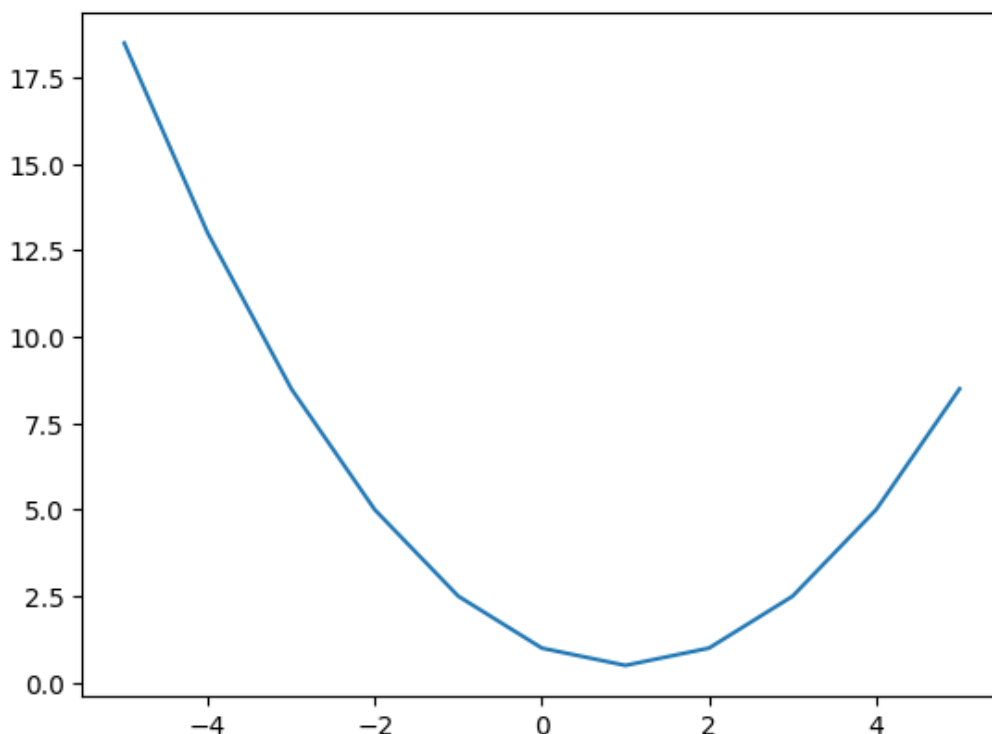


図 4.4 Notebook figure 03-04

今回は 2 次関数を例にグラフを縦横に平行移動させましたが、同様に x や y を置き換えることで、他の関数も平行移動させることができます。

4.8 三角関数

xy 平面上の単位円周上で、点 $(1, 0)$ から反時計回りに角度 θ だけ回転した点を $P(x, y)$ とします。このとき、三角関数 $\sin \theta, \cos \theta$, および $\tan \theta$ は、点 P の座標を用いて以下のように定義されます。

$$\sin \theta = y$$

$$\cos \theta = x$$

$$\tan \theta = \frac{y}{x} = \frac{\sin \theta}{\cos \theta}$$

このことを図に表すと以下のとおりです。

三角関数を使うと、角度から三角形の辺の長さを計算できるようになり、大変便利です。

日常生活において角度を表す際は、1 周を 360° とする度数法を用います。

しかし、数学の分野では、1 周を $2\pi[\text{rad}]$ (単位はラジアンと読みます) とする弧度法を用いることが一般的です。

度数法で表された角度 ϕ から弧度法で表された角度 θ への変換は次式で行なえます。

$$\theta[\text{rad}] = \frac{2\pi}{360} \phi[^\circ]$$

いくつかの角度に対する度数法と弧度法の対応を次の表に示します。

度数法 $[^\circ]$	0	30	45	60	90	180	360
弧度法 $[\text{rad}]$	0	$\frac{\pi}{6}$	$\frac{\pi}{4}$	$\frac{\pi}{3}$	$\frac{\pi}{2}$	π	2π

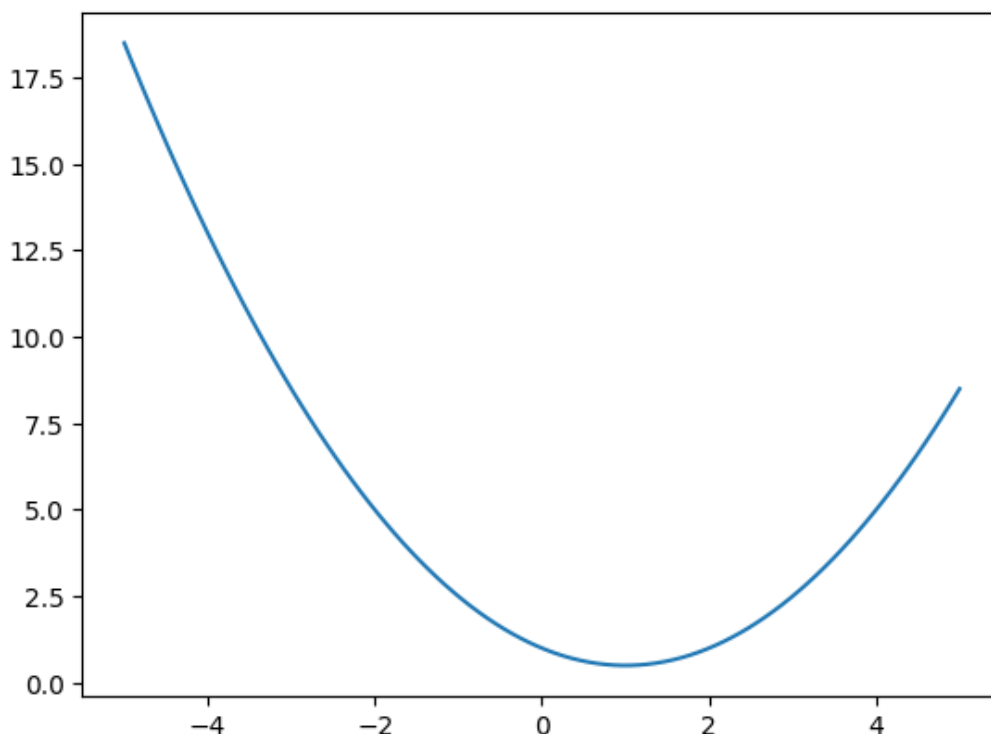


図 4.5 Notebook figure 03-05

NumPy にも三角関数の値を計算する関数が用意されていますが、引数には弧度法で表された角度を渡す必要があります。

`np.sin` は引数として配列が渡されると、各要素ごとに `sin` の値を計算します。

$0 \leq \theta < 4\pi$ の範囲で、 $\sin \theta$ と $\cos \theta$ をプロットするプログラムは、以下のように書けます。

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(0, 4 * np.pi, 100) # 0 から  $\pi$  2 までの範囲を 100 等分した次元配列
5 y1 = np.sin(x)
6 y2 = np.cos(x)
7
8 plt.plot(x, y1)
9 plt.plot(x, y2)
10 plt.show()
```

$\sin \theta$ および $\cos \theta$ は、 2π の周期で同じ値が現れる波状の関数です（このような関数を周期関数といいます）。

このため、これらの関数は、波を数式で表現したいときによく用いられます。

また、 $\sin \theta$ と $\cos \theta$ は、互いに θ 軸に沿った平行移動の関係にあります。

すなわち、

$$\sin \theta = \cos \left(\theta - \frac{\pi}{2} \right)$$

が成り立ちます。

$0 \leq \theta < 4\pi$ の範囲で $\tan \theta$ のグラフをプロットするプログラムは以下のとおりです。

$\tan \theta$ は $\theta = (2n + 1)\pi$ (n は整数) のとき、 $\cos \theta = 0$ になってしまうため値を定義できません。

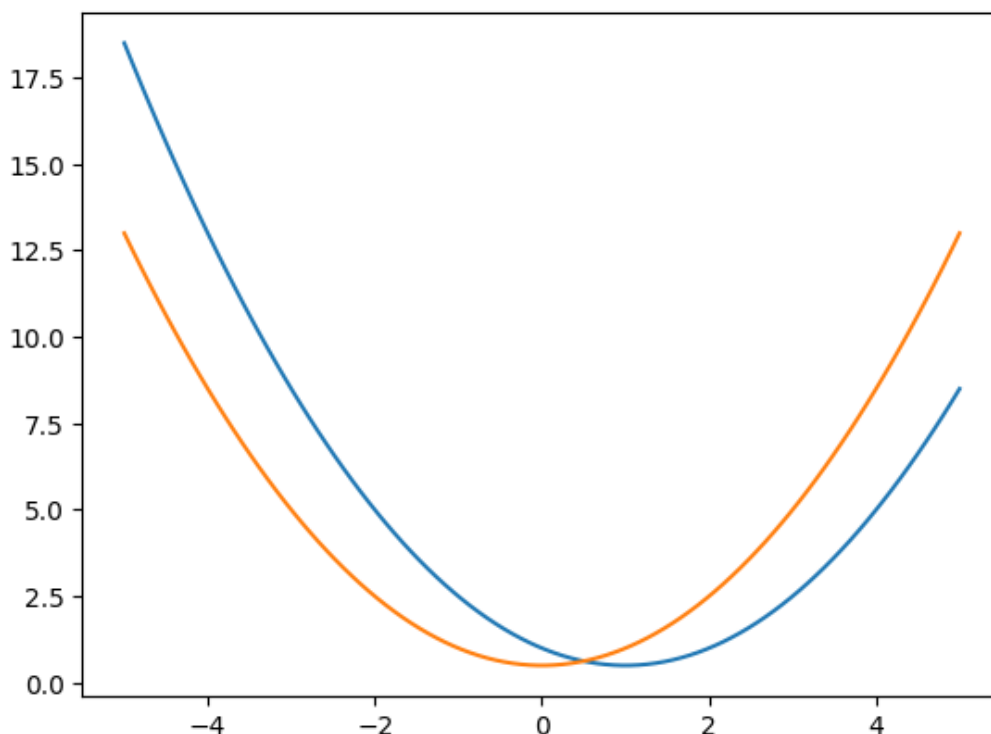


図 4.6 Notebook figure 03-06

```

1 x = np.linspace(0, 4 * np.pi, 1000) # 0 から π 4 までの範囲を 1000 等分した次元配列
2 y = np.tan(x)
3 threshold = 10
4 y[y > threshold] = np.inf
5 y[y < -threshold] = -np.inf
6
7 plt.plot(x, y)
8 plt.show()

```

4.9 指数関数

a を x 回掛け合わせるべき乗（累乗） a^x は、以下の指数法則を満たします。

$$a^m a^n = a^{m+n}, \quad (4.1)$$

$$(a^m)^n = a^{mn}, \quad (4.2)$$

$$(ab)^n = a^n b^n \quad (4.3)$$

ここで、 a^x における a を底と呼ばれ、 x を指数と呼ばれます。

a を x 回掛け合わせるという考え方では、指数として自然数の場合しか考えることができません。

そこで、この考え方を脱却し、べき乗とは上の指数法則を満たす計算というように考え直すと、指数を実数まで拡張することができます。

こうして拡張した指数を使って定義される関数を指数関数といい、次式で表します。

$$y = a^x (a > 0, a \neq 1)$$

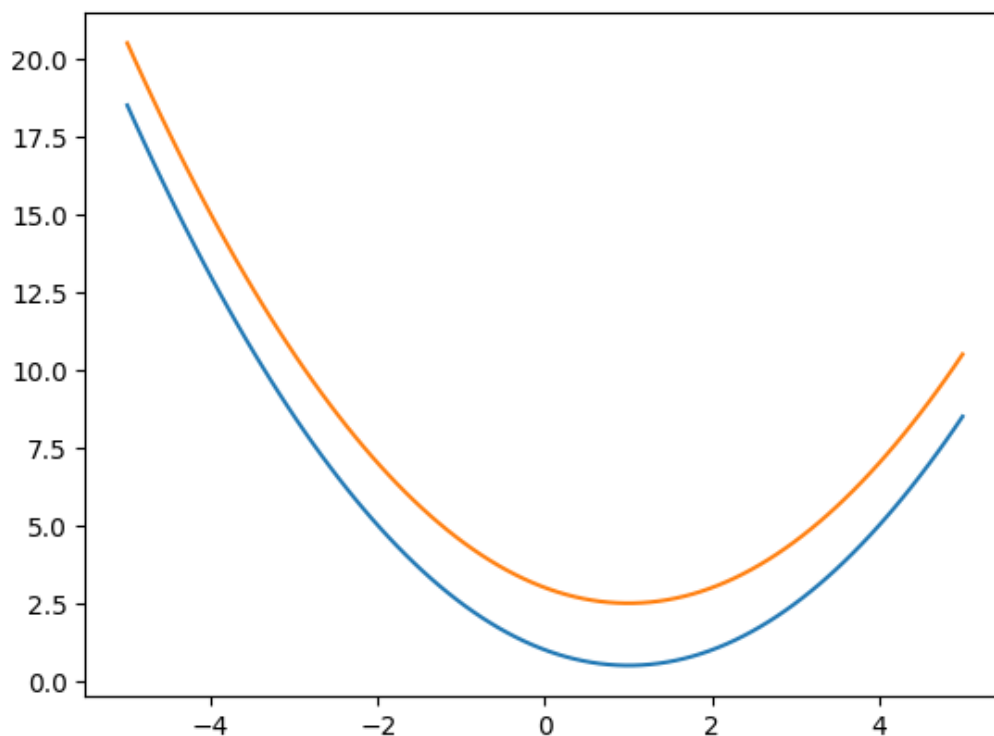


図 4.7 Notebook figure 03-07

指数関数 $y = 2^x$ のグラフを $-5 \leq x < 5$ の範囲でプロットするプログラムを以下に示します。

```

1 x = np.linspace(-5, 5, 100) # -5 から 5 までの範囲を 100 等分した次元配列 1
2 a = 2
3 y = a**x
4
5 plt.plot(x, y)
6 plt.show()
```

グラフを見ると、指数関数は x の値が少し増加すると y の値が急激に増加することがわかります。

$$y = e^x = \exp(x)$$

```

1 x = np.linspace(-5, 5, 100) # -5 から 5 までの範囲を 100 等分した次元配列 1
2 y = np.exp(x)
3
4 plt.plot(x, y)
5 plt.show()
```

4.10 対数関数

対数関数は、指数関数 $y = a^x$ の逆関数です。

つまり、 a と y がわかっているときに、 $y = a^x$ を満たすような x を計算する関数が対数関数で

$$x = \log_a y \quad (a > 0, a \neq 1, y > 0)$$

と書きます。

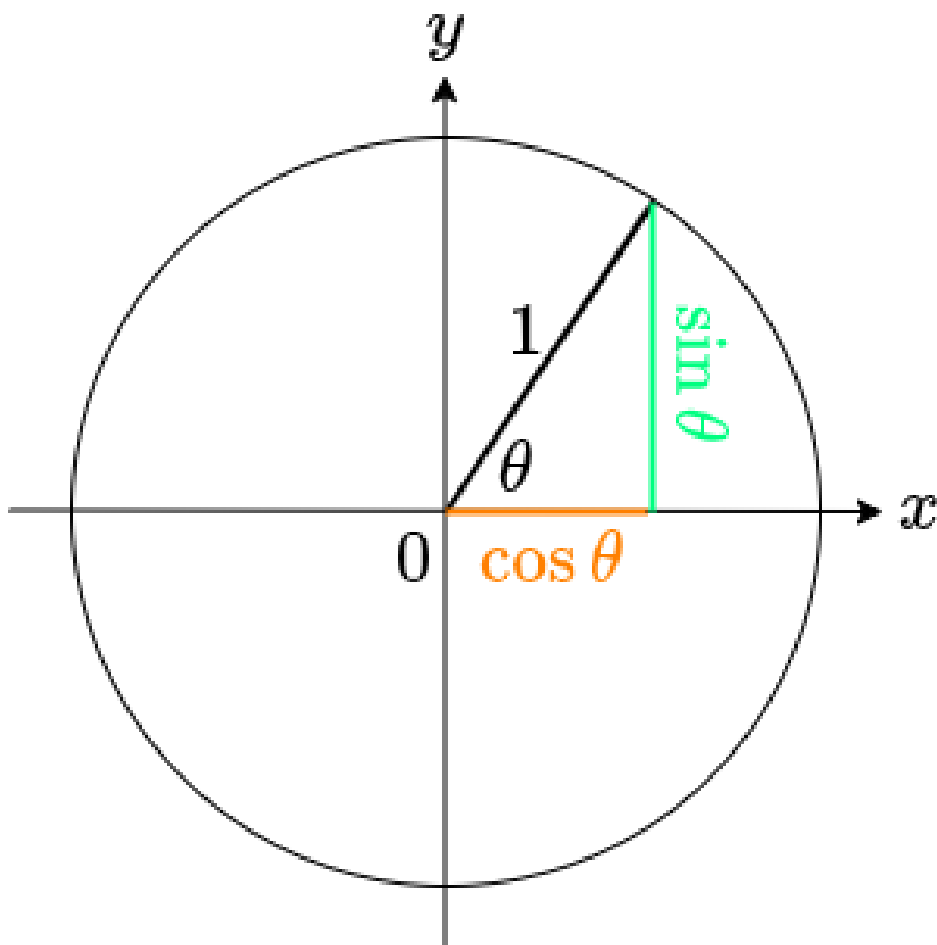


図 4.8 三角関数の定義

指数関数はどんな x についても y が正の値となるため、
対数関数に与える y も正である必要があります。
これまでの関数と同じように左辺の変数を y で書くと、

$$y = \log_a x \quad (a > 0, a \neq 1, x > 0)$$

となります。

具体的な例を挙げましょう。

$$y = \log_2 8$$

という式は、8は2を何乗したものでしょう？という質問を考えていることになります。

$2^3 = 8$ ですから、8は2を3乗したものです。

すなわち、 $y = 3$ ということになります。

対数関数は指数関数と深く関連するため、指数法則に対応する法則の対数関数バージョンを考えることができます。

この法則は対数法則と呼ばれ、具体的には以下の式で表されます。

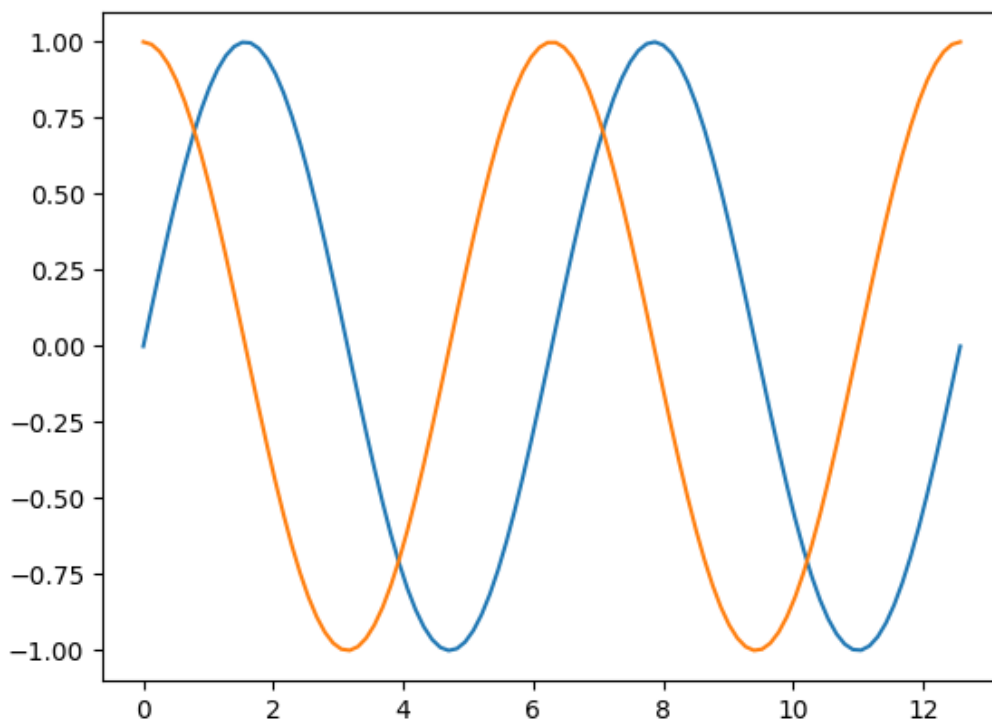


図 4.9 Notebook figure 03-08

$$\log_a MN = \log_a M + \log_a N \quad (4.4)$$

$$\log_a \frac{M}{N} = \log_a M - \log_a N \quad (4.5)$$

$$\log_a M^p = p \log_a M \quad (4.6)$$

この対数法則は、対数を使うと MN のような掛け算や $\frac{M}{N}$ のような割り算をそれぞれ足し算や引き算にできるということを意味しています。

これがとても便利で、コンピュータのない時代には、大きい数同士の掛け算を対数法則で足し算に変換してから計算するなど、とても重宝されました。現在でも、コンピュータでたくさんの掛け算を計算させたいときに、対数法則を使って足し算として計算することがよくあります。

NumPy には、底が 2 の対数を計算する関数 `np.log2`、底が 10 の対数（常用対数と呼ばれます）を計算する関数 `np.log10`、および底がネイピア数 e の対数（自然対数と呼ばれます。詳しくは 4 章で説明します）を計算する関数 `np.log` が用意されています。

2 を底とする対数関数 $y = \log_2 x$ のグラフを $0 \leq x < 5$ の範囲でプロットするプログラムを以下に示します。

```
1 x = np.linspace(0.01, 5, 100) # -5 から 5 までの範囲を 100 等分した次元配列 1
2 y = np.log2(x)
3
4 plt.plot(x, y)
5 plt.show()
```

そのほかの底を持つ対数を計算するための関数は、NumPy には用意されていません。

しかし、対数関数は、底を a から 1 でない任意の正数 b に次式の通り変換することができます。

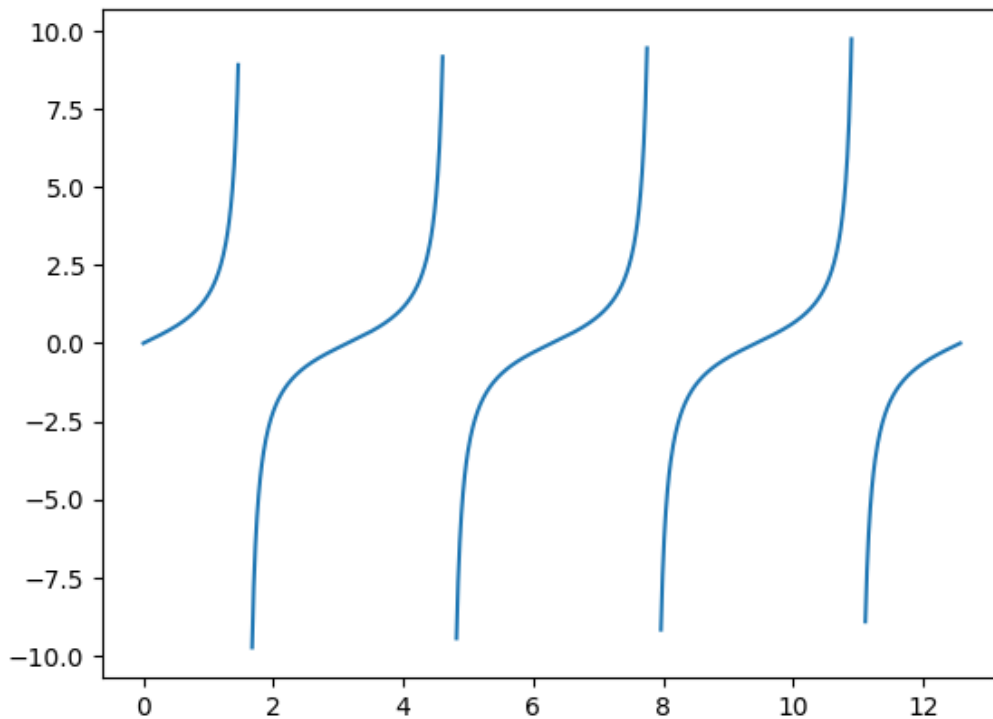


図 4.10 Notebook figure 03-09

$$\log_a M = \frac{\log_b M}{\log_b a}$$

この公式を使うと、任意の底を持つ対数を、NumPy に用意されている関数のみで計算することができます。

4.11 ブロードキャスト

行列同士の要素ごとの四則演算は、通常は行列の形が同じでなければ定義できません。

しかし、前節の最後では 3×3 行列に 3 次元ベクトルを足す計算が実行できました。

これが要素ごとの計算と同じように実行できる理由は、NumPy が自動的に 3 次元ベクトル **b** を 3 つ並べてできる 3×3 行列を想定し、**a** と同じ形に揃える操作を暗黙に行っているからです。

この操作を、**ブロードキャスト**と呼びます。

算術演算を異なる形の配列同士で行う場合、NumPy は自動的に小さい方の配列を**ブロードキャスト**し、大きい方の配列と形を合わせます。

ただし、この自動的に行われるブロードキャストでは、行いたい算術演算が、大きい方の配列の一部に対して**繰り返し行われる**ことで実現されるため、実際に小さい方の配列のデータをコピーして大きい配列をメモリ上に作成することは可能な限り避けられます。

また、この繰り返しの計算は NumPy の内部の C 言語によって実装されたループで行われるため、高速です。

よりシンプルな例で考えてみましょう。

以下のような配列 **a** があり、この全ての要素を 2 倍にしたいとします。

```
1 a = np.array([1, 2, 3])
2
3 a
```

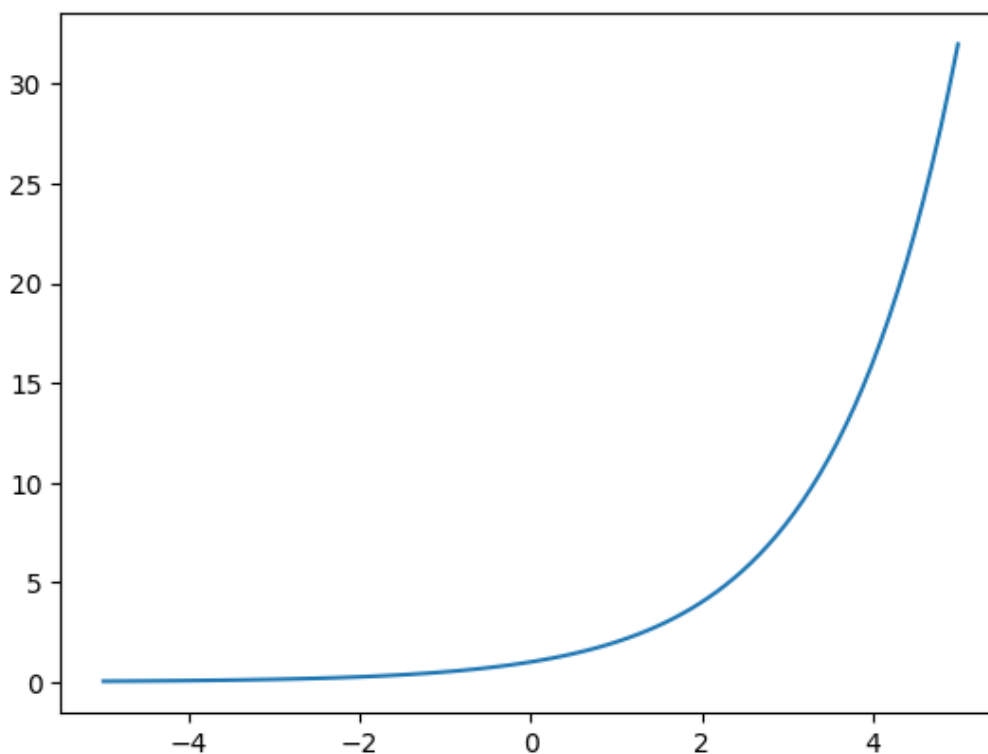


図 4.11 Notebook figure 03-10

```
array([1, 2, 3])
```

このとき、一つの方法は以下のように同じ形で要素が全て 2 である別の配列を定義し、これと要素ごとの積を計算するやり方です。

```
1 b = np.array([2, 2, 2])
2
3 c = a * b
4
5 c
```

```
array([2, 4, 6])
```

しかし、スカラの 2 をただ a に掛けるだけでも同じ結果が得られます。

```
1 c = a * 2
2
3 c
```

```
array([2, 4, 6])
```

* 2 という計算が、c の 3 つの要素の**どの要素に対する計算なのか**が明示されていないため、NumPy はこれを**全ての要素に対して行うという意味**だと解釈して、スカラの 2 を a の要素数 3 だけ引き伸ばしてから掛けてくれます。

形の異なる配列同士の計算がブロードキャストによって可能になるためにはルールがあります。

それは、「**2 つの配列の各次元が同じ大きさになっているか、どちらかが 1 であること**」です。

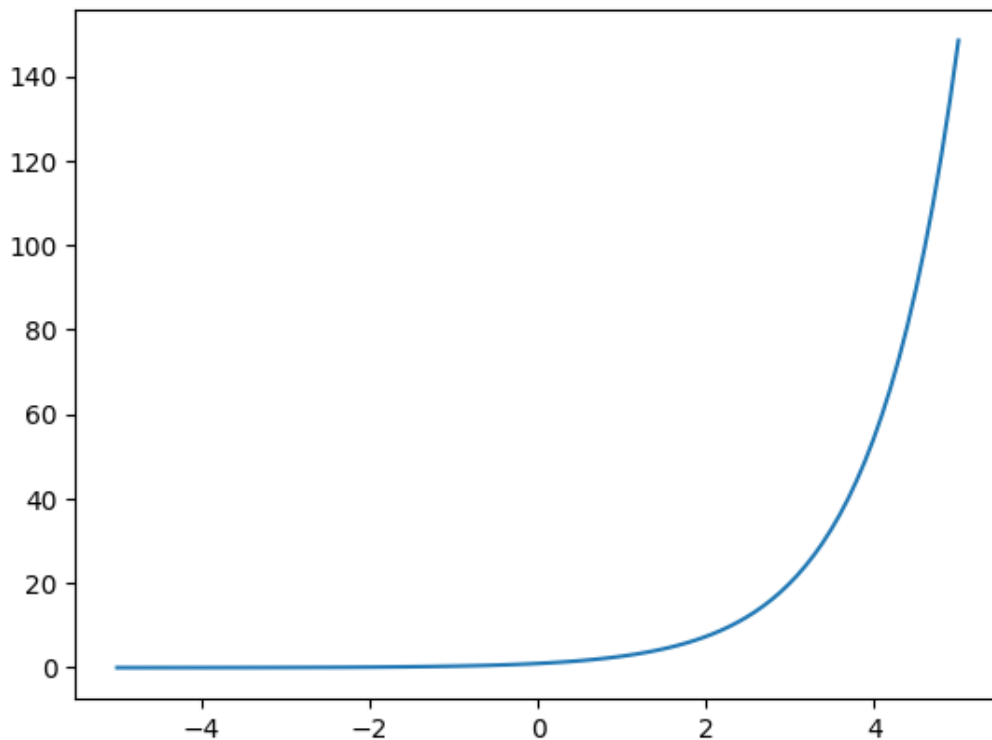


図 4.12 Notebook figure 03-11

このルールを満たさない場合、NumPy は "ValueError: operands could not be broadcast together with shapes (1 目目の配列の形) (2 目目の配列の形)" というエラーを出します。

ブロードキャストされた配列の各次元のサイズ (注釈 4) は、入力された配列のその次元のサイズの中で最大の値と同じになっています。

入力された配列は、各次元のサイズが入力のうち大きい方のサイズと同じになるようブロードキャストされ、その拡張されたサイズで計算されます。

もう少し具体例を見てみましょう。

以下のような 2 つの配列 `a` と `b` を定義し、足します。

```

1  # 0 ~ 9 の範囲の値をランダムに用いて埋められた (2, 1, 3) と (3, 1) という大きさの配列を作る
2  a = np.random.randint(0, 10, (2, 1, 3))
3  b = np.random.randint(0, 10, (3, 1))
4
5  print('a:\n', a)
6  print('\na.shape:', a.shape)
7  print('\nb:\n', b)
8  print('\nb.shape:', b.shape)
9
10 # 加算
11 c = a + b
12
13 print('\na_+b:\n', c)
14 print('\n(a_+b).shape:', c.shape)

```

```

a:
[[[9 1 9]]

```

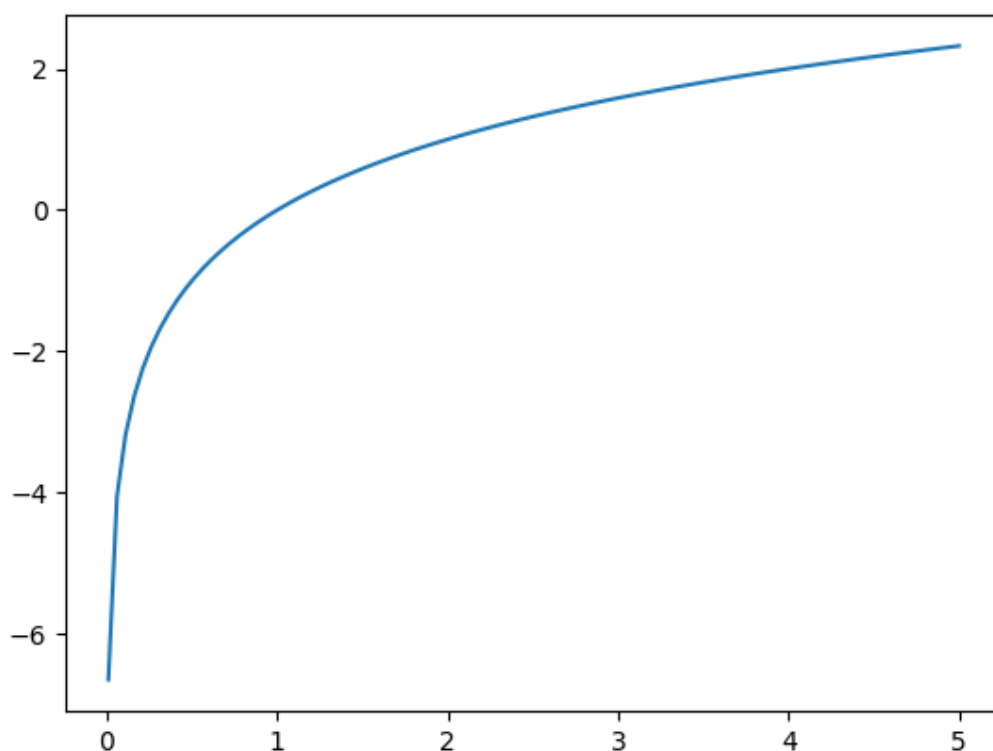


図 4.13 Notebook figure 03-12

```
[[7 3 4]]

a.shape: (2, 1, 3)

b:
[[3]
 [3]
 [8]]

b.shape: (3, 1)

a + b:
[[[12  4 12]
  [12  4 12]
  [17  9 17]]

 [[10  6  7]
  [10  6  7]
  [15 11 12]]]

(a + b).shape: (2, 3, 3)
```

a の形は (2, 1, 3) で、b の形は (3, 1) でした。

この 2 つの配列の**末尾次元 (trailing dimension)** (注釈 5) はそれぞれ 3 と 1 なので、ルールにあった「次元が同じサイズであるか、どちらかが 1 であること」を満たしています。

次に、各配列の第 2 次元に注目してみましょう。

それぞれ 1 と 3 です。

これもルールを満たしています。

ここで、`a` は 3 次元配列ですが、`b` は 2 次元配列です。

つまり、次元数が異なっています。

このような場合は、`b` は**一番上の次元にサイズが 1 の次元が追加された形** (1, 3, 1) として扱われます。

そして 2 つの配列の各次元ごとのサイズの最大値をとった形 (2, 3, 3) にブロードキャストされ、足し算が行われます。

このように、もし 2 つの配列のランクが異なる場合は、次元数が小さい方の配列が大きい方と同じ次元数になるまでその形の先頭に新たな次元が追加されます。

サイズが 1 の次元がいくつ追加されても、要素の数は変わらないことに注意してください。

要素数 (size 属性で取得できる値) は、各次元のサイズの掛け算になるので、1 を何度かけても値は変わらないことから、これが成り立つことが分かります。

NumPy がブロードキャストのために自動的に行う新しい次元の挿入は、`[]` を使った以下の表な表記を用いることで**手動で行うこともできます**。

```
1 print('Original shape:', b.shape)
2
3 b_expanded = b[np.newaxis, :, :]
4
5 print('Added new axis to the top:', b_expanded.shape)
6
7 b_expanded2 = b[:, np.newaxis, :]
8
9 print('Added new axis to the middle:', b_expanded2.shape)
```

```
Original shape: (3, 1)
Added new axis to the top: (1, 3, 1)
Added new axis to the middle: (3, 1, 1)
```

`np.newaxis` が指定された位置に、新しい次元が挿入されます。

配列が持つ数値の数は変わっていません。

そのため、挿入された次元のサイズは必ず 1 になります。

```
1 b
```

```
array([[3],
       [3],
       [8]])
```

```
1 b_expanded
```

```
array([[[3],
        [3],
        [8]]])
```

```
1 b_expanded2
```

```
array([[[3]],
        [[3]],
        [[8]])
```

NumPy のブロードキャストは慣れるまで直感に反するように感じる場合があるかもしれません。

しかし、使いこなすと同じ計算が Python のループを使って行うよりも高速に行えるため、ブロードキャストを理解することは非常に重要です。

一つ具体例を見てみます。

5×5 行列 a に、3 次元ベクトル b を足します。

まず、 a 、 b および結果を格納する配列 c を定義します。

```
1 a = np.array([
2     [0, 1, 2, 1, 0],
3     [3, 4, 5, 4, 3],
4     [6, 7, 8, 7, 6],
5     [3, 4, 5, 4, 4],
6     [0, 1, 2, 1, 0]
7 ])
8
9 b = np.array([1, 2, 3, 4, 5])
10
11 # 結果を格納する配列を先に作る
12 c = np.empty((5, 5))
```

`%%timeit` という Jupyter Notebook で使用できるそのセルの実行時間を計測するためのマジックを使って、 a の各行 (1 次元目) に b の値を足していく計算を Python のループを使って 1 行ずつ処理していくコードの実行時間を測ってみます。

```
1 %%timeit
2 for i in range(a.shape[0]):
3     c[i, :] = a[i, :] + b
```

10.3 μ s $\pm$ 2.03 μ s per loop (mean $\pm$ std. dev. of 7 runs, 100,000 loops each)

```
1 c
```

```
array([[ 1.,  3.,  5.,  5.,  5.],
       [ 4.,  6.,  8.,  8.,  8.],
       [ 7.,  9., 11., 11., 11.],
       [ 4.,  6.,  8.,  8.,  9.],
       [ 1.,  3.,  5.,  5.,  5.]])
```

次に、NumPy のブロードキャストを活用した方法で同じ計算を行ってみます。

```
1 %%timeit
2 c = a + b
```

1.16 μ s $\pm$ 35.7 ns per loop (mean $\pm$ std. dev. of 7 runs, 1,000,000 loops each)

```
1 c
```

```
array([[ 1.,  3.,  5.,  5.,  5.],
       [ 4.,  6.,  8.,  8.,  8.],
       [ 7.,  9., 11., 11., 11.],
       [ 4.,  6.,  8.,  8.,  9.],
       [ 1.,  3.,  5.,  5.,  5.]])
```

計算結果は当然同じになります。

しかし、実行時間が数倍短くなっています。

このように、ブロードキャストを理解して活用することで、記述が簡単になるだけでなく、実行速度という点においても有利になります。

4.12 基本的な統計量の求め方

本節では、多次元配列に含まれる値の平均・分散・標準偏差・最大値・最小値といった統計値を計算する方法を紹介します。

8×10 の行列を作成し、この中に含まれる値全体に渡るこれらの統計値を計算してみましょう。

```
1 x = np.random.randint(0, 10, (8, 10))
2
3 x
```

```
array([[2, 3, 9, 3, 2, 6, 1, 6, 4, 7],
       [4, 9, 7, 6, 8, 5, 3, 6, 1, 6],
       [3, 4, 4, 8, 9, 0, 9, 3, 7, 1],
       [8, 5, 3, 1, 9, 6, 5, 9, 3, 8],
       [6, 7, 9, 4, 5, 6, 9, 7, 2, 7],
       [3, 6, 4, 8, 8, 1, 7, 9, 3, 5],
       [6, 2, 8, 6, 1, 9, 3, 7, 4, 8],
       [6, 2, 5, 8, 8, 2, 9, 4, 0, 4]])
```

```
1 # 平均値
2 x.mean()
```

```
5.2625
```

```
1 # 分散
2 x.var()
```

```
6.918593749999999
```

```
1 # 標準偏差
2 x.std()
```

```
2.6303219859933495
```

```
1 # 最大値
2 x.max()
```

```
3.0
```

```
1 # 最小値
2 x.min()
```

```
1.0
```

ここで、`x` は 2 次元配列なので、各次元に沿ったこれらの統計値の計算も行えます。

例えば、最後の次元内だけで平均をとると、8 個の平均値が得られるはずです。

平均を計算したい軸（何次元目に沿って計算するか）を `axis` という引数に指定します。

```
1 x.mean(axis=1)
```

```
-----
AxisError                                Traceback (most recent call last)
Cell 123 line 1
----> 1 x.mean(axis=1)

File ~/initial-training/venv/lib/python3.9/site-packages/numpy/core/_methods
.py:106, in _mean(a, axis, dtype, out, keepdims, where)
    102 arr = asanyarray(a)
    104 is_float16_result = False
--> 106 rcount = _count_reduce_items(arr, axis, keepdims=keepdims, where=
where)
    107 if rcount == 0 if where is True else umr_any(rcount == 0, axis=None)
    :
    108     warnings.warn("Mean of empty slice.", RuntimeWarning, stacklevel
=2)

File ~/initial-training/venv/lib/python3.9/site-packages/numpy/core/_methods
.py:77, in _count_reduce_items(arr, axis, keepdims, where)
    75     items = 1
    76     for ax in axis:
--> 77         items *= arr.shape[mu.normalize_axis_index(ax, arr.ndim)]
    78     items = nt.intp(items)
    79 else:
    80     # TODO: Optimize case when 'where' is broadcast along a non-
reduction
    81     # axis and full sum is more excessive than needed.
    82
    83     # guarded to protect circular imports

AxisError: axis 1 is out of bounds for array of dimension 1
```

これは、以下のように 1 次元目の値の平均を計算していったものを並べているのと同じことです。

（ゼロベースインデックスで考えています。`x` の形は (8, 10) なので、0 次元目のサイズが 8、1 次元目のサイズが 10 です。）

```
1 np.array([
2     x[0, :].mean(),
3     x[1, :].mean(),
4     x[2, :].mean(),
5     x[3, :].mean(),
6     x[4, :].mean(),
7     x[5, :].mean(),
8     x[6, :].mean(),
9     x[7, :].mean(),
10 ])
```

```
array([5. , 4.3, 6.2, 5.2, 6.2, 4.2, 5.3, 3.5])
```

数式を NumPy による配列の計算に落とし込むことに慣れていくには少し時間がかかりますが、慣れると少ない量のコードで記述できるだけでなく、高速に計算が行なえるため、大きな恩恵があります。

4.13 注釈

4.13.1 注釈 1

ライブラリとは、汎用性の高い複数の関数やクラスなどを再利用可能な形でひとまとまりにしたもので、Python の世界では**パッケージ**とも呼ばれます。また、Python で関数やクラスの定義、文などが書かれたファイルのことを**モジュール**と呼び、パッケージはモジュールが集まったものです。

4.13.2 注釈 2

NumPy には `matrix` というクラスも存在しますが、本チュートリアルでは基本的に多次元配列を表す `ndarray` をベクトルや行列を表すために用います。

4.13.3 注釈 3

これは、その多次元配列が表すテンソルの**階数 (rank、以下ランク)**と対応します。

4.13.4 注釈 4

「次元のサイズ」と言った場合はその次元の大きさを意味し、配列の `size` 属性とは異なるものを指しています。

4.13.5 注釈 5

末尾次元 (trailing dimension) とは、その配列の形を表すタプルの一番最後の値のことを指します。

第 5 章

微分の基礎

5.1 微分と関数最小化の関係

本節ではまず、微分が関数の最小化に役立つことを、具体例を用いて直感的に理解しましょう。

例として、下図のような下向きにくぼんだ形をした関数がどこで最小値をとるかを探す問題を考えます。

適当な点 θ_1 でこの関数のグラフに接する直線（接線）を考えます（注釈 1）。

仮にこの接線の傾きが $+3$ 、すなわち正の値であったとしましょう。この時、接線は右に進むほど高さが上がり、逆に左に進むほど高さが下がります。

θ_1 の周辺では、関数のグラフと接線は非常に近く、両者はほとんど見分けることができません。

すると、関数も接線と同じように θ_1 右に進むと増加し、左に進むと減少していることがわかります。

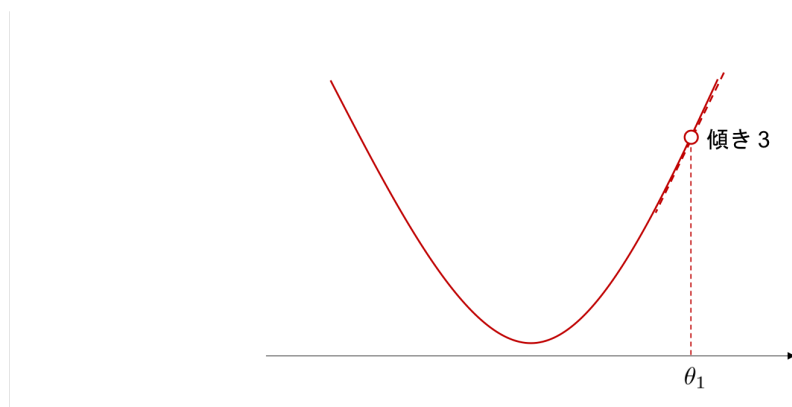


図 5.1 微分 1

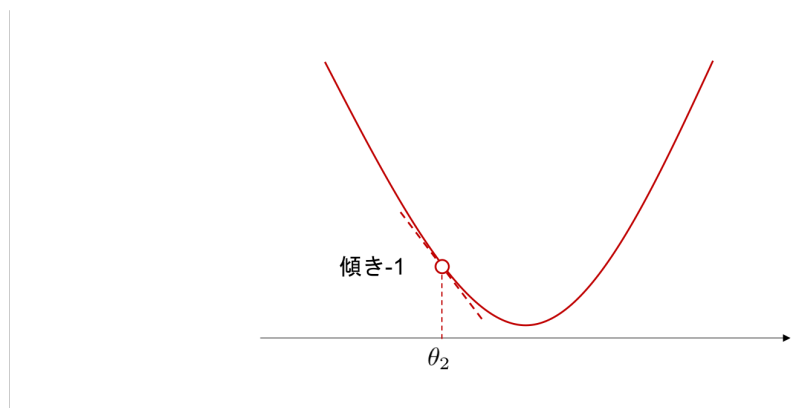


図 5.2 微分 2

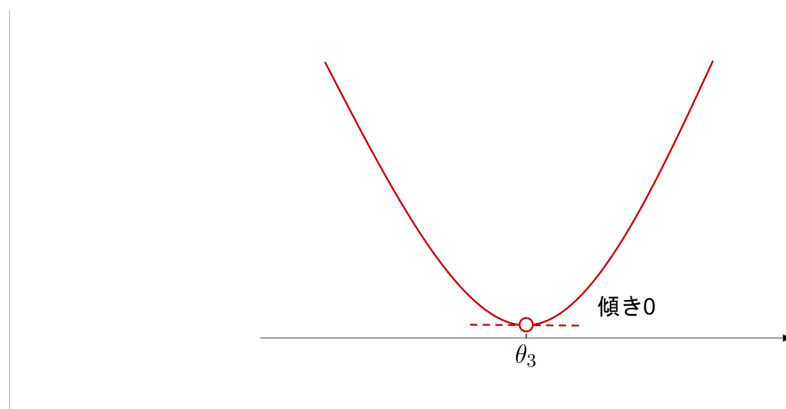


図 5.3 微分 3

次に、グラフ上の別の点 θ_2 においてこのグラフに接する接線を考えます。

今度は接線の傾きが -1 であるとします。傾きが負のため、接線は右肩下がりの直線です。

グラフは接線と非常に近いため、グラフも θ_2 の周辺では、同じように右肩下がりであるとわかります。

最後にちょうどグラフの谷となっている点 θ_3 を考えます。

図を見て分かる通り、関数はこの点で最小値を取ります。

一方、点 θ_3 でグラフに接する接線は水平、すなわち**傾きが 0** です。

これまでの観察をまとめると、接線の傾きと関数の振る舞いには次の関係があることがわかります。

- ある点で接する接線の傾きが正ならば、その点の近くでグラフは右肩上がり（＝ 左に進むと高さが下がる）
- ある点で接する接線の傾きが負ならば、その点の近くでグラフは右肩下がり（＝ 右に進むと高さが下がる）
- 関数が最小値をとる点に接する接線の傾きは 0 である

つまり、関数の最小値をとる点を求める問題では、接線の傾きが 0 となる点が答えの候補（[注釈 2](#)）となることがわかります。

本章で説明するように、**微分を用いると接線の傾きを計算することができます。**

このことから、微分が関数の最小化問題に有用なツールであることがわかります。

以降では、微分の定義と微分に関する公式を紹介します。さらに、入力が多変数の関数での微分（偏微分）についても解説します。

5.2 2 点間を通る直線の傾き

接線の傾きと微分の関係を調べるため、まずは 2 点を通る直線の傾きを求める問題を考えます。

直線の傾きは「 y （縦方向）の増加量 / x （横方向）の増加量」で計算できるので、上図の直線の傾き a は、

$$a = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$$

で求められます。

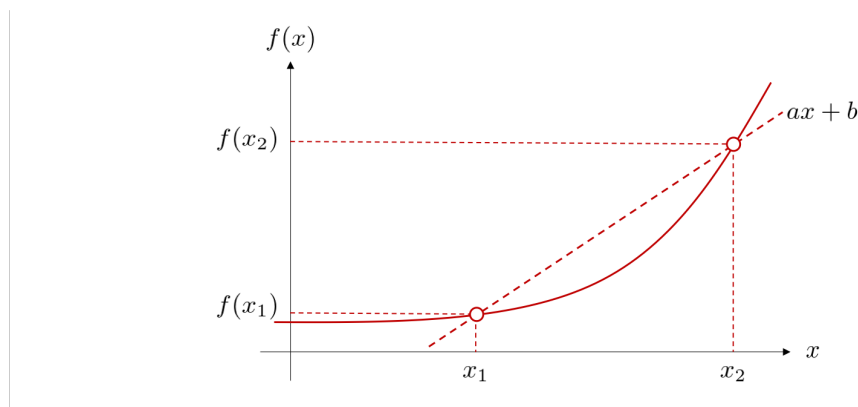


図 5.4 2 点間を通る直線

5.3 接線の傾き

上図での点 x_1 における接線の傾きを求めるために、もう一方の点 x_2 を x_1 に近づけていきます。ただし、 x_1 と x_2 が完全に同じになってしまうと、 x 方向、 y 方向の増加量がどちらも 0 になってしまうため、傾きは $0/0$ で計算することができません。

そこで 2 点は異なる点でありつつ、 x_2 を限りなく x_1 に近づけていった時、直線の傾きがどのように振る舞うかを見る必要があります。これを数式的に表現するには、**極限** の考えが必要になります。

極限では、変数がある値に限りなく近づくとき、その変数によって記述される関数がどのような値に近づくかを考えます。

関数 f に対し、 h という変数を a に近づけていったときの $f(h)$ が近づく値を $\lim$ という記号を用いて

$$\lim_{h \rightarrow a} f(h)$$

と書きます。例えば、 h を限りなく 0 に近づけた時に、 $3h$ も限りなく 0 に近づいていきます。

従って

$$\lim_{h \rightarrow 0} 3h = 0$$

です。もっと一般に n を自然数、 c を定数として、

$$\lim_{h \rightarrow a} ch^n = ca^n$$

が成立します。これだけ見ると単に関数 $\lim_{h \rightarrow a} f(h)$ は h に a を代入した値 $f(a)$ のように思えるかも知れませんが、今考えている直線の傾きの場合、そのような代入操作を行うと先程のように $0/0$ の形が現れてしまうため、単純な代入では極限を求めることはできません。

極限の計算方法は後ほど詳しく解説します。

それでは、下図のある点 x における接線の傾き a を求めていきましょう。

2 点を通る直線の傾きの式と極限を組み合わせ、接線の傾きを求めることができます。

はじめに、 x から h だけ離れた点 $x+h$ を考え、2 点を通る直線の傾きを求めてみます。

次に h を 0 に限りなく近づけていけば、1 点 x で接する接線を考えることができます。

これを式でみると

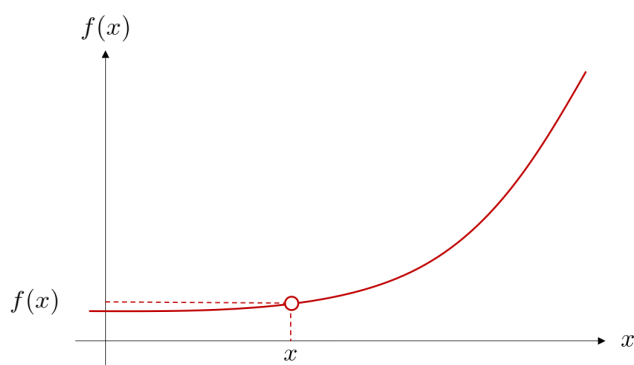


図 5.5 1点での接線

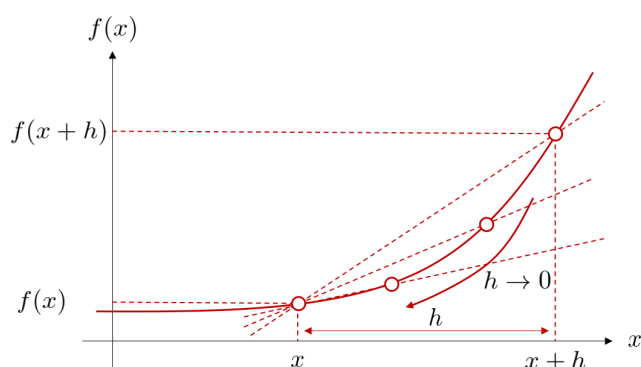


図 5.6 1点での接線 2

$$\begin{aligned} a &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{(x+h) - x} \\ &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \end{aligned}$$

となります。よく見るとこの式は x を決めるごとにある 1 つの値を定めています、すなわちこの式は x の関数です (h は $h \rightarrow 0$ の極限を取っているので h の関数にはなっていないことに注意してください)。

この式を f の **導関数 (derivative)** と呼び、 $f'(x)$ と書きます。すなわち、

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

です。導関数を求めることを **微分 (differentiation)** するといいます。

記号の使い方として、 $f'(x)$ を

$$\frac{d}{dx} f(x)$$

または

$$\frac{df}{dx}(x)$$

と書いても構いません。

この d という記号は増分をとる操作を表しており、例えば dx は x の変化量を表します。

この記法は煩雑ですが、 x, y など変数が複数ある場合、どの変数で微分しているかが明確になるため、表現を正確にすることができます。

5.4 微分の公式

ある関数の導関数を計算するには、導関数の定義通りに計算するのが最も愚直な方法です。

しかし、これから紹介する種々の公式を組み合わせることで、わざわざ定義に戻らなくても、複雑な関数の導関数を計算できます。

以下では、 c は定数、 x は変数を表します。

まずはじめに、以下の 3 つの公式を紹介します。

$$(c)' = 0 \quad (5.1)$$

$$(x)' = 1 \quad (5.2)$$

$$(x^2)' = 2x \quad (5.3)$$

1 つ目の左辺は定数関数 $f(x) = c$ を x で微分した導関数を表しています。

定数関数のグラフを書くと x 軸に平行になるため、どの点で接線を引いてもその傾きは 0 です（グラフと接線は同一の直線となります）。これは導関数は 0 であることを意味します。これが 1 つ目の式の図形的な理解です。

同様に 2 つ目の公式は「 $y = x$ のグラフではどの点で接線を引いても傾きが 1」であることを意味します（この場合もグラフと接線は同一の直線です）。

極限の扱いに慣れるために、3 番目の公式を証明しましょう。導関数の定義から左辺は

$$\lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{(x+h) - x}$$

です。これは

$$\lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{(x+h) - x} = \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} = \lim_{h \rightarrow 0} (2x + h) \quad (5.4)$$

と計算できます。ここで $h \rightarrow 0$ 、すなわち h を限りなく 0 に近づけると、 $2x + h$ は $2x$ に限りなく近づいていきます。従って、 $(x^2)' = 2x$ が導かれました。

以下の公式も頻繁に利用します。

$$(x^n)' = nx^{n-1} \quad (5.5)$$

$$(e^{ax})' = ae^{ax} \quad (5.6)$$

ここで、 a は定数、 e は**自然対数の底**、もしくは**ネイピア数**と呼ばれる特別な定数で、およそ 2.71828... です。1 つ目の公式で $n = 1, 2$ とすると、本節の最初に紹介した x, x^2 の微分の公式に帰着されることに注意してください。

e^{ax} は $\exp(ax)$ のように表記されることがよくあります。その表記を用いると最後の公式は

$$(\exp(ax))' = a \exp(ax)$$

とも書けます。

5.5 線形性

微分は**線形性**という性質を持っています。

それがどのような性質なのか、具体例を挙げて見ていきましょう。

微分には線形性という性質によって、

$$(3x)' = 3 \times (x)'$$

のように定数項を微分の演算の外側に出すことができます。

また、

$$(3x^2 + 4x - 5)' = (3x^2)' + (4x)' - (5)'$$

のように、加算や減算はそれぞれ項ごとに独立に微分の演算を行うことができます。

この2つの特性を合わせて線形性と呼びます。

もう少し微分の計算を練習してみましょう。

$$\begin{aligned} (3x^2 + 4x + 5)' &= (3x^2)' + (4x)' - (5)' \\ &= 3 \times (x^2)' + 4 \times (x)' - 5 \times (1)' \\ &= 3 \times 2x + 4 \times 1 - 5 \times 0 \\ &= 6x + 4 \end{aligned}$$

この線形性に関しては、下記のように公式としてまとめることができます。

$$(cf(x))' = cf'(x) \tag{5.7}$$

$$(f(x) + g(x))' = f'(x) + g'(x) \tag{5.8}$$

$$\tag{5.9}$$

2つの関数の積の形で書かれている関数に関しては次の公式が成り立ちます。

$$(f(x)g(x))' = f'(x)g(x) + f(x)g'(x)$$

関数 f の導関数と関数 g の導関数がわかれば、関数 fg を計算できることがわかります。

5.6 合成関数の微分

関数 $y = f(x)$ と $z = g(y)$ の**合成**とは f を適用したあとに g を適用する関数、すなわち $z = g(f(x))$ のことを指します。

ディープラーニングで用いるニューラルネットワークは、層を何層も重ねて複雑な関数を表現します。各々の層を1つの関数とみなすと、ニューラルネットワークは多くの関数（層）を合成した**合成関数**と見ることができます。

合成関数の微分を考える時には次に紹介する公式（合成関数の微分の公式）が有用です。

この公式は**連鎖律 (chain rule)**とも呼ばれています。

連鎖律は合成関数を微分を簡単に計算するための公式というだけではなく、ニューラルネットワークの訓練方法である誤差逆伝播法を理解する上で本質的な役割を果たします。

簡単な例として、

$$\{(3x+4)^2\}'$$

を計算することを考えます。

この式は、 $3x+4$ という内側の部分と $(\cdot)^2$ という外側の部分で構成されています。

この式を $(9x^2 + 24x + 16)'$ のように展開してから微分を計算しても良いのですが、3 乗や 4 乗とべき数が増えると式を展開するのが大変になります。

ここで役に立つ考え方が合成関数の微分です。

合成関数の微分は、内側の微分と外側の微分をそれぞれ行い、その結果を掛け合わせることで求めることができます。

外側の微分の際には関数の引数を入力とみなし、その入力についての微分を計算します。

それでは、具体的にこの $(3x+4)^2$ という関数の微分を考えてみます。

まず内側の関数を $u = (3x+4)$ とおいて、

$$\{(3x+4)^2\}' = (u^2)'$$

と見ます。ここで、 $(\cdot)'$ をもう少し厳密に考える必要が出てきます。

今変数は x, u の 2 つあるため、 $(\cdot)'$ という表記では、 x で微分しているのか u で微分しているのかの区別が付きません。

そこで、多少複雑に見えますが、先程紹介した d を使った記法を用いて微分する変数を明示します。

合成関数の微分を公式としてまとめると次のようになります。

$$\frac{d}{dx} f(g(x)) = \frac{df(u)}{du} \frac{du}{dx}$$

ここで $u = g(x)$ です。

公式を見るよりも実際の適用例を見た方が理解しやすいかも知れません。

合成関数の微分の公式を用いて、先程の $(3x+4)^2$ の微分を計算すると次のようになります。

2 行目で合成関数の微分の公式を利用していることに注目してください。

$$\begin{aligned} \{(3x+4)^2\}' &= \frac{d}{dx} \{(3x+4)^2\} \\ &= \frac{du}{dx} \frac{d}{du} (u^2) \\ &= \frac{d}{dx} (3x+4) \cdot \frac{d}{du} (u^2) \\ &= 3 \cdot 2u \\ &= 6u = 6(3x+4) = 18x + 24 \end{aligned}$$

気になる人は、 $(3x+4)^2$ を展開してから各項を微分した場合と結果が一致していることを確かめてみてください。

5.7 偏微分

機械学習では、1 つの入力変数 x から出力変数 y を予測するケースは稀であり、多くの場合、複数の入力変数 $x_1, x_2, \dots, x_M$ を用いて y を予測する**多変数関数**が扱われます。

例えば、家賃を予測する場合、部屋の広さだけではなく、駅からの距離や周辺の犯罪発生率なども同時に考慮した方がより正確に予測ができると期待されます。

複数の入力 $x_1, x_2, \dots, x_M$ をとる関数 $f(x_1, x_2, \dots, x_M)$ を多変数関数とよびます。

この多変数関数において、ある入力 x_m にのみ注目して微分することを**偏微分**とよび、

$$\frac{\partial}{\partial x_m} f(x_1, x_2, \dots, x_M)$$

と表します。微分を意味する記号が、 d から ∂ に変わっています。こうすると、 $\frac{\partial}{\partial x_m}$ は x_m 以外を定数と考え、 x_m にのみ着目して微分を行うという意味となります（[注釈 3](#)）。

以下の例で具体的な計算の流れを確認しましょう。

$$\begin{aligned} \frac{\partial}{\partial x_1} (3x_1 + 4x_2) &= \frac{\partial}{\partial x_1} (3x_1) + \frac{\partial}{\partial x_1} (4x_2) \\ &= 3 \times \frac{\partial}{\partial x_1} (x_1) + 4 \times \frac{\partial}{\partial x_1} x_2 \\ &= 3 \times 1 + 4 \times 0 \\ &= 3 \end{aligned}$$

偏微分でも微分と同じ公式を適用できます。今回のケースでは、 x_1 にだけ着目しており、 x_2 は定数として扱っています。そのため、上式の 2 行目から 3 行目で x_2 を x_1 で偏微分した値を 0 としています（定数の微分は 0 であったことを思い出してください）。

5.8 注釈

5.8.1 注釈 1

ここで考えている関数のグラフでは、グラフ上のどの点を取ってもその点で接する接線がただ 1 本だけ引ける状況を考えています。例えば関数のグラフが谷の部分で「尖った」形をしていると、谷の底で複数の接線が引けてしまいます。ここではそのようなケースは考えず、関数のグラフは図のような「滑らか」なカーブになっている場合をイメージしてください。

5.8.2 注釈 2

今は関数のグラフの「谷」を考えましたが、「山」でも同様に接線の傾きが 0 となるため、ある点での接線の傾きが 0 だからと言って、必ずしも関数がある点で最小値をとるとは限りません。

5.8.3 注釈 3

入力変数がある他の入力変数と独立でない場合は定数と考えることはできません。しかし本資料ではそのようなケースは出てきません。

第 6 章

積分の基礎

6.1 この章の目的

微分が「ある点での変化の速さ」を扱うのに対し、積分は「小さな量を足し合わせて全体量を求める」ための道具です。機械学習や信号処理では、

- 面積や総量の評価
- 平均値や期待値の計算
- 連続時間信号のエネルギー評価
- 確率密度関数の正規化

などで積分が現れます。

本章の到達目標は次の通りです。

- 不定積分を「微分の逆演算」として理解できる
- 定積分を「区間上の総和の極限」として理解できる
- 広義積分が必要になる場면을説明できる
- 二重積分の意味を面積・体積の観点から説明できる
- 数値積分で近似的に積分値を求められる

6.2 不定積分

関数 $F(x)$ が関数 $f(x)$ の原始関数であるとは、

$$F'(x) = f(x)$$

が成り立つことを言います。

このとき、 $f(x)$ の不定積分を

$$\int f(x) dx = F(x) + C$$

と書きます。ここで C は積分定数です。

積分定数が必要なのは、微分すると定数項が消えるからです。例えば

$$\frac{d}{dx}(x^3 + 7) = 3x^2, \quad \frac{d}{dx}(x^3 - 2) = 3x^2$$

なので、 $3x^2$ の原始関数は 1 つではありません。

6.2.1 基本公式

微分の公式を逆向きに使うと、以下の不定積分公式が得られます。

$$\int x^n dx = \frac{x^{n+1}}{n+1} + C \quad (n \neq -1)$$

$$\int \cos x dx = \sin x + C$$

$$\int \sin x dx = -\cos x + C$$

$$\int \frac{1}{x} dx = \log |x| + C$$

ここで最後の式だけ形が違うのは、 x^{-1} に対して上のべき乗則をそのまま適用できないからです。

6.2.2 線形性

不定積分も線形性を持ちます。定数 a, b と関数 f, g に対して

$$\int (af(x) + bg(x)) dx = a \int f(x) dx + b \int g(x) dx$$

が成り立ちます。

複雑な積分を基本公式の組み合わせに分解する際には、まず線形性で項をばらすのが基本です。

6.3 定積分

6.3.1 面積としての理解

区間 $[a, b]$ 上で非負の関数 $f(x)$ を考えるとき、

$$\int_a^b f(x) dx$$

は、グラフと x 軸に挟まれた領域の面積として理解できます。

ただし積分は単なる「面積」より少し一般的で、関数が負になる部分は負の寄与として数えます。したがって定積分は、**符号付き面積**と考えると正確です。

6.3.2 リーマン和

定積分の本質は、「区間を細かく分けて長方形の面積を足し合わせ、分割を無限に細かくした極限を取る」ことです。区間 $[a, b]$ を N 分割し、幅を

$$\Delta x = \frac{b-a}{N}$$

とします。各小区間の代表点を x_k とすると、リーマン和

$$\sum_{k=0}^{N-1} f(x_k) \Delta x$$

が定積分の近似になり、

$$\int_a^b f(x) dx = \lim_{N \rightarrow \infty} \sum_{k=0}^{N-1} f(x_k) \Delta x$$

で定義されます。

この見方は数値積分や確率論で極めて重要です。期待値や連続時間信号のエネルギーも、本質的には「無数の小さな寄与の総和」です。

6.3.3 微積分の基本定理

定積分と不定積分をつなぐ最重要結果が**微積分の基本定理**です。 $F'(x) = f(x)$ を満たす原始関数 F があるとき、

$$\int_a^b f(x) dx = F(b) - F(a)$$

が成り立ちます。

この定理により、面積の極限として定義された量を、原始関数の差として効率よく計算できます。

例えば

$$\int_0^1 x^2 dx = \left[\frac{x^3}{3} \right]_0^1 = \frac{1}{3}$$

です。

6.4 置換積分と部分積分の考え方

本章では詳細な技巧を網羅しませんが、よく使う発想だけは押さえておきます。

6.4.1 置換積分

合成関数の積分では、新しい変数に置き換えて簡単な形に直すことがあります。例えば

$$\int 2x \cos(x^2) dx$$

では $u = x^2$ と置くと $du = 2x dx$ なので、

$$\int \cos u du = \sin u + C = \sin(x^2) + C$$

と計算できます。

6.4.2 部分積分

積の微分公式

$$(fg)' = f'g + fg'$$

を変形すると

$$\int f(x)g'(x) dx = f(x)g(x) - \int f'(x)g(x) dx$$

が得られます。

これは「微分すると簡単になる部分」と「積分しやすい部分」を分ける操作です。例えば多項式と指数関数、 $x \sin x$ のような積で有効です。

6.5 広義積分

普通の定積分は有限区間上の有界な関数を想定していますが、実際には

- 積分区間が無限に広い
- 被積分関数が途中で発散する

場合もあります。このような積分を**広義積分**と呼びます。

例えば

$$\int_1^{\infty} \frac{1}{x^2} dx$$

は上端が無限大なので、極限で

$$\int_1^{\infty} \frac{1}{x^2} dx = \lim_{R \rightarrow \infty} \int_1^R \frac{1}{x^2} dx = \lim_{R \rightarrow \infty} \left[-\frac{1}{x} \right]_1^R = 1$$

と定義します。

一方,

$$\int_1^{\infty} \frac{1}{x} dx$$

は

$$\lim_{R \rightarrow \infty} \int_1^R \frac{1}{x} dx = \lim_{R \rightarrow \infty} \log R$$

となり発散します。

「無限区間だから必ず発散する」わけではなく、**どれくらい速く減衰するか**が重要です。

6.6 多重積分

2変数関数 $f(x, y)$ の積分では、平面上の微小面積要素を足し合わせます。長方形領域 $a \leq x \leq b$, $c \leq y \leq d$ 上では,

$$\iint f(x, y) dx dy$$

を考えます。

もし $f(x, y)$ がその点での高さを表すなら、二重積分は曲面の下の方の体積に対応します。

計算上は、1変数ずつ順に積分することが多く、

$$\iint_R f(x, y) dx dy = \int_a^b \left(\int_c^d f(x, y) dy \right) dx$$

のように扱います。

例えば単位正方形上の $f(x, y) = x + y$ なら

$$\int_0^1 \int_0^1 (x + y) dy dx = \int_0^1 \left(x + \frac{1}{2} \right) dx = 1$$

です。

画像処理では2次元信号、確率論では同時密度関数、機械学習では複変数にまたがる平均化で多重積分が現れます。

6.7 Python で数値積分を確かめる

解析的に原始関数が求まらない場合や、離散データしかない場合には数値積分を使います。以下は台形公式で積分値を近似する例です。

```
1 from pathlib import Path
2
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 output_dir = Path("outputs/ch05")
7 output_dir.mkdir(parents=True, exist_ok=True)
8
9 x = np.linspace(0.0, 1.0, 1001)
10 y = x ** 2
11
12 approx = np.trapezoid(y, x)
13 print(f"approx_integral={approx:.6f}")
14
15 plt.figure(figsize=(4, 3))
16 plt.plot(x, y, label="y=x^2")
17 plt.fill_between(x, y, alpha=0.3)
18 plt.xlabel("x")
19 plt.ylabel("y")
20 plt.tight_layout()
21 plt.savefig(output_dir / "integral_x_squared.png", dpi=150)
22 plt.close()
```

この例では、 $\int_0^1 x^2 dx = 1/3$ に近い値が得られるはずです。分割数を増やすと近似誤差が小さくなることも確認してください。

6.8 この章で押さえるべき点

- 不定積分は原始関数を求める操作であり、積分定数が必要である
- 定積分は区間上の総和の極限であり、符号付き面積として理解できる
- 微積分の基本定理により、定積分は原始関数の差で計算できる
- 広義積分では無限区間や特異点を極限で扱う
- 多重積分は高次元での総和であり、画像・確率・信号処理で広く使う

次の線形代数・確率統計の章では、積分で定義される平均値や期待値が頻出します。そのとき、積分を「連続量の足し合わせ」として捉えられるかが理解の分かれ目です。

第 7 章

線形代数の基礎

機械学習の理論では線形代数で用いられる概念が多く登場します。

これらの概念を利用することで、複数の値や変数をまとめて扱うことができるようになり、数式を簡潔に表現できるようになります。

本章では、特に以下の概念を順番に紹介します

- スカラ・ベクトル・行列・テンソル
- ベクトル・行列の演算（加減算・スカラ倍・内積・行列積）
- 特別な行列（単位行列・逆行列）
- 多変数関数（線形結合，二次形式）とその微分

7.1 スカラ・ベクトル・行列・テンソル

まず始めに、スカラ、ベクトル、行列、テンソルという 4 つの言葉を解説します。

スカラ (scalar) は、2.5、-1、 $\sqrt{2}$ 、 π といった 1 つの値もしくは変数のことを指します。スカラは温度や身長といった単一の数量を表します。スカラ変数を表すには

$$x, y, M, N$$

のように、太字や斜体にされていない文字を利用するのが一般的です。

$2.4 + 3.2i$ のような複素数でも、値が 1 つならばスカラ（複素数のスカラ）と呼びますが、本資料では特に明示しなければ実数のスカラを扱います。後述するベクトル、行列、テンソルについても同様です。

x が実数のスカラであることをしばしば「 $x \in \mathbb{R}$ 」と書きます。

$\mathbb{R}$ は実数のスカラからなる集合を表し、「 $A \in B$ 」は「 A は B に属する」という意味です。従って、 $x \in \mathbb{R}$ で「 x は実数のスカラからなる集合に属する」、すなわち「 x は実数のスカラである」と解釈できます。

ベクトル (vector) は、スカラを 1 方向に並べたものです。例えば、

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$$

のように表します。ベクトルを構成するスカラ達（ x_1 や y_2 など）のことを**要素**や**成分**と呼びます。

ベクトルを表すのに用いられる文字は、スカラと区別しやすいよう太字とするのが一般的です。

上の 2 つの例のように、その要素を**縦方向に並べたものは列ベクトル**と呼びます。

一方、

$$\mathbf{z} = [z_1 \quad z_2 \quad z_3]$$

のように、要素を横方向に並べたものは**行ベクトル**と呼びます。

本資料では、特に明示しない限り、単にベクトルと表現した場合には列ベクトルを指すものとします。

ベクトルに含まれるスカラの数のことを、そのベクトルの**次元**と呼びます。

例えば、 $\mathbf{x}$ は 3 次元の列ベクトル、 $\mathbf{z}$ は 3 次元の行ベクトルです。

スカラの場合と同様に、ベクトル $\mathbf{x}$ が N 次元のベクトルで、ベクトルを構成するスカラが実数であることを、数式を用いて「 $\mathbf{x} \in \mathbb{R}^N$ 」とも書きます。

行列 (matrix) は同じサイズのベクトルを複数個並べたものです。例えば、

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \\ x_{31} & x_{32} \end{bmatrix}$$

は行列の一例です。行列は大文字、または大文字の太文字で表記することでスカラやベクトルと区別します。行列の形 (サイズ) は行数と列数で表現します。

例えば、上に挙げた $\mathbf{X}$ は行ベクトルが 3 つ並んだものと見るができます。

そのため、 $\mathbf{X}$ の行数は 3 です。

一方、見方を変えると $\mathbf{X}$ は列ベクトルが 2 つ並んだものと見ることもできますので、 $\mathbf{X}$ の列数は 2 です。そこで、 $\mathbf{X}$ を「3 行 2 列の行列」と呼びます。

「サイズが (3, 2) である行列」や「サイズが 3×2 の行列」と呼んでも構いません。

N 次元の列ベクトルはサイズが $(N, 1)$ の行列、 N 次元の行ベクトルはサイズが $(1, N)$ の行列と見るができます。

スカラ、ベクトルの場合と並行して、行列 $\mathbf{X}$ のサイズが (N, M) であり、各要素が実数であることを、「 $\mathbf{X} \in \mathbb{R}^{N \times M}$ 」とも書きます。例えば、先程の $\mathbf{X}$ については $\mathbf{X} \in \mathbb{R}^{3 \times 2}$ です。

テンソル (tensor) はベクトルや行列を一般化した概念です。

例えば、ベクトルは 1 方向に、行列は 2 方向にスカラが並んでいます。これは「ベクトルは 1 階のテンソルで、行列は 2 階のテンソルである」であることを意味します。

この考え方をさらに進めて、下図のように行列を奥行き方向にさらに並べたものを 3 階のテンソルと呼びます。例えば、カラー画像をデジタル表現する場合、1 枚の画像は RGB (Red Green Blue) の 3 枚のレイヤー (チャンネルと呼びます) を持つのが一般的です。

各チャンネルは行列として表され、その行列がチャンネル方向に複数積み重なっているため、画像は 3 階テンソルとみなすことができます。

3 階のテンソルは、特定の要素を指定するのに「上から 3 番目、左から 2 番目、手前から 5 番目」のように整数 (インデックス) を 3 個必要とします。

同様に、4 次元以上の場合でも、 N 次元にスカラを並べたもの (つまり、要素を指定するのに N 個のインデックスが必要なもの) を N 階のテンソルと言います。例えば、多くのディープラーニングフレームワークでは、複数枚の画像の集まりを「画像のインデックス 1 つ」+「各画像のインデックス 3 つ (幅、高さ、チャンネル)」の 4 階テンソルとして表現します。

前述のようにベクトルや行列はテンソルの一種とみなすことができますが、本資料では単に「テンソル」と言った場合は 3 階以上のテンソルを指します。

7.1.1 変数の形と字体の関係

多くの教科書 (特に線形代数の教科書) では、変数を一目見てその型が何かがわかるように、特定の形 (ベクトル、行列など) の変数には特定の字体を用いるという工夫がされています。

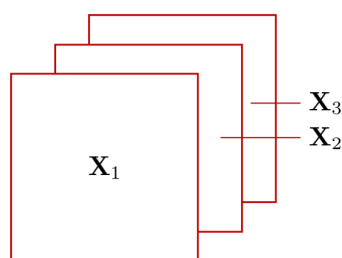


図 7.1 テンソル

本資料に限らず、数式を追う時には変数の字体を見て、その変数の形を意識するのがおすすめです。

変数の形と字体の対応関係は教科書によって流派がありますが、以下にその一例を紹介します（本資料もこの規約に従います）。

字体	小文字	大文字
細字	スカラの変数 (a, b など)	スカラの定数 (A, B など)
太字	ベクトル ($\mathbf{x}, \mathbf{y}$ など)	行列、テンソル ($\mathbf{A}, \mathbf{B}$ など)

7.1.2 加算・減算・スカラ倍

次に、ベクトル、行列、テンソルの演算について解説します。

加算（足し算） 及び **減算（引き算）** は同じサイズのベクトル同士、行列同士、テンソル同士の間だけで成立します。

以下に、行列およびベクトル同士の加算を具体例で示します。

ベクトル同士の加算

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 1+4 \\ 2+5 \\ 3+6 \end{bmatrix} = \begin{bmatrix} 5 \\ 7 \\ 9 \end{bmatrix}$$

行列同士の加算

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix} = \begin{bmatrix} 1+7 & 2+8 & 3+9 \\ 4+10 & 5+11 & 6+12 \end{bmatrix} = \begin{bmatrix} 8 & 10 & 12 \\ 14 & 16 & 18 \end{bmatrix}$$

このように、加算、減算ではベクトル（行列/テンソル）の同じ位置にある要素同士に演算を行います。

このような計算は、**要素ごとの (element-wise) 計算**とも呼ばれています。

ベクトル（行列/テンソル）のサイズが異なる場合、計算が定義できないことに注意してください。

スカラ倍とはベクトル、行列、テンソルにスカラを掛ける演算です。

例えば、スカラ k に対し、「ベクトル $\mathbf{x}$ の k 倍」とは、ベクトルの各要素に k を掛ける操作です。

行列やテンソルの場合も同様に要素ごとに同じスカラ k を掛けます。

言葉で説明するよりも以下の具体例を見る方がわかりやすいかもしれません。

ベクトルのスカラ倍

$$10 \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 10 \times 1 \\ 10 \times 2 \\ 10 \times 3 \end{bmatrix} = \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix}$$

$$\begin{array}{ccc}
 \mathbf{A} & \mathbf{B} & \mathbf{C} \\
 \begin{bmatrix} \boxed{1} & \boxed{2} \\ 3 & 4 \end{bmatrix} & \begin{bmatrix} \boxed{5} & \boxed{6} \\ 7 & 8 \end{bmatrix} & = \begin{bmatrix} \boxed{1 \times 5 + 2 \times 7} & 1 \times 6 + 2 \times 8 \\ 3 \times 5 + 4 \times 7 & 3 \times 6 + 4 \times 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix} \\
 \text{行} & \text{列} &
 \end{array}$$

図 7.2 行列積

行列のスカラー倍

$$10 \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} 10 \times 1 & 10 \times 2 & 10 \times 3 \\ 10 \times 4 & 10 \times 5 & 10 \times 6 \end{bmatrix} = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$$

スカラー倍を行う前後でベクトル（行列/テンソル）のサイズは変化しないことに注意してください。

7.1.3 内積

同じサイズの2つのベクトルには、**内積 (inner product)** という演算が定義できます。

これは、それぞれのベクトルの同じ位置に対応する要素同士を掛け、それらを足し合わせる計算です。

$\mathbf{x}$ と $\mathbf{y}$ の内積は $\mathbf{x} \cdot \mathbf{y}$ で表されます。

例えば、以下の例では3次元のベクトルの内積を計算しています。

$$\begin{aligned}
 \mathbf{x} &= \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} \\
 \mathbf{x} \cdot \mathbf{y} &= 1 \times 4 + 2 \times 5 + 3 \times 6 = 32
 \end{aligned}$$

2つのベクトルの内積はスカラーになることに注意してください。

7.1.4 行列積

行列の乗算には、**行列積**、**外積**、**要素積 (アダマール積)** など複数の方法があります。

ここではそのうち、線形代数や機械学習の多くの問題で登場する**行列積**について説明します。

以降では明示しない限り、「行列の掛け算」と言ったときには行列積を指すこととします。

行列 $\mathbf{A}$ と行列 $\mathbf{B}$ の行列積は $\mathbf{AB}$ と書き、 $\mathbf{A}$ の各行と $\mathbf{B}$ の各列の内積を並べたものとして定義されます。

例えば、行列 $\mathbf{A}$ の1行目の行ベクトルと、行列 $\mathbf{B}$ の1列目の列ベクトルの内積の結果は、 $\mathbf{A}$ と $\mathbf{B}$ の行列積の結果を表す行列 $\mathbf{C}$ の1行1列目に対応します。

内積が定義される条件はベクトルのサイズが等しくなければなりません。ここでもそれが成り立たなければなりません。具体的には、 $\mathbf{A} \in \mathbb{R}^{N \times M}$ 、 $\mathbf{B} \in \mathbb{R}^{O \times P}$ の時、 $\mathbf{AB} = \mathbf{C}$ の行列積を定義するためには、 $\mathbf{A}$ の**列数** M と $\mathbf{B}$ の**行数** O が一致する必要があります（[注釈 1](#)）。

つまり、

$$M = O$$

でなければなりません。

そして、結果の行列 $\mathbf{C}$ の行数と列数は $\mathbf{A}$ の行数 N と $\mathbf{B}$ の列数 P とそれぞれ等しくなります。

すなわち、 $\mathbf{C} \in \mathbb{R}^{N \times P}$ です。

2つの N 次元ベクトル $\mathbf{x}, \mathbf{y}$ の内積は、 $\mathbf{x}$ をサイズ $(1, N)$ の行列（すなわち行ベクトル）、 $\mathbf{y}$ をサイズ $(N, 1)$ の行列（すなわち列ベクトル）とみなして行列積を行うことと考えることができます。

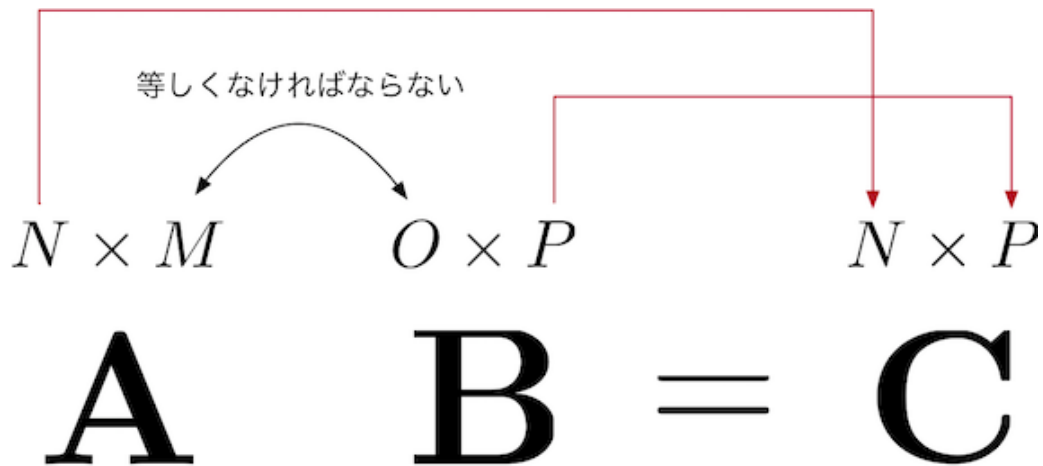


図 7.3 matrix product

2つのベクトルの行列積はサイズが $(1, 1)$ の行列、すなわちスカラとなります。これは2つのベクトルの内積の結果がスカラとなることと一致しています。

行列積は $\mathbf{AB}$ と $\mathbf{BA}$ が等しいとは限らないという点でスカラの掛け算と大きく異なります。例えば、

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

とすると、

$$\mathbf{AB} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 1*5+2*7 & 1*6+2*8 \\ 3*5+4*7 & 3*6+4*8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

$$\mathbf{BA} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 5*1+6*3 & 5*2+6*4 \\ 7*1+8*3 & 7*2+8*4 \end{bmatrix} = \begin{bmatrix} 23 & 34 \\ 31 & 46 \end{bmatrix}$$

となり、 $\mathbf{AB} \neq \mathbf{BA}$ です。

掛け算の順序の違いを明示的に表現する場合、行列 $\mathbf{A}$ に行列 $\mathbf{B}$ を左から掛けること（ $\mathbf{BA}$ の計算）を行列 $\mathbf{B}$ を行列 $\mathbf{A}$ に**左乗**すると言い、右から掛ける場合は**右乗**すると言います。

$\mathbf{A}$ と $\mathbf{B}$ の選び方によっては $\mathbf{AB}$ と $\mathbf{BA}$ が等しくなる場合もあります。

例えば、

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \mathbf{B} = \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix}$$

とすると、

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 9 & 12 \end{bmatrix}$$

となり、 $\mathbf{AB}$ と $\mathbf{BA}$ は一致します。ここで述べている注意は「**どんな** $\mathbf{A}, \mathbf{B}$ でも $\mathbf{AB} = \mathbf{BA}$ が成立する」という主張は正しくないことを意味しています。

それでは、行列の計算条件も確認しながら、下記の3つの練習問題の計算を行ってください。

$$\begin{aligned}
 (1) \quad & \begin{bmatrix} \text{ベクトル} \end{bmatrix} \begin{bmatrix} \text{ベクトル} \end{bmatrix} = \text{スカラー} & \longleftarrow & \mathbf{x}^T \mathbf{y} \\
 (2) \quad & \begin{bmatrix} \text{行} \\ \text{列} \end{bmatrix} \begin{bmatrix} \text{ベクトル} \end{bmatrix} = \begin{bmatrix} \text{ベクトル} \end{bmatrix} & \longleftarrow & \mathbf{X}^T \mathbf{y} \\
 (3) \quad & \begin{bmatrix} \text{ベクトル} \end{bmatrix} \begin{bmatrix} \text{行} \\ \text{列} \end{bmatrix} \begin{bmatrix} \text{ベクトル} \end{bmatrix} = \text{スカラー} & \longleftarrow & \mathbf{x}^T \mathbf{A} \mathbf{y}
 \end{aligned}$$

図 7.4 演算後の行列

$$\begin{aligned}
 (1) \quad & \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \end{bmatrix} \\
 (2) \quad & \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} \\
 (3) \quad & \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix}
 \end{aligned}$$

こちらが解答です。

$$\begin{aligned}
 (1) \quad & \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = 1 \times 3 + 2 \times 4 = 11 \\
 (2) \quad & \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 1 \times 5 + 2 \times 6 \\ 3 \times 5 + 4 \times 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix} \\
 (3) \quad & \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 3 \times 3 + 4 \times 1 \\ 5 \times 3 + 6 \times 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 13 \\ 21 \end{bmatrix} = 1 \times 13 + 2 \times 21 = 55
 \end{aligned}$$

このような計算は、機械学習の基礎を学習していく過程でよく登場します。行列積では、演算前と後の行数・列数の変化に注意しましょう。

7.1.5 行列積によるベクトル・行列のサイズ変化

行列の前後では行列の形が変化します。

具体的にはサイズが (L, M) と (M, N) の行列の行列積の結果はサイズが (L, N) の行列となります。

先ほどの 3 つの練習問題では、行列やベクトルの形・サイズがどのように変化していたかを確認してみましょう。

特に (3) では、一番左端のベクトルと、真ん中の行列の乗算結果が、行ベクトルであるため、サイズの変化が (1) と同じケースに帰着することに注意してください。

また、例えば (3, 1) の行列のようにある次元のサイズが 1 となった場合、その次元を削除する場合があります。例えば (2) の計算結果はサイズが (2, 1) の行列ですが、これは 2 次元のベクトルとして扱えます。同様に、(1), (3) の答えはサイズ (1, 1) の行列ですが、スカラとみなすことができます。このようにサイズが 1 になった次元をつぶす操作をしばしば **squeeze** と呼びます。

7.1.6 転置

ベクトルは縦方向に要素が並んだ列ベクトルを基本としていましたが、横方向に要素が並んだ行ベクトルを使いたい場合もあります。そこで列ベクトルを行ベクトルに、行ベクトルを列ベクトルに変換する操作を**転置 (transpose)** と呼びます。転置はベクトルの右肩に T と書くことで表します。例えば、 $\mathbf{x}$ が 3 次元のベクトルならば、

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \mathbf{x}^T = [1 \quad 2 \quad 3]$$

です。

転置は行列に対しても定義できます。例えば、

$$\mathbf{X} = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}, \mathbf{X}^T = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

です。行列に対する転置では、行と列が入れ替わるため、サイズが (N, M) だった行列は、転置するとサイズが (M, N) の行列となります。つまり、 i 行 j 列目の値が転置後には j 行 i 列目の値になります。転置の公式として次を覚えておきましょう。

- (1) $(\mathbf{A}^T)^T = \mathbf{A}$
- (2) $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$
- (3) $(\mathbf{ABC})^T = \mathbf{C}^T \mathbf{B}^T \mathbf{A}^T$

転置を用いると、2 つの列ベクトル $\mathbf{x}, \mathbf{y}$ の**内積** **@@TOK2@@** は、**行列積を用いて** **@@TOK3@@** と書けることに注意してください。

7.1.7 単位行列

スカラ値の 1 は、 $10 \times 1 = 10$ のように、任意の数を 1 に乗じても値が変化しないという性質を持ちます。行列の演算において、これと同様の働きをする行列が**単位行列 (identity matrix, unit matrix)** です ([注釈 2](#))。

単位行列は

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

という形をしており、記号 $\mathbf{I}$ で表すのが一般的です。行列の斜めの要素を**対角要素**とよび、それ以外の要素を**非対角要素**とよびます。単位行列は、対角要素が全て 1 で、非対角要素が全て 0 であるような**正方行列** (行数と列数が等しい行列) です。例えば、 2×2 の単位行列は、

$$\mathbf{I} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

であり、 3×3 の単位行列は、

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

です。行列のサイズを明示したい場合には、 I_n （ $n \times n$ の単位行列の意味）と添字でサイズを表記することがあります。

単位行列はサイズが等しい任意の正方行列 $\mathbf{A}$ に対して以下の計算が成立します。

$$\mathbf{AI} = \mathbf{A}$$

$$\mathbf{IA} = \mathbf{A}$$

適当な行列で、単位行列を掛けても元の行列と値が変わらないことを確認してみましょう。

$$\begin{aligned} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} &= \begin{bmatrix} 1 \times 1 + 2 \times 0 & 1 \times 0 + 2 \times 1 \\ 3 \times 1 + 4 \times 0 & 3 \times 0 + 4 \times 1 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \end{aligned}$$

計算結果から分かる通り、元の行列と全ての要素が一致しました。

$\mathbf{I}$ を左から掛けても同様の結果となることを確かめてみてください。

7.1.8 逆行列

0 でないスカラー x には、逆数 x^{-1} を考えることができます。

行列で逆数に対応するものが **逆行列 (inverse matrix)** です。

厳密な定義は次の通りです。行列 $\mathbf{A}$ に対し、 $\mathbf{AB} = \mathbf{I}$, $\mathbf{BA} = \mathbf{I}$ を満たす行列 $\mathbf{B}$ のことを $\mathbf{A}$ の逆行列といいます。

このような条件を満たす行列 $\mathbf{B}$ は行列 $\mathbf{A}$ によって存在したり存在しなかったりします。

しかし、もし $\mathbf{A}$ に対しこの条件を満たす行列が存在するならば、そのような行列はただ 1 つであることが知られています。

そこで、行列 $\mathbf{A}$ の逆行列を $\mathbf{A}^{-1}$ と書きます。

逆行列の定義から、

$$\mathbf{AA}^{-1} = \mathbf{I}$$

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$$

が成立することに注意してください。ここで、 $\mathbf{I}$ は単位行列です。

逆行列が存在するような行列のことを**正則行列**と呼びます。

正則行列となるには少なくとも行列積 $\mathbf{AA}^{-1}$ と $\mathbf{A}^{-1}\mathbf{A}$ の両方が定義されていなければなりません。

これは $\mathbf{A}$ は正方行列（つまり、行数と列数が同じ行列）であることを意味します（なぜか考えてみてください）。

しかし、 $\mathbf{A}$ の行数と列数が同じであるからと言って、常に $\mathbf{A}$ が正則行列とは限りません。

行列が正則であるための条件に関する詳細な説明はここでは省略します。

サイズが 2×2 や 3×3 といった小さな行列の場合には、手計算でも可能な逆行列計算の方法が知られていますが、機械学習ではより大きなサイズの行列（ 1000×1000 など）を扱うことがあり、そういった大きな行列の逆行列を効率的または近似的にコンピュータを使って計算する手法が研究されています。

7.1.9 線形結合と二次形式

機械学習の数式で頻出する形式として、 $\mathbf{b}^T \mathbf{x}$ と $\mathbf{x}^T \mathbf{A} \mathbf{x}$ の 2 つがあります。

前者は**線形結合**もしくは**一次結合**、後者は**二次形式**と呼ばれています。

スカラの一次式 $(ax + b)$ や二次式 $(ax^2 + bx + c)$ をベクトルに拡張したものと捉えると良いでしょう。

線形結合の計算を要素ごとに見てみると、

$$\mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\mathbf{b}^T \mathbf{x} = \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1 + 2x_2$$

のように $\mathbf{x}$ の要素である x_1 および x_2 に関して一次式となっています。

また、二次形式も同様に要素ごとに確認してみると、

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\mathbf{x}^T \mathbf{A} \mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$= \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} x_1 + 2x_2 \\ 3x_1 + 4x_2 \end{bmatrix}$$

$$= x_1(x_1 + 2x_2) + x_2(3x_1 + 4x_2)$$

$$= x_1^2 + 5x_1x_2 + 4x_2^2$$

となり、各要素に関して二次式となっています。

一般に、 $x_1, \dots, x_N$ に関する二次関数は、

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

として、

$$\mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{b}^T \mathbf{x} + c$$

と表記できます。ここで、 $\mathbf{A}$ は 2 次の項の係数を表すサイズ (n, n) の 行列、 $\mathbf{b}$ は 1 次の項の係数を表す n 次元のベクトル、 c はスカラの定数項です。

7.1.10 ベクトルによる微分と勾配

微分は入力に変化した場合の関数値の変化量から求められました。

これは関数の入力にベクトルである場合も同様です。

ベクトルを入力にとる関数の微分を考えてみましょう。

入力ベクトルの要素毎に出力に対する偏微分を計算し、それらを並べてベクトルにしたものが**勾配 (gradient)** です。

まず勾配に関する計算の具体例を見てみましょう。

$$\mathbf{b} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\mathbf{b}^T \mathbf{x} = \begin{bmatrix} 3 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = 3x_1 + 4x_2$$

この $\mathbf{b}^T \mathbf{x}$ をベクトル $\mathbf{x}$ で微分したものを、

$$\frac{\partial}{\partial \mathbf{x}} (\mathbf{b}^T \mathbf{x})$$

と表します。

「ベクトルで微分」とは、ベクトルのそれぞれの要素で対象を微分し、その結果を要素に対応する位置に並べてベクトルを作ることです。

前述の例では、以下のように計算を行います。

$$\begin{aligned}\frac{\partial}{\partial \mathbf{x}} (\mathbf{b}^T \mathbf{x}) &= \frac{\partial}{\partial \mathbf{x}} (3x_1 + 4x_2) \\ &= \left[\frac{\partial}{\partial x_1} (3x_1 + 4x_2) \quad \frac{\partial}{\partial x_2} (3x_1 + 4x_2) \right]\end{aligned}$$

各要素の計算を進めると、以下のようになります。本チュートリアルでは、ベクトルは原則として列ベクトルを用いるルールを採用していますが、ここでは微分した結果を行ベクトルで表していることに注意してください。これは、後で登場するベクトル値関数（ベクトルを出力とする関数）の微分を考えた場合との整合性をとるためです。

$$\begin{aligned}\frac{\partial}{\partial x_1} (3x_1 + 4x_2) &= \frac{\partial}{\partial x_1} (3x_1) + \frac{\partial}{\partial x_1} (4x_2) \\ &= 3 \times \frac{\partial}{\partial x_1} (x_1) + 4x_2 \times \frac{\partial}{\partial x_1} (1) \\ &= 3 \times 1 + 4x_2 \times 0 \\ &= 3\end{aligned}$$

$$\begin{aligned}\frac{\partial}{\partial x_2} (3x_1 + 4x_2) &= \frac{\partial}{\partial x_2} (3x_1) + \frac{\partial}{\partial x_2} (4x_2) \\ &= 3x_1 \times \frac{\partial}{\partial x_2} (1) + 4 \times \frac{\partial}{\partial x_2} (x_2) \\ &= 3x_1 \times 0 + 4 \times 1 \\ &= 4\end{aligned}$$

従って、計算結果を整理すると、以下のようになります。

$$\frac{\partial}{\partial \mathbf{x}} (\mathbf{b}^T \mathbf{x}) = \left[\frac{\partial}{\partial x_1} (3x_1 + 4x_2) \quad \frac{\partial}{\partial x_2} (3x_1 + 4x_2) \right] = [3 \quad 4] = \mathbf{b}^T$$

もう一つ別の例を考えてみましょう。今度は定数スカラーをベクトルで微分します。

$$\begin{aligned}c = 3, \mathbf{x} &= \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \\ \frac{\partial}{\partial \mathbf{x}} (c) &= \left[\frac{\partial}{\partial x_1} (3) \quad \frac{\partial}{\partial x_2} (3) \right] = [0 \quad 0] = \mathbf{0}\end{aligned}$$

偏微分を行う対象に変数が含まれていない場合、その偏微分は 0 となります。要素が 0 のみで構成されたベクトルを**ゼロベクトル**と言い、数字の 0 を太字にした **0** で表します。

これらを踏まえて、機械学習で頻出する計算結果をまとめて覚えておきましょう。

$$\begin{aligned}(1) \quad \frac{\partial}{\partial \mathbf{x}} (c) &= \mathbf{0} \\ (2) \quad \frac{\partial}{\partial \mathbf{x}} (\mathbf{b}^T \mathbf{x}) &= \mathbf{b}^T \\ (3) \quad \frac{\partial}{\partial \mathbf{x}} (\mathbf{x}^T \mathbf{A} \mathbf{x}) &= \mathbf{x}^T (\mathbf{A} + \mathbf{A}^T)\end{aligned}$$

ここで、(1) と (2) はすでに導出済みです。(3) は導出が少し複雑なので省略しますが、数値を代入して確認してみてください。

7.2 ベクトルを入力とする関数

前節で考えた $\mathbf{b}^T \mathbf{x}$ や $\mathbf{x}^T \mathbf{A} \mathbf{x}$ はベクトル $\mathbf{x}$ を受け取って、スカラー $\mathbf{b}^T \mathbf{x}$, $\mathbf{x}^T \mathbf{A} \mathbf{x}$ を出力する関数と考えることができます。 $\mathbf{x}$ を成分表示して

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

と書くと、この関数は 2 つのスカラー変数 x_1, x_2 を入力とする多変数関数です。

同様に、入力が M 個のスカラー変数の関数は、 M 次元のベクトル 1 個を入力とする関数とも解釈することができます。

多変数関数 $f(x_1, \dots, x_M)$ は、入力をまとめて

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_M \end{bmatrix}$$

と書き、 $f(\mathbf{x})$ と表記しても構いません。 $f(\mathbf{x})$ の微分は、前章で説明した偏微分を各変数 $x_1, \dots, x_M$ に対して行い

$$\frac{\partial}{\partial \mathbf{x}} f(\mathbf{x}) = \left[\frac{\partial}{\partial x_1} f(\mathbf{x}) \quad \dots \quad \frac{\partial}{\partial x_M} f(\mathbf{x}) \right]$$

と計算します。

7.3 ベクトル値関数

次に入力ではなく出力が多変数の関数を考えます。

まず、1 つのスカラー変数 x を入力とし、 N 個のスカラー（つまり、 N 次元のベクトルを 1 個）を出力する関数 $\mathbf{f}(x)$ を考えます。

ベクトル変数は $\mathbf{y}$ などの太字で書いていました。それと同様に、この関数も出力がベクトルであることを強調して $\mathbf{f}$ と太字で表します。

$\mathbf{f}$ のように出力がベクトルである関数を **ベクトル値関数 (vector-valued function)** と呼びます。それに対し、出力がスカラーである関数を（そのことを強調したい場合には）**スカラー値関数 (scalar-valued function)** と呼びます。

$\mathbf{f}$ の出力の各成分に注目すると、 $\mathbf{f}$ は N 個のスカラー値関数の集まりと考えることができます。

つまり、 x を受け取り、 $\mathbf{f}$ の第 n 成分を出力する関数を f_n と書くと、

$$\mathbf{f}(x) = \begin{bmatrix} f_1(x) \\ \vdots \\ f_N(x) \end{bmatrix}$$

です。

入力が多変数でも同様に成分表示することができます。すなわち、入力変数が M 次元ベクトル、出力変数が N 次元ベクトルである関数 $\mathbf{g}$ は

$$\mathbf{g}(\mathbf{x}) = \begin{bmatrix} g_1(\mathbf{x}) \\ \vdots \\ g_N(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} g_1(x_1, \dots, x_M) \\ \vdots \\ g_N(x_1, \dots, x_M) \end{bmatrix}$$

と成分表示できます。誤解が生じないならば、一番右の表式のように入力変数 $\mathbf{x}$ を成分表示しても構いません。

7.4 ベクトル値関数の微分

ベクトル値関数の微分を行うには、各成分ごとにスカラ値関数と同様の方法で微分します。例えば前節の $\mathbf{f}$ の微分は

$$\mathbf{f}'(x) = \begin{bmatrix} f'_1(x) \\ \vdots \\ f'_N(x) \end{bmatrix}$$

です。出力がベクトルであることに対応して、微分もベクトルであることに注意してください。

微小量を表す d を用いて

$$\frac{d}{dx}\mathbf{f}(x) = \begin{bmatrix} \frac{d}{dx}f_1(x) \\ \vdots \\ \frac{d}{dx}f_N(x) \end{bmatrix}$$

と書いても構いません。

入出力が共に多変数である関数を微分する場合も、出力の成分ごとに微分します。

ただし、今度は入力も多変数であるため、入力の成分ごとに偏微分を行わなければなりません。

例えば、前節の $\mathbf{g}$ を微分すると、前節の成分表示を用いて

$$\frac{\partial \mathbf{g}}{\partial \mathbf{x}}(\mathbf{x}) = \begin{bmatrix} \frac{\partial g_1}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial g_1}{\partial x_M}(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \frac{\partial g_N}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial g_N}{\partial x_M}(\mathbf{x}) \end{bmatrix}$$

となります。 M 個の入力変数と N 個の出力変数の組み合わせを考える必要があるので、微分はサイズ (N, M) の行列となることに注意してください。この、すべての偏微分の組み合わせをまとめて作った行列のことを **ヤコビ行列** と呼びます。

7.5 合成関数の微分（多変数バージョン）

先程 1 変数入力、1 変数出力の合成関数の微分（連鎖律）を説明しました。

類似の定理が多変数入力、多変数出力の場合にも成立します。

本節では多変数バージョンの合成関数の微分の公式を紹介します。

$\mathbf{g}$ を M 変数入力 N 変数出力、 $\mathbf{f}$ を N 変数入力 L 変数出力の関数とします。

これらの関数の合成 $\mathbf{h} = \mathbf{f}(\mathbf{g}(\mathbf{x}))$ は M 変数入力 L 変数出力の関数であることに注意してください。

これを x で微分します。

以下の式が多変数関数バージョンの合成関数の微分の公式です。

$$\frac{\partial \mathbf{h}}{\partial \mathbf{x}}(\mathbf{x}) = \frac{\partial \mathbf{f}}{\partial \mathbf{u}}(\mathbf{u}) \frac{\partial \mathbf{g}}{\partial \mathbf{x}}(\mathbf{x}).$$

ここで、 $\mathbf{u} = \mathbf{g}(\mathbf{x})$ です。

1 変数入力、1 変数出力の場合の合成関数の微分の公式と見比べると細字だった変数や関数が太字となり d であった所が偏微分を表す ∂ に置き換えられています。すなわち、1 変数の場合との最も大きな違いは

$$\frac{\partial \mathbf{h}}{\partial \mathbf{x}}(\mathbf{x}), \frac{\partial \mathbf{f}}{\partial \mathbf{u}}(\mathbf{u}), \frac{\partial \mathbf{g}}{\partial \mathbf{x}}(\mathbf{x})$$

が行列であることです。行列のサイズはそれぞれ (L, M) , (L, N) , (N, M) です。

それに伴い、公式の右辺の掛け算は行列積です。サイズが (L, N) と (N, M) なので、行列積がきちんと定義できることに注意してください。

また、行列の積は交換可能ではないので、右辺を交換して $\frac{\partial \mathbf{g}}{\partial \mathbf{x}}(\mathbf{x}) \frac{\partial \mathbf{f}}{\partial \mathbf{u}}(\mathbf{u})$ とすると別の意味になってしまう（そもそも定義することができないかもしれない）ことにも注意してください。

$M = N = L = 1$ とすると、前節の合成関数の微分と全く同じ公式です。

その意味で多変数関数の合成関数の微分は 1 変数の場合を一般化した公式です。

逆に、1 変数の公式は多変数の公式の特別な場合と言うこともできます。

7.5.1 計算例 1

合成関数の微分を使って、多変数関数の微分を具体的に計算してみましょう。

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix}$$

として、

$$\mathbf{g}(\mathbf{x}) = \begin{bmatrix} 3x_1 + 1 \\ 2x_2 + 4 \\ x_1x_2 \\ 5 \end{bmatrix}, \mathbf{f}(\mathbf{u}) = \begin{bmatrix} 2u_1 + 1 \\ u_2^2 + u_3^2 \\ u_3u_4 \end{bmatrix}$$

とします。

つまり、 $\mathbf{g}$ は 2 入力 4 出力、 $\mathbf{f}$ は 4 入力 3 出力です。

これらについて $\frac{\partial \mathbf{f}(\mathbf{g}(\mathbf{x}))}{\partial \mathbf{x}}$ を計算します。

偏微分を定義どおり計算すると

$$\begin{aligned} \frac{\partial \mathbf{g}}{\partial \mathbf{x}}(\mathbf{x}) &= \begin{bmatrix} \frac{\partial g_1}{\partial x_1} & \frac{\partial g_1}{\partial x_2} \\ \frac{\partial g_2}{\partial x_1} & \frac{\partial g_2}{\partial x_2} \\ \frac{\partial g_3}{\partial x_1} & \frac{\partial g_3}{\partial x_2} \\ \frac{\partial g_4}{\partial x_1} & \frac{\partial g_4}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 2 \\ x_2 & x_1 \\ 0 & 0 \end{bmatrix} \\ \frac{\partial \mathbf{f}}{\partial \mathbf{u}}(\mathbf{u}) &= \begin{bmatrix} \frac{\partial f_1}{\partial u_1} & \frac{\partial f_1}{\partial u_2} & \frac{\partial f_1}{\partial u_3} & \frac{\partial f_1}{\partial u_4} \\ \frac{\partial f_2}{\partial u_1} & \frac{\partial f_2}{\partial u_2} & \frac{\partial f_2}{\partial u_3} & \frac{\partial f_2}{\partial u_4} \\ \frac{\partial f_3}{\partial u_1} & \frac{\partial f_3}{\partial u_2} & \frac{\partial f_3}{\partial u_3} & \frac{\partial f_3}{\partial u_4} \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 2u_2 & 2u_3 & 0 \\ 0 & 0 & u_4 & u_3 \end{bmatrix} \end{aligned}$$

となります（行列中の $\frac{\partial g_1}{\partial x_1}$ などは本来は $\frac{\partial g_1}{\partial x_1}(\mathbf{x})$ などと書くべきですが、式が煩雑になるため省略しました）。

よって、合成関数の微分の公式より、

$$\begin{aligned}
\frac{\partial \mathbf{f}(\mathbf{g}(\mathbf{x}))}{\partial \mathbf{x}} &= \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 2u_2 & 2u_3 & 0 \\ 0 & 0 & u_4 & u_3 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 2 \\ x_2 & x_1 \\ 0 & 0 \end{bmatrix} \\
&= \begin{bmatrix} 6 & 0 \\ 2u_3x_2 & 4u_2 + 2u_3x_1 \\ u_4x_2 & u_4x_1 \end{bmatrix} \\
&= \begin{bmatrix} 6 & 0 \\ 2x_1x_2^2 & 2x_1^2x_2 + 8x_2 + 16 \\ 5x_2 & 5x_1 \end{bmatrix}
\end{aligned}$$

となります。公式を適用する際に $\mathbf{u} = \mathbf{g}(\mathbf{x})$ 、すなわち

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} = \begin{bmatrix} 3x_1 + 1 \\ 2x_2 + 4 \\ x_1x_2 \\ 5 \end{bmatrix}$$

を用いました。

7.5.2 計算例 2

もう 1 つの例として、合成した関数の入出力が 1 変数となる場合、つまり、 $M = 1, L = 1$ の場合を考えてみましょう。

式を簡単にするために、 $N = 2$ とします。

1 変数であることを強調して、 $\mathbf{u} = \mathbf{g}(x)$ 、 $y = f(\mathbf{u})$ とスカラーとなる変数には小文字を用いることにします。

f と $\mathbf{g}$ を合成した関数を $h(x) = f(\mathbf{g}(x))$ とします。

h は入出力が共に 1 変数なので、その微分を簡便に $h'(x)$ と書くことにすると、合成関数の微分の公式は

$$\begin{aligned}
h'(x) &= \frac{\partial f}{\partial \mathbf{u}}(\mathbf{u}) \frac{d\mathbf{g}}{dx}(x) \\
&= \begin{bmatrix} \frac{\partial f}{\partial u_1}(\mathbf{u}) & \frac{\partial f}{\partial u_2}(\mathbf{u}) \end{bmatrix} \begin{bmatrix} \frac{dg_1}{dx}(x) \\ \frac{dg_2}{dx}(x) \end{bmatrix} \\
&= \frac{\partial f}{\partial u_1}(\mathbf{u}) \frac{dg_1}{dx}(x) + \frac{\partial f}{\partial u_2}(\mathbf{u}) \frac{dg_2}{dx}(x)
\end{aligned}$$

と書き下すことができます（微分する変数が 1 つしかない場合は偏微分記号 ∂ ではなく 1 変数微分 d を用いて書きました）。

本資料で紹介した合成関数の微分の公式の他にも行列やベクトルの計算で用いられる公式はたくさんあります。

それらの公式は [The Matrix Cookbook](#) などにまとまっていますので、論文等を読む際などにも必要に応じてこれらを参照すると良いでしょう。

また、ヤコビ行列の計算方法は [The Matrix Calculus You Need For Deep Learning](#) などにまとまっています。

7.6 注釈

7.6.1 注釈 1

$N \times M$ 行列、などと言われたときに、 N と M のどちらが行で、どちらが列だろう？と迷ったときは、「行列」という言葉を再度思い浮かべて、「行→列」つまり先にくる N が行数で、 M が列数だ、と思い出すのがおすすめです。

7.6.2 注釈 2

文献によっては単位行列ではなく、すべての成分が 1 の行列のことを unit matrix と呼ぶこともあります。

第8章

確率・統計の基礎

本節では機械学習で用いる確率論、統計学の概念や用語を解説します。

世の中には「ランダム」に起こる出来事、もしくは背景のメカニズムがわからないため「ランダム」として扱わざるを得ない出来事が存在します。そのようなランダムな出来事を理論的に扱うには、出来事そのものや、ある出来事と別の出来事の関連を数学的に記述できなければなりません。確率論ではランダムな出来事のことを**事象 (event)**と呼びます（厳密な定義は本資料では省略します）。

本節では、まず事象を記述する道具として確率分布を導入し、それに関連する概念（周辺確率、条件付き確率、確率変数の独立）を解説します。さらにこれらの概念を用いてベイズの定理を説明します。ベイズの定理により、ある出来事が観測された時に、その原因となる出来事が起こった確率の計算ができるようになります。

次に、確率分布の中から観測データに適合した最適な確率分布を選択する方法である、最尤推定と事後分布最大化推定（MAP 推定）を紹介します。

機械学習の文脈では、これらは訓練モデルの「最適な」パラメータを決定することに対応します。

最後に、機械学習で頻繁に利用される統計学の用語（平均、分散、標準偏差、相関係数など）を解説します。

8.1 確率・統計と機械学習の関係

機械学習システムが学習に用いるのは限られた数の観測データですが、多くの機械学習タスクでやりたいことは、与えられた観測データの背後にある普遍性や法則を捉え、将来の出来事に対し予測を行えることです。機械学習ではそれを達成するために統計学の考え方を利用します。

統計学では、ある集団すべてについて調査ができない時、集団からランダムサンプリングを行い元の集団の性質を推定します。手元にある観測データが、何らかの法則に従って得られる確率的なサンプルであると考えすることで、機械学習と統計学がつながります。

統計学を利用することで、あるデータが未知のデータ源から発生しやすいか、データが外れ値かどうか、どういった方法でモデルを学習させればよいか、といった問題を客観的に判断することができるようになります。また、学習させたモデルの性能に対して理論的な保証を与えることも統計学により可能となります。

このような、機械学習の統計学としての側面を強調する場合、機械学習を特に「統計的機械学習」と呼びます。

8.2 確率変数と確率分布

現代数学で広く利用されている「確率」の概念を定式化するには様々な準備が必要になるため、この資料では「確率」という言葉を数学的に厳密には定義しません。代わりに次のように考えます。ある対象としている現象の中で、様々な事象があり得るとき、それぞれの事象ごとに、それが「どの程度起きそうか」という度合いを考えます。確率とはその度合いのこととします。そして、その確率に従って、色々な値をとりうる**確率変**

数 (random variable) を考えます。確率変数は、名前に「変数」とついていますが、「事象」を「数値」に変換する関数と考えると理解しやすくなります。例えば、「コインを投げて表が出る」という「事象」を、「1」という「数値」に変換し、「コインを投げて裏が出る」という「事象」を、「0」という「数値」に変換する関数を考えると、これは「1」か「0」という値のどちらかをとりうる確率変数だ（注釈1）ということになります。

それでは、確率的現象の具体例を考えてみます。ある歪んだサイコロがあり、「サイコロを投げて x という目が出た」という事象（注釈2）を、 x という数値に対応させる確率変数 X があるとします。そして、この確率変数がとりうる全ての値が、それぞれどのような確率で出現するかを表した以下のような表があります。

確率変数 X の値	その値をとる確率
1	0.3
2	0.1
3	0.1
4	0.2
5	0.1
6	0.2

このような表を**確率分布 (probability distribution)** と呼びます。確率分布には重要な制約があり、「確率変数がとりうるあらゆる値の確率をすべて足すと和が必ず 1 になること」及び「全ての確率は 0 以上の値であること」の両者を常に満たします。上の表の左の列に並ぶ数値を**実現値**と呼び、小文字の x で表します。そして、右の列に並ぶそれぞれの x に対応する確率を $p(x)$ と書くことにします。すなわち上の表から $p(1) = 0.3$, $p(2) = 0.1$, ... です。この表記を用いると、確率分布が持つ 2 つの制約は、以下のように表せます。

$$\sum_x p(x) = 1 \quad (8.1)$$

$$\forall x, p(x) \geq 0 \quad (8.2)$$

ここで、 $\sum_x$ は全てのありうる x の値にわたる和を表し、例えば上のサイコロの例では、 $\sum_{x=1}^6$ と同じことを意味します。 $\forall x$ は、あり得る x の値すべてにおいて、右の条件（ $p(x) \geq 0$ ）が成り立つ、ということの意味をしています。

$p(1) = 0.3$ というのは、確率変数 X が 1 という値をとる確率ということですが、これを $p(X = 1) = 0.3$ とも書きます。前述の $p(x)$ は、確率変数 X の存在を暗に仮定し、 $p(X = x)$ を簡単に表記したもの、すなわち X という確率変数がある値 x をとる確率と考えることができます。一方、 $p(X)$ と確率変数だけを引数にとる場合、これは上の表のような確率分布を意味します。

8.3 同時分布・周辺確率

前節では、1 つの確率変数について、その分布（確率分布のこと）とは何かと、分布が持つ制約について説明しました。この節では、複数の確率変数が登場する場合について考えます。

まず具体例を用いて考えてみます。ここに 2 つのサイコロがあり、それぞれのサイコロの出目を 2 つの確率変数 X, Y で表します。この 2 つのサイコロを同時に振って、1 つ目のサイコロが x という値をとり、2 つ目のサイコロが y という値をとったという事象の確率は、以下のように書き表します。

$$p(X = x, Y = y)$$

x, y はいずれも 1, 2, 3, 4, 5, 6 の 6 つの数字のどれかです。例えば、「3」と「5」の目が出る、という事象が起こる確率は

$$p(X = 3, Y = 5)$$

と表されます。このように、 $X = 3$ となる**かつ** $Y = 5$ となる、といった複数の条件を指定したときに、それらが全て同時に成り立つ確率のことを、**同時確率 (joint probability)** と呼びます。

では次に、この 2 つのサイコロを別々に見てみましょう。例えば、「1 つ目のサイコロが 3 の目を出した」という事象が起こる確率 $p(X = 3)$ は、1 つ目のサイコロが 3 **かつ** 2 つ目のサイコロが 1 のとき / 2 のとき / 3 のとき / ... / 6 のとき、の 6 つのパターンが発生する確率を全て足したものになります。つまり、

$$p(X = 3) = \sum_y p(X = 3, Y = y)$$

と書けます。このとき、 $\sum_y$ は「 Y のとり得るあらゆる値 y についての和」という意味です。これを、「(2 つ目のサイコロの目がなんであれ) 1 つ目のサイコロの目が x である確率」と一般化すると、以下ようになります。

$$p(X = x) = \sum_y p(X = x, Y = y)$$

同様に、「(1 つ目のサイコロの目がなんであれ) 2 つ目のサイコロの目が y である確率」は、1 つ目のサイコロについてあり得る値すべての確率の和を取れば良いので、

$$p(Y = y) = \sum_x p(X = x, Y = y)$$

となります。このように、同時確率が与えられたとき、着目していない方の確率変数がとり得る全ての値について同時確率を計算しその和をとることを**周辺化 (marginalization)** と呼び、結果として得られる確率を**周辺確率 (marginal probability)** と呼びます。

また、周辺確率をその着目している確率変数がとり得る全ての値について並べて一覧にしたものが**周辺確率分布 (marginal probability distribution)** です。

さらに、上の例のように 2 つの確率変数の同時確率を考えると、とり得る全ての組み合わせの確率を一覧にしたものが、**同時分布 (joint distribution)** です。

ここで、2 つのサイコロの同時分布の表は大きくなってしまいますので、より簡単な例として、表が出る確率と裏が出る確率が異なる 2 つのコインを考えてみましょう。この 2 つのコインを同時に投げたときの表裏の組み合わせについての同時分布が、以下ようになったとします。

	Y = 表	Y = 裏
X = 表	1 / 5	2 / 5
X = 裏	1 / 5	1 / 5

ここで、1 つ目のコインの表裏を表す確率変数を X 、2 つ目のコインの表裏を表す確率変数を Y としています。表の中身がどういう意味を持っているか確認してください。2 つのコインが両方表になる確率は $p(X = \text{表}, Y = \text{表}) = 1/5$ となっています。他のマスに書かれている同時確率の値の意味も確認してください。

では、この表の中の数字を、行ごとに合計してみましょう。1 行目は、

$$p(X = \text{表}, Y = \text{表}) + p(X = \text{表}, Y = \text{裏}) = 1/5 + 2/5 = 3/5$$

です。

これは、 $\sum_y p(X = \text{表}, Y = y)$ (注釈 3) を計算していることになるので、**周辺化によって $p(X = \text{表})$ という周辺確率を求めていることに相当します。**

同様に、1 列目の値を合計してみると、今度は $\sum_x p(X = x, Y = \text{表})$ (注釈 4) を計算することに相当し、これは周辺化によって $P(Y = \text{表})$ という周辺確率を計算していることになります。

こうして計算される周辺確率を、上の同時分布の表に書き入れてみます。

	Y = 表	Y = 裏	p(X)
X = 表	1 / 5	2 / 5	3 / 5
X = 裏	1 / 5	1 / 5	2 / 5
p(Y)	2 / 5	3 / 5	

このように、周辺確率はしばしば同時分布表の周辺に記述されます。

8.4 条件付き確率

前節では、複数の確率変数を同時に考える方法として、同時確率および同時分布という概念と、同時確率と一つ一つの確率変数のみに着目した際の確率（周辺確率）の間の関係を、周辺化という計算で説明しました。

本節では、「とある条件下での着目事象の確率」を考える**条件付き確率 (conditional probability)** という概念を説明します。

これまでの節で扱っていた確率では、まず対象とする現象について、考えうる全ての事象を考え、そのうち対象とする事象が起きる確率はいくらか、を考えていました。

一方、条件付き確率は、それらの考えうる事象のうち、特定の条件を満たした事象のみをまず抜き出し、その中で、さらに着目する特定の事象が起きる確率を考えるためのものです。

例えば、あるお店にやってきた人（以下、客）が傘を持っているときに 1、持っていなかったときに 0 をとる確率変数 X を考えます。

また、ある客がやってきたときに外で雨が降っていたときに 1、降っていなかったときに 0 をとる確率変数 Y を考えます。

客が傘を持っていたかと、その瞬間に雨が降っていたかどうか、という 2 つの情報が組となったデータが 16 個あるとします。

16 人の客を観測してデータを集めた、ということです。

それは以下の様なものでした。

傘 (X)	雨 (Y)
1	1
1	1
1	1
1	1
1	1
1	1
0	1
0	1
0	1
1	0
0	0
0	0
0	0
0	0
0	0
0	0

これらのデータから、確率変数 X, Y の同時分布表を作成すると、以下のようになります。この表の作り方は、例えば表の左上のマスは $X = 0$ かつ $Y = 0$ となっているデータが 16 個中何個あるかを数えて、その割合を記入すればよく、他のマスについても同様となります。

	Y = 0	Y = 1
X = 0	6 / 16	3 / 16
X = 1	1 / 16	6 / 16

では、「客が傘を持っていた ($X = 1$)」という**条件の下で**、「そのとき雨が降っていなかった ($Y = 0$)」という確率は、どのようになるでしょうか？

まず、「客が傘を持っていた ($X = 1$)」という状況下のデータだけを、上の同時分布表から抜き出してみよう。

傘 (X)	雨 (Y)
1	1
1	1
1	1
1	1
1	1
1	1
1	0

全部で 7 個のデータがあり、この中で「雨が降っていなかった ($Y = 0$)」となっているデータは 1 個です。すなわち、「客が傘を持っていたという**条件の下で**、そのとき雨が降っていなかった確率」は $1/7$ となります。これを、以下のように書きます。

$$p(Y = 0 | X = 1) = \frac{1}{7}$$

これが**条件付き確率**と呼ばれるもので、条件となる事象を縦棒 | の後に書き、その条件下で注目している事象を縦棒 | の前に書きます。同様に、「客が傘を持っていたという**条件の下で**、そのとき雨が降っていた確率」

は、上の表から $6/7$ なので、以下が成り立ちます。

$$p(Y = 1|X = 1) = \frac{6}{7}$$

これらの2つの確率が、それぞれ $p(Y = 0, X = 1) = 1/16$, $p(Y = 1, X = 1) = 6/16$ という同時確率とは異なっていることを、上の同時分布表と見比べて確認してみましょう。

「客が傘を持っていてかつ外で雨が降っている」という同時確率は $6/16$ となっていますが、「客が傘を持っているという条件の下で外で雨が降っている」という条件付き確率は $6/7$ でした。

前者は「客が傘を持っておらず外で雨が降っていた」「客が傘を持っておらず外で雨が降っていなかった」「客が傘を持っていたが外で雨が降っていなかった」を含むあり得る全ての事象を考えた上で、その中で対象としている事象「客が傘を持っていて外で雨が降っている」が起こる確率を意味しています。一方で、後者の条件付き確率は、今条件となっている事象「客が傘を持っている」が成り立っている事象だけをまず対象とし、その中で着目している事象「外で雨が降っている」が起こる確率を考えます。それぞれの確率の値を計算する際の分母が異なっていることに注目してください。

また、上記の2つの条件付き確率を足すと、1になります。「ある条件下で」という制約された世界で、対象とする確率変数がとり得る値全ての確率を並べて一覧したものは**条件付き分布 (conditional distribution)**と呼ばれ、それらの確率を全て足すと必ず1になります。上の例では、以下の計算で確かめられます。

$$p(Y = 0|X = 1) + p(Y = 1|X = 1) = \frac{1}{7} + \frac{6}{7} = 1$$

ここで、確率変数 X がとりうる実際の値のうちいずれかを表す文字として x を、確率変数 Y がとりうる実際の値のうちいずれかを表す文字として y を用いると、一般的に、条件付き確率は以下のように定義されます。

$$p(Y = y|X = x) = \frac{p(X = x, Y = y)}{p(X = x)}$$

このとき、条件付き分布が満たす条件は以下のように表せます。

$$\sum_y p(Y = y|X = x) = 1$$

ここで、 $\sum_x P(Y = y|X = x)$ は必ずしも1にならないということに注意してください。

では、上の条件付き確率の定義を用いて、同時確率と周辺確率から条件付き確率を、再度計算してみましょう。

$$p(Y = 0|X = 1) = \frac{p(X = 1, Y = 0)}{p(X = 1)} \quad (8.3)$$

$$= \frac{\frac{1}{16}}{\frac{7}{16}} \quad (8.4)$$

$$= \frac{1}{7} \quad (8.5)$$

確かに、上で別の方法で求めた条件付き確率と一致しました。

さて、上で触れた、同時確率と条件付き確率の関係について再度考えてみましょう。前述の「条件付き確率の定義」の式を変形すると、以下のようになります。以降では、簡単のために、確率変数の記述を省き、 $p(X = x)$ を $p(x)$ と表記します。確率変数 X によって x という値に対応付けられている事象が生じる確率、という意味です。同様に、 $p(Y = y)$ を $p(y)$ と表記します。

$$p(x, y) = p(y|x)p(x)$$

同時確率は、条件付き確率に、その条件となっている事象が生じる確率（周辺確率）を掛け合わせることで得られることが分かります。

今回の例では、「雨が降っているかどうか」によって「客が傘を持っていたかどうか」は影響を受けるため、2つの確率変数 Y, X は**従属 (dependent)**、と言います。一方、例えば「客が来たとき雨が降っていたかどうか」と「私が今お腹が空いているかどうか」が全く関係していません。後者について、お腹が空いているという事象を 1 に、お腹が空いていないという事象を 0 に対応付ける確率変数 Z を導入すると、 Y と Z が今**独立 (independent)** であるという状況です。このとき、 Z の実現値を z として、条件付き確率 $p(y|z)$ は、 Y がどうなるかが Z がどうなるかに全く関係がないことから、 $p(y)$ と等しくなります。条件付けたとしても、条件付けなかったとしても、結果が変わらないためです。これを用いると、 y と z の同時確率は以下のように変形できます。

$$p(y, z) = p(y|z)p(z) \quad (8.6)$$

$$p(y, z) = p(y)p(z) \quad (8.7)$$

Y と Z が独立であることから、 $p(y|z) = p(y)$ となることを用いました。この式から、**2つの確率変数が独立であるとき、同時確率はそれぞれの周辺確率の積で書ける**ということが言えます。

8.5 ベイズの定理

前節で説明した条件付き確率は、ある前提が成り立っている状況下で着目する事象が生じる確率を記述するためのものでした。これは、「ある”原因”が生じたという条件の下で、ある”結果”が生じる確率」を考えていると捉えることができます。では、逆に「ある”結果”が観測されたというとき、ある事象が”原因”である確率」を考えるにはどうしたら良いでしょうか。今、原因となり得る事象が確率変数 X で表され、結果となり得る事象が確率変数 Y で表されるとします。このとき、原因として実際に x が観測されたもとで結果が y となる確率が条件付き確率

$$p(Y = y|X = x)$$

で表されました。前節の定義から、これに原因 x がそもそも発生する確率 $p(X = x)$ を掛け合わせると、同時確率 $p(Y = y, X = x)$ になります。すなわち、

$$p(Y = y|X = x)p(X = x) = p(Y = y, X = x) \quad \dots (1)$$

です。では、結果が y であったという観測の下で、原因が x である確率はどのように表されるでしょうか。これは上の「条件の事象」と「着目する事象」を入れ替えた条件付き確率として、

$$p(X = x|Y = y)$$

と書くことができます。 y という結果が観測されたもとで、原因が x であった確率という意味です。これに結果 y がそもそも発生する確率 $p(Y = y)$ をかけ合わせると、条件付き確率の定義から

$$p(X = x|Y = y)p(Y = y) = p(X = x, Y = y) \quad \dots (2)$$

が導かれます。ここで、式 (1) と式 (2) の右辺が同じになっていることに着目します。すると、式 (1) と式 (2) の左辺同士を等号で結ぶことができ、

$$p(Y = y|X = x)p(X = x) = p(X = x|Y = y)p(Y = y)$$

が成り立つことが分かります。この式を変形して、求めたい「結果 y が生じたという条件の下で原因が x である確率」を左辺に残すと、

$$p(X = x|Y = y) = \frac{p(Y = y|X = x)p(X = x)}{p(Y = y)}$$

が導かれます。これを**ベイズの定理 (Bayes' theorem)**と言います。これによって、ある結果が観測されたときに、ある事象が原因である確率を、

- 原因として考えている事象が（結果と関係なく）そもそも生じる確率（ $p(X = x)$ ）
- 結果として観測した事象が（原因に関係なく）そもそも生じる確率（ $p(Y = y)$ ）
- 考えている原因が実際に生じたという下で観測された結果が生じる確率（ $p(Y = y|X = x)$ ）

の3つから、求めることができます。ベイズの定理に登場する左辺の条件付き確率と右辺の分子にある周辺確率には、特別な呼び方があります。まず右辺の分子にある周辺確率（上式では $p(X = x)$ ）は、「結果」が観測する前（事前）に、原因 x がそもそも生じる確率を表すので、**事前確率 (prior probability)**と呼ばれます。一方、左辺の条件付き確率（上式では $p(X = x|Y = y)$ ）は、ある「結果 y 」が観測されたという条件の下で（事後に）、原因 x が生じていたという確率を表すので、**事後確率 (posterior probability)**と呼ばれます。

ベイズの定理の応用事例として、スパムメールフィルターがあります。まず、 N 個の着目する単語 $w_i (i = 1, \dots, N)$ を決めます（"sale", "buy", "free", ... など）。そして、メールにある単語 w_i が含まれるとき 1 となり、含まれないとき 0 となる確率変数を W_i とし、「メールにある単語 w_i が含まれる確率」を $p(W_i = 1)$ とします。次に、メールがスパムメールであるとき 1 となり、スパムメールでないとき 0 となる確率変数を Y とし、「スパムメールの存在確率」を $p(Y = 1)$ とします。このとき、「あるメールがスパムメールであったという状況の下で単語 w_i が含まれていた」という確率は $p(W_i = 1|Y = 1)$ となります。すると、例えば w_i と w_j が両方含まれていた場合は、同時確率 $p(W_i = 1, W_j = 1|Y = 1)$ を考えることになりますが、各単語が独立に現れると仮定すると、これは $p(W_i = 1|Y = 1)p(W_j = 1|Y = 1)$ と分解できます。ここで、世の中にある大量のメールを集めて、各単語がどのくらいの確率で現れるかを調べると、 $p(W_i = 1) (i = 1, \dots, N)$ が求まります。また、集めたメールからスパムメールだけを抜き出します。すると、メールがスパムメールである確率 $p(Y = 1)$ が求まります。さらに、それらのスパムメール中に含まれる単語を調べ、どの単語がどういう確率で出現しているかを、着目している全単語について求めると、 $p(W_i = 1|Y = 1) (i = 1, \dots, N)$ が全て求まります。これらを用いると、「あるメールに単語 $w_i, w_j, w_k, \dots$ が含まれているとき、そのメールがスパムメールである確率」を以下のように計算できます。

$$p(Y = 1|W_i = 1, W_j = 1, W_k = 1, \dots) = \frac{p(W_i = 1, W_j = 1, W_k = 1, \dots|Y = 1)p(Y = 1)}{p(W_i = 1, W_j = 1, W_k = 1, \dots)}$$

8.6 尤度と最尤推定

これまでの節では、「**確率変数がとりうる様々な値に確率を対応させる**」確率分布は、表の形で表されていました。ここで、確率変数 X が x という値をとる確率 $p(X = x)$ を、表の代わりに、あるパラメータ θ で特徴づけられた関数 $f(x; \theta)$ によって表してみます。この関数は、「**確率変数 X がとりうる様々な値 x に確率を対応させる**」ので、 X の確率分布を表しています。このような関数は**確率モデル (probabilistic model)**とも呼ばれ、特にパラメータによって形状が決定される関数を用いる場合は**パラメトリックモデル (parametric model)**と呼ばれます。

このような確率モデルのパラメータをうまく決定することで、データの分布を表現できれば、未知のデータに対してもそれがどのくらいの確率で発生するのかといった予測が可能となったり、便利です。推定方法には

様々なものがあり、どうやって推定するかは推定する人の自由ですが、本節では最も一般的な方法の一つである最尤推定を説明します。最尤推定は確率論から正当化できるという長所があります。

さて、確率モデルがどのくらい実際の観測データに即しているかを**尤度 (ゆうど) (likelihood)** と言い、観測されたデータをモデルに入力し、その出力を掛け合わせたもので定義されます。

例えば、 N 個のデータ x_i ($i = 1, \dots, N$) が独立に観測されたとき、この観測データの下での確率モデル $f(x; \theta)$ の**尤度**は

$$L(\theta) = \prod_{i=1}^N f(x_i; \theta)$$

と計算されます。与えられたデータから、「どのようなデータがどのくらいよく観測されるか」を表す確率分布を推定 (注釈 5) するために、予め用意した確率モデルのパラメータを、それらの観測データの下での**尤度が最大**になるように決定する方法を、**最尤 (さいゆう) 推定 (maximum likelihood estimation)** と言います。

上式の $\prod$ という記号は、 $\sum$ の掛け算版で、全ての値を掛け合わせるという意味です。複数データに対する尤度は、1 より小さな値の積となるため、結果は非常に小さな数になり、コンピュータでこの計算を行う際にはアンダーフローという問題が発生しやすくなります。また尤度を最大化したい場合、積の形の式の最大化は難しいことが知られています。そこで尤度の対数をとった**対数尤度 (log likelihood)** を考えます。 $y = \log(x)$ は単調増加であるため、対数をとる前後で大小関係は変わりません。そのため対数尤度を最大にするときの θ は、元の尤度も最大にします。

$$\log L(\theta) = \sum_{i=1}^N \log f(x_i; \theta)$$

よって、この対数尤度を最大化するパラメータ θ を求めることができれば、その値が観測データ x_i ($i = 1, \dots, N$) の分布を最もよく表現する確率モデルのパラメータとなります。

8.6.1 例：コイントスのパラメータ推定

ここで、具体例として、コインの表・裏が出る確率を推定する問題を考えてみます。コインを投げて「表が出る」事象を 1 に、「裏が出る」事象を 0 に対応付ける確率変数を X とします。このとき、 $p(X)$ をシンプルな確率モデル

$$f(x; \theta) = \theta^x (1 - \theta)^{1-x}$$

で表現することにします (注釈 6)。これは $x = 1$ のとき、すなわちコインが表であるとき θ をそのまま返し、 $x = 0$ のとき、すなわちコインが裏であるとき $1 - \theta$ を返すような関数となっています。ただし、 $0 \leq \theta \leq 1$ を満たすこととします。今、コインを 10 回投げ、以下の観測結果が得られたとします。

	1 回目	2 回目	3 回目	4 回目	5 回目	6 回目	7 回目	8 回目	9 回目	10 回目
X	1	0	1	1	1	0	0	1	0	0

これらの観測データの下での確率モデルの尤度は

$$\begin{aligned} L(\theta) &= \theta \cdot (1 - \theta) \cdot \theta \cdot \theta \cdot \theta \cdot (1 - \theta) \cdot (1 - \theta) \cdot \theta \cdot (1 - \theta) \cdot (1 - \theta) \\ &= \theta^5 \cdot (1 - \theta)^5 \end{aligned}$$

です。対数尤度は、

$$\log L(\theta) = 5 \log \theta + 5 \log (1 - \theta)$$

となります。これが最大となるときの θ は、この式を θ で微分した式を 0 とおいて解析的に解けばよく、

$$\frac{5}{\theta} - \frac{5}{(1-\theta)} = 0 \quad (8.8)$$

$$\therefore \theta = \frac{1}{2} \quad (8.9)$$

と決定します。これが最尤推定です。

8.7 事後確率最大化推定 (MAP 推定)

前節で解説した最尤推定は、多くの場合で有用ですが、求めるパラメータに何らかの**事前知識**があっても、それを活かすことができません。そのため、サンプルサイズが小さい場合に、実際にはほとんどあり得ない結果が求まることがあります。具体例を見てみましょう。

再び、コインの表・裏が出る確率を推定する例を考えます。コインを 5 回投げたところ、**たまたま 5 回とも表が出た**とします。この場合、前節と同じ確率モデルを用いて尤度を計算し、最尤推定でモデルのパラメータを決定すると、 $0 \leq \theta \leq 1$ を満たしながら、今回の観測データの下での尤度 θ^5 を最大にする θ は 1 となります。これは「表が出る確率が 1」すなわち「**裏面が出る確率は 0**」という推定結果が得られてしまったことを意味します。もし「裏が出る確率も 0 よりは大きいだろう」という事前知識を持っておりそれを活用できれば、今回のような場合でもより良い推定ができそうです。

事前知識も考慮しながら、観測データに基づいて確率モデルのパラメータを推定する方法には、**事後確率最大化 (maximum a posteriori, 以降 MAP) 推定**があります。MAP 推定でも、対象のデータの確率分布を表す確率モデルのパラメータを、データを利用して決定しますが、データだけでなく**パラメータも確率変数だと考える**点が最尤推定とは異なっています。パラメータを確率変数 Θ とすると、このパラメータがどのような値をとるかを表す分布 $p(\Theta)$ が導入できます。これを**事前分布 (prior distribution)**と呼び、ここに「求めたいパラメータに対する事前知識」を反映させることができます。ここで、 $\Theta = \theta$ となる確率 $p(\Theta = \theta)$ (以降は $p(\theta)$ と略記) をデータの確率モデル $f(x; \theta)$ とは別の確率モデル $g(\theta; \beta)$ で表せば、 β を人為的に与えるか、なんらかの方法で決定しておくことで、「どのような θ が起こりやすいか」という事前知識を表現することができます。

MAP 推定では、観測データ x_i ($i = 1, \dots, N$) が観測されたという**条件の下で**最も確率が大きくなる θ を求めます。すなわち、 $p(\theta|x_1, \dots, x_N)$ を最大とする θ を求めます。これは「 x_i ($i = 1, \dots, N$) が観測された」という**結果**のもとで「データの確率分布を表すモデルのパラメータが θ であったことが**原因**である」確率を意味するので、**事後確率**です。ベイズの定理から、事後確率は

$$p(\theta|x_1, \dots, x_N) = \frac{p(x_1, \dots, x_N|\theta)p(\theta)}{p(x_1, \dots, x_N)}$$

と書けました。右辺に着目してください。分子にある $p(x_1, \dots, x_N|\theta)$ を、確率モデル $f(x; \theta)$ を用いて $\prod_{i=1}^N f(x_i; \theta)$ と表すと、これは観測データ x_i ($i = 1, \dots, N$) の下での確率モデル $f(x; \theta)$ の**尤度**です。また、その右の $p(\theta)$ は、パラメータが θ という値になる**事前確率**でした。これが $g(\theta; \beta)$ という確率モデルで表されるとします。

今、事後確率を最大にするパラメータ θ を求めるために、右辺を最大化する θ を考えます。このとき、パラメータと無関係に決まる $p(x_1, \dots, x_N)$ を無視し

$$p(x_1, \dots, x_N|\theta)p(\theta)$$

だけに注目してこれを最大化するようにパラメータ θ を決定すると、その θ は求めたい左辺の事後確率を

最大にする θ に一致します。上式は、尤度と事前分布の積になっています。このことから、**事後分布は尤度と事前分布の積に比例する**ということが分かります。

8.7.1 例：コイントスでのパラメータ推定

前節の最尤推定で用いたコイントスの例を再び考えます。

今度は MAP 推定を用いてパラメータを推定してみましょう。用いるデータは先程と同様の 10 回のコイントス結果です。

また、確率モデル $f(x; \theta)$ も同様に最尤推定の場合と同様のものを用います。

MAP 推定では確率モデル $f(x; \theta)$ の他に、事前分布 $p(\theta)$ を設計する必要があります。

$p(\theta)$ の決め方は推定を行う人がどのような事前知識（もしくは信念）を持っているかによって自由に選択することができますが、ここでは事前分布を

$$p(\theta) = 12\theta(1 - \theta)^2$$

と設定します（[注釈 7](#)）。グラフを書くと $p(\theta)$ は $\theta = 1/3$ で最大値をとることがわかります。これは「コイントスが行われる前には、実験者は $\theta = 1/3$ が最もあり得る可能性である」すなわち、「このコインは表よりも裏の方が出やすい」と信じている（事前知識を持っている）と解釈することができます。

事後分布は θ の関数として $p(x_1, \dots, x_N | \theta)p(\theta)$ に比例するのです。

これを具体的に計算すると

$$p(x_1, \dots, x_N | \theta)p(\theta) = \theta^5(1 - \theta)^5 \times 12\theta(1 - \theta)^2 = 12\theta^6(1 - \theta)^7$$

となります。これを θ の関数として最大化します。

最尤推定の時と同様に、この値そのものではなく、対数を取った値を最大とする θ を求めます。

$$\log p(x_1, \dots, x_N | \theta)p(\theta) = 6 \log \theta + 7 \log(1 - \theta) + \text{const.}$$

ここで、 const. は θ に依存しない定数（具体的には $\log 12$ ）で、値を最大にする θ を決定するのに影響を及ぼしません。

これを θ で微分したものが 0 に等しいという方程式を立てると、

$$\frac{6}{\theta} - \frac{7}{1 - \theta} = 0 \tag{8.10}$$

$$\therefore \theta = \frac{6}{13} \tag{8.11}$$

と解くことができます。これが MAP 推定で得られた推定値です。

最尤推定で得られた推定値は $\theta = 1/2$ だったので、MAP 推定は最尤推定よりも低めに θ を推定していることがわかります。

これは事前分布 $p(\theta)$ により θ は $1/2$ より小さい値が出やすいという事前知識を入れていたことを反映しています。逆に事前分布で大きい θ が出やすいという事前知識を入れれば、MAP 推定の値を最尤推定よりも大きくすることができます。

この例からも分かる通り、MAP 推定は事前分布の設定により得られる推定値が変化します。

この事は、MAP 推定の方が事前知識により柔軟なモデリングができる半面、誤った事前知識に基づいて事前分布を設計すると、推定値が実際のデータと乖離する可能性を持つことを意味します。

8.8 統計量

本節では、いくつかよく用いられる統計量を説明します。統計量とは、**観測されたデータの特徴を要約する数値**のことを指します。代表的な統計量として平均、分散、標準偏差を紹介します。

8.8.1 平均

平均 (mean) は、観測された数値を合計し、その数値の個数で割ったもののことを言います。たとえば、300 円、400 円、500 円の平均は、

$$\frac{300 + 400 + 500}{3} = 400$$

です。一般に、 N 個のデータ x_i ($i = 1, \dots, N$) が観測されたとき、その平均は

$$\bar{x} = \frac{x_1 + x_2 + \dots + x_N}{N} = \frac{1}{N} \sum_{n=1}^N x_n$$

と定義されます。

平均を表す記号として、 $\bar{x}$ や μ がよく用いられます。

データの分布において、平均はその重心に相当する値です。

8.8.2 分散

次に、**分散 (variance)** を紹介します。分散はよく σ^2 と表され、

$$\sigma^2 = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})^2$$

と定義されます。各データ点 x_i ($i = 1, \dots, N$) の値から、それらの平均 $\bar{x}$ を引き二乗した値 ($(x_i - \bar{x})^2$ ($i = 1, \dots, N$)) の平均を計算しています。これは、「データが自らの平均より平均的にどのくらいばらついているか」を表します。平均が同じデータでも、全てのデータが平均付近にある場合は分散は小さく、平均よりも極めて大きなデータ点や小さなデータ点が多数ある場合、分散は大きくなります。分散にはもう一種類あります。以下のようなものです。

$$s^2 = \frac{1}{N-1} \sum_{n=1}^N (x_n - \bar{x})^2$$

始めに登場した σ^2 の定義では $\frac{1}{N}$ となっていた部分が $\frac{1}{N-1}$ に変わっています。

前者は**標本分散 (sample variance)** といい、後者は**不偏分散 (unbiased variance)** といいます。

これらの式の導出は他の文献に譲るとして、ここではその使い分けについて説明します。

例えば、全国の小学生の身長と体重の分散を調べたいとします。このとき、全国の小学生を一人の抜け漏れもなく調べたなら、集まったデータは**母集団 (population)** と呼ばれます。

一方、各都道府県の小学生を 100 人ずつ調べた場合、そのデータは**標本集団 (sample population)** と言います。

すなわち、母集団とは解析を行いたいデータ全ての集合を指し、標本集団とは母集団から抽出された一部のデータの集合を指します。一般に、標本集団のデータ数が少ないとき、標本分散は母集団の分散よりも小さくなることが知られています。不偏分散は、その差を補正することで、母集団の分散をより正確に推定するため

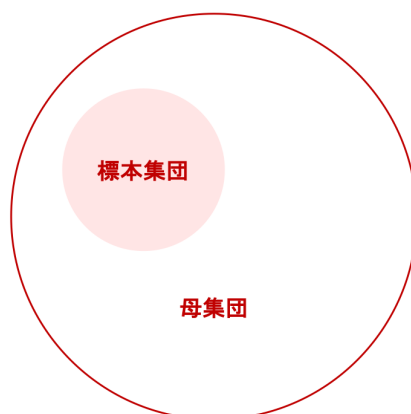


図 8.1 母集団と標本集団

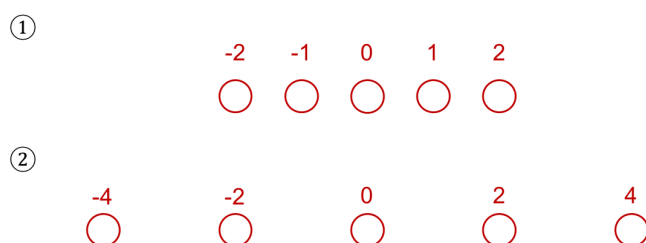


図 8.2 練習問題

に用います。不偏分散は抽出するデータの数 (N) を増やせば、いずれ母集団の分散に一致します。また、 N が大きいと標本分散と不偏分散はほぼ一致するので、たくさんのデータ数をとることができる状況では、両者に大きな違いが生じないことも多々あります。

分散を利用すると、データのばらつきを定量評価することができるようになります。例えば、同じ現象を複数回観測する実験を行った際、結果のばらつきが大きければ、その観測方法には問題がある可能性があります。もしくは、同じだと思っていた現象は、実は観測のたびに異なる現象であった可能性があります。このように、多数の試行の結果がある値に集まっていることが望ましいような状況において、ばらつきの度合いを定量し評価することは重要です。

8.8.3 標準偏差

次に**標準偏差 (standard deviation)** を紹介します。

分散はデータの平均からの差の**二乗**の平均でした。そのため単位は元の単位を二乗したものになります。例えばデータの単位が kg であれば、分散の単位は kg^2 になります。そこで、分散 σ^2 の平方根 σ を計算することで、データと単位が等しくなり、解釈が容易になります。この σ を標準偏差と呼びます。

練習問題で具体的な計算手順の確認を行きましょう。以下の①と②のデータに対して、平均、分散、標準偏差を求めてください。ただし、今回は標本分散を使用することとします。

①の解答は以下の通りです。

$$\begin{aligned}
 \bar{x} &= \frac{1}{5}(-2 - 1 + 0 + 1 + 2) = 0 \\
 \sigma^2 &= \frac{1}{5}\{(-2 - 0)^2 + (-1 - 0)^2 + (0 - 0)^2 + (1 - 0)^2 + (2 - 0)^2\} \\
 &= \frac{1}{5} \times 10 = 2 \\
 \sigma &= \sqrt{2}
 \end{aligned}$$

②の解答は以下の通りです。

$$\begin{aligned}
 \bar{x} &= \frac{1}{5}(-4 - 2 + 0 + 2 + 4) = 0 \\
 \sigma^2 &= \frac{1}{5}\{(-4 - 0)^2 + (-2 - 0)^2 + (0 - 0)^2 + (2 - 0)^2 + (4 - 0)^2\} \\
 &= \frac{1}{5} \times 40 = 8 \\
 \sigma &= \sqrt{8} = 2\sqrt{2}
 \end{aligned}$$

これより、②のケースの方が分散が大きく、データのばらつきが大きいことがわかります。

8.8.4 相関係数

最後に**相関係数 (correlation coefficient)**を紹介します。

2種類のデータが得られている状況では、両者の関係性の解析がしばしば重要となります。

相関係数は両者の関連の度合いを定量的に測るのに用います。

2種類のデータ点がスカラー値で N 個ずつある状況を考えます。それぞれを x_n, y_n ($n = 1, \dots, N$) とした場合、相関係数の中でも良く用いられるピアソンの相関係数は以下のように定義されます。

$$r = \frac{\sum_{n=1}^N (x_n - \bar{x})(y_n - \bar{y})}{\sqrt{\left(\sum_{n=1}^N (x_n - \bar{x})^2\right) \left(\sum_{n=1}^N (y_n - \bar{y})^2\right)}}$$

上式のように相関係数を表す記号は r を用いるのが一般的です。

相関係数 r は常に $-1 \leq r \leq 1$ になり、相関が認められるとき、 r の値が正の場合は**正の相関**があるといい、逆に負の値のときは**負の相関**があるといいます。

2種類のデータ間の相関が強いほど r の絶対値は大きくなります。

しかし、「 r がいくつ以上ならば相関があると思って問題ない」という閾値はタスクごとに異なり、例えば $r = 0.2$ から相関ありと判断してよいかは一概には言えません。本資料では説明を省略しますが、無相関検定などの手法を利用することで客観的な判断を行うことができます。

8.9 注釈

8.9.1 注釈 1

ここでは概念の説明を簡単にするため、この例のように離散的な値をとる確率変数を考え、特に明示しない限り連続値の確率変数は考えないことにします。

8.9.2 注釈 2

x は 1, 2, 3, 4, 5, 6 のいずれか。すなわち $x \in \{1, 2, 3, 4, 5, 6\}$ です。

8.9.3 注釈 3

y は 2 つ目のコインがとりうる状態で、この場合、「表」と「裏」という値のいずれか。

8.9.4 注釈 4

x は 1 つ目のコインがとりうる状態で、この場合、「表」と「裏」という値のいずれか。

8.9.5 注釈 5

以降、このことを「データの分布を推定する」と言うことがあります。また、観測されたデータのみから各データの発生確率（頻度とも捉えられる）を求めたものは**経験分布（empirical distribution）**とも呼ばれ、本節で説明しているのは正確にはこの経験分布を確率モデルで近似する方法です。

8.9.6 注釈 6

この関数には、ベルヌーイ分布という名前がついています。

8.9.7 注釈 7

先頭の 12 は $p(\theta)$ 0 から 1 まで積分した値が 1 となるように決めています。これにより、 $p(\theta)$ が $0 \leq \theta \leq 1$ の上の確率分布となります。

第 9 章

Pandas 入門

Pandas はデータ操作によく用いられるパッケージであり、CSV などの一般的なデータ形式で保存されたデータの読み込みや、条件を指定しての一部データの抽出など、機械学習手法で取り扱うデータを整理するのに便利です。

今回は Pandas の以下の代表的な機能の使い方を説明します。

- CSV ファイルの読み書き
- 統計量の算出
- 並べ替え
- データの選択
- 条件指定による選択
- 欠損値の除去 / 補間
- ndarray とデータフレームを相互に変換
- グラフの描画

まず Pandas パッケージを読み込みましょう。

pandas は pd という別名を与えて用いるのが一般的です。

```
1 import pandas as pd
```

9.1 CSV ファイルの読み書き

データセットは Google Colaboratory で用意されているサンプルデータを使用します。

Google Colaboratory 以外で実行する場合は、[こちら](#)をダウンロードして、使用してください。

Pandas では CSV ファイルを読み込むための `pd.read_csv()` という関数が用意されています。

こちらを使って CSV ファイルを読み込みます。

```
1 # データセットの読み込み
2 df = pd.read_csv('sample_data/california_housing_train.csv')
```

`df` という変数名は、**データフレーム (data frame)** という Pandas で中心的に用いられる**データ構造 (data structure)**を表すクラスの名前の頭文字に由来しています。

`pd.read_csv()` 関数は、CSV ファイルの内容を `DataFrame` オブジェクトに読み込みます。

```
1 # 型の確認
2 type(df)
```

```
pandas.core.frame.DataFrame
```

9.2 DataFrame の表示

df に読み込まれたデータの中身の確認してみましょう。

Jupyter Notebook 上では、大きな DataFrame を表示しようとすると自動的に一部が省略されることがあります。

```
1 df
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
0	-114.31	34.19	15.0	5612.0	1283.0
1	-114.47	34.40	19.0	7650.0	1901.0
2	-114.56	33.69	17.0	720.0	174.0
3	-114.57	33.64	14.0	1501.0	337.0
4	-114.57	33.57	20.0	1454.0	326.0
5	-114.58	33.63	29.0	1387.0	236.0
6	-114.58	33.61	25.0	2907.0	680.0
7	-114.59	34.83	41.0	812.0	168.0
8	-114.59	33.61	34.0	4789.0	1175.0
9	-114.60	34.83	46.0	1497.0	309.0
10	-114.60	33.62	16.0	3741.0	801.0
11	-114.60	33.60	21.0	1988.0	483.0
12	-114.61	34.84	48.0	1291.0	248.0
13	-114.61	34.83	31.0	2478.0	464.0
14	-114.63	32.76	15.0	1448.0	378.0
15	-114.65	34.89	17.0	2556.0	587.0
16	-114.65	33.60	28.0	1678.0	322.0
17	-114.65	32.79	21.0	44.0	33.0
18	-114.66	32.74	17.0	1388.0	386.0
19	-114.67	33.92	17.0	97.0	24.0
20	-114.68	33.49	20.0	1491.0	360.0
21	-114.73	33.43	24.0	796.0	243.0
22	-114.94	34.55	20.0	350.0	95.0
23	-114.98	33.82	15.0	644.0	129.0
24	-115.22	33.54	18.0	1706.0	397.0
25	-115.32	32.82	34.0	591.0	139.0
26	-115.37	32.82	30.0	1602.0	322.0
27	-115.37	32.82	14.0	1276.0	270.0
28	-115.37	32.81	32.0	741.0	191.0
29	-115.37	32.81	23.0	1458.0	294.0
...	...	...	...	...	...
16970	-124.17	40.80	52.0	1606.0	419.0
16971	-124.17	40.80	52.0	1557.0	344.0
16972	-124.17	40.79	43.0	2285.0	479.0
16973	-124.17	40.78	39.0	1606.0	330.0
16974	-124.17	40.77	30.0	1895.0	366.0
16975	-124.17	40.76	26.0	1776.0	361.0
16976	-124.17	40.75	13.0	2171.0	339.0
16977	-124.17	40.62	32.0	1595.0	309.0
16978	-124.18	40.79	39.0	1836.0	352.0
16979	-124.18	40.78	37.0	1453.0	293.0
16980	-124.18	40.78	34.0	1592.0	364.0

16981	-124.18	40.78	33.0	1076.0	222.0
16982	-124.18	40.62	35.0	952.0	178.0
16983	-124.19	41.78	15.0	3140.0	714.0
16984	-124.19	40.78	37.0	1371.0	319.0
16985	-124.19	40.77	30.0	2975.0	634.0
16986	-124.19	40.73	21.0	5694.0	1056.0
16987	-124.21	41.77	17.0	3461.0	722.0
16988	-124.21	41.75	20.0	3810.0	787.0
16989	-124.21	40.75	32.0	1218.0	331.0
16990	-124.22	41.73	28.0	3003.0	699.0
16991	-124.23	41.75	11.0	3159.0	616.0
16992	-124.23	40.81	52.0	1112.0	209.0
16993	-124.23	40.54	52.0	2694.0	453.0
16994	-124.25	40.28	32.0	1430.0	419.0
16995	-124.26	40.58	52.0	2217.0	394.0
16996	-124.27	40.69	36.0	2349.0	528.0
16997	-124.30	41.84	17.0	2677.0	531.0
16998	-124.30	41.80	19.0	2672.0	552.0
16999	-124.35	40.54	52.0	1820.0	300.0

	population	households	median_income	median_house_value
0	1015.0	472.0	1.4936	66900.0
1	1129.0	463.0	1.8200	80100.0
2	333.0	117.0	1.6509	85700.0
3	515.0	226.0	3.1917	73400.0
4	624.0	262.0	1.9250	65500.0
5	671.0	239.0	3.3438	74000.0
6	1841.0	633.0	2.6768	82400.0
7	375.0	158.0	1.7083	48500.0
8	3134.0	1056.0	2.1782	58400.0
9	787.0	271.0	2.1908	48100.0
10	2434.0	824.0	2.6797	86500.0
11	1182.0	437.0	1.6250	62000.0
12	580.0	211.0	2.1571	48600.0
13	1346.0	479.0	3.2120	70400.0
14	949.0	300.0	0.8585	45000.0
15	1005.0	401.0	1.6991	69100.0
16	666.0	256.0	2.9653	94900.0
17	64.0	27.0	0.8571	25000.0
18	775.0	320.0	1.2049	44000.0
19	29.0	15.0	1.2656	27500.0
20	1135.0	303.0	1.6395	44400.0
21	227.0	139.0	0.8964	59200.0
22	119.0	58.0	1.6250	50000.0
23	137.0	52.0	3.2097	71300.0
24	3424.0	283.0	1.6250	53500.0
25	327.0	89.0	3.6528	100000.0
26	1130.0	335.0	3.5735	71100.0
27	867.0	261.0	1.9375	80900.0
28	623.0	169.0	1.7604	68600.0
29	866.0	275.0	2.3594	74300.0
...	...	...	...	...
16970	891.0	367.0	1.5850	75500.0
16971	758.0	319.0	1.8529	62500.0

16972	1169.0	482.0	1.9688	70500.0
16973	731.0	327.0	1.6369	68300.0
16974	990.0	359.0	2.2227	81300.0
16975	992.0	380.0	2.8056	82800.0
16976	951.0	353.0	4.8516	116100.0
16977	706.0	277.0	2.8958	86400.0
16978	883.0	337.0	1.7450	70500.0
16979	867.0	310.0	2.5536	70200.0
16980	950.0	317.0	2.1607	67000.0
16981	656.0	236.0	2.5096	72200.0
16982	480.0	179.0	3.0536	107000.0
16983	1645.0	640.0	1.6654	74600.0
16984	640.0	260.0	1.8242	70000.0
16985	1367.0	583.0	2.4420	69000.0
16986	2907.0	972.0	3.5363	90100.0
16987	1947.0	647.0	2.5795	68400.0
16988	1993.0	721.0	2.0074	66900.0
16989	620.0	268.0	1.6528	58100.0
16990	1530.0	653.0	1.7038	78300.0
16991	1343.0	479.0	2.4805	73200.0
16992	544.0	172.0	3.3462	50800.0
16993	1152.0	435.0	3.0806	106700.0
16994	434.0	187.0	1.9417	76100.0
16995	907.0	369.0	2.3571	111400.0
16996	1194.0	465.0	2.5179	79000.0
16997	1244.0	456.0	3.0313	103600.0
16998	1298.0	478.0	1.9797	85800.0
16999	806.0	270.0	3.0147	94600.0

[17000 rows x 9 columns]

9.3 先頭の数件だけを表示

データを数件のみ確認したい場合は、データフレームがもつ `df.head()` メソッドを使用します。

`df.head()` はデフォルトで先頭から 5 件のデータを表示しますが、`df.head(3)` のように引数に表示したいデータ件数を指定すると、指定された件数だけを表示することもできます。

それでは、`df.head()` を実行してみましょう。

```
1 df.head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-114.31	34.19	15.0	5612.0	1283.0	
1	-114.47	34.40	19.0	7650.0	1901.0	
2	-114.56	33.69	17.0	720.0	174.0	
3	-114.57	33.64	14.0	1501.0	337.0	
4	-114.57	33.57	20.0	1454.0	326.0	
	population	households	median_income	median_house_value		
0	1015.0	472.0	1.4936	66900.0		
1	1129.0	463.0	1.8200	80100.0		
2	333.0	117.0	1.6509	85700.0		
3	515.0	226.0	3.1917	73400.0		

4	624.0	262.0	1.9250	65500.0
---	-------	-------	--------	---------

```
1 df.head(3)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-114.31	34.19	15.0	5612.0	1283.0	
1	-114.47	34.40	19.0	7650.0	1901.0	
2	-114.56	33.69	17.0	720.0	174.0	

	population	households	median_income	median_house_value
0	1015.0	472.0	1.4936	66900.0
1	1129.0	463.0	1.8200	80100.0
2	333.0	117.0	1.6509	85700.0

特定の列を抽出したい場合は、df に対し、Python の辞書オブジェクトに行うように [] を使って取り出したい列の名前を指定します。

```
1 df['longitude'].head(3)
```

```
0    -114.31
1    -114.47
2    -114.56
Name: longitude, dtype: float64
```

9.4 CSV ファイルの保存

Pandas ではデータフレームオブジェクトの内容を CSV ファイルとして保存するための df.to_csv() というメソッドが用意されています。

```
1 df.to_csv('sample.csv')
```

```
1 !ls sample.csv
```

```
sample.csv
```

9.5 データフレームの形

データフレームオブジェクトの行数と列数を確認するには、df.shape() メソッドを用います。

```
1 # 形の確認
2 df.shape
```

```
(17000, 9)
```

9.6 統計量の算出

データフレームには、中のデータに対し統計量を計算するためのメソッドも用意されています。代表的なものを紹介します。

```

1 # 平均
2 df.mean()

```

```

longitude      -119.562108
latitude        35.625225
housing_median_age    28.589353
total_rooms      2643.664412
total_bedrooms    539.410824
population      1429.573941
households       501.221941
median_income      3.883578
median_house_value 207300.912353
dtype: float64

```

```

1 # 分散
2 df.var()

```

```

longitude      4.020692e+00
latitude        4.568221e+00
housing_median_age    1.584310e+02
total_rooms      4.752169e+06
total_bedrooms    1.776618e+05
population      1.317566e+06
households       1.478563e+05
median_income      3.641061e+00
median_house_value  1.345223e+10
dtype: float64

```

```

1 # 各列の None, NaN, NaT のいずれでもない値の数
2 df.count()

```

```

longitude      17000
latitude        17000
housing_median_age    17000
total_rooms      17000
total_bedrooms    17000
population      17000
households       17000
median_income      17000
median_house_value  17000
dtype: int64

```

ここで、データの特徴をおおまかに調べるために便利な `df.describe()` メソッドを実行してみましょう。

```

1 # データの概要
2 df.describe()

```

	longitude	latitude	housing_median_age	total_rooms	\
count	17000.000000	17000.000000	17000.000000	17000.000000	
mean	-119.562108	35.625225	28.589353	2643.664412	
std	2.005166	2.137340	12.586937	2179.947071	
min	-124.350000	32.540000	1.000000	2.000000	

```

25%      -121.790000      33.930000      18.000000      1462.000000
50%      -118.490000      34.250000      29.000000      2127.000000
75%      -118.000000      37.720000      37.000000      3151.250000
max       -114.310000      41.950000      52.000000      37937.000000

      total_bedrooms      population      households      median_income  \
count      17000.000000      17000.000000      17000.000000      17000.000000
mean         539.410824      1429.573941      501.221941         3.883578
std          421.499452      1147.852959      384.520841         1.908157
min           1.000000         3.000000         1.000000         0.499900
25%          297.000000         790.000000      282.000000         2.566375
50%          434.000000      1167.000000      409.000000         3.544600
75%          648.250000      1721.000000      605.250000         4.767000
max          6445.000000      35682.000000      6082.000000        15.000100

      median_house_value
count      17000.000000
mean        207300.912353
std         115983.764387
min          14999.000000
25%          119400.000000
50%          180400.000000
75%          265000.000000
max           500001.000000

```

また、もうひとつ便利なメソッドに相関係数を算出する `df.corr()` があります。

入力変数間や入出力間の相関係数を確認することができます。

```

1 # 相関係数の算出
2 df.corr()

```

```

      longitude      latitude      housing_median_age      total_rooms  \
longitude         1.000000      -0.925208         -0.114250         0.047010
latitude         -0.925208         1.000000          0.016454        -0.038773
housing_median_age -0.114250         0.016454          1.000000        -0.360984
total_rooms         0.047010        -0.038773        -0.360984         1.000000
total_bedrooms         0.071802        -0.069373        -0.320434         0.928403
population          0.101674        -0.111261        -0.295890         0.860170
households           0.059628        -0.074902        -0.302754         0.919018
median_income        -0.015485        -0.080303        -0.115932         0.195383
median_house_value   -0.044982       -0.144917          0.106758         0.130991

      total_bedrooms      population      households      median_income  \
longitude          0.071802         0.101674         0.059628        -0.015485
latitude          -0.069373        -0.111261        -0.074902        -0.080303
housing_median_age -0.320434        -0.295890        -0.302754        -0.115932
total_rooms         0.928403         0.860170         0.919018         0.195383
total_bedrooms         1.000000         0.881169         0.980920        -0.013495
population          0.881169         1.000000         0.909247        -0.000638
households           0.980920         0.909247         1.000000         0.007644
median_income        -0.013495        -0.000638         0.007644         1.000000
median_house_value     0.045783        -0.027850         0.061031         0.691871

      median_house_value

```

longitude	-0.044982
latitude	-0.144917
housing_median_age	0.106758
total_rooms	0.130991
total_bedrooms	0.045783
population	-0.027850
households	0.061031
median_income	0.691871
median_house_value	1.000000

9.7 並べ替え

データフレームのある列を抽出し、`df.sort_values()` メソッドを呼び出すことで値の**並べ替え (sort)** を行うことができます。

なお、このメソッドは並べ替えが終わったあとの値でもとのデータフレーム内の値を置き換えることまでは行わず、結果を返します。

そこで、別の変数で結果を受け取り、始めの 5 行を表示することで並べ替えが行われたことを確認してみましょう。

`df.sort_values()` は、デフォルトでは**昇順 (ascending)** に並べ替えを行います。昇順とは、だんだん値が大きくなっていくように並べ替えるときの並べ方のことで、逆にだんだん値が小さくなっていくように並べ替えるときは、**降順 (descending)** に並べると言います。

`df.sort_values()` は並べ替えを行いたい列の名前を `by` という引数で受け取ります。また、デフォルトでは昇順に並べ替えを行います。

```
1 # total_rooms 列の値を昇順に並べ替え
2 df_as = df.sort_values(by='total_rooms')
```

```
1 df_as.head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
2990	-117.79	35.21	4.0	2.0	2.0
16309	-122.50	37.79	52.0	8.0	1.0
8188	-118.44	34.28	46.0	11.0	11.0
15369	-122.29	37.81	46.0	12.0	4.0
15656	-122.37	37.60	26.0	15.0	3.0

	population	households	median_income	median_house_value
2990	6.0	2.0	2.3750	137500.0
16309	13.0	1.0	15.0001	500001.0
8188	24.0	13.0	2.8750	162500.0
15369	18.0	7.0	0.4999	67500.0
15656	11.0	3.0	5.0480	350000.0

降順に並べ替える場合は、`ascending=False` という引数の指定を行います。

```
1 # total_rooms 列の値を降順に並べ替え
2 df_de = df.sort_values(by='total_rooms', ascending=False)
```

```
1 df_de.head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
2871	-117.74	33.89	4.0	37937.0	5471.0
12772	-121.79	36.64	11.0	32627.0	6445.0
2969	-117.78	34.03	8.0	32054.0	5290.0
8881	-118.78	34.16	9.0	30405.0	4093.0
1116	-117.12	33.52	4.0	30401.0	4957.0

	population	households	median_income	median_house_value
2871	16122.0	5189.0	7.4947	366300.0
12772	28566.0	6082.0	2.3087	118800.0
2969	15507.0	5050.0	6.0191	253900.0
8881	12873.0	3931.0	8.0137	399200.0
1116	13251.0	4339.0	4.5841	212300.0

9.8 データの選択

着目したい要素や、行、列を選択する方法を紹介します。

[scikit-learn 入門](#)の章で行ったように、入力値 x と目標値 t が別の列として同じ配列に格納されている場合は、指定した列だけを取り出す操作を行って、結果を別の変数に代入する操作を行います。

このように、特定の列の選択や列を範囲指定して選択する機能が Pandas のデータフレームにも用意されています。

今回は最後の列 `median_house_value` と、それ以外の列をそれぞれ取り出して、別々の変数に格納してみましょう。

列や行の選択を行う方法は複数あります。

ここでは、整数インデックスを用いてデータの部分選択を行う `df.iloc[]` を紹介します。

```
1 # データの確認
2 df.head(3)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-114.31	34.19	15.0	5612.0	1283.0	
1	-114.47	34.40	19.0	7650.0	1901.0	
2	-114.56	33.69	17.0	720.0	174.0	

	population	households	median_income	median_house_value
0	1015.0	472.0	1.4936	66900.0
1	1129.0	463.0	1.8200	80100.0
2	333.0	117.0	1.6509	85700.0

```
1 # df.iloc行[, 列]
2 # 0 行目 longitude 列の選択
3 df.iloc[0, 0]
```

```
-114.31
```

```
1 # 1 行目 latitude 列の選択
2 df.iloc[1, 1]
```

```
34.4
```

`iloc` には NumPy の `ndarray` の中の値を部分的に選択するのと同様のスライス表記を用いることができます。

負のインデックスを使い、末尾の要素からの個数を用いて位置指定を行うこともできます。

```
1 # すべての行の、最後の列を選択
2 t = df.iloc[:, -1]
```

```
1 # 先頭件の表示3
2 t.head(3)
```

```
0    66900.0
1    80100.0
2    85700.0
Name: median_house_value, dtype: float64
```

```
1 # 型の確認
2 type(t)
```

```
pandas.core.series.Series
```

1 行だけ、もしくは 1 列だけ抽出した場合は、**シリーズ (series)** オブジェクトが返されます。

```
1 # すべての行の、先頭の列から末尾の列のひとつ手前までを選択
2 x = df.iloc[:, 0:-1]
```

```
1 # 先頭の件の表示3
2 x.head(3)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-114.31	34.19	15.0	5612.0	1283.0	
1	-114.47	34.40	19.0	7650.0	1901.0	
2	-114.56	33.69	17.0	720.0	174.0	

	population	households	median_income
0	1015.0	472.0	1.4936
1	1129.0	463.0	1.8200
2	333.0	117.0	1.6509

[NumPy 入門](#)で紹介したようなスライス記法を用いる際の先頭位置の省略も行えます。

```
1 # すべての行の、先頭の列から末尾の列のひとつ手前までを選択
2 x = df.iloc[:, :-1]
```

```
1 # 先頭の件の表示3
2 x.head(3)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-114.31	34.19	15.0	5612.0	1283.0	
1	-114.47	34.40	19.0	7650.0	1901.0	
2	-114.56	33.69	17.0	720.0	174.0	

	population	households	median_income
0	1015.0	472.0	1.4936
1	1129.0	463.0	1.8200
2	333.0	117.0	1.6509

行の数、列の数両方を複数選択した場合、データフレームオブジェクトが返ります。

```
1 # 型の確認
2 type(x)
```

```
pandas.core.frame.DataFrame
```

9.9 条件指定による要素の選択

次に値に対する条件を指定してデータの選択を行う方法を紹介します。

簡単のため、まず `median_house_value` 列を選択し、返ってきたシリーズオブジェクトに対して、比較演算子を使って**各要素に対する条件**を指定し、条件を満たすかどうかを全要素に対して調べた結果を取得してみましょう。

```
1 # median_house_value 列を選択し、全要素に対し 70000 より大きいかどうかを計算
2 mask = df['median_house_value'] > 70000
```

```
1 mask.head()
```

```
0    False
1     True
2     True
3     True
4    False
Name: median_house_value, dtype: bool
```

このように、比較演算子の片方の辺にデータフレームやシリーズをおくと、指定された条件を満たすかどうかを全ての要素に対して計算することができます。

結果は、各要素が条件を満たすか、満たさないかを表す `True`、`False` が各要素の位置に格納されたデータフレームやシリーズとなります。

これを**マスク (mask)** と呼ぶことがあります。

そして、データフレームやシリーズも NumPy の `ndarray` と同様に、マスクを使って要素を選択することができます。

上の `mask` を `df` に `[]` を使って与えることで、指定した条件を満たす要素だけを取り出すことができます。

```
1 # df[mask] の先頭 5 件を表示
2 df[mask].head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
1	-114.47	34.40	19.0	7650.0	1901.0	
2	-114.56	33.69	17.0	720.0	174.0	
3	-114.57	33.64	14.0	1501.0	337.0	
5	-114.58	33.63	29.0	1387.0	236.0	
6	-114.58	33.61	25.0	2907.0	680.0	

	population	households	median_income	median_house_value
1	1129.0	463.0	1.8200	80100.0
2	333.0	117.0	1.6509	85700.0
3	515.0	226.0	3.1917	73400.0
5	671.0	239.0	3.3438	74000.0
6	1841.0	633.0	2.6768	82400.0

9.9.1 複数の条件指定による要素の選択

複数の条件を組み合わせる要素を選択することも出来ます。

その場合は条件式を () でくくって用います。

論理和 (or) は |、**論理積 (and)** は & を用いて表します。

```
1 # 70000 より小さいまたは 80000 より大きい
2 mask2 = (df['median_house_value'] < 70000) | (df['median_house_value'] >
    80000)
```

```
1 mask2.head()
```

```
0    True
1    True
2    True
3   False
4    True
Name: median_house_value, dtype: bool
```

```
1 df[mask2].head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-114.31	34.19	15.0	5612.0	1283.0	
1	-114.47	34.40	19.0	7650.0	1901.0	
2	-114.56	33.69	17.0	720.0	174.0	
4	-114.57	33.57	20.0	1454.0	326.0	
6	-114.58	33.61	25.0	2907.0	680.0	

	population	households	median_income	median_house_value
0	1015.0	472.0	1.4936	66900.0
1	1129.0	463.0	1.8200	80100.0
2	333.0	117.0	1.6509	85700.0
4	624.0	262.0	1.9250	65500.0
6	1841.0	633.0	2.6768	82400.0

```
1 # 70000 より大きいかつ 80000 より小さい
2 mask3 = (df['median_house_value'] > 70000) & (df['median_house_value'] <
    80000)
```

```
1 mask3.head()
```

```
0    False
1    False
2    False
```



```
3     True
4     False
Name: median_house_value, dtype: bool
```

```
1 df[mask3].head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
3	-114.57	33.64	14.0	1501.0	337.0	
5	-114.58	33.63	29.0	1387.0	236.0	
13	-114.61	34.83	31.0	2478.0	464.0	
23	-114.98	33.82	15.0	644.0	129.0	
26	-115.37	32.82	30.0	1602.0	322.0	

	population	households	median_income	median_house_value
3	515.0	226.0	3.1917	73400.0
5	671.0	239.0	3.3438	74000.0
13	1346.0	479.0	3.2120	70400.0
23	137.0	52.0	3.2097	71300.0
26	1130.0	335.0	3.5735	71100.0

条件に当てはまる要素を調べる操作と、条件に当てはまる要素の選択まで、1行にまとめて書くこともできます。

```
1 df[(df['median_house_value'] > 70000) & (df['median_house_value'] < 80000)].head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
3	-114.57	33.64	14.0	1501.0	337.0	
5	-114.58	33.63	29.0	1387.0	236.0	
13	-114.61	34.83	31.0	2478.0	464.0	
23	-114.98	33.82	15.0	644.0	129.0	
26	-115.37	32.82	30.0	1602.0	322.0	

	population	households	median_income	median_house_value
3	515.0	226.0	3.1917	73400.0
5	671.0	239.0	3.3438	74000.0
13	1346.0	479.0	3.2120	70400.0
23	137.0	52.0	3.2097	71300.0
26	1130.0	335.0	3.5735	71100.0

9.9.2 条件指定による要素の置換

条件を指定して選択した要素に対し、値の書き換えを行うことができます。

例えば、median_house_value 列に対していくつかの条件を別々に調べ、それぞれの条件を満たしている場合に特定の値を持つような新しい列を df に追加してみます。

median_house_value が

- 60000 より小さい場合は 0
- 60000 以上 70000 未満は 1
- 70000 以上 80000 未満は 2
- 80000 以上は 3

となる値を持つ `target` という列を追加します。

まず、何も値の入っていない `target` という列を `df` に追加します。

```
1 # 新しい列 target を None で初期化
2 df['target'] = None
```

```
1 df.head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-114.31	34.19	15.0	5612.0	1283.0	
1	-114.47	34.40	19.0	7650.0	1901.0	
2	-114.56	33.69	17.0	720.0	174.0	
3	-114.57	33.64	14.0	1501.0	337.0	
4	-114.57	33.57	20.0	1454.0	326.0	
	population	households	median_income	median_house_value	target	
0	1015.0	472.0	1.4936	66900.0	None	
1	1129.0	463.0	1.8200	80100.0	None	
2	333.0	117.0	1.6509	85700.0	None	
3	515.0	226.0	3.1917	73400.0	None	
4	624.0	262.0	1.9250	65500.0	None	

`target` という列の全ての要素は `None` となっています。

この値を、条件指定によって書き換えます。

まず各条件に対応するマスクを作成します。

```
1 mask1 = df['median_house_value'] < 60000
2 mask2 = (df['median_house_value'] >= 60000) & (df['median_house_value'] <
3         70000)
4 mask3 = (df['median_house_value'] >= 70000) & (df['median_house_value'] <
5         80000)
6 mask4 = df['median_house_value'] >= 80000
```

行や列を整数インデックスで選択する場合は `df.iloc[]` を使用しましたが、**列を名前で指定する**場合には `df.loc[]` を用います。

それでは、上で計算したマスクと名前による列指定を組み合わせ、各条件を満たす行の `target` 列の値を書き換えます。

```
1 df.loc[mask1, 'target'] = 0
2 df.loc[mask2, 'target'] = 1
3 df.loc[mask3, 'target'] = 2
4 df.loc[mask4, 'target'] = 3
```

結果を確認してみましょう。

```
1 # 先頭から 5 番目までを表示
2 df.head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-114.31	34.19	15.0	5612.0	1283.0	
1	-114.47	34.40	19.0	7650.0	1901.0	
2	-114.56	33.69	17.0	720.0	174.0	
3	-114.57	33.64	14.0	1501.0	337.0	
4	-114.57	33.57	20.0	1454.0	326.0	

	population	households	median_income	median_house_value	target
0	1015.0	472.0	1.4936	66900.0	1
1	1129.0	463.0	1.8200	80100.0	3
2	333.0	117.0	1.6509	85700.0	3
3	515.0	226.0	3.1917	73400.0	2
4	624.0	262.0	1.9250	65500.0	1

9.10 欠損値の除去・補間

欠損値を含むデータの場合、一部の行の値が欠損している列に NaN (Not a Number)、None、NaT (Not a Time) などが含まれる場合があります。

欠損値への対策としては、欠損値を含む行、または列を除去するか、欠損値を特定の値で補完するという方法が考えられます。

まずは、欠損値の除去の方法を紹介します。

```
1 # 欠損値を人為的に作成
2 df.iloc[0, 0] = None
```

```
1 # (0, 'longitude') の要素が NaN になっていることを確認
2 df.head(3)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	NaN	34.19	15.0	5612.0	1283.0	
1	-114.47	34.40	19.0	7650.0	1901.0	
2	-114.56	33.69	17.0	720.0	174.0	

	population	households	median_income	median_house_value	target
0	1015.0	472.0	1.4936	66900.0	1
1	1129.0	463.0	1.8200	80100.0	3
2	333.0	117.0	1.6509	85700.0	3

```
1 # 欠損値のあるレコードを削除
2 df_dropna = df.dropna()
3
4 # 先頭から 3 件を表示
5 df_dropna.head(3)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
1	-114.47	34.40	19.0	7650.0	1901.0	
2	-114.56	33.69	17.0	720.0	174.0	
3	-114.57	33.64	14.0	1501.0	337.0	

	population	households	median_income	median_house_value	target
1	1129.0	463.0	1.8200	80100.0	3
2	333.0	117.0	1.6509	85700.0	3
3	515.0	226.0	3.1917	73400.0	2

上の結果と見比べると、NaN を含んでいた 0 行目のデータが取り除かれていることが分かります。

次に、平均を使った欠損値の補完を行ってみましょう。

まずは、補完に使用する平均値の計算を行います。

```
1 mean = df.mean()
2 mean
```

```
longitude          -119.562417
latitude            35.625225
housing_median_age  28.589353
total_rooms         2643.664412
total_bedrooms      539.410824
population          1429.573941
households          501.221941
median_income       3.883578
median_house_value  207300.912353
target              2.808412
dtype: float64
```

計算した各列の値の平均が格納されている `mean` を、`df.fillna()` メソッドに渡すことで、`mean` を用いた欠損値の補完を行うことができます。

```
1 # 欠損値を mean で補完
2 df_fillna = df.fillna(mean)
3
4 # 先頭から 3 件を表示
5 df_fillna.head(3)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-119.562417	34.19	15.0	5612.0	1283.0	
1	-114.470000	34.40	19.0	7650.0	1901.0	
2	-114.560000	33.69	17.0	720.0	174.0	

	population	households	median_income	median_house_value	target
0	1015.0	472.0	1.4936	66900.0	1
1	1129.0	463.0	1.8200	80100.0	3
2	333.0	117.0	1.6509	85700.0	3

0 行目のデータの `longitude` 列に、`mean` の `longitude` 行の値が表示されていることが分かります。

今回は欠損値が 1 箇所だけにあったデータを用いましたが、`df.dropna()` や `df.fillna()` は、対象の全ての欠損値に対して上記のような操作を行うメソッドです。

9.11 ndarray とデータフレームを相互に変換

`scikit-learn` では、データフレームやシリーズをそのまま扱うことができます。

しかし、`Chainer` を含む他のライブラリやフレームワークではそのままでは扱うことができない場合があります。

そこで、データフレームを NumPy の `ndarray` に変換する方法を紹介します。

まず、`df` がデータフレームであることを確認します。

```
1 type(df)
```

```
pandas.core.frame.DataFrame
```

次に、`df` の `values` という属性の型を調べてみます。

```
1 type(df.values)
```

```
numpy.ndarray
```

NumPy の ndarray になっています。

データフレームやシリーズは、`values` という属性に値を ndarray として格納しています。

```
1 df.values
```

```
array([[      nan,  3.4190e+01,  1.5000e+01, ...,  1.4936e+00,
        6.6900e+04,  1.0000e+00],
       [-1.1447e+02,  3.4400e+01,  1.9000e+01, ...,  1.8200e+00,
        8.0100e+04,  3.0000e+00],
       [-1.1456e+02,  3.3690e+01,  1.7000e+01, ...,  1.6509e+00,
        8.5700e+04,  3.0000e+00],
       ...,
       [-1.2430e+02,  4.1840e+01,  1.7000e+01, ...,  3.0313e+00,
        1.0360e+05,  3.0000e+00],
       [-1.2430e+02,  4.1800e+01,  1.9000e+01, ...,  1.9797e+00,
        8.5800e+04,  3.0000e+00],
       [-1.2435e+02,  4.0540e+01,  5.2000e+01, ...,  3.0147e+00,
        9.4600e+04,  3.0000e+00]])
```

```
1 type(df['longitude'])
```

```
pandas.core.series.Series
```

```
1 type(df['longitude'].values)
```

```
numpy.ndarray
```

逆に、Python のリストや ndarray からシリーズやデータフレームを作ることができます。

NumPy で乱数を要素にもつ ndarray を生成し、これをデータフレームに変換してみましょう。

`pd.DataFrame` のインスタンス化の際に、`data` 引数に元にした ndarray を与えます。

```
1 import numpy as np
2
3 # ndarray -> pd.DataFrame
4 df = pd.DataFrame(
5     data=np.random.randn(10, 10)
6 )
7
8 df
```

	0	1	2	3	4	5	6	\
0	0.235076	-0.025952	0.635551	-1.233459	1.875983	0.471026	-0.846990	
1	1.232096	0.327020	0.073412	-0.291659	-0.835771	-1.806214	-0.840970	
2	0.540820	0.140009	1.351297	2.709533	0.469056	-0.573746	-0.135772	
3	1.527199	-1.129577	0.240729	-0.352903	-0.052596	-0.013748	-0.025188	
4	-1.171764	0.503813	2.424457	0.262115	2.149916	0.984207	-1.187516	
5	0.278999	-0.892611	0.429661	-2.511540	0.536308	0.811684	1.774007	
6	0.724094	0.804228	1.049159	0.348018	-2.106214	0.185539	-0.556945	

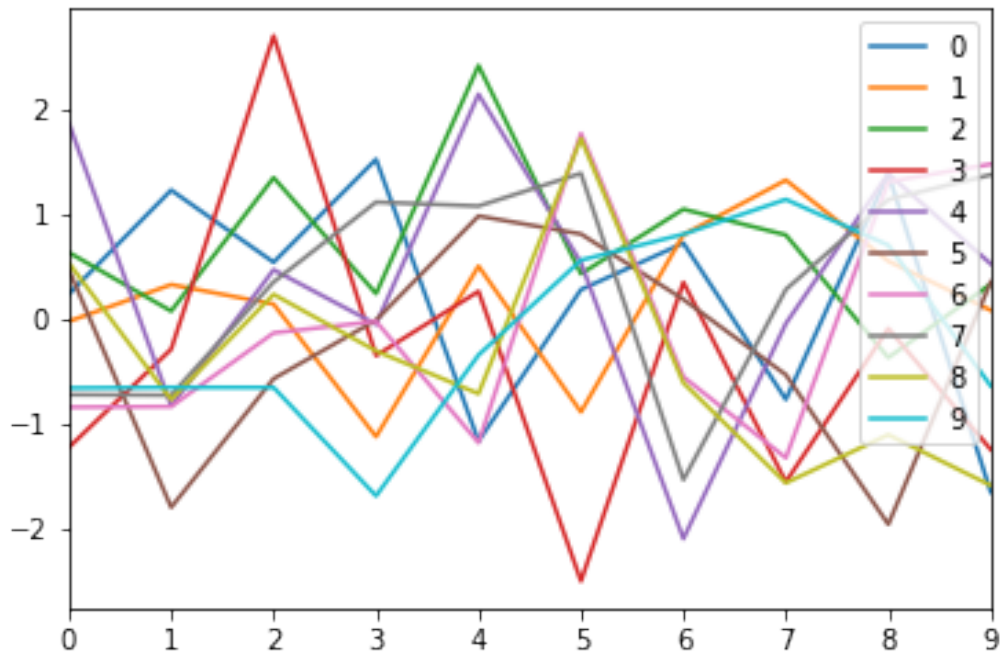


図 9.1 Notebook figure 07-01

7	-0.773019	1.327983	0.802475	-1.562232	-0.046259	-0.536498	-1.331944
8	1.388867	0.551502	-0.372442	-0.093690	1.388269	-1.969065	1.312570
9	-1.660362	0.081545	0.332048	-1.252397	0.525961	0.352583	1.480100

	7	8	9
0	-0.727128	0.535281	-0.662937
1	-0.731787	-0.773722	-0.657064
2	0.352596	0.233631	-0.658191
3	1.117134	-0.308465	-1.696524
4	1.079167	-0.717449	-0.347563
5	1.392216	1.726091	0.561226
6	-1.541576	-0.614728	0.809802
7	0.280377	-1.572351	1.142880
8	1.141520	-1.110659	0.704427
9	1.378975	-1.596351	-0.642919

9.12 グラフの描画

データフレームオブジェクトから直接可視化のための機能呼び出すことができます。

次の章で紹介する Matplotlib というグラフ描画ライブラリを `df.plot()` 機能を用いて利用することができます。

```
1 # グラフの描画
2 df.plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x121ee8cc0>
```

Matplotlib の使い方は、次章で説明します。

第 10 章

Matplotlib 応用

10.1 この章の目的

Matplotlib の基本が分かっていても、独学では次の段階で詰まりやすくなります。

- どのデータにどのグラフを使うべきか
- ノートブック上で見えた図を、どう保存して再利用するか
- 軸ラベル、凡例、色、複数図の配置をどう整えるか

本章では、単に「こう描ける」と列挙するのではなく、**目的に応じて図を選び、保存可能な形で出力すること**を目標にします。

本章の到達目標は次の通りです。

- 散布図、ヒストグラム、箱ひげ図、折れ線グラフの使い分けを説明できる
- `plt.figure()` や `plt.subplots()` で図の構成を制御できる
- `xlabel`, `ylabel`, `title`, `legend` を使って図を読める形にできる
- `savefig` を使って提出物や再現用の図を保存できる
- `seaborn` を併用して分布や相関を効率的に確認できる

10.2 読み込みと保存先の準備

まずは Matplotlib, NumPy, Pandas を読み込み、図の保存先ディレクトリを作ります。ノートブックに表示されるだけでは後で見返せないのが、**必ず保存先を決める運用**にします。

```
1 from pathlib import Path
2
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import pandas as pd
6
7 output_dir = Path("outputs/ch08")
8 output_dir.mkdir(parents=True, exist_ok=True)
```

この章では California Housing のサンプルデータを使います。

```
1 df = pd.read_csv("sample_data/california_housing_train.csv")
2 print(df.shape)
3 print(df.columns.tolist())
```

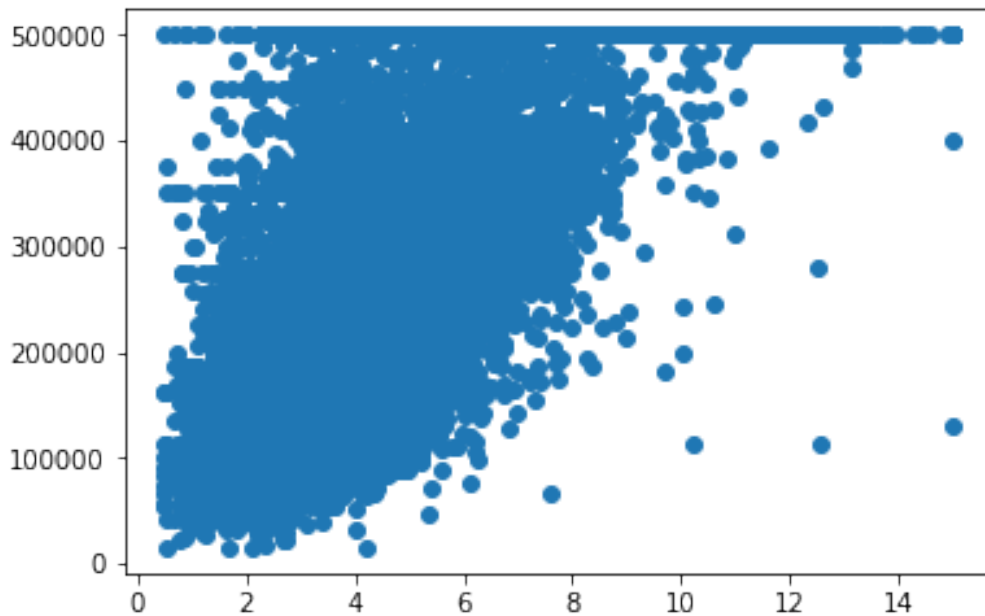


図 10.1 収入と住宅価格の散布図の例

図を描く前に、どの列が何を表しているかをざっと確認してください。可視化は「図を出すこと」よりも、「どの変数を比較すべきかを定めること」の方が本質です。

10.3 散布図

10.3.1 何を見るための図か

散布図 (scatter plot) は、2 つの連続値変数の関係を見るための基本図です。特に、

- 正の相関・負の相関がありそうか
- 直線的な関係か、曲線的な関係か
- 外れ値やクラスタがあるか

を確認するのに向いています。

例えば収入と住宅価格の関係を見るなら、横軸に `median_income`、縦軸に `median_house_value` を取ります。

```
1 plt.figure(figsize=(6, 4))
2 plt.scatter(df["median_income"], df["median_house_value"], s=8, alpha=0.4)
3 plt.xlabel("median_income")
4 plt.ylabel("median_house_value")
5 plt.title("Income vs house value")
6 plt.tight_layout()
7 plt.savefig(output_dir / "scatter_income_value.png", dpi=150)
8 plt.close()
```

この図では、収入が高いほど住宅価格も上がる傾向がある一方で、ばらつきもかなり大きいことが読み取れます。

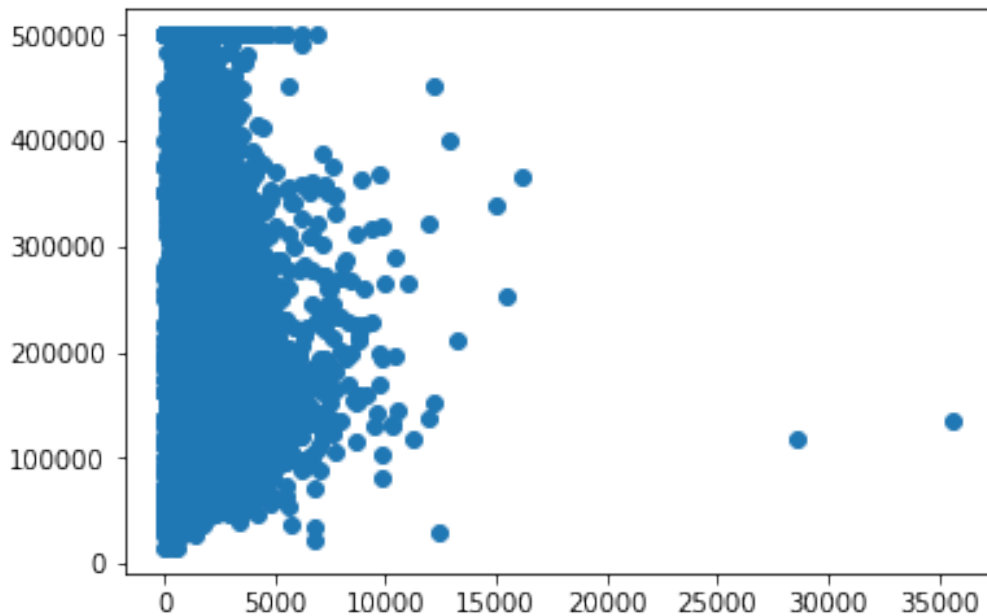


図 10.2 人口と住宅価格の散布図の例

10.3.2 散布図で注意すべき点

点が多すぎると、重なって密度が見えにくくなります。その場合は

- 点を小さくする
- `alpha` で透明度を下げる
- サンプルングして一部だけ描く

といった工夫が有効です。

また、元 notebook には `pupulation` という誤記がありましたが、正しくは `population` 列です。列名の typo に気付けるかどうか、実務では重要です。

```
1 plt.figure(figsize=(6, 4))
2 plt.scatter(df["population"], df["median_house_value"], s=8, alpha=0.3)
3 plt.xlabel("population")
4 plt.ylabel("median_house_value")
5 plt.tight_layout()
6 plt.savefig(output_dir / "scatter_population_value.png", dpi=150)
7 plt.close()
```

10.4 ヒストグラム

10.4.1 何を見るための図か

ヒストグラム (histogram) は、1 つの変数の分布を見るための図です。散布図が「2 変数の関係」を見る図であるのに対し、ヒストグラムは

- 値がどのあたりに集中しているか
- 裾が長いかな

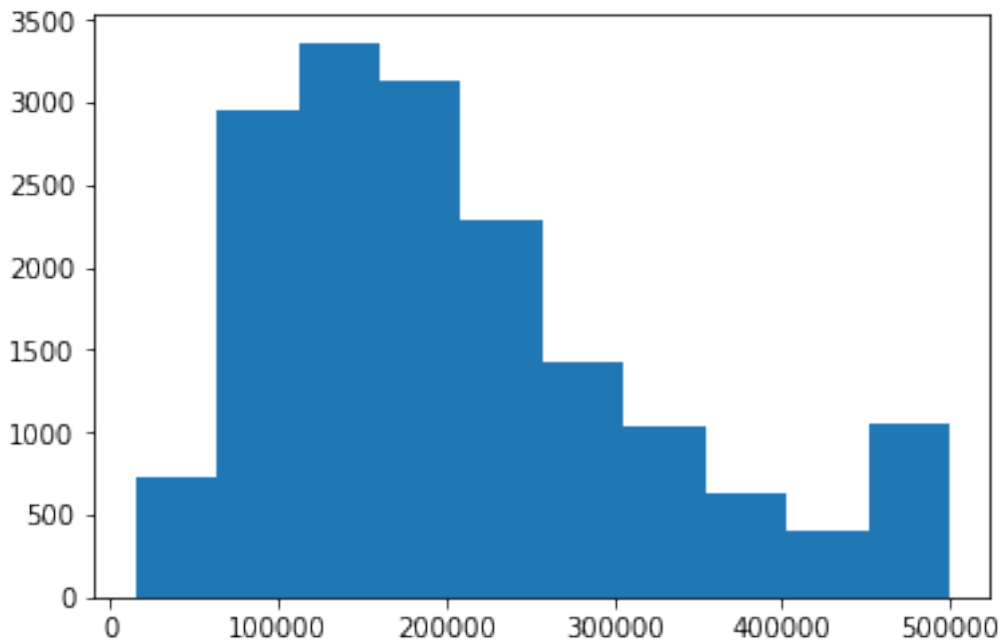


図 10.3 住宅価格のヒストグラムの例

- 分布が左右対称か
- 極端な山があるか

を確認するために使います。

```

1 plt.figure(figsize=(6, 4))
2 plt.hist(df["median_house_value"], bins=30, edgecolor="black")
3 plt.xlabel("median_house_value")
4 plt.ylabel("count")
5 plt.title("Distribution of house values")
6 plt.tight_layout()
7 plt.savefig(output_dir / "hist_house_value.png", dpi=150)
8 plt.close()

```

10.4.2 bins の意味

ヒストグラムでは、棒 1 本 1 本が**ビン (bin)** と呼ばれる区間を表します。bins を大きくすると細かい構造が見えやすくなり、小さくすると全体傾向は見やすくなります。したがって、bins は「正解が 1 つある設定値」ではなく、**どの粒度で分布を見たいか**という判断です。

```

1 plt.figure(figsize=(6, 4))
2 plt.hist(df["median_house_value"], bins=50, edgecolor="black")
3 plt.xlabel("median_house_value")
4 plt.ylabel("count")
5 plt.tight_layout()
6 plt.savefig(output_dir / "hist_house_value_bins50.png", dpi=150)
7 plt.close()

```

このデータでは、500000 付近に不自然な山があり、上限で打ち切られているような分布に見えます。こうした「データ収集や前処理の制約」を見つけるのも可視化の重要な役割です。

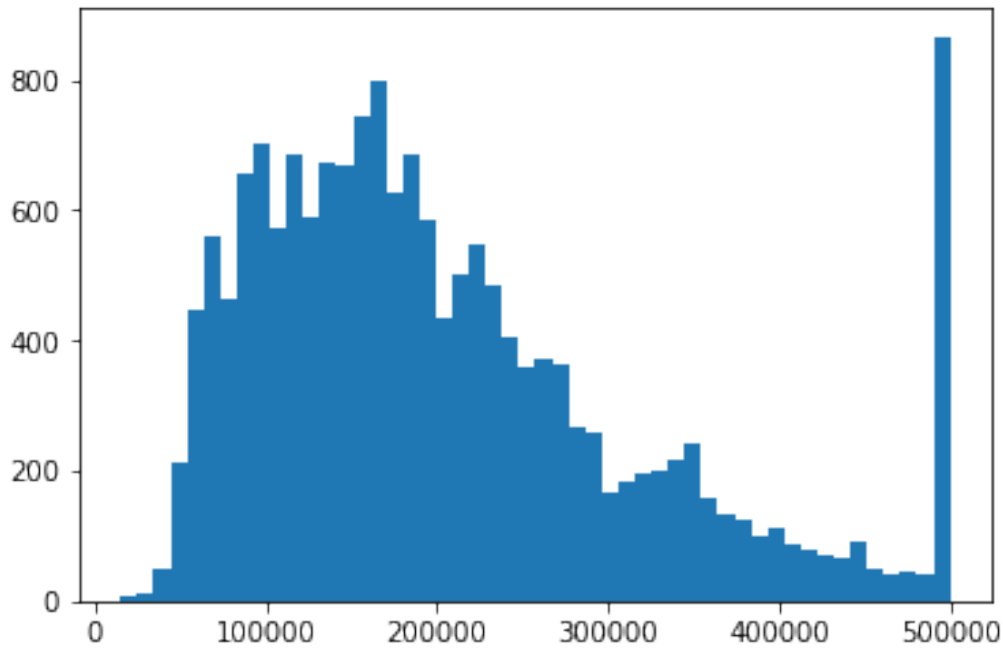


図 10.4 ビン数を増やしたヒストグラムの例

10.5 箱ひげ図

10.5.1 何を見るための図か

箱ひげ図 (box plot) は、分布を五数要約で要約した図です。

- 最小値
- 第 1 四分位点
- 中央値
- 第 3 四分位点
- 最大値

をもとに、ばらつきや外れ値をコンパクトに確認できます。

```

1 plt.figure(figsize=(4, 4))
2 plt.boxplot(df["median_house_value"])
3 plt.ylabel("median_house_value")
4 plt.tight_layout()
5 plt.savefig(output_dir / "box_house_value.png", dpi=150)
6 plt.close()

```

10.5.2 複数系列の比較

複数の分布を横並びに比較したいとき、箱ひげ図は特に便利です。ただし、単位や桁が極端に違う変数同士を並べると読みづらくなります。

```

1 plt.figure(figsize=(5, 4))
2 plt.boxplot((df["total_bedrooms"], df["population"]),

```

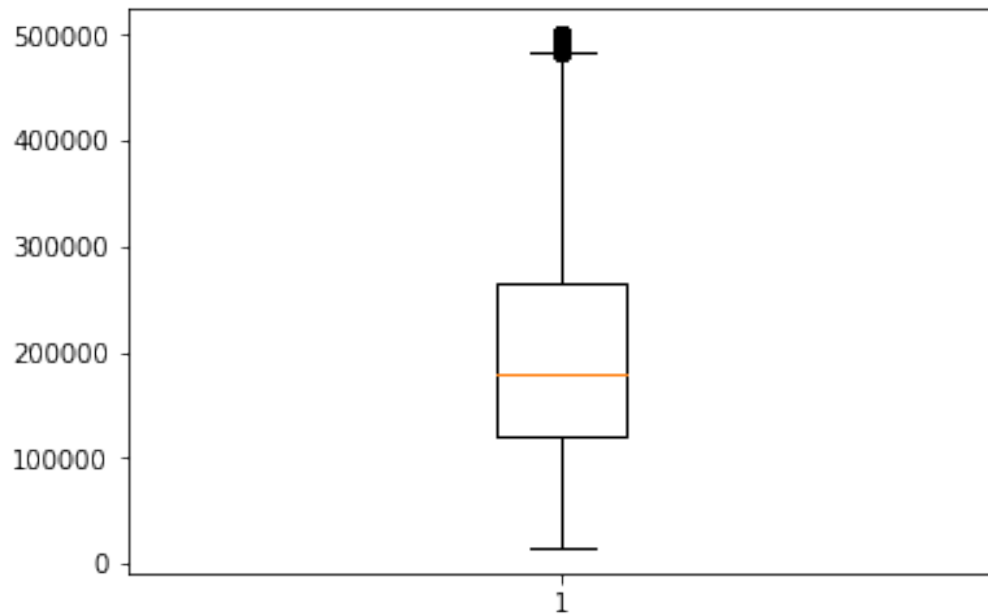


図 10.5 住宅価格の箱ひげ図の例

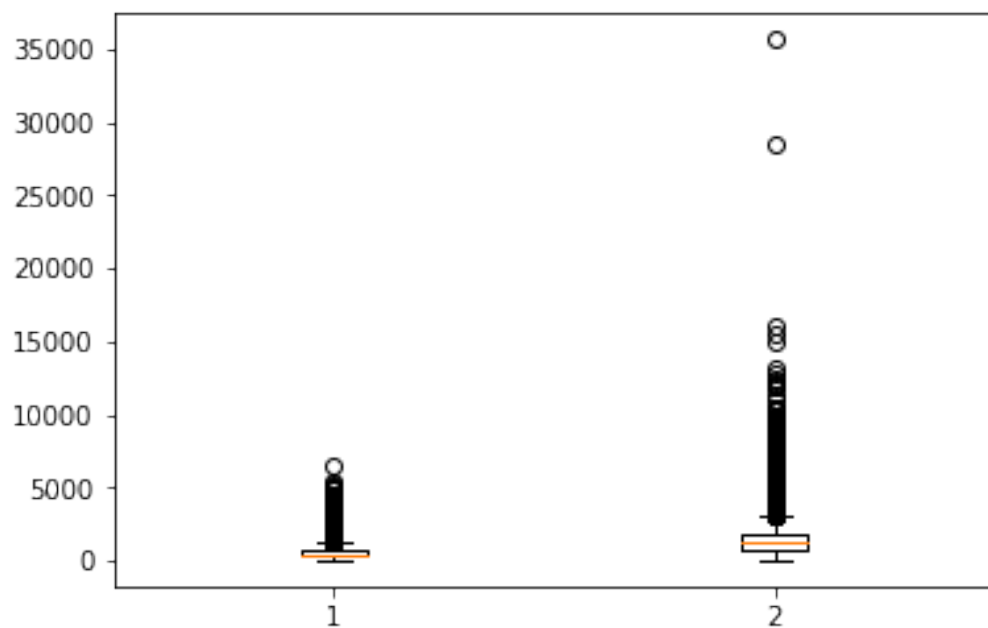


図 10.6 複数変数の箱ひげ図の例

```

3         labels=["total_bedrooms", "population"])
4 plt.tight_layout()
5 plt.savefig(output_dir / "box_multiple.png", dpi=150)
6 plt.close()

```

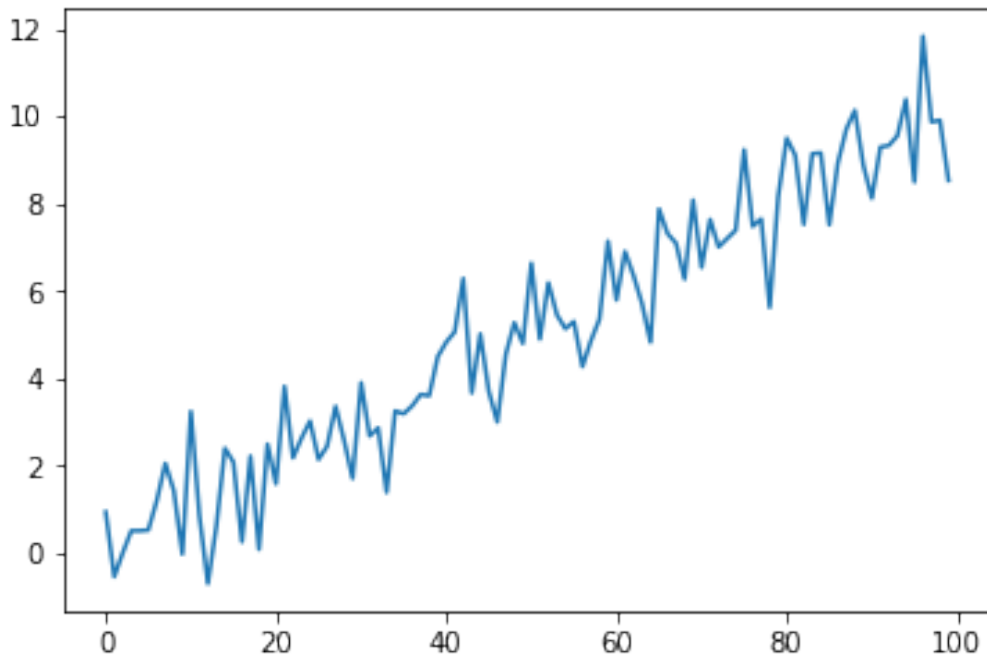


図 10.7 インデックスを横軸にした折れ線グラフの例

10.6 折れ線グラフ

10.6.1 何を見るための図か

折れ線グラフは、時系列や連続的な変化を表すときに向いています。並び順に意味がないカテゴリデータにはあまり向きません。

`plt.plot(y)` のように 1 引数で描くと、横軸はインデックスになります。

```
1 rng = np.random.default_rng(0)
2 x = np.linspace(0, 10, 100)
3 y = x + rng.normal(scale=1.0, size=100)
4
5 plt.figure(figsize=(6, 4))
6 plt.plot(y)
7 plt.xlabel("index")
8 plt.ylabel("value")
9 plt.tight_layout()
10 plt.savefig(output_dir / "line_index_only.png", dpi=150)
11 plt.close()
```

横軸に意味のある変数があるなら、`plt.plot(x, y)` のように明示した方が良いです。

```
1 plt.figure(figsize=(6, 4))
2 plt.plot(x, y, label="noisy_line")
3 plt.xlabel("x")
4 plt.ylabel("y")
5 plt.legend()
6 plt.tight_layout()
7 plt.savefig(output_dir / "line_xy.png", dpi=150)
8 plt.close()
```

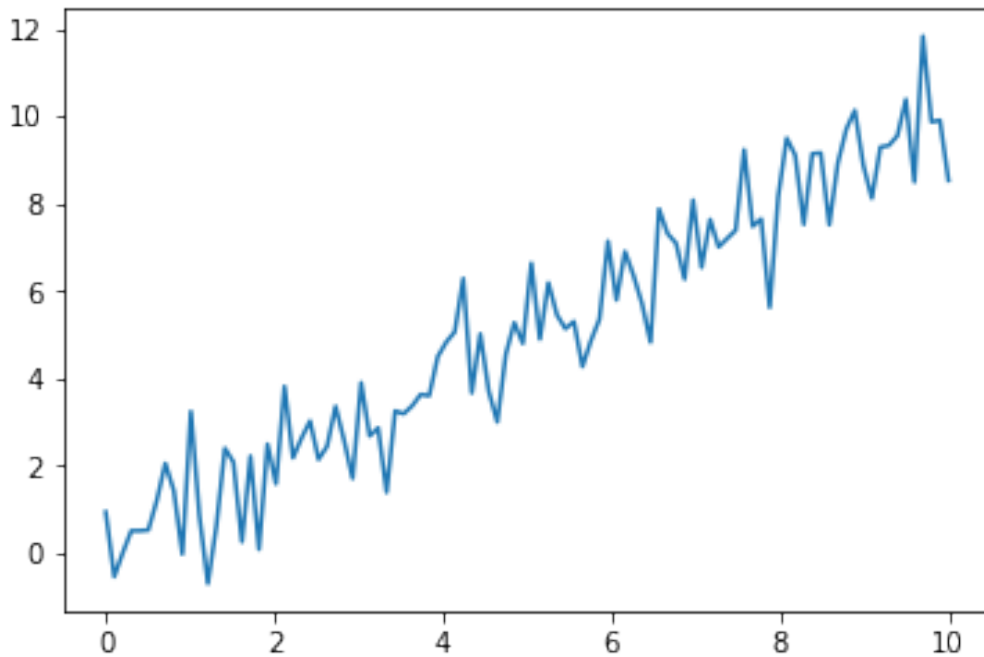


図 10.8 横軸を明示した折れ線グラフの例

10.7 図を読む形に整える

グラフは描けるだけでは不十分で、**第三者が見て意味を読む**必要があります。最低限そろえるべき要素は次の通りです。

- 横軸ラベル
- 縦軸ラベル
- 必要ならタイトル
- 複数系列があるなら凡例
- 保存時の解像度と余白調整

代表的な設定は次です。

```
1 plt.figure(figsize=(6, 4))
2 plt.plot(x, y, color="tab:blue", linewidth=2, label="signal")
3 plt.xlabel("x")
4 plt.ylabel("y")
5 plt.title("Example_plot")
6 plt.legend()
7 plt.grid(True, alpha=0.3)
8 plt.tight_layout()
9 plt.savefig(output_dir / "styled_plot.png", dpi=150)
10 plt.close()
```

`tight_layout()` を入れておくと、ラベルのはみ出しをかなり防げます。また、Jupyter で見えるだけで満足せず、`savefig` で保存してから終えるのがこの教材での基本です。

10.8 subplots で複数図を並べる

比較したい図を1枚にまとめるときは、`plt.subplots()` が便利です。探索的解析では、1変数の分布と2変数の関係を並べて見るのがよくあります。

```

1 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
2
3 axes[0].hist(df["median_income"], bins=30, edgecolor="black")
4 axes[0].set_title("median_income")
5 axes[0].set_xlabel("value")
6 axes[0].set_ylabel("count")
7
8 axes[1].scatter(df["median_income"], df["median_house_value"], s=8, alpha
9                 =0.3)
10 axes[1].set_title("income vs house value")
11 axes[1].set_xlabel("median_income")
12 axes[1].set_ylabel("median_house_value")
13
14 fig.tight_layout()
15 fig.savefig(output_dir / "subplots_example.png", dpi=150)
16 plt.close(fig)

```

`plt.figure()` ベースより少し記述が増えますが、複数図を明示的に管理しやすくなります。

10.9 seaborn の使いどころ

seaborn は Matplotlib を土台にした高水準可視化ライブラリです。分布や相関を素早く見るのに便利ですが、内部では Matplotlib を使っているため、軸や図の基本概念は同じです。

```

1 import seaborn as sns

```

10.9.1 ヒストグラムと KDE

古い資料では `sns.distplot()` がよく紹介されますが、現在は非推奨です。代わりに `sns.histplot()` を使う方が安全です。

```

1 plt.figure(figsize=(6, 4))
2 sns.histplot(df["population"], bins=30, kde=True)
3 plt.tight_layout()
4 plt.savefig(output_dir / "sns_histplot_population.png", dpi=150)
5 plt.close()

```

10.9.2 複数変数の相関を俯瞰する

`sns.pairplot()` は複数列の関係を一気に見たいときに便利です。ただし列数が多いと非常に重くなるので、必要な列だけ絞って使うべきです。

```

1 cols = ["median_income", "median_house_value", "population", "households"]
2 grid = sns.pairplot(df[cols].sample(1000, random_state=0))
3 grid.savefig(output_dir / "pairplot_selected_columns.png", dpi=150)
4 plt.close("all")

```

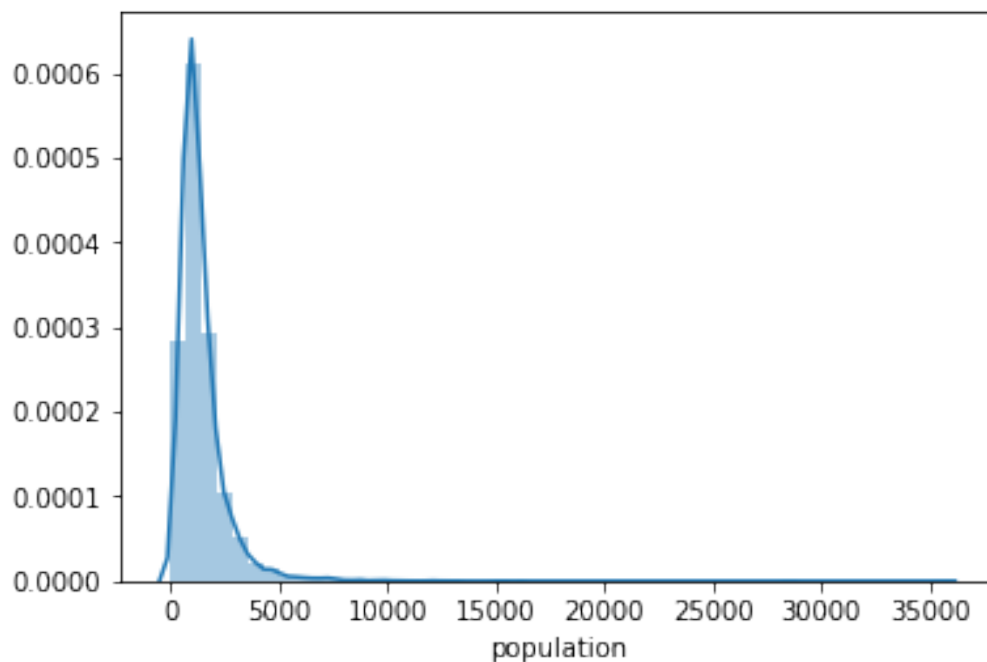


図 10.9 分布確認に使う seaborn 図の例

pairplot は便利ですが、毎回フルデータで回す必要はありません。サンプリングして全体傾向だけ見た方が実用的な場面も多いです。

10.10 この章で押さえるべき点

- 散布図は 2 変数の関係、ヒストグラムは 1 変数の分布、箱ひげ図は分布の要約、折れ線グラフは順序ある変化を見る図である
- グラフの種類を覚えるだけでなく、「何を確認したいからこの図を選ぶのか」を説明できることが重要である
- `xlabel`, `ylabel`, `legend`, `tight_layout`, `savefig` は基本セットである
- 複数図を扱うときは `subplots` を使うと管理しやすい
- seaborn は便利だが、Matplotlib の基本概念を理解した上で使うべきである

可視化は「見た目をきれいにする作業」ではなく、データの構造を読み取るための観測手段です。その意識を持つと、どの図を選ぶべきかがかなり判断しやすくなります。

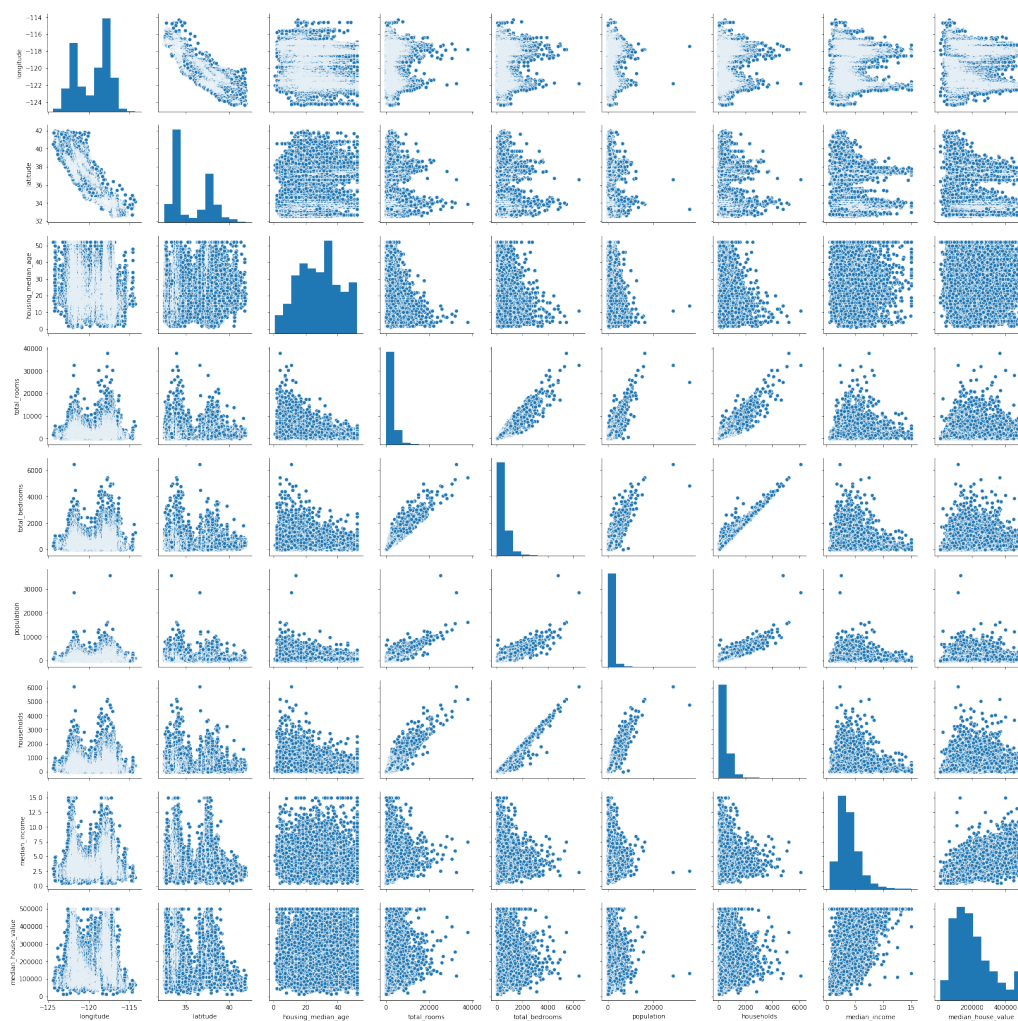


図 10.10 複数変数の関係を俯瞰する pairplot の例

第 11 章

PySoundFile 入門

11.1 この章の目的

音響信号処理では、波形データを NumPy 配列として読み書きできることが出発点になります。PySoundFile はそのための軽量なライブラリであり、

- WAV や FLAC などの音声ファイルを読む
- NumPy 配列として波形を扱う
- 加工結果を音声ファイルとして保存する

という基本作業をシンプルに行えます。

本章の到達目標は次の通りです。

- `soundfile.read` と `soundfile.write` の基本的な使い方を説明できる
- モノラルとステレオで返る配列形状の違いを理解できる
- サンプルレートと波形長の間係を説明できる
- 一部分だけ読み込む、`dtype` を指定するなどの実用的な操作を使える
- 保存した波形を Python で検証する最小手順を持てる

11.2 PySoundFile とは何か

PySoundFile は `libsndfile` を土台にした Python 用の音響ファイル入出力ライブラリです。WAV, FLAC, OGG など複数の形式に対応し、音声データを NumPy 配列として返します。

詳細は [公式ドキュメント](#) を参照してください。

信号処理や機械学習の前処理では、音声を

- 何 Hz でサンプリングされたか
- 何チャンネルあるか
- 何サンプル続くか

というメタデータと一緒に扱う必要があります。PySoundFile は、波形そのものとサンプルレートをセットで返すので、この最初の入口として使いやすいです。

11.3 インストール

soundfile は教材 repo の共有環境に含まれる。教材 repo の共有 Python 環境を使う。準備・更新は教材 repo でいい、演習は作業 repo のルートで教材側の `.venv` を有効にして実行する。手順と保存先は [共通手順](#)を参照する。

次の OS 側ライブラリの準備は、Python 環境の同期とは別の操作である。WSL や Linux 環境で `libsndfile` が不足している場合は、先にシステム側のライブラリを入れます。

```
1 sudo apt update
2 sudo apt install libsndfile1
```

読み込み時は慣例的に `sf` という別名を使います。

```
1 import soundfile as sf
```

11.4 音声を読む

もっとも基本的な読み込みは次の通りです。

```
1 data, samplerate = sf.read("path/to/audio.wav")
```

ここで

- `data` は NumPy 配列
- `samplerate` は整数のサンプリング周波数

です。

例えば `samplerate = 16000` なら、1 秒間に 16000 個のサンプルが並んでいます。この情報がないと、波形の横軸を秒として解釈できません。

11.5 モノラルとステレオの配列形状

PySoundFile が返す配列形状はチャンネル数で変わります。

- モノラル: `(num_samples,)`
- ステレオ: `(num_samples, 2)`
- 一般の多チャンネル: `(num_samples, num_channels)`

つまり、**時間方向が先頭軸**です。画像処理でよく見る `(height, width, channel)` と同様に、最後の軸がチャンネルです。

モノラルとステレオを同じコードで扱うときは、次のように `shape` をまず確認する癖を付けると安全です。

```
1 print(data.shape)
2 print(data.dtype)
3 print(samplerate)
```

ステレオ音声の左チャンネルだけを使いたい場合は

```
1 left = data[:, 0]
```

のように取り出します。

11.6 dtype と振幅範囲

`sf.read` は既定では浮動小数点配列を返すことが多く、典型的には `float64` です。このとき振幅はしばしば $[-1, 1]$ 付近に正規化されています。

ただし、ファイル形式や指定によっては整数型で読むこともできます。例えば 16 bit PCM をそのまま整数で読みたい場合は、

```
1 data_int16, samplerate = sf.read("path/to/audio.wav", dtype="int16")
```

のように dtype を指定します。

機械学習や可視化では `float` の方が扱いやすく、波形保存や既存ツールとの互換性確認では PCM 整数の意味が重要になることがあります。どの表現を使っているかを曖昧にしないことが重要です。

11.7 一部分だけ読む

長い音声ファイルを丸ごと読む必要がない場面もあります。例えば最初の 3 秒だけ確認したいなら、サンプル数に直して `frames` を指定します。

```
1 seconds = 3
2 frames = 16000 * seconds
3 segment, sr = sf.read("path/to/audio.wav", frames=frames)
```

また、途中から読むには `start` を使えます。

```
1 start = 16000 * 5 # 5 秒地点
2 frames = 16000 * 2 # そこから 2 秒分
3 segment, sr = sf.read("path/to/audio.wav", start=start, frames=frames)
```

これは長時間録音の確認や、学習データセットから必要区間だけ切り出す前処理で便利です。

11.8 音声を書き出す

加工した NumPy 配列は `sf.write` で保存できます。

```
1 sf.write("path/to/output.wav", data, samplerate)
```

保存時に重要なのは、**配列の shape と samplerate が妥当か**です。例えばモノラルとして保存したいのに shape が `(num_samples, 1)` や `(1, num_samples)` になっていると、想定と違う結果になることがあります。

また、別名で保存するときは拡張子からフォーマットが決まることが多いです。

```
1 sf.write("denoised.flac", denoised, samplerate)
```

11.9 Python で最小の読み書きと検証を行う

以下は、音声を読み込み、先頭 1 秒を切り出して保存し、同時に波形を図として確認する最小例です。

```
1 from pathlib import Path
2
3 import matplotlib.pyplot as plt
4 import numpy as np
```

```

5 import soundfile as sf
6
7 input_path = "path/to/audio.wav"
8 output_dir = Path("outputs/ch11")
9 output_dir.mkdir(parents=True, exist_ok=True)
10
11 data, sr = sf.read(input_path, dtype="float32")
12
13 if data.ndim == 2:
14     mono = data.mean(axis=1)
15 else:
16     mono = data
17
18 segment = mono[:sr] # 先頭 1 秒
19 sf.write(output_dir / "segment.wav", segment, sr)
20
21 t = np.arange(len(segment)) / sr
22 plt.figure(figsize=(8, 3))
23 plt.plot(t, segment, linewidth=1)
24 plt.xlabel("time [s]")
25 plt.ylabel("amplitude")
26 plt.tight_layout()
27 plt.savefig(output_dir / "segment_waveform.png", dpi=150)
28 plt.close()

```

この例で確認すべき点は次の通りです。

- `data.ndim` を見ればモノラルか多チャンネルか判定できる
- `len(segment) / sr` で長さを秒に変換できる
- 保存した `.wav` と保存した波形図の両方を残すと検証しやすい

11.10 情報だけ見たいときは SoundFile オブジェクトを使う

ファイル全体をすぐ読み込まずに、メタデータだけ確認したいこともあります。その場合は `SoundFile` オブジェクトを使います。

```

1 with sf.SoundFile("path/to/audio.wav") as f:
2     print(f.samplerate)
3     print(f.channels)
4     print(len(f))

```

大量ファイルの前処理前に、サンプリング周波数やチャンネル数が混在していないかを点検する用途に向いています。

11.11 この章で押さえるべき点

- `PySoundFile` は音声ファイルを NumPy 配列として読み書きするための基本ライブラリである
- `sf.read` は波形配列とサンプリング周波数を返す
- モノラルは 1 次元、ステレオは `(num_samples, 2)` のような 2 次元配列になる
- `start` や `frames` を使うと必要な区間だけ読める
- 保存時は `dtype`, `shape`, `samplerate` を意識して検証する

次に STFT やスペクトログラムを扱うときも、入力は結局この波形配列です。そのため、「どういう shape と意味を持つ配列を読んでいるか」をここで曖昧にしないことが重要です。

第 12 章

Pillow 入門

12.1 この章の目的

画像処理を Python で始めるとき、最初に必要なのは「画像をファイルとして読み込み、配列や画素として扱い、加工結果を保存する」ことです。Pillow はそのための基本ライブラリであり、

- 画像の読み込みと保存
- サイズ変更, 回転, 切り出し
- 色空間の変換
- 簡単なフィルタ処理
- NumPy 配列との相互変換

を手軽に行えます。

本章の到達目標は次の通りです。

- Pillow の Image オブジェクトが何を表しているか説明できる
- 画像の `size`, `mode`, `format` を確認できる
- `resize`, `crop`, `rotate`, `flip`, `filter` を使って前処理を行える
- 画像を NumPy 配列に変換して `shape` や画素値を確認できる
- 結果を `save()` で保存し、提出や再現に使える形で残せる

12.2 Pillow とは何か

Pillow は、かつての PIL (Python Imaging Library) を引き継ぐ形で整備されている Python の画像処理ライブラリです。PNG, JPEG, BMP, TIFF など多くの画像形式に対応し、画像ファイルを Image オブジェクトとして扱えます。

ここで重要なのは、Pillow は「画像を表示するための GUI ライブラリ」ではなく、**画像データを操作するためのライブラリ**だという点です。この教材では、`show()` で一時表示するよりも、`save()` で結果を保存する流れを基本とします。

12.3 インストール

Pillow は教材 repo の共有環境に含まれる。教材 repo の共有 Python 環境を使う。準備・更新は教材 repo で行い、演習は作業 repo のルートで教材側の `.venv` を有効にして実行する。手順と保存先は [共通手順](#)を参照する。

読み込みは通常次のように行います。

```
1 from PIL import Image, ImageDraw, ImageFilter
```

12.4 画像を開いて属性を確認する

画像ファイルを開くには `Image.open()` を使います。

```
1 from pathlib import Path
2 from PIL import Image
3
4 output_dir = Path("outputs/ch12")
5 output_dir.mkdir(parents=True, exist_ok=True)
6
7 img = Image.open("path/to/image.jpg")
8 print(img.format)
9 print(img.mode)
10 print(img.size)
```

ここで確認すべき属性は次の通りです。

- `format`: JPEG や PNG などのファイル形式
- `mode`: RGB, L, RGBA などの画素表現
- `size`: (width, height) の順のサイズ

初学者がよく混乱する点として、`size` は NumPy 配列の (height, width, channel) とは順序が違います。Pillow では (width, height)、NumPy では一般に (height, width, channel) です。

12.5 結果を保存する流れを最初に作る

授業や演習では、「その場で見えた」ではなく「後で確認できる」ことが重要です。したがって、画像処理では最初に保存先を決めておく方が安全です。

```
1 img.save(output_dir / "original_copy.png")
```

以後の各操作でも、結果を新しいファイル名で保存する癖を付けると比較しやすくなります。

12.6 サイズ変更

`resize()` は画像を指定サイズへ変換します。

```
1 img_resized = img.resize((256, 256))
2 img_resized.save(output_dir / "resized_256.png")
```

ただし、単に (256, 256) にすると元画像の縦横比が崩れることがあります。機械学習の前処理では、**縦横比を保つか、固定サイズにそろえるか**を明示的に決める必要があります。

12.7 回転と反転

`rotate()` は反時計回りに画像を回転させます。

```
1 img_rotated = img.rotate(45, expand=True)
2 img_rotated.save(output_dir / "rotated_45.png")
```

`expand=True` を付けると、回転後の全体が収まるようにキャンパスが広がります。これを付けないと、画像の端が切れることがあります。

左右反転・上下反転には `transpose()` を使います。

```
1 img_flip_lr = img.transpose(Image.FLIP_LEFT_RIGHT)
2 img_flip_tb = img.transpose(Image.FLIP_TOP_BOTTOM)
3
4 img_flip_lr.save(output_dir / "flip_left_right.png")
5 img_flip_tb.save(output_dir / "flip_top_bottom.png")
```

画像分類では左右反転はよく使いますが、上下反転はタスクによって意味が変わることがあります。`augmentation` は「何でも増やせば良い」わけではありません。

12.8 切り出し

`crop()` は `(left, upper, right, lower)` を指定して部分画像を切り出します。

```
1 crop_box = (50, 50, 200, 200)
2 img_cropped = img.crop(crop_box)
3 img_cropped.save(output_dir / "cropped.png")
```

ここでの座標系は、左上が原点で、右方向に x 、下方向に y が増える画像座標です。これは数学でよく使う座標系と上下が逆なので注意してください。

12.9 描画と注釈

Pillow では画像の上に矩形や文字を重ねることもできます。例えばアノテーション確認用に矩形を描くなら次のようにします。

```
1 annotated = img.copy()
2 draw = ImageDraw.Draw(annotated)
3 draw.rectangle(((50, 50), (200, 200)), outline="red", width=3)
4 annotated.save(output_dir / "annotated_box.png")
```

元画像を壊したくない場合は、`copy()` してから描画の方が安全です。

12.10 フィルタ処理

`filter()` を使うと、ぼかしや輪郭強調などの簡単な処理ができます。

```
1 img_blurred = img.filter(ImageFilter.BLUR)
2 img_blurred.save(output_dir / "blurred.png")
```

Pillow のフィルタは教育用や軽い前処理には便利ですが、本格的な画像処理では OpenCV や scikit-image を使うこともあります。それでも、「畳み込み的な処理で画像の見え方が変わる」ことを掴む入口としては十分です。

12.11 色空間の変換

画像処理では、RGB のまま扱うだけでなく、グレースケールや HSV に変換した方が都合のよいことがあります。

```

1  img_gray = img.convert("L")
2  img_hsv = img.convert("HSV")
3
4  img_gray.save(output_dir / "gray.png")
5  img_hsv.save(output_dir / "hsv.png")

```

L は 1 チャンネルのグレースケール画像です。色ではなく明るさだけを使うタスクでは、最初からグレースケールに落とす方が扱いやすい場合があります。

12.12 NumPy 配列との相互変換

機械学習では、最終的に画像を NumPy 配列や Tensor に変換して扱うことがほとんどです。

```

1  import numpy as np
2
3  img_array = np.array(img)
4  print(img_array.shape)
5  print(img_array.dtype)

```

RGB 画像なら、通常は (height, width, 3) の shape になります。ここで Pillow と NumPy の軸順の違いを再確認してください。

NumPy 配列から Pillow 画像へ戻すこともできます。

```

1  img_from_array = Image.fromarray(img_array)
2  img_from_array.save(output_dir / "from_array.png")

```

12.13 特定画素の値を調べる

特定の画素値を確認したい場合は `getpixel()` を使えます。

```

1  rgb = img.getpixel((50, 50))
2  print(rgb)

```

ただし、画像全体の処理では画素を 1 点ずつ Python でループするより、NumPy 配列へ変換してベクトル化した方が効率的です。`getpixel()` は確認用や学習用と考えた方がよいです。

12.14 ヒストグラム

画像の明るさや色の分布を見たいときはヒストグラムを使います。まずグレースケールへ変換してからヒストグラムを見ると解釈しやすいです。

```

1  import matplotlib.pyplot as plt
2
3  gray = img.convert("L")
4  histogram = gray.histogram()
5
6  plt.figure(figsize=(6, 4))
7  plt.bar(range(256), histogram)
8  plt.xlabel("pixel_value")
9  plt.ylabel("count")
10 plt.tight_layout()

```

```
11 plt.savefig(output_dir / "gray_histogram.png", dpi=150)
12 plt.close()
```

元 notebook には `img` が未定義のままヒストグラムを取って失敗していたセルがありましたが、ここでは最初に画像を開いた流れの上で一貫して処理しています。

12.15 二値化

二値化は、画素値をしきい値で 0/255 のような 2 値へ分ける処理です。文書画像や簡単なマスク生成の入口としてよく使います。

```
1 gray = img.convert("L")
2 threshold = 127
3 img_binarized = gray.point(lambda x: 0 if x < threshold else 255)
4 img_binarized.save(output_dir / "binarized.png")
```

しきい値 127 が常に正しいわけではありません。画像ごとに明るさが違うので、ヒストグラムを見ながら妥当な閾値を考える必要があります。

12.16 最小の前処理パイプライン

以下は、画像を読み込み、RGB からグレースケールへ変換し、サイズをそろえて保存する最小例です。

```
1 from pathlib import Path
2 from PIL import Image
3
4 input_path = "path/to/image.jpg"
5 output_dir = Path("outputs/ch12")
6 output_dir.mkdir(parents=True, exist_ok=True)
7
8 img = Image.open(input_path)
9 img = img.convert("L")
10 img = img.resize((128, 128))
11 img.save(output_dir / "preprocessed_128_gray.png")
```

この程度の流れでも、

- 入力形式の統一
- 色空間の統一
- サイズの統一

という前処理の基本を一通り含んでいます。

12.17 この章で押さえるべき点

- Pillow は画像ファイルを `Image` オブジェクトとして扱うための基本ライブラリである
- `size`, `mode`, `format` を最初に確認すると、後の処理で混乱しにくい
- `show()` に頼るより、`save()` で結果を残す方が教材・演習では重要である
- `resize`, `crop`, `rotate`, `flip`, `filter`, `convert` は画像前処理の基本操作である
- NumPy 配列へ変換すると、機械学習や数値処理へつなげやすい

この章の内容を押さえておくと、画像分類や画像変換タスクの前処理を「ブラックボックスの API 呼び出し」ではなく、どの形のデータをどのように整えているか意識しながら扱えるようになります。

第 13 章

信号処理の基礎

13.1 この章の目的

本章では、信号処理を学び始めるときに必要な最小限の概念を整理します。ここでいう**信号**とは、時間、空間、周波数などの軸に沿って並んだ数値列です。音声波形、画像の画素値列、センサの時系列、ニューラルネットワークに入力する特徴量列は、いずれも信号として扱えます。

機械学習の前処理では、生の観測値をそのままモデルに入れるのではなく、

- どの軸に沿って値が並んでいるか
- どのくらい細かく観測されているか
- どのような操作で情報が強調または失われるか

を理解しておく必要があります。

本章の到達目標は次の通りです。

- 離散時間信号を数列として表現できる
- サンプルング周期とサンプルング周波数の意味を説明できる
- インパルス、ステップ、正弦波といった基本信号を説明できる
- シフト、反転、スケーリングなどの基本操作が何をするか理解できる
- エネルギーと平均電力の違いを説明できる

13.2 信号とは何か

連続時間で変化する物理量を t の関数 $x(t)$ と表すことがあります。例えばマイクで観測される空気圧の変化は連続時間信号として考えられます。一方、コンピュータ上では通常、信号は一定間隔で標本化された数列として扱います。このとき離散時間信号を

$$x[n], \quad n = 0, 1, 2, \dots$$

と書きます。

n は**サンプル番号**であり、実際の時間そのものではありません。サンプルング周期を T_s 秒とすると、 n 番目のサンプルは実時間では

$$t = nT_s$$

に対応します。したがって、離散時間信号は「時間の情報を失った数列」ではなく、**一定間隔で観測した連続時間信号の近似表現**です。

13.3 サンプリング周期とサンプリング周波数

サンプリング周期 T_s は、隣り合う 2 つのサンプルの時間間隔です。その逆数

$$f_s = \frac{1}{T_s}$$

を**サンプリング周波数**と呼びます。単位は Hz です。

例えば $f_s = 16000$ Hz なら、1 秒間に 16000 点観測していることになります。音声処理では 16 kHz や 44.1 kHz、画像処理では画素間隔やフレームレートが同様の役割を持ちます。

ここで重要なのは、離散信号だけを見ても「隣り合うサンプルが何秒離れているか」は自動では分からないことです。同じ数列でも、 f_s が変われば意味する物理現象は変わります。したがって、信号データを扱うときは、値そのもののだけでなく**サンプリング条件も一緒に管理する**必要があります。

13.4 基本信号

13.4.1 単位インパルス

単位インパルスは

$$\delta[n] = \begin{cases} 1 & (n = 0) \\ 0 & (n \neq 0) \end{cases}$$

で定義されます。

一見すると単純な信号ですが、インパルスはシステムの性質を調べるための基準信号として非常に重要です。後の LTI 系の章では、インパルス応答から系全体の振る舞いを記述できることを学びます。

13.4.2 単位ステップ

単位ステップは

$$u[n] = \begin{cases} 1 & (n \geq 0) \\ 0 & (n < 0) \end{cases}$$

で定義されます。

信号がある時刻から立ち上がる状況や、累積和のような振る舞いを考えるときに役立ちます。

13.4.3 正弦波

離散時間の正弦波は

$$x[n] = A \sin(\omega_0 n + \phi)$$

と書けます。

- A は振幅
- ω_0 は離散時間角周波数
- ϕ は初期位相

です。

連続時間での周波数 f Hz とサンプリング周波数 f_s Hz の関係は

$$\omega_0 = 2\pi \frac{f}{f_s}$$

です。この式から、離散時間での正弦波の見え方は、元の周波数 f だけでなくサンプリング周波数 f_s にも依存することが分かります。

13.5 基本操作

13.5.1 時間シフト

信号を n_0 サンプルだけ遅らせる操作は

$$y[n] = x[n - n_0]$$

です。逆に $x[n + n_0]$ は n_0 サンプルだけ進めた信号です。

離散時間では、シフト量は通常整数サンプルで扱います。音響では遅延、画像では平行移動に対応する基本操作です。

13.5.2 反転

$$y[n] = x[-n]$$

は時間反転です。畳み込みを理解する際に必ず現れる操作なので、単なる見た目の変換としてではなく、後で出てくる演算の一部として覚える必要があります。

13.5.3 振幅スケーリング

$$y[n] = ax[n]$$

は振幅を a 倍する操作です。 $|a| > 1$ なら増幅、 $|a| < 1$ なら減衰です。 $a < 0$ なら上下反転も同時に起こります。

13.6 エネルギーと平均電力

信号の大きさを要約する尺度として、**エネルギー**と**平均電力**があります。

離散時間信号 $x[n]$ のエネルギーを

$$E = \sum_{n=-\infty}^{\infty} |x[n]|^2$$

と定義します。

有限長の信号や、時間とともに十分小さくなる信号ではエネルギーが有限になります。一方で、ずっと振動し続ける正弦波のような信号は、無限に長く観測するとエネルギーが発散します。

そのような信号には平均電力

$$P = \lim_{N \rightarrow \infty} \frac{1}{2N+1} \sum_{n=-N}^N |x[n]|^2$$

を使います。

有限長パルスはエネルギー信号、定常的に続く正弦波は電力信号として扱うのが典型です。どちらを使うべきかは、対象が「有限のイベント」なのか「定常的に続く現象」なのかで決まります。

13.7 Python で基本信号を確かめる

以下のコードは、正弦波とノイズを生成し、サンプル番号と実時間の両方を意識して可視化する最小例です。

```

1  from pathlib import Path
2
3  import matplotlib.pyplot as plt
4  import numpy as np
5
6  output_dir = Path("outputs/ch13")
7  output_dir.mkdir(parents=True, exist_ok=True)
8
9  fs = 16000
10 duration = 0.01
11 n = np.arange(int(fs * duration))
12 t = n / fs
13
14 clean = np.sin(2 * np.pi * 440 * t)
15 rng = np.random.default_rng(0)
16 noise = 0.1 * rng.standard_normal(size=n.shape)
17 observed = clean + noise
18
19 plt.figure(figsize=(8, 3))
20 plt.plot(n, observed, label="observed", linewidth=1)
21 plt.plot(n, clean, label="clean", linewidth=2)
22 plt.xlabel("sample_index_n")
23 plt.ylabel("amplitude")
24 plt.legend()
25 plt.tight_layout()
26 plt.savefig(output_dir / "signal_and_noise.png", dpi=150)
27 plt.close()

```

このコードで確認したい点は 2 つあります。

- n を横軸にすると離散信号は数列として見える
- $t = n/f_s$ を使えば、同じ信号を実時間軸でも解釈できる

モデルに入力する直前のデータが「何番目のサンプルか」だけでなく「何秒に対応するか」まで追えるようになると、フィルタ設計、窓長の設定、特徴量抽出の妥当性を判断しやすくなります。

13.8 この章で押さえるべき点

- 離散時間信号 $x[n]$ は、サンプル番号に対応づけられた数列である
- サンプリング周波数 f_s がなければ、信号の時間的な意味は確定しない
- インパルス、ステップ、正弦波は後の解析の基準になる基本信号である
- シフト、反転、スケーリングは畳み込みやフィルタ理解の前提になる
- 有限長信号と定常信号では、エネルギーと平均電力のどちらを見るべきかが異なる

次章では、正弦波を基底として信号を周波数ごとに分解する考え方、すなわちスペクトル解析を導入します。

第 14 章

スペクトル解析の基礎

14.1 なぜ周波数領域で考えるのか

時間領域で見た信号は、「いつ大きいか」は分かっても、「どのような速さで変動しているか」は必ずしも直感的ではありません。例えば同じ音声波形でも、

- ゆっくり変化する成分
- 細かく振動する成分
- 特定の周波数だけが強い成分

を分けて見たいことがあります。

このとき役立つのが**スペクトル解析**です。スペクトル解析では、信号を複数の正弦波成分の重ね合わせとして見て、各周波数成分がどれだけ含まれているかを調べます。

音響では音の高さや倍音構造、画像では周期的な模様やエッジ方向、機械学習では特徴抽出の前処理として重要です。

14.2 正弦波は分解の基底になる

長さ N の離散信号を考えると、

$$\exp\left(j2\pi\frac{k}{N}n\right)$$

の形をした複素指数関数を基底として使うと、信号を周波数成分に分解できます。

ここで

- $n = 0, 1, \dots, N - 1$ はサンプル番号
- $k = 0, 1, \dots, N - 1$ は周波数ビン番号
- j は虚数単位

です。

複素数を使うと一見難しく見えますが、本質は「余弦と正弦をまとめて扱うための便利な表記」です。実際、

$$\exp(j\theta) = \cos\theta + j\sin\theta$$

が成り立つので、複素指数関数は余弦波と正弦波の組を表しています。

異なる k に対応する基底同士は直交するため、ある周波数成分がどれだけ含まれるかを**内積による射影**として取り出せます。この「直交基底への分解」が、スペクトル解析の計算的な核心です。

14.3 離散フーリエ変換

長さ N の信号 $x[n]$ に対する**離散フーリエ変換 (DFT)** を

$$X[k] = \sum_{n=0}^{N-1} x[n] \exp\left(-j2\pi \frac{k}{N}n\right)$$

と定義します。

$X[k]$ は一般に複素数です。その大きさ $|X[k]|$ は、 k 番目の周波数成分の強さに対応し、偏角 $\angle X[k]$ は位相に対応します。

逆変換は

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \exp\left(j2\pi \frac{k}{N}n\right)$$

で与えられます。すなわち DFT は、信号を周波数成分へ分解するだけでなく、その情報から元の信号を復元できる可逆変換です。

14.4 周波数ビンと実際の周波数

DFT の添字 k はそのまま Hz ではありません。サンプリング周波数を f_s Hz とすると、 k 番目のビンが表す周波数は

$$f_k = \frac{k}{N} f_s$$

です。

例えば $f_s = 16000$ Hz, $N = 1024$ なら、ビン幅は

$$\Delta f = \frac{f_s}{N} = 15.625 \text{ Hz}$$

です。

ここから分かる重要な点は次の 2 つです。

- 観測長 N を長くすると周波数分解能は高くなる
- サンプリング周波数 f_s を上げると観測できる周波数帯域は広がる

実信号では、正の周波数側だけ見れば十分なことが多いです。特に実数値信号ではスペクトルに共役対称性があり、負の周波数側は正の周波数側から決まります。

14.5 DC 成分とナイキスト周波数

$k = 0$ の成分は **DC 成分**と呼ばれ、信号の平均値に対応します。信号全体が正に持ち上がっているか、平均 0 の周りで振動しているかはここで分かります。

また、離散時間信号で区別可能な最高周波数は

$$\frac{f_s}{2}$$

であり、これを**ナイキスト周波数**と呼びます。

もし元の連続時間信号にこの周波数を超える成分が含まれているのに、十分高い f_s でサンプリングしなければ、異なる高周波成分が低周波成分に見えてしまいます。これを**エイリアシング**と呼びます。

したがって、サンプリング前には必要に応じてアナログのローパスフィルタをかけ、観測したい帯域だけを残しておく必要があります。

14.6 スペクトルを見るときに注意

14.6.1 リークエージ

有限長の区間だけを切り出して DFT を取ると、信号がちょうどその区間内で周期的に閉じていない限り、1 つの周波数に集中していたはずのエネルギーが周辺ビンに広がって見えます。これを**スペクトルリークエージ**と呼びます。

14.6.2 窓関数

リークエージを和らげるために、切り出した信号に窓関数を掛けてから DFT を取ります。代表例には Hann 窓や Hamming 窓があります。

ただし窓を掛けると、ピークの鋭さや振幅の見え方も変わります。窓関数は「魔法の補正」ではなく、リークエージ低減と分解能低下のトレードオフとして理解する必要があります。

14.6.3 ゼロ埋め

信号の末尾に 0 を足して DFT 点数を増やすと、スペクトルの見た目は滑らかになります。しかし、これは**新しい情報を増やしているわけではありません**。ゼロ埋めは周波数軸の補間に近い処理であり、真の分解能を改善するものではないことに注意してください。

14.7 Python で DFT を確かめる

以下のコードでは、440 Hz と 1000 Hz の 2 つの正弦波を重ね合わせた信号を作り、FFT を使って振幅スペクトルを確認します。

```
1 from pathlib import Path
2
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 output_dir = Path("outputs/ch14")
7 output_dir.mkdir(parents=True, exist_ok=True)
8
9 fs = 16000
10 duration = 0.064
11 t = np.arange(int(fs * duration)) / fs
12 x = 0.8 * np.sin(2 * np.pi * 440 * t)
13 x += 0.3 * np.sin(2 * np.pi * 1000 * t)
14
15 window = np.hanning(len(x))
16 xw = x * window
17
18 X = np.fft.rfft(xw)
19 freq = np.fft.rfftfreq(len(xw), d=1 / fs)
20 amp = np.abs(X)
21
22 plt.figure(figsize=(8, 3))
23 plt.plot(freq, amp)
24 plt.xlim(0, 2000)
```

```
25 plt.xlabel("frequency [Hz]")
26 plt.ylabel("magnitude")
27 plt.tight_layout()
28 plt.savefig(output_dir / "spectrum.png", dpi=150)
29 plt.close()
```

この例で確認すべき点は次の通りです。

- 時間波形では見えにくい 2 つの周波数成分が、周波数領域ではピークとして見える
- 窓関数を掛けるかどうかでスペクトルの広がり方が変わる
- ‘np.fft.rfft’ と ‘np.fft.rfftfreq’ を使うと、実信号の正の周波数側だけを効率よく扱える

14.8 この章で押さえるべき点

- DFT は信号を周波数成分へ分解する可逆変換である
- スペクトルの横軸はビン番号ではなく、 $f_k = kf_s/N$ で実周波数に変換して読む
- $k = 0$ は平均値に対応する DC 成分である
- ナイキスト周波数を超える成分はエイリアシングを起こす
- リークエージ、窓関数、ゼロ埋めの意味を混同しない

次章では、入力信号に対して出力信号を返す**システム**の見方を導入し、線形時不変 (LTI) 系と畳み込みの関係を整理します。

第 15 章

LTI 系の基礎

15.1 システムとして信号処理を見る

これまでの章では、主に信号そのものを見てきました。本章では、信号を入力すると別の信号を出力する仕組みを**システム**として考えます。

システムを

$$y[n] = \mathcal{T}\{x[n]\}$$

と書くことにします。ここで $\mathcal{T}$ は変換規則そのものを表します。

例えば、

- ノイズを減らす平滑化フィルタ
- 画像のぼかしやエッジ抽出
- 音声の残響付与
- ニューラルネットワークの 1 層

は、いずれも広い意味で入力信号を出力信号へ写すシステムです。

その中でも特に重要なのが、**線形時不変 (Linear Time-Invariant; LTI) 系**です。LTI 系は解析しやすく、多くのフィルタや信号処理アルゴリズムの基礎になります。

15.2 線形性

システム $\mathcal{T}$ が**線形**であるとは、任意の信号 $x_1[n], x_2[n]$ と定数 a, b に対して

$$\mathcal{T}\{ax_1[n] + bx_2[n]\} = a\mathcal{T}\{x_1[n]\} + b\mathcal{T}\{x_2[n]\}$$

が成り立つことです。

これは

- 入力を足したとき、出力も足し合わせで表せる
- 入力を定数倍したとき、出力も同じ定数倍になる

という 2 つの性質をまとめたものです。

線形性があると、複雑な入力を簡単な成分に分けて考えられます。スペクトル解析や畳み込みが有効なのは、この分解可能性のおかげです。

15.3 時不変性

システム $\mathcal{T}$ が**時不変**であるとは、入力を n_0 サンプル遅らせたとき、出力も同じだけ遅れることです。式で書くと、

$$\mathcal{T}\{x[n - n_0]\} = y[n - n_0]$$

です。

この性質は、「いつ入力したか」ではなく「どのような形の入力か」にだけシステムが反応することを意味します。同じ拍手音でも、1 秒後に入れても 2 秒後に入れても、応答波形の形は変わらず位置だけがずれる、というのが時不変の直感です。

15.4 インパルス応答

LTI 系に単位インパルス $\delta[n]$ を入力したときの出力を

$$h[n] = \mathcal{T}\{\delta[n]\}$$

と定義し、これを**インパルス応答**と呼びます。

インパルス応答が重要なのは、LTI 系では任意の入力に対する出力を $h[n]$ だけで決められるからです。これは、任意の離散信号が

$$x[n] = \sum_{m=-\infty}^{\infty} x[m]\delta[n - m]$$

と表せることから分かります。

右辺は「各時刻 m に置かれたシフト付きインパルスを、重み $x[m]$ で足し合わせたもの」です。LTI 系にこの表現を入れると、線形性と時不変性によって

$$y[n] = \sum_{m=-\infty}^{\infty} x[m]h[n - m]$$

が得られます。

これが**畳み込み和**です。

15.5 畳み込み

離散時間の畳み込みを

$$y[n] = (x * h)[n] = \sum_{m=-\infty}^{\infty} x[m]h[n - m]$$

と書きます。

畳み込みの計算では、 $h[n]$ を反転して平行移動し、入力 $x[m]$ と重ね合わせながら総和を取る、という見方が有用です。公式だけ暗記するのではなく、

- 反転
- シフト
- 各点での積
- 総和

の流れを頭の中で再現できるようにしておく必要があります。

有限長の FIR フィルタなら、実際には和の範囲は有限個の項だけになります。例えば

$$h[0] = \frac{1}{3}, \quad h[1] = \frac{1}{3}, \quad h[2] = \frac{1}{3}$$

だけが非ゼロなら、

$$y[n] = \frac{x[n] + x[n-1] + x[n-2]}{3}$$

となり、3 点移動平均フィルタになります。

15.6 差分方程式による表現

LTI 系は、差分方程式で書かれることも多いです。例えば

$$y[n] = x[n] + 0.5x[n-1]$$

は FIR 系です。

一方、

$$y[n] = 0.8y[n-1] + x[n]$$

のように過去の出力を使う系は IIR 系です。

FIR は有限長のインパルス応答を持ち、実装や安定性の扱いが比較的簡単です。IIR は少ない係数で鋭い周波数特性を作りやすい一方で、安定性や初期条件の扱いが重要になります。

15.7 因果性と BIBO 安定性

15.7.1 因果性

実時間で動作するシステムでは、現在の出力が未来の入力に依存してはいけません。この性質を**因果性**と呼びます。

LTI 系が因果的であるためには、

$$h[n] = 0 \quad (n < 0)$$

である必要があります。つまり、インパルスを入れる前に応答が始まってはいけません。

15.7.2 BIBO 安定性

有界な入力に対して常に有界な出力が得られる性質を **BIBO 安定** と呼びます。離散時間 LTI 系では

$$\sum_{n=-\infty}^{\infty} |h[n]| < \infty$$

であれば BIBO 安定です。

直感的には、インパルス応答の総量が無限に大きくならないなら、入力を有限範囲に抑えたとき出力も暴走しない、ということです。

15.8 周波数応答

複素指数信号

$$x[n] = \exp(j\omega n)$$

を LTI 系に入れると、出力は同じ周波数のまま定数倍されます。すなわち

$$y[n] = H(e^{j\omega}) \exp(j\omega n)$$

と書けます.

ここで

$$H(e^{j\omega}) = \sum_{n=-\infty}^{\infty} h[n] \exp(-j\omega n)$$

を**周波数応答**と呼びます.

$|H(e^{j\omega})|$ は周波数ごとの増幅率, $\angle H(e^{j\omega})$ は位相のずれを表します. ローパスフィルタ, 高域強調, 帯域通過といった設計は, この周波数応答を狙って行います.

15.9 Python で移動平均と IIR を比べる

以下のコードでは, ノイズを含む信号に対して, FIR の移動平均と 1 次 IIR を適用します.

```

1  from pathlib import Path
2
3  import matplotlib.pyplot as plt
4  import numpy as np
5
6  output_dir = Path("outputs/ch15")
7  output_dir.mkdir(parents=True, exist_ok=True)
8
9  rng = np.random.default_rng(0)
10 n = np.arange(200)
11 x = np.sin(2 * np.pi * 0.03 * n) + 0.4 * rng.standard_normal(size=n.shape)
12
13 h = np.ones(5) / 5
14 y_fir = np.convolve(x, h, mode="same")
15
16 y_iir = np.zeros_like(x)
17 alpha = 0.8
18 for i in range(len(x)):
19     prev = y_iir[i - 1] if i > 0 else 0.0
20     y_iir[i] = alpha * prev + (1 - alpha) * x[i]
21
22 plt.figure(figsize=(8, 3))
23 plt.plot(n, x, label="input", alpha=0.5)
24 plt.plot(n, y_fir, label="FIR moving average", linewidth=2)
25 plt.plot(n, y_iir, label="first-order IIR", linewidth=2)
26 plt.xlabel("sample index n")
27 plt.ylabel("amplitude")
28 plt.legend()
29 plt.tight_layout()
30 plt.savefig(output_dir / "fir_vs_iir.png", dpi=150)
31 plt.close()

```

この例では, どちらも平滑化を行いますが, 性質は同じではありません.

- FIR 移動平均は有限個の過去サンプルだけを見る
- IIR は過去の出力を再帰的に使うので, より長い記憶を持つ
- 同じ「なめらかにする」処理でも, 遅延や立ち上がりの鋭さが異なる

この差は, 周波数応答を見るとさらに明確になります.

15.10 この章で押さえるべき点

- LTI 系は線形性と時不変性を持つシステムである
- LTI 系はインパルス応答 $h[n]$ だけで記述できる
- 任意入力への出力は畳み込み和 $y[n] = (x * h)[n]$ で与えられる
- 因果性は $h[n] = 0$ for $n < 0$, BIBO 安定性は $\sum_n |h[n]| < \infty$ と関係する
- 周波数応答 $H(e^{j\omega})$ を見ると、フィルタが各周波数をどう変えるか分かる

本章まで理解できれば、離散信号、スペクトル、LTI 系という信号処理の中核概念が一通りつながります。この基盤の上に、STFT、画像フィルタ、音響特徴量抽出、畳み込みニューラルネットワークの理解を積み上げていきます。

第 16 章

単回帰分析と重回帰分析

本章では、基礎的な機械学習手法として代表的な**単回帰分析**と**重回帰分析**の仕組みを、数式を用いて説明します。

また次章では、本章で紹介した数式を Python によるプログラミングで実装する例も紹介します。本章と次章を通じて、数学とプログラミングの結びつきを体験して理解することができます。

本チュートリアル主題であるディープラーニングの前に、単回帰分析と重回帰分析を紹介することには 2 つの理由があります。

1 つ目は、単回帰分析と重回帰分析の数学がニューラルネットワーク含めたディープラーニングの数学の基礎となるためです。

2 つ目は、単回帰分析のアルゴリズムを通して微分、重回帰分析のアルゴリズムを通して線形代数に関する理解を深めることができるためです。

機械学習手法は、**教師あり学習 (supervised learning)**、**教師なし学習 (unsupervised learning)**、**強化学習 (reinforcement learning)** に大別され、単回帰分析は教師あり学習に含まれます。

本チュートリアルで扱う多くの手法が教師あり学習です。

教師あり学習の中でも典型的な問題設定は 2 つに大別されます。

与えられた入力変数から、10 や 0.1 といった実数値を予測する **回帰 (regression)** と、「赤ワイン」、「白ワイン」といったカテゴリを予測する **分類 (classification)** の 2 つです。

単回帰分析は回帰を行うための手法であり、1 つの入力変数から 1 つの出力変数を予測します。

それに対し、重回帰分析は、複数の入力変数から 1 つの出力変数を予測します。

この両手法は教師あり学習であるため、訓練の際には、入力変数 x と目的変数 t がペアで準備されている必要があります。

回帰分析を行うアルゴリズムでは、以下の 3 ステップを順番に考えていきます。

- Step 1: モデルを決める
- Step 2: 目的関数を決める
- Step 3: 最適なパラメータを求める

16.1 単回帰分析

16.1.1 問題設定 (単回帰分析)

単回帰分析では、1 つの入力変数から 1 つの出力変数を予測します。

今回は身近な例として、部屋の広さ x から家賃 y を予測する問題を考えてみます。

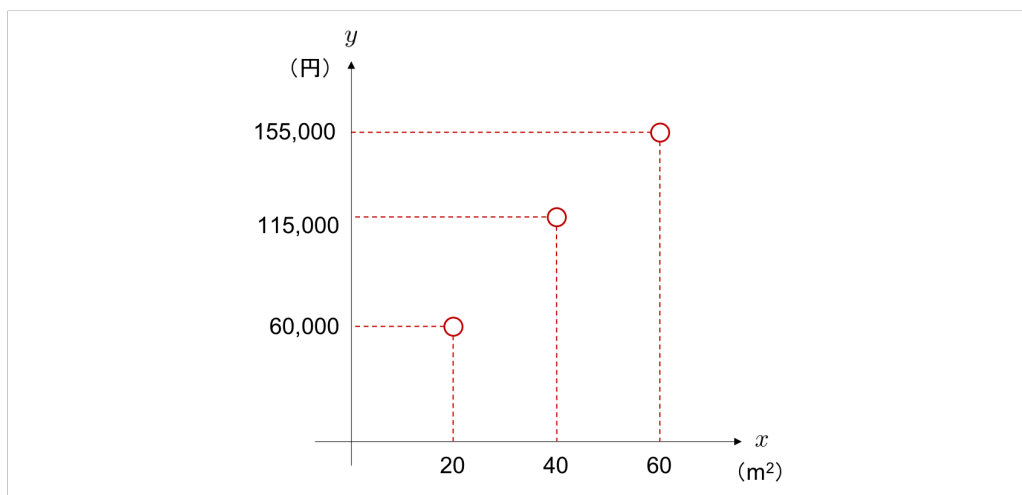


図 16.1 部屋の広さと家賃

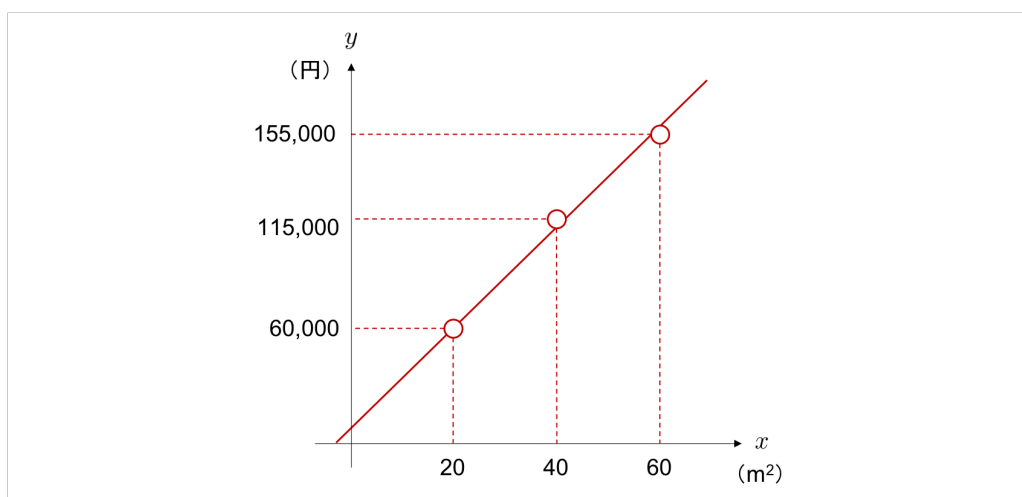


図 16.2 部屋の広さと家賃の関係

16.1.2 Step 1: モデルを決める (単回帰分析)

まずはじめに、入力変数 x と出力変数 y との関係をどのように定式化するかを決定します。

この定式化したものを **モデル** もしくは **数理モデル** と呼びます。

単回帰分析におけるモデルを具体的に考えていきましょう。

例えば、家賃と部屋の広さの組で表されるデータを 3 つ集め、「家賃」を y 軸に、「部屋の広さ」を x 軸にとってそれらをプロットしたとき、次のようになっていたとします。

この場合、部屋が広くなるほど、家賃が高くなるという関係が予想されます。

また、この 2 変数間の関係性は直線によって表現を行うことができそうだと考えられます。

そこで、2 つのパラメータ w と b によって特徴づけられる直線の方程式

$$y = wx + b$$

によって、部屋の広さと家賃の関係を表すことを考えます。

ここで、 w は **重み (weight)**、 b は **バイアス (bias)** の頭文字を採用しています。

単回帰分析では、このようにモデルとして直線 $y = wx + b$ を用います。

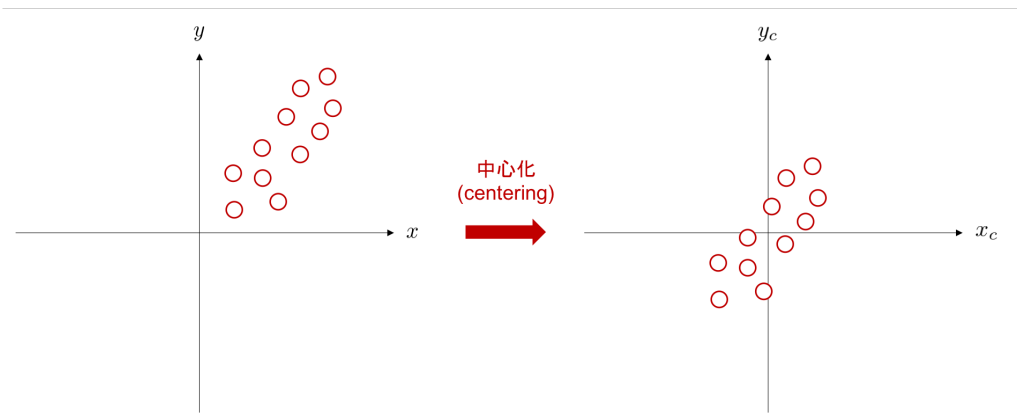


図 16.3 データの中心化

そして、2つのパラメータ w と b を、直線がデータによくフィットするように調整します。

パラメータで特徴づけられたモデルを用いる場合、与えられた **データセット** に適合するように最適なパラメータを求めることが目標となります。

今回はデータセットとして部屋の広さ x と家賃 t の組からなるデータの集合を用います。

全部で N 個のデータがあり、 n 番目のデータが (x_n, t_n) と表されるとき、データセットは

$$\mathcal{D} = \{(x_1, t_1), (x_2, t_2), \dots, (x_N, t_N)\} \quad (16.1)$$

$$= \{(x_n, t_n)\}_{n=1}^N \quad (16.2)$$

と表すことができます (注釈 1)。

これを用いて、新しい x を入力すると、それに対応する t を予測するモデルを訓練します。

16.1.3 前処理

次のステップに進む前に、データの**前処理 (preprocessing)** をひとつ紹介します。

データの **中心化 (centering)** は、データの平均値が 0 になるように全てのデータを平行移動する処理を言います。

下図は、データ集合 (x_n, y_n) ($n = 1, \dots, 11$) を、平均が $(0, 0)$ になるように平行移動する例です。

中心化によるデータ前処理の利点の一つとして、調整すべきパラメータを削減できることが挙げられます。中心化処理を行うことで切片を考慮する必要がなくなるため、データ間の関係性を表現する直線の方程式を、 $y_c = wx_c$ のように、簡潔に表現可能となります。

データセット内の入力変数と目的変数の平均をそれぞれ $\bar{x}$, $\bar{t}$ としたとき、中心化後の入力変数と目的変数は、

$$x_c = x - \bar{x}$$

$$t_c = t - \bar{t}$$

となります。

以降は記述を簡単にするため、 $_c$ という添え字を省略し、事前に中心化を行っている前提でデータを扱います。

また、そのデータにフィットさせたいモデルは、

$$y = wx$$

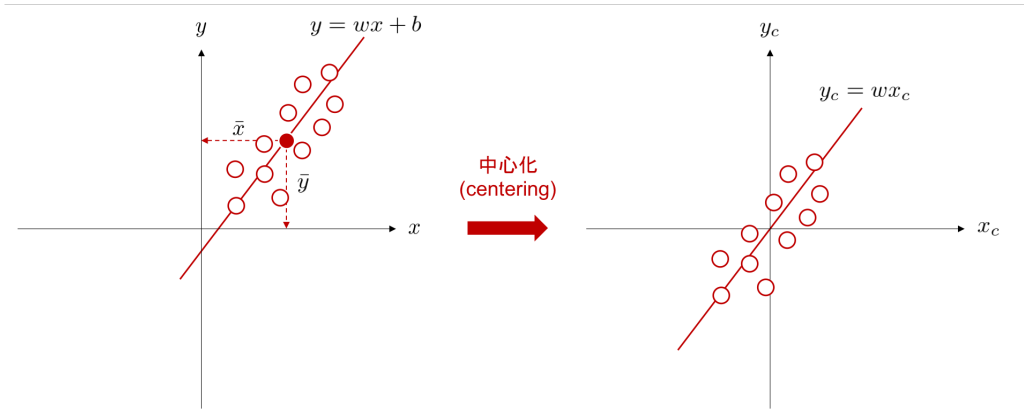


図 16.4 中心化によるパラメータ削減

と、こちらも添え字 c を省略して説明を行います。

16.1.4 Step 2：目的関数を決める（単回帰分析）

ここでの目標は、部屋の広さと家賃の関係を直線の方程式によってモデル化することです。

このために、予め収集されたいくつかのデータセットを使って、モデルが部屋の広さの値から予測した家賃（予測値）と、その部屋の広さに対応する実際の家賃（目標値）の差が小さくなるように、モデルのパラメータを決定します。

今回は、目的関数として [こちらの章](#) ですでに紹介した予測値と目標値との**二乗和誤差** (sum-of-squares error) を用います。

二乗和誤差が 0 のとき、またその時のみ予測値 y は目標値 t と完全に一致 ($t = y$) しています。

n 個目のデータの部屋の広さ x_n が与えられたときのモデルの予測値 y_n と、対応する目標値 t_n との間の二乗誤差は、

$$(t_n - y_n)^2$$

となります。これを全データに渡って合計したものが以下の二乗和誤差です。

$$\begin{aligned} L &= (t_1 - y_1)^2 + (t_2 - y_2)^2 + \cdots + (t_N - y_N)^2 \\ &= \sum_{n=1}^N (t_n - y_n)^2 \end{aligned}$$

今回用いるモデルは

$$y_n = wx_n$$

であるため、目的関数は

$$L = \sum_{n=1}^N (t_n - wx_n)^2$$

と書くこともできます。

16.1.5 Step 3：最適なパラメータを求める（単回帰分析）

この目的関数を最小化するようなパラメータを求めます。

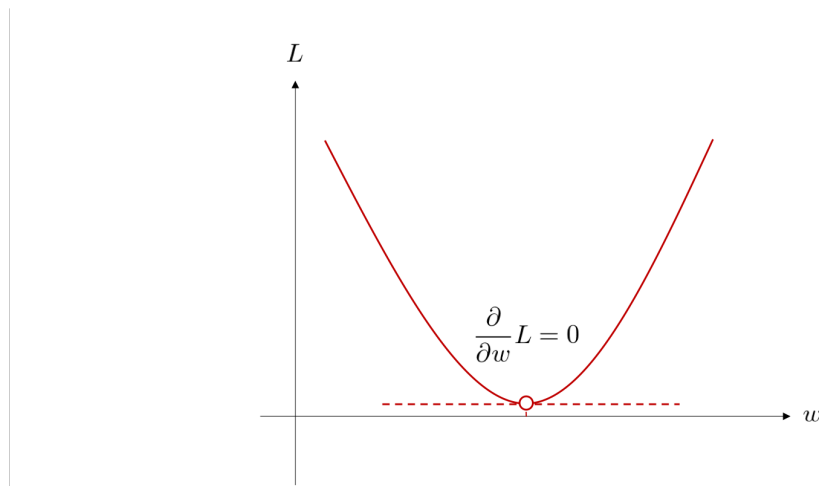


図 16.5 目的関数を最小化するパラメータ

ここで、目的関数は差の二乗和であり、常に正の値または 0 をとるような、下に凸な二次関数となっています。

(一般的には多くの場合において、最適なパラメータを用いてもモデルがすべてのデータを完全に表現できず、目的関数の値は 0 にはなりません。)

目的関数の値が最小となる点を求める際には、微分の知識が有用です。

微分では、対象とする関数の接線の傾きを求めることができます。凸関数では、この接線の傾きが 0 である点において、関数の最小値、もしくは最大値が得られます。

今回は、目的関数が x に関する二次関数となっているため、下図のように重み w に関する接線の傾きが 0 であるときに、目的関数の値が最小となります。

それでは、具体的に今回定めた目的関数 L をパラメータである w で微分してみましょう。

微分に関する基本的な計算や性質は [こちらの章](#)で紹介しました。

$$\frac{\partial}{\partial w} L = \frac{\partial}{\partial w} \sum_{n=1}^N (t_n - wx_n)^2$$

ここで、微分の**線形性**から、和の微分は、微分の和となるため、

$$\frac{\partial}{\partial w} L = \sum_{n=1}^N \frac{\partial}{\partial w} (t_n - wx_n)^2$$

と変形できます。

次に、総和 ($\sum$) の中の各項に着目すると、

$$\frac{\partial}{\partial w} (t_n - wx_n)^2$$

となっており、この部分は $t_n - wx_n$ と $(\cdot)^2$ という関数の**合成関数**になっています。

そこで、 $u_n = t_n - wx_n$ 、 $f(u_n) = u_n^2$ とおいて計算すると、

$$\begin{aligned} \frac{\partial}{\partial w} (t_n - wx_n)^2 &= \frac{\partial}{\partial w} f(u_n) \\ &= \frac{\partial u_n}{\partial w} \frac{\partial f(u_n)}{\partial u_n} \\ &= -x_n (2u_n) \\ &= -2x_n (t_n - wx_n) \end{aligned}$$

が得られます。

これを $\partial L / \partial w$ の式に戻すと、

$$\begin{aligned}\frac{\partial}{\partial w} L &= \sum_{n=1}^N \frac{\partial}{\partial w} (t_n - wx_n)^2 \\ &= - \sum_{n=1}^N 2x_n(t_n - wx_n)\end{aligned}$$

となります。

この導関数の値が 0 となるときの w が、目的関数を最小にするパラメータです。

そこで、 $\frac{\partial}{\partial w} L = 0$ とおいてこれを w について解きます。

$$\begin{aligned}\frac{\partial}{\partial w} L &= 0 \\ -2 \sum_{n=1}^N x_n(t_n - wx_n) &= 0 \\ -2 \sum_{n=1}^N x_n t_n + 2 \sum_{n=1}^N wx_n^2 &= 0 \\ -2 \sum_{n=1}^N x_n t_n + 2w \sum_{n=1}^N x_n^2 &= 0 \\ w \sum_{n=1}^N x_n^2 &= \sum_{n=1}^N x_n t_n\end{aligned}$$

以上より、

$$w = \frac{\sum_{n=1}^N x_n t_n}{\sum_{n=1}^N x_n^2}$$

と求まりました。これを最適な w と呼びます。

この値は、与えられたデータセット $\mathcal{D} = \{x_n, t_n\}_{n=1}^N$ のみから決定されています。

16.1.6 数値例

例題にあげていた数値でパラメータ w を求めてみましょう。

まずはデータの中心化を行うために、平均の値を事前に算出します。

$$\begin{aligned}\bar{x} &= \frac{1}{3}(20 + 40 + 60) = 40 \\ \bar{t} &= \frac{1}{3}(60000 + 115000 + 155000) = 110000\end{aligned}$$

この平均の値を使い、全変数に対して中心化を行うと、

$$\begin{aligned}x_1 &= 20 - 40 = -20 \\ x_2 &= 40 - 40 = 0 \\ x_3 &= 60 - 40 = 20 \\ t_1 &= 60000 - 110000 = -50000 \\ t_2 &= 115000 - 110000 = 5000 \\ t_3 &= 155000 - 110000 = 45000\end{aligned}$$

となります。

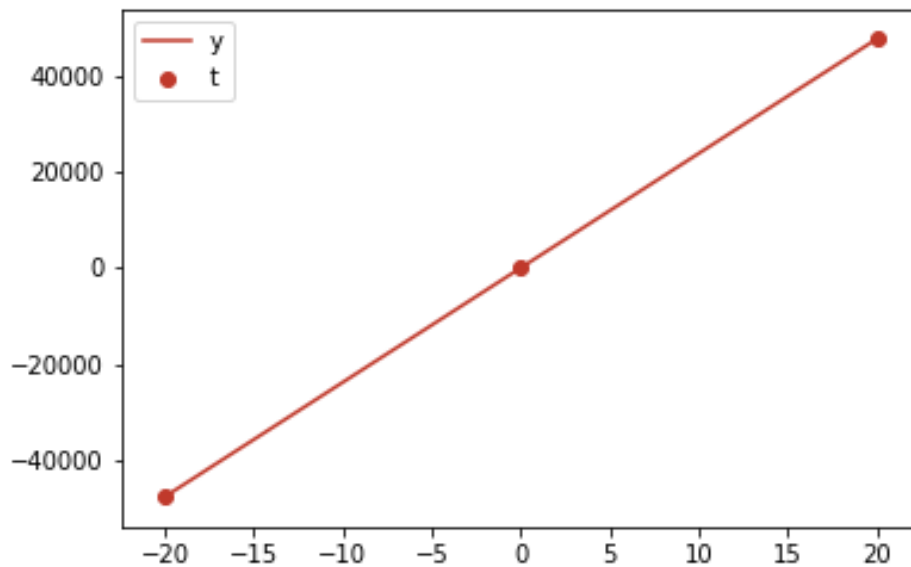


図 16.6 家賃の予測値と目標値（中心化後）

これらの中心化後の値を用いて、最適なパラメータ w を計算すると、

$$\begin{aligned}
 w &= \frac{\sum_{n=1}^N x_n t_n}{\sum_{n=1}^N x_n^2} \\
 &= \frac{x_1 t_1 + x_2 t_2 + x_3 t_3}{x_1^2 + x_2^2 + x_3^2} \\
 &= \frac{-20 \times (-50000) + 0 \times 5000 + 20 \times 45000}{(-20)^2 + 0^2 + 20^2} \\
 &= 2375
 \end{aligned}$$

と求められます。

したがって、家賃が 1 m^2 増えるごとに、2375 円家賃が上昇しているとわかります。

この w を用いて決定される直線 $y = 2375x$ と、学習データとして用いた 3 つの点をプロットした図が以下です。

この直線上の点の y の値が、対応する x の値に対するここで訓練したモデルによる予測値です。

ここで、 x 軸で負の値をとっていますが、これは中心化後であることに注意してください。

訓練済みのモデルを使って新しいサンプル x_q に対する家賃の予測を行います。

例えば、部屋の広さが 50 m^2 の場合の家賃に対する予測値を計算する推論を行いましょう。

$$\begin{aligned}
 y_c &= w x_c \\
 y_q - \bar{t} &= w(x_q - \bar{x}) \\
 \Rightarrow y_q &= w(x_q - \bar{x}) + \bar{t} \\
 &= 2375(50 - 40) + 110000 \\
 &= 133750
 \end{aligned}$$

このように、部屋の広さが 50 m^2 の場合の家賃が 133,750 円であると予測することができました。

上記のように、モデル化の際に中心化を行っていた処理を推論の際には元に戻して計算しましょう。

16.2 重回帰分析

16.2.1 問題設定（重回帰分析）

重回帰分析も単回帰分析の場合と同様に家賃を予測する問題を題材にして説明します。

単回帰分析の場合と異なり、入力変数として「部屋の広さ」だけでなく、「駅からの距離」や「犯罪発生率」などの変数を合わせて考慮することにします。

部屋の広さを x_1 、駅からの距離を x_2 、…、犯罪発生率を x_M といった形で表し、 M 個の入力変数を扱うことを考えてみましょう。

16.2.2 Step 1：モデルを決める（重回帰分析）

単回帰分析では、

$$y = wx + b$$

のように直線の方程式をモデルとして用いていました。重回帰分析でも、

$$y = w_1x_1 + w_2x_2 + \cdots + w_Mx_M + b$$

のように、単回帰分析と似た形のモデルを定義します。

単回帰分析の際は二次元平面を考え、その平面上に存在するデータに最もよくフィットする直線を考えましたが、今回は M 次元空間に存在するデータに最もよくフィットする直線を考えることになります。

重回帰分析のモデルは総和の記号を使って表記すると、

$$y = \sum_{m=1}^M w_mx_m + b$$

と書くことができます。

ここでバイアス b の扱い方を改めて考えます。

単回帰分析では、中心化を前処理として施し、バイアス b を省略することで、簡潔に定式化することができました。

重回帰分析では、 M 個の重み $w_1, w_2, \dots, w_M$ と 1 個のバイアス b があり、合わせて $M+1$ 個のパラメータが存在します。これらのパラメータをうまく定式化することを考えます。

そこで、今回は $x_0 = 1$ 、 $w_0 = b$ とおくことで、

$$\begin{aligned} y &= w_1x_1 + w_2x_2 + \cdots + w_Mx_M + b \\ &= w_1x_1 + w_2x_2 + \cdots + w_Mx_M + w_0x_0 \\ &= w_0x_0 + w_1x_1 + \cdots + w_Mx_M \\ &= \sum_{m=0}^M w_mx_m \end{aligned}$$

のように b を総和の内側の項に含めて、簡潔に表記できるようにします。

（ここで、 $\sum$ 記号の下部が $m=1$ ではなく $m=0$ となっていることに注意してください。）

さらに、ここから線形代数で学んだ知識を活かして、数式を整理していきます。

上式をベクトルの内積を用いて表記しなおすと、

$$\begin{aligned}
 y &= w_0 x_0 + w_1 x_1 + \cdots + w_M x_M \\
 &= \begin{bmatrix} w_0 & w_1 & \cdots & w_M \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_M \end{bmatrix} \\
 &= \mathbf{w}^T \mathbf{x}
 \end{aligned}$$

のように、シンプルな形式で表現することができます。

このモデルが持つパラメータは前述の通り $M+1$ 個あり、 $M+1$ 次元のベクトル $\mathbf{w}$ で表されています。重回帰分析では、この $\mathbf{w}$ のすべての要素について最適値を求めます。

16.2.3 Step 2：目的関数を決める（重回帰分析）

単回帰分析の例と比べると、入力変数は増えましたが、家賃を目標値としている点は変わっていません。そこで、単回帰分析と同じ目的関数

$$L = (t_1 - y_1)^2 + (t_2 - y_2)^2 + \cdots + (t_N - y_N)^2$$

を用います。

この目的関数は、ベクトルの内積を用いて表記し直すと、

$$\begin{aligned}
 L &= (t_1 - y_1)^2 + (t_2 - y_2)^2 + \cdots + (t_N - y_N)^2 \\
 &= \begin{bmatrix} t_1 - y_1 & t_2 - y_2 & \cdots & t_N - y_N \end{bmatrix} \begin{bmatrix} t_1 - y_1 \\ t_2 - y_2 \\ \vdots \\ t_N - y_N \end{bmatrix} \\
 &= (\mathbf{t} - \mathbf{y})^T (\mathbf{t} - \mathbf{y})
 \end{aligned}$$

と書くことができます。

ここで、内積には交換法則が成り立つため、 $\mathbf{w}^T \mathbf{x}$ は $\mathbf{x}^T \mathbf{w}$ と書くこともできます。これを利用して、モデルの方程式 $\mathbf{y} = \mathbf{w}^T \mathbf{x}$ を、以下のように変形します。

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1^T \mathbf{w} \\ \mathbf{x}_2^T \mathbf{w} \\ \vdots \\ \mathbf{x}_N^T \mathbf{w} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix} \mathbf{w}$$

さらに、 $\mathbf{x}_n^T = [x_{n0}, x_{n1}, x_{n2}, \dots, x_{nM}]$ ($n = 1, \dots, N$) と展開すると、

$$\begin{aligned}
 \mathbf{y} &= \begin{bmatrix} x_{10} & x_{11} & x_{12} & \cdots & x_{1M} \\ x_{20} & x_{21} & x_{22} & \cdots & x_{2M} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{N0} & x_{N1} & x_{N2} & \cdots & x_{NM} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_M \end{bmatrix} \\
 &= \mathbf{X} \mathbf{w}
 \end{aligned}$$

と表記できます。

ここで、 $N \times M$ 行列 $\mathbf{X}$ は、各行が各データを表しており、各列が各入力変数を表しています。

このような行列を、**デザイン行列 (design matrix)** と呼びます。

各列はそれぞれ入力変数の種類に対応しており、例えば、部屋の広さや、駅からの距離などです。

各行が表すデータ点がどのように表されているかを説明するため、具体的な数値例を挙げます。

部屋の広さ = 50m²、駅からの距離 = 600m、犯罪発生率 = 2% という 3 つの入力変数を考える場合、 $M = 3$ であり、これが n 個目のデータのとき、 $\mathbf{x}_n^T$ は、

$$\mathbf{x}_n^T = [1 \quad 50 \quad 600 \quad 0.02]$$

となります。先頭に 1 があるのは、Step 1 で $x_0 = 1$ と定めたためです。

16.2.4 Step 3：パラメータを最適化する（重回帰分析）

それでは、目的関数 L を最小化するモデルのパラメータベクトル $\mathbf{w}$ を求めましょう。

単回帰分析と同様に、目的関数をパラメータで微分して 0 とおき、 $\mathbf{w}$ について解きます。

まずは目的関数に登場している予測値 $\mathbf{y}$ を、パラメータ $\mathbf{w}$ を用いた表記に置き換えます。

$$\begin{aligned} L &= (\mathbf{t} - \mathbf{y})^T (\mathbf{t} - \mathbf{y}) \\ &= (\mathbf{t} - \mathbf{X}\mathbf{w})^T (\mathbf{t} - \mathbf{X}\mathbf{w}) \\ &= \{\mathbf{t}^T - (\mathbf{X}\mathbf{w})^T\} (\mathbf{t} - \mathbf{X}\mathbf{w}) \\ &= (\mathbf{t}^T - \mathbf{w}^T \mathbf{X}^T) (\mathbf{t} - \mathbf{X}\mathbf{w}) \end{aligned}$$

ここで、転置の公式 $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$ を用いました。

さらに分配法則を使って展開すると、

$$L = \mathbf{t}^T \mathbf{t} - \mathbf{t}^T \mathbf{X}\mathbf{w} - \mathbf{w}^T \mathbf{X}^T \mathbf{t} + \mathbf{w}^T \mathbf{X}^T \mathbf{X}\mathbf{w}$$

となります。この目的関数に対しパラメータの $\mathbf{w}$ についての偏微分を計算します。

その前に、この式をもう少し整理します。

まず、 $(1)^T = 1$ のように、スカラは転置しても変化しません。

また、 $\mathbf{w} \in \mathbb{R}^{M+1}$ 、 $\mathbf{X} \in \mathbb{R}^{N \times (M+1)}$ であり、 $\mathbf{X}\mathbf{w} \in \mathbb{R}^N$ となることから、 $\mathbf{t} \in \mathbb{R}^N$ との間の内積 $\mathbf{t}^T \mathbf{X}\mathbf{w} \in \mathbb{R}$ は、スカラになります。

したがって、

$$(\mathbf{t}^T \mathbf{X}\mathbf{w})^T = \mathbf{t}^T \mathbf{X}\mathbf{w}$$

が成り立ちます。

さらに、転置の公式 $(\mathbf{ABC})^T = \mathbf{C}^T \mathbf{B}^T \mathbf{A}^T$ より、

$$(\mathbf{t}^T \mathbf{X}\mathbf{w})^T = \mathbf{w}^T \mathbf{X}^T \mathbf{t}$$

も成り立ちます。以上から、

$$(\mathbf{t}^T \mathbf{X}\mathbf{w})^T = \mathbf{t}^T \mathbf{X}\mathbf{w} = \mathbf{w}^T \mathbf{X}^T \mathbf{t}$$

が導かれます。目的関数 L は、この式を利用して、

$$L = \mathbf{t}^T \mathbf{t} - 2\mathbf{t}^T \mathbf{X}\mathbf{w} + \mathbf{w}^T \mathbf{X}^T \mathbf{X}\mathbf{w}$$

と変形できます。

ここで、 $\mathbf{w}$ に関する偏微分を行いやすくするため、 $\mathbf{w}$ 以外の定数項を一つにまとめます。

$$\begin{aligned}
L &= \mathbf{t}^T \mathbf{t} - 2\mathbf{t}^T \mathbf{X} \mathbf{w} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} \\
&= \mathbf{t}^T \mathbf{t} - 2(\mathbf{X}^T \mathbf{t})^T \mathbf{w} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} \\
&= c + \mathbf{b}^T \mathbf{w} + \mathbf{w}^T \mathbf{A} \mathbf{w}
\end{aligned}$$

すると、 $\mathbf{w}$ に関する二次形式で表現できました。

ここで、

$$\mathbf{A} = \mathbf{X}^T \mathbf{X} \quad (16.3)$$

$$\mathbf{b} = -2\mathbf{X}^T \mathbf{t} \quad (16.4)$$

$$c = \mathbf{t}^T \mathbf{t} \quad (16.5)$$

とおいっていることに注意してください。

それでは、目的関数を最小にするパラメータ $\mathbf{w}$ の求め方を考えます。

目的関数はパラメータ $\mathbf{w}$ に関して二次関数になっています。

まずは、 $\mathbf{w}$ 以外のベクトルや行列に、具体的な数値を当てはめて考えてみましょう。

例えば、

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, c = 1$$

とおきます。すると、目的関数の値は

$$\begin{aligned}
L &= \mathbf{w}^T \mathbf{A} \mathbf{w} + \mathbf{b}^T \mathbf{w} + c \\
&= \begin{bmatrix} w_1 & w_2 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} + \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} + 1 \\
&= \begin{bmatrix} w_1 & w_2 \end{bmatrix} \begin{bmatrix} w_1 + 2w_2 \\ 3w_1 + 4w_2 \end{bmatrix} + w_1 + 2w_2 + 1 \\
&= w_1(w_1 + 2w_2) + w_2(3w_1 + 4w_2) + w_1 + 2w_2 + 1 \\
&= w_1^2 + 5w_1w_2 + 4w_2^2 + w_1 + 2w_2 + 1
\end{aligned}$$

となります。これを w_1, w_2 に関して整理すると、

$$\begin{aligned}
L &= w_1^2 + (5w_2 + 1)w_1 + (4w_2^2 + 2w_2 + 1) \\
&= 4w_2^2 + (5w_1 + 2)w_2 + (w_1^2 + w_1 + 1)
\end{aligned}$$

となり、 w_1, w_2 それぞれについて二次関数になっていることが分かります。

目的関数 L を w_1 の二次関数、 w_2 の二次関数と見たとき、 L は、下図のような概形となっています。

さらに、各次元が w_1, w_2, L を表す 3 次元空間上においては、 L の概形は下図のようになっています。

ここでは 2 つのパラメータ w_1 と w_2 について図示していますが、目的関数が 任意の M 個の変数 $w_0, w_1, w_2, \dots, w_M$ によって特徴づけられている場合でも、目的関数がそれぞれのパラメータについて二次形式になっている限り、同様のことが言えます。

すなわち、 $M + 1$ 個の連立方程式、

$$\begin{cases} \frac{\partial}{\partial w_0} L = 0 \\ \frac{\partial}{\partial w_1} L = 0 \\ \vdots \\ \frac{\partial}{\partial w_M} L = 0 \end{cases}$$

を解けば良いということになります。

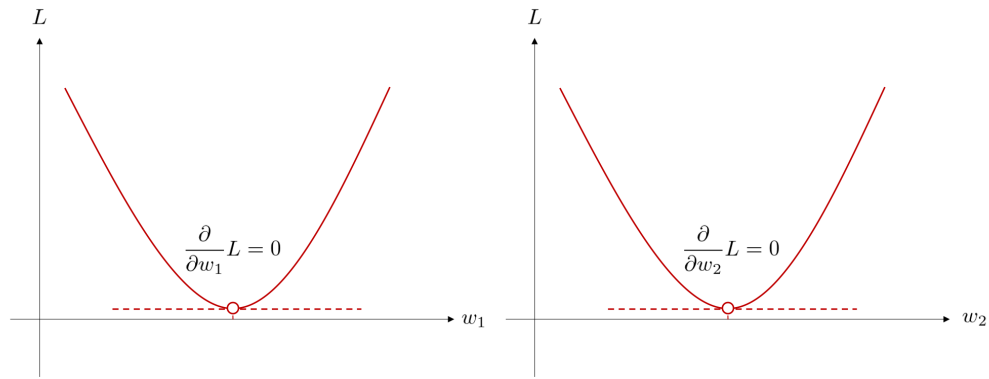


図 16.7 目的関数の概形 (2 次元)

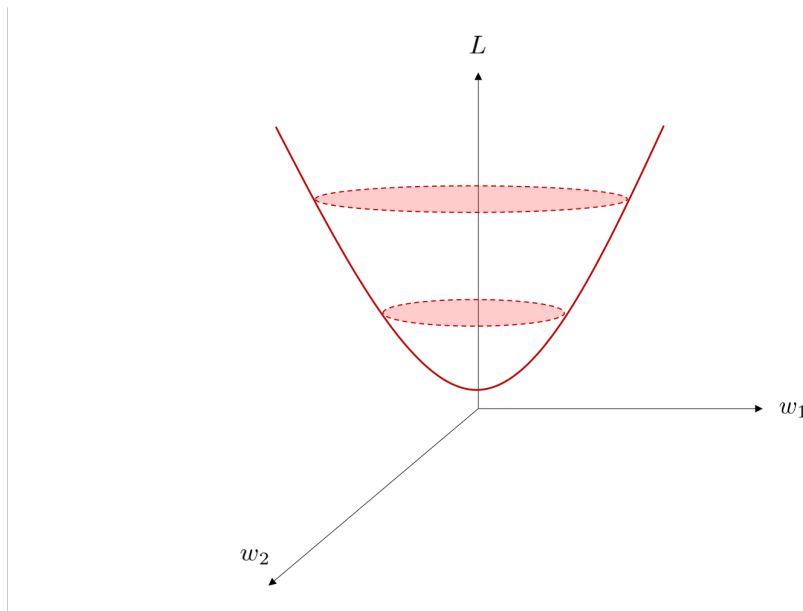


図 16.8 目的関数の概形 (3 次元)

これはベクトルによる微分を用いて表記すると、以下ようになります。

$$\begin{bmatrix} \frac{\partial}{\partial w_0} L \\ \frac{\partial}{\partial w_1} L \\ \vdots \\ \frac{\partial}{\partial w_M} L \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \\ \Rightarrow \frac{\partial}{\partial \mathbf{w}} L = \mathbf{0}$$

上式を $\mathbf{w}$ について解くために、以下のような式変形を行います。

式変形の途中で理解できない部分があった場合は、[こちらの章](#) を読み返してみてください。

まずは、左辺について整理を行います。

$$\begin{aligned} \frac{\partial}{\partial \mathbf{w}} L &= \frac{\partial}{\partial \mathbf{w}} (c + \mathbf{b}^T \mathbf{w} + \mathbf{w}^T \mathbf{A} \mathbf{w}) \\ &= \frac{\partial}{\partial \mathbf{w}} (c) + \frac{\partial}{\partial \mathbf{w}} (\mathbf{b}^T \mathbf{w}) + \frac{\partial}{\partial \mathbf{w}} (\mathbf{w}^T \mathbf{A} \mathbf{w}) \\ &= \mathbf{0} + \mathbf{b}^T + \mathbf{w}^T (\mathbf{A} + \mathbf{A}^T) \end{aligned}$$

これを 0 とおき、 $\mathbf{A}$ 、 $\mathbf{b}$ を展開すると

$$\begin{aligned} -2(\mathbf{X}^T \mathbf{t})^T + \mathbf{w}^T \{\mathbf{X}^T \mathbf{X} + (\mathbf{X}^T \mathbf{X})^T\} &= \mathbf{0} \\ -2\mathbf{t}^T \mathbf{X} + 2\mathbf{w}^T \mathbf{X}^T \mathbf{X} &= \mathbf{0} \\ \mathbf{w}^T \mathbf{X}^T \mathbf{X} &= \mathbf{t}^T \mathbf{X} \end{aligned}$$

のように式変形できます。

ここで両辺を転置して、 $\mathbf{w}^T$ の転置を直しておきましょう。

$$\begin{aligned} (\mathbf{w}^T \mathbf{X}^T \mathbf{X})^T &= (\mathbf{t}^T \mathbf{X})^T \\ \mathbf{X}^T \mathbf{X} \mathbf{w} &= \mathbf{X}^T \mathbf{t} \end{aligned}$$

ここで、 $\mathbf{X}^T \mathbf{X}$ に**逆行列が存在すると仮定**して、両辺に左側から $(\mathbf{X}^T \mathbf{X})^{-1}$ を掛けると、

$$\begin{aligned} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X} \mathbf{w} &= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t} \\ \mathbf{I} \mathbf{w} &= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t} \\ \mathbf{w} &= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t} \end{aligned}$$

が導かれます。特に、この最後の式を**正規方程式 (normal equation)** と呼びます。

上式は、与えられたデータを並べたデザイン行列 $\mathbf{X}$ と、各データの目標値を並べたベクトル $\mathbf{t}$ から、最適なパラメータ $\mathbf{w}$ を計算しています。

$\mathbf{I}$ は単位行列を表します。

$\mathbf{w}$ を求める際に気をつけたいこととして、以下の誤った式変形をしてしまう例が挙げられます。

$$\begin{aligned} \mathbf{X}^T \mathbf{X} \mathbf{w} &= \mathbf{X}^T \mathbf{t} \\ (\mathbf{X}^T)^{-1} \mathbf{X}^T \mathbf{X} \mathbf{w} &= (\mathbf{X}^T)^{-1} \mathbf{X}^T \mathbf{t} \\ \mathbf{X} \mathbf{w} &= \mathbf{t} \\ \mathbf{X}^{-1} \mathbf{X} \mathbf{w} &= \mathbf{X}^{-1} \mathbf{t} \\ \mathbf{w} &= \mathbf{X}^{-1} \mathbf{t} \end{aligned}$$

上記の式変形は一般には成立しません。

この式変形が可能かどうかは、 $\mathbf{X}^{-1}$ が存在するかどうか、に関わっています。

サンプルサイズ N と独立変数の数 $M+1$ が等しくない場合、 $\mathbf{X} \in \mathbb{R}^{N \times (M+1)}$ は**正方行列ではない**ため、逆行列 $\mathbf{X}^{-1}$ を持ちません。

従って、上式の 2 行目の変形を行うことはできません（逆行列が求まるためのより厳密な条件についてはここでは省略します）。

一方、 $\mathbf{X}^T \mathbf{X}$ は $(M+1) \times (M+1)$ 行列であり、その形はサンプルサイズ N に依存することなく、常に正方行列となるため、これを利用して式変形を行います。

新しい入力変数の値 $\mathbf{x}_q = [x_1, \dots, x_M]^T$ に対して、対応する目標値 y_q を予測するためには、訓練により決定されたパラメータ $\mathbf{w}$ を用いて、

$$y_q = \mathbf{w}^T \mathbf{x}_q$$

のように計算します。

16.3 注釈

16.3.1 注釈 1

データセット中の各データのことをデータ点 (datum) ということがあります。具体的には、上の説明で同上した $\mathcal{D}$ 中の各 (x_1, t_1) などのことです。

第 17 章

scikit-learn 入門

scikit-learn は Python のオープンソース機械学習ライブラリです。

様々な機械学習の手法が統一的なインターフェースで利用できるようになっています。

scikit-learn では NumPy の ndarray でデータやパラメータを取り扱うため、他のライブラリとの連携もしやすくなっています。

本章では、この scikit-learn というライブラリを用いて、**データを使ってモデルを訓練し、評価するという一連の流れを解説**し、Chainer を使ったディープラーニングの解説に入る前に、共通する重要な項目について説明します。

機械学習の様々な手法を用いる際には、**データを使ってモデルを訓練するまでに、以下の 5 つのステップ**がよく共通して現れます。

- Step 1: **データセットの準備**
- Step 2: **モデルを決める**
- Step 3: **目的関数を決める**
- Step 4: **最適化手法を選択する**
- Step 5: **モデルを訓練する**

前章では、**ステップ 2 → ステップ 3 → ステップ 4 & 5** という **3 ステップ**に分けて説明を行いました。この 5 つのステップに分けて考える方法は、後の章で解説する **Chainer を用いたニューラルネットワークの訓練においても共通しています**。

また、上記の 5 つが完了した後は、通常、訓練済みモデルによるテストデータを用いた精度検証を行います。この点も共通しています。

本章では、これらの 5 つのステップ + テストデータでの精度検証までを、scikit-learn の機能を使って簡潔に紹介します。

17.1 scikit-learn を用いた重回帰分析

前章で NumPy を用いて実装した重回帰分析を、scikit-learn を使ってより大きなデータセットに対し適用してみましょう。

17.1.1 Step 1: データセットの準備

本章では、前章までのような人工データではなく、米国ボストンの 506 の地域ごとの住環境の情報などと家賃の中央値の情報を収集して作られた Boston house prices dataset というデータセットを使用します。

このデータセットには 506 件のサンプルが含まれており、各サンプルは以下の情報を持っています。

属性名	説明
CRIM	人口 1 人あたりの犯罪発生率
ZN	25,000 平方フィート以上の住宅区画が占める割合
INDUS	小売業以外の商業が占める面積の割合
CHAS	チャールズ川の川沿いかどうか (0 or 1)
NOX	窒素酸化物の濃度
RM	住居の平均部屋数
AGE	1940 年より前に建てられた持ち主が住んでいる物件の割合
DIS	5 つのボストン雇用施設からの重み付き距離
RAD	環状高速道路へのアクセシビリティ指標
TAX	\$10,000 あたりの固定資産税率
PTRATIO	町ごとにみた教師 1 人あたりの生徒数
B	町ごとにみた黒人の比率を Bk としたときの $(Bk - 0.63)^2$ の値
LSTAT	給与の低い職業に従事する人口の割合
MEDV	物件価格の中央値

このデータセットを用いて、最後の MEDV 以外の 13 個の指標から、MEDV を予測する回帰問題に取り組んでみましょう。

このデータセットは、scikit-learn の `load_boston()` という関数を呼び出すことで読み込むことができます。

```
1 from sklearn.datasets import load_boston
2
3 dataset = load_boston()
```

読み込んだデータセットは、`data` という属性と `target` という属性を持っており、それぞれに入力値と目標値を並べた `ndarray` が格納されています。

これらを取り出して、それぞれ `x` と `t` という変数に格納しておきましょう。

```
1 x = dataset.data
2 t = dataset.target
```

入力値が格納されている `x` は、506 個の 13 次元ベクトルを並べたものになっています。形を確認してみましょう。

```
1 x.shape
```

```
(506, 13)
```

一方 `t` は、各データ点ごとに 1 つの値を持つため、506 次元のベクトルになっています。形を確認してみましょう。

```
1 t.shape
```

```
(506,)
```

データセットの分割

ここで、まずこのデータセットを 2 つに分割します。

それは、モデルの訓練に用いるためのデータと、訓練後のモデルのパフォーマンスをテストするために用いるデータは、異なるものになっている必要があるためです。

これは、[機械学習に使われる数学](#)の章で少しだけ触れた汎化性能というものに関わる重要なことです。

ここで、例え話を使ってなぜデータセットを分割する必要があるかを説明します。

例えば、大学受験の準備のために 10 年分の過去問を購入し、一部を**勉強のために**、一部を**勉強の成果をはかる**ために使用したいとします。

10 年分という限られた数の問題を使って、結果にある程度の信頼のおけるような方法で実力をチェックするには、下記の 2 つのうちどちらの方法がより良いでしょうか。

- 10 年分の過去問全てを使って勉強したあと、もう一度同じ問題を使って実力をはかる
- 5 年分の過去問だけを使って勉強し、残りの 5 年分の未だ見たことがない問題を使って実力をはかる

一度勉強した問題を再び解くことができると確認できても、大学受験の当日に未知の問題が出たときにどの程度対処できるかを事前にチェックするには不十分です。

よって、後者のような方法で数限られた問題を活用する方が、本当の実力をはかるには有効でしょう。

これは機械学習におけるモデルの訓練と検証でも同様に言えることです。

実力をつけるための勉強に使うデータの集まりを、訓練用データセット (training dataset) といい、実力をはかるために使うデータの集まりを、テスト用データセット (test dataset) と言います。

このとき、訓練用データセットとテスト用データセットに含まれるデータの間には、**重複がないようにします。**

早速、さきほど用意した x と t を、訓練用データセットとテスト用データセットに分割しましょう。

どのように分けるかには色々な方法がありますが、単純に全体の何割かを訓練用データセットとし、残りをテスト用データセットとする、といった分割を行う方法は**ホールドアウト法 (holdout method)**と呼ばれます。

scikit-learn では、データセットから指定された割合（もしくは個数）のデータをランダムに抽出して訓練用データセットを作成し、残りをテスト用データセットとする処理を行う関数が提供されています。

```
1 # データセットを分割する関数の読み込み
2 from sklearn.model_selection import train_test_split
3
4 # 訓練用データセットとテスト用データセットへの分割
5 x_train, x_test, t_train, t_test = train_test_split(x, t, test_size=0.3,
6                                                    random_state=0)
```

ここで、`train_test_split()` の `test_size` という引数に 0.3 を与えています。

これはテスト用データセットを全体の 30% のデータを用いて作成することを意味しています。

自動的に残りの 70% は訓練用データセットとなります。

上のコードは全サンプルの中から**ランダムに** 70% を訓練データとして抽出し、残った 30% をテストデータとして返します。

例えば、データセット中のサンプルが、目標値が 1 のサンプルが 10 個、2 のサンプルが 8 個、3 のサンプルが 12 個…というように、カテゴリごとにまとめられて並んでいることがあります。

そのとき、データセットの先頭から 18 個目のところで訓練データとテストデータに分割したとすると、訓練データには目標値が 3 のデータが 1 つも含まれないこととなります。

そこで、ランダムにデータセットを分割する方法が採用されています。

`random_state` という引数に毎回同じ整数を与えることで、実行のたびに結果が変わることを防いでいます。

それでは、分割後の訓練データを用いてモデルの訓練、精度の検証を行きましょう。

17.1.2 Step 2 4: モデル・目的関数・最適化手法を決める

scikit-learn で重回帰分析を行う場合は、`LinearRegression` クラスを使用します。

`sklearn.linear_model` 以下にある `LinearRegression` クラスを読み込んで、インスタンスを作成しましょう。

```
1 from sklearn.linear_model import LinearRegression
2
3 # モデルの定義
4 reg_model = LinearRegression()
```

上記のコードは、前述の 2 ～ 4 までのステップを行います。

`LinearRegression` は最小二乗法を行うクラスで、目的関数や最適化手法もあらかじめ内部で用意されたものが使用されます。

詳しくはこちらのドキュメントを参照してください。

参考: [sklearn.linear_model.LinearRegression](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html)

17.1.3 Step 5: モデルの訓練

次にモデルの訓練を行います。

scikit-learn に用意されている手法の多くは、共通して `fit()` というメソッドを持っており、再利用可能なコードが書きやすくなっています。

`reg_model` を用いて訓練を実行するには、`fit()` の引数に入力値 `x` と目標値 `t` を与えます。

```
1 # モデルの訓練
2 reg_model.fit(x_train, t_train)
```

```
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,
                  normalize=False)
```

モデルの訓練が完了しました。

求まったパラメータの値を確認してみましょう。

重回帰分析では、重み `w` とバイアス `b` の 2 つがパラメータでした。

求まった重み `w` の値は `model.coef_` に、バイアス `b` の値は `model.intercept_` に格納されています。

```
1 # 訓練後のパラメータ w
2 reg_model.coef_
```

```
array([-1.21310401e-01,  4.44664254e-02,  1.13416945e-02,  2.51124642e+00,
        -1.62312529e+01,  3.85906801e+00, -9.98516565e-03, -1.50026956e+00,
         2.42143466e-01, -1.10716124e-02, -1.01775264e+00,  6.81446545e-03,
        -4.86738066e-01])
```

```
1 # 訓練後のバイアス b
2 reg_model.intercept_
```

```
37.93710774183255
```

モデルの訓練が完了したら、精度の検証を行います。

`LinearRegression` クラスは `score()` メソッドを提供しており、入力値と目標値を与えると訓練済みのモデルを用いて計算した**決定係数 (coefficient of determination)** という指標を返します。

これは、使用するデータセットのサンプルサイズを N 、 n 個目の入力値に対する予測値を y_n 、目標値を t_n 、そしてそのデータセット内の全ての目標値の平均値を $\bar{t}$ としたとき、

$$R^2 = 1 - \frac{\sum_{n=1}^N (t_n - y_n)^2}{\sum_{n=1}^N (t_n - \bar{t})^2}$$

で表される指標です。

決定係数の最大値は 1 であり、値が大きいほどモデルが与えられたデータに当てはまっていることを表します。

```
1 # 精度の検証
2 reg_model.score(x_train, t_train)
```

```
0.7645451026942549
```

訓練済みモデルを用いて訓練用データセットで計算した決定係数は、およそ 0.765 でした。

17.1.4 新しい入力値に対する予測の計算 (推論)

訓練が終わったモデルに、新たな入力値を与えて、予測値を計算させるには、`predict()` メソッドを用います。

訓練済みのモデルを使ったこのような計算は、**推論 (inference)** と呼ばれることがあります。

今回は、訓練済みモデル `reg_model` を用いて、テスト用データセットからサンプルを 1 つ取り出し、推論を行ってみましょう。

このとき、`predict()` メソッドに与える入力値の `ndarray` の形が (**サンプルサイズ, 各サンプルの次元数**) となっている必要があることに気をつけてください。

```
1 reg_model.predict(x_test[:1])
```

```
array([24.9357079])
```

この入力に対する目標値を見てみましょう。

```
1 t_test[0]
```

```
22.6
```

22.6 という目標値に対して、およそ 24.94 という予測値が返ってきました。

17.1.5 テスト用データセットを用いた評価

それでは、訓練済みモデルの性能を、テスト用データセットを使って決定係数を計算することで評価してみましょう。

```
1 reg_model.score(x_test, t_test)
```

```
0.6733825506400194
```

訓練用データセットを用いて算出した値 (およそ 0.765) よりも、低い値がでてしまいました。

教師あり学習の目的は、訓練時には見たことがない新しいデータ、ここではテスト用データセットに含まれているデータに対しても、高い性能を発揮するように、モデルを訓練することです。

逆に、訓練時に用いたデータに対してはよく当てはまっていますが、訓練時に用いなかったデータに対しては予測値と目標値の差異が大きくなってしまいう現象を、**過学習 (overfitting)** と言います。

過学習を防ぐために、色々な方法が研究されています。

ここでは、データに前処理を行い、テスト用データセットを用いて計算した決定係数を改善します。

17.2 各ステップの改善

17.2.1 Step 1 の改善：前処理

前処理 (preprocessing) とは、欠損値の補完、外れ値の除去、特徴量選択、正規化などの処理を訓練を開始する前にデータセットに対して行うことです。

手法やデータに合わせた前処理が必要となるため、適切な前処理を行うためには手法そのものについて理解している必要があるだけでなく、使用するデータの特性についてもよく調べておく必要があります。

今回のデータは、入力値の値の範囲が CRIM, ZN, INDUS, ... といった指標ごとに大きく異なっています。

そこで、各入力変数ごとに平均が 0、分散が 1 となるように値をスケーリングする**標準化 (standardization)**をおこなってみましょう。

scikit-learn では `sklearn.preprocessing` というモジュール以下に `StandardScaler` というクラスが定義されています。

今回は、これを用いてデータセットに対し標準化を適用します。

それでは、`StandardScaler` クラスを読み込み、インスタンスを作成します。

```
1 from sklearn.preprocessing import StandardScaler
2
3 scaler = StandardScaler()
```

標準化を行うためには、データセットの各入力変数ごとの平均と分散の値を計算する必要があります。

この計算は、`scaler` オブジェクトがもつ `fit()` メソッドを用いて行います。

引数には、平均・分散を計算したい入力値の `ndarray` を渡します。

```
1 scaler.fit(x_train)
```

```
StandardScaler(copy=True, with_mean=True, with_std=True)
```

すべてのサンプルではなく、訓練用データセットのみを用いてこれらの値を算出します。

先ほどの `fit()` の実行の結果、算出された平均値が `mean_` 属性に、分散が `var_` 属性に格納されているので、確認してみましょう。

```
1 # 平均
2 scaler.mean_
```

```
array([3.35828432e+00, 1.18093220e+01, 1.10787571e+01, 6.49717514e-02,
        5.56098305e-01, 6.30842655e+00, 6.89940678e+01, 3.76245876e+00,
        9.35310734e+00, 4.01782486e+02, 1.84734463e+01, 3.60601186e+02,
        1.24406497e+01])
```

```
1 # 分散
2 scaler.var_
```



```
array([6.95792305e+01, 5.57886665e+02, 4.87753572e+01, 6.07504229e-02,
       1.33257561e-02, 4.91423928e-01, 7.83932705e+02, 4.26314655e+00,
       7.49911344e+01, 2.90195600e+04, 4.93579208e+00, 7.31040807e+03,
       4.99634123e+01])
```

これらの平均・分散の値を使って、データセットに含まれる値に標準化を施すには、`transform()` メソッドを使用します。

```
1 x_train_scaled = scaler.transform(x_train)
2 x_test_scaled  = scaler.transform(x_test)
```

それでは、標準化を行ったデータを使って、同じモデルを訓練してみましょう。

```
1 reg_model = LinearRegression()
2
3 # モデルの訓練
4 reg_model.fit(x_train_scaled, t_train)
```

```
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,
                  normalize=False)
```

```
1 # 精度の検証 (訓練データ)
2 reg_model.score(x_train_scaled, t_train)
```

```
0.7645451026942549
```

```
1 # 精度の検証 (テストデータ)
2 reg_model.score(x_test_scaled, t_test)
```

```
0.6733825506400195
```

結果は変わりませんでした。

べき変換をする別の前処理を適用し、再度同じモデルの訓練を行ってみましょう。

```
1 from sklearn.preprocessing import PowerTransformer
2
3 scaler = PowerTransformer()
4 scaler.fit(x_train)
5
6 x_train_scaled = scaler.transform(x_train)
7 x_test_scaled  = scaler.transform(x_test)
8
9 reg_model = LinearRegression()
10 reg_model.fit(x_train_scaled, t_train)
```

```
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,
                  normalize=False)
```

```
1 # 訓練データでの決定係数
2 reg_model.score(x_train_scaled, t_train)
```

```
0.7859862562650238
```

```

1 # テストデータでの決定係数
2 reg_model.score(x_test_scaled, t_test)

```

```
0.7002856552456189
```

結果が改善しました。

パイプライン化

前処理用の `scaler` と重回帰分析を行う `reg_model` は、両方 `fit()` メソッドを持っていました。scikit-learn には、パイプラインと呼ばれる一連の処理を統合できる機能があります。これを用いて、これらの処理をまとめてみましょう。

```

1 from sklearn.pipeline import Pipeline
2
3 # パイプラインの作成 (scaler -> svr)
4 pipeline = Pipeline([
5     ('scaler', PowerTransformer()),
6     ('reg', LinearRegression())
7 ])

```

```

1 # scaler および reg を順番に使用
2 pipeline.fit(x_train, t_train)

```

```

Pipeline(memory=None,
       steps=[('scaler', PowerTransformer(copy=True, method='yeo-johnson',
       standardize=True)), ('reg', LinearRegression(copy_X=True,
       fit_intercept=True, n_jobs=None,
       normalize=False))])

```

```

1 # 訓練用データセットを用いた決定係数の算出
2 pipeline.score(x_train, t_train)

```

```
0.7859862562650238
```

```

1 # テスト用データセットを用いた決定係数の算出
2 linear_result = pipeline.score(x_test, t_test)
3
4 linear_result

```

```
0.7002856552456189
```

パイプライン化を行うことで、`x_train_scaled` のような中間変数を作成することなく、同じ処理が行えるようになりました。

これによってコード量が減らせるだけでなく、評価を行う前にテスト用データセットに対しても訓練用データセットに対して行ったのと同様の前処理を行うことを忘れてしまうといったミスを防ぐことができます。

第 18 章

ニューラルネットワークの基礎

18.1 事前に読んでおく必要がある章

- 2 章 [Python 入門](#)
- 4 章 [微分の基礎](#)
- 5 章 [線形代数の基礎](#)
- 6 章 [確率・統計の基礎](#)
- 7 章 [単回帰分析と重回帰分析](#)
- 8 章 [NumPy 入門](#)

18.2 ニューラルネットワークとは

ニューラルネットワークは、**微分可能な変換を繋げて作られた計算グラフ (computational graph)** です。本章では、まずは下の図のような、円で表された**ノード (node)** に値が入っていて、ノードとノードが**エッジ (edge)** で繋がれているようなものを考えます。

この図でいうノードの縦方向の集まりのことを**層 (layer)** と呼びます。

そして**ディープラーニング (deep learning)** とは、層の数が非常に多いニューラルネットワークを用いた機械学習の手法や、その周辺の研究領域のことを指します。

18.2.1 層 (layer)

上の図は、ニューラルネットワークを用いて、ワインに関するいくつかの情報から、そのワインが「白ワイン」なのか「赤ワイン」なのか、というカテゴリを予測する分類問題を解く例を表しています。

左側から、最初の層を**入力層 (input layer)**、最後の層を**出力層 (output layer)** といい、その間にある層は**中間層 (intermediate layer)** もしくは**隠れ層 (hidden layer)** といいます。

上のニューラルネットワークは、入力層、中間層、出力層がそれぞれ一つずつあるので、合計 **3 層の構造 (アーキテクチャ; architecture)** となっています。複数の中間層を持たせれば、さらに多層のニューラルネットワークとすることができます。

ニューラルネットワークは、層から層へ、値を変換していきます。

そのため、ニューラルネットワークとはこの変換がいくつも連なってできる**一つの大きな関数**だと考えることができます。

従って、基本的には、入力を受け取って、何か出力を返すものです。

そして、どのようなデータを入力し、どのような出力を作りたいかによって、入力層と出力層のノード数が決定されます。

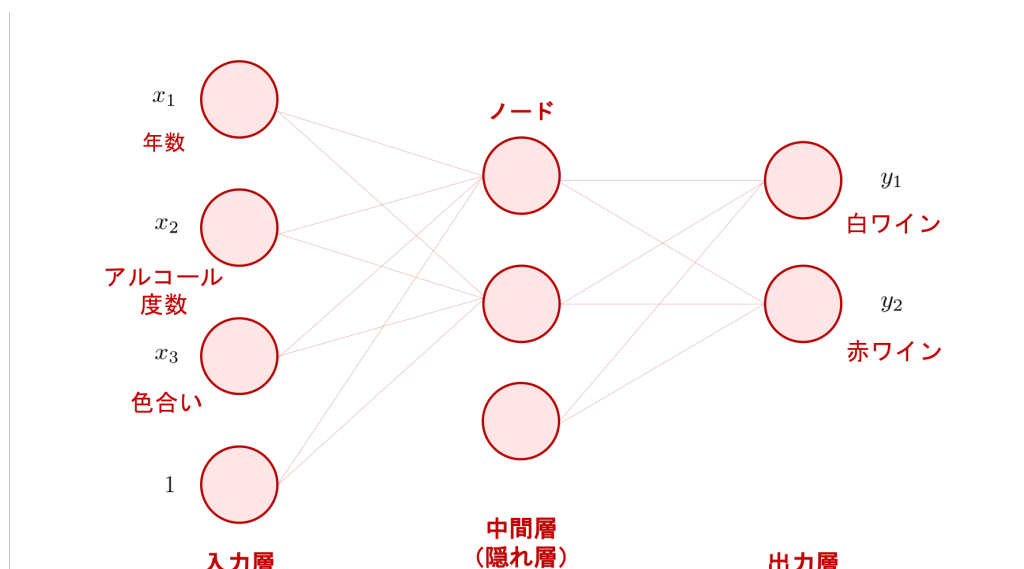


図 18.1 ニューラルネットワークの基本構造

上の例では、入力変数 x_1 に年数、 x_2 にアルコールの度数、 x_3 に色合い、といったあるワインを表す定量的な情報が与えられています。

そのため、入力層のノードの数は入力変数の数 M （上の図では、 $M = 3$ ）によって決定されます。

ここで、層と層の間にあるノード間の結合は、一つ一つが重みを持っており、上のような**全結合型ニューラルネットワーク**の場合は、それらの重みをまとめて、一つの行列で表現します。

この際、[単回帰分析と重回帰分析](#)の章で解説した重回帰分析の例と同じように、バイアスをその重み行列に含めて扱うため、入力層の最後に常に 1 を持つノードが追加されていることに注意してください。

このため、図では入力層のノード数が $M + 1 = 3 + 1 = 4$ 個となっています。

また、今回は**ワインの情報を入力して、それが「赤ワイン」か「白ワイン」かを予測する**という場合を表しているため、カテゴリ数が 2 の分類問題と言えます。

そのため、出力層のノード数は 2 となっています。

この出力層のノード数を変化させれば、3 つや 4 つのカテゴリ数の分類問題にも対応することができます。

また、離散的なカテゴリではなく、連続的な実数値を予測する回帰問題の場合は、**目標値の種類に合わせて出力層のノード数を決定**します。

18.2.2 構造 (architecture)

下の図のように、層数が同じでも、中間層のノード数を 3 にするか、5 にするか、などには任意性があります。

こういった値をどのように決めるかは、ニューラルネットワークを設計する人に委ねられています。

また、中間層自体をいくつ重ねるか、その個数も自分で決める必要があります。

こういった自分で決定するパラメータを、**ハイパーパラメータ**と呼びます。

中間層のノード数が 3 の場合

中間層のノード数が 5 の場合

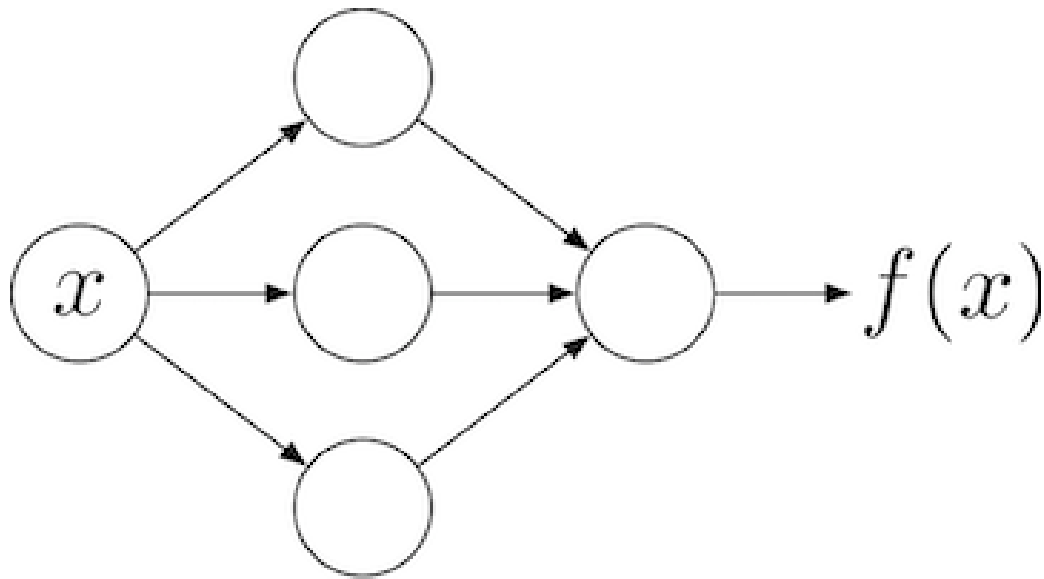


図 18.2 中間層のノード数が 3 の場合

18.2.3 色々な種類 (type)

ニューラルネットワークには、上図のような層間のノードが互いに密に結合した**全結合型 (fully-connected)** のものだけでなく、画像処理などでよく用いられる**畳み込み型 (convolutional)** のもの、系列データの扱いによく用いられる**再帰型 (recurrent)** のものなど、いくつかの種類があります。

また、基本的にそれらの違いは層間の結合の仕方であり、別々の層間には別々の種類の結合の仕方を用いることができます。

つまり、**1 つのニューラルネットワークの中に複数の種類の結合が混ざって現れることもあります。**

例えば、入力に近いところでは畳み込み型の結合を用い、出力に近いところでは全結合型の結合を用いる、といったことがよく行われています。

本章では、**まず最も基本となる全結合型のみに着目して、解説を行います。**

その他の種類については、後の章で解説を行います。

18.3 ニューラルネットワークの計算

18.3.1 基本的な計算の流れ

それでは、「ワインの分類」を題材に、ニューラルネットワークにある入力を与えられたとき、どのように計算結果が得られ、その結果をどう使えば分類が行えるのかを、まずは数式を用いず、以下の図を見ながら理解しましょう。

例えば、**年数が 3 年、アルコール度数が 14 度、色合いが 0.2** で表されるワインがあるとします。

このワインの情報をニューラルネットワークの入力層に与えたところ、そこから中間層、そして出力層へと**順番に計算が進み**、最終的に $y_1 = 0.15$, $y_2 = 0.85$ という 2 つの数値が得られたとします。

この **2 つの数値**は、入力されたワインが「赤ワイン」「白ワイン」という **2 つのカテゴリ**のそれぞれに属する確率を表しています。

分類問題では、**出力された数値のうちどのノードが最も大きな値を持っているかによって、ニューラルネットワークの予測が決まります。**

今回は、2 つの出力のうち $y_2 = 0.85$ が最も大きな値となっているため、 y_2 に対応する「赤ワイン」が予測

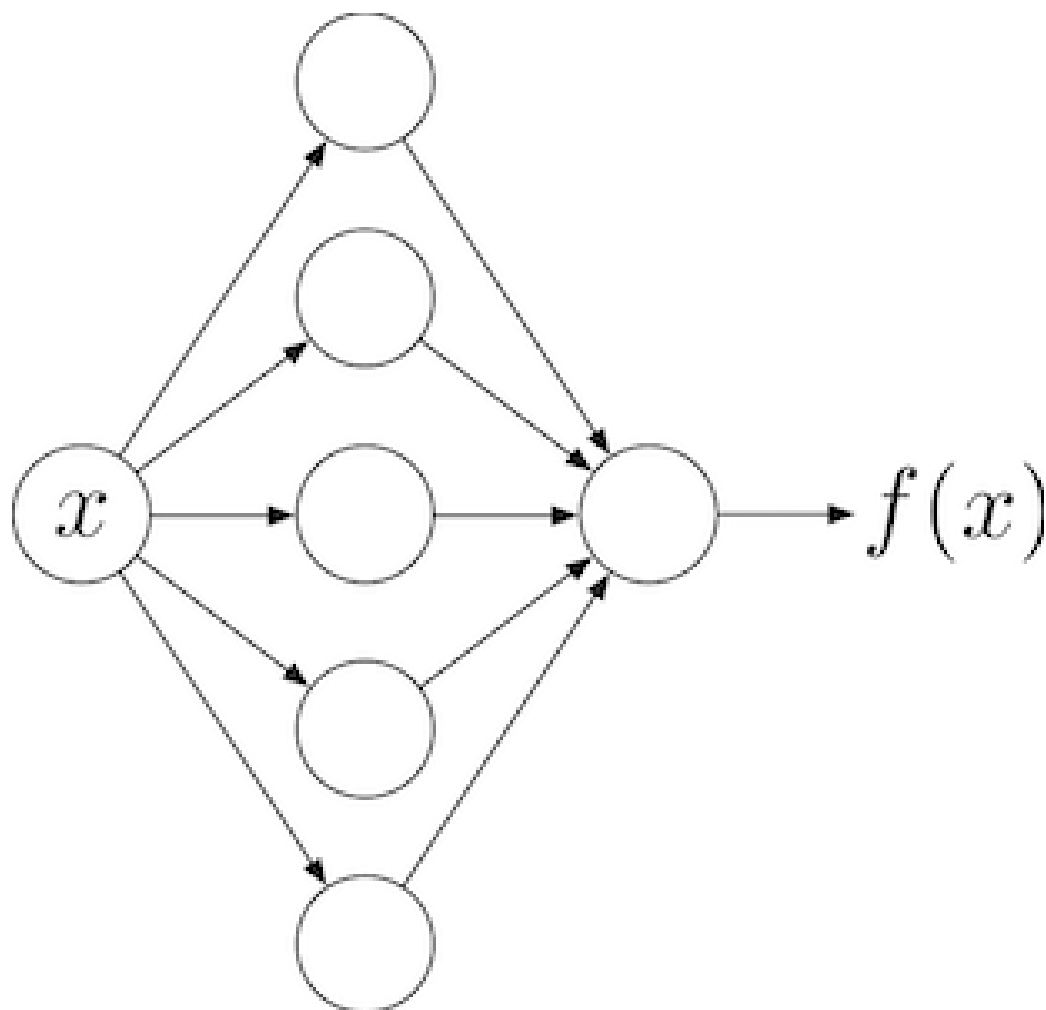


図 18.3 中間層のノード数が 5 の場合

されたカテゴリとなります。

このように、入力を与えられたとき、ニューラルネットワークの各層を順番に計算していき、出力まで計算を行うことを、**順伝播 (forward propagation)** と言います。

それでは、ここからはこのような計算がニューラルネットワークの内部でどのように行われているのか、詳しく見ていきましょう。

ポイントは、ニューラルネットワークの各層では、**前の層の出力値に「線形変換」と「非線形変換」を順番に施している**、というところです。

それでは、**線形変換**とは何か、**非線形変換**とは何か、を順番に説明します。

18.3.2 線形変換

線形変換（[注釈 1](#)）とは、入力ベクトルを $\mathbf{h}_0$ としたとき（[注釈 2](#)）、重み行列 $\mathbf{W}_{10}$ と、バイアスベクトル $\mathbf{b}_1$ の 2 つを使って、次のような変換を行うことを言います。

$$\mathbf{u}_1 = \mathbf{W}_{10}\mathbf{h}_0 + \mathbf{b}_1$$

これを踏まえて、以下の 2 層の**全結合型ニューラルネットワーク**の図を見てみましょう。

このニューラルネットワークは、**上の式と同じ計算を行います**。

2 つの層の間にはノードとノードを接続する線（エッジ）が書かれています。

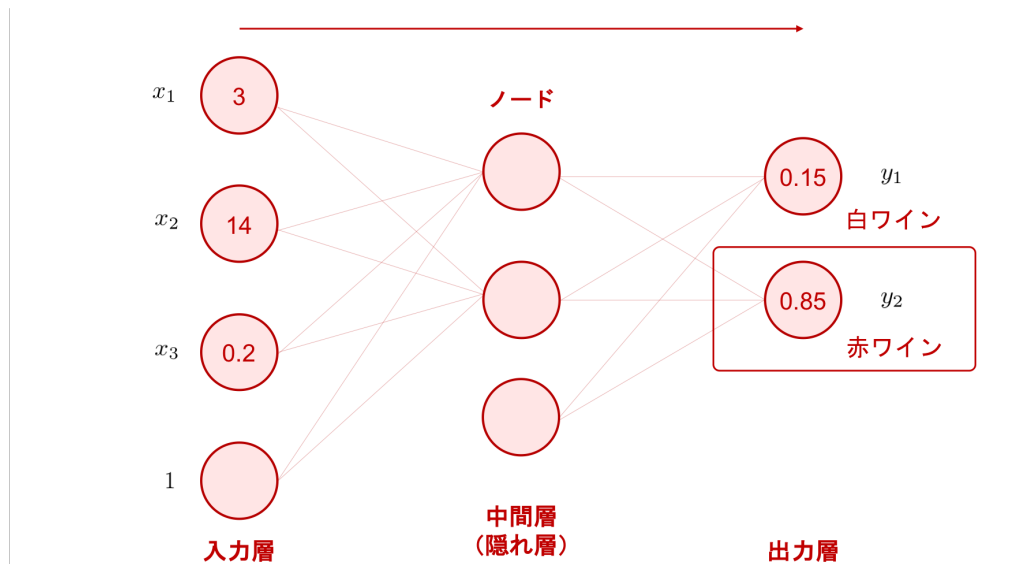


図 18.4 ニューラルネットワークの出力値

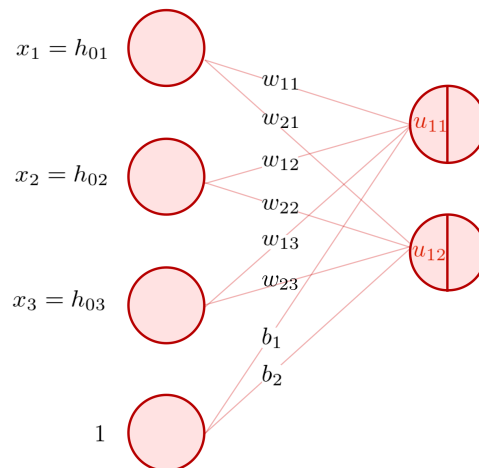


図 18.5 線形変換

そして、エッジの上には、 $w_{11}, w_{21}, \dots$ といった文字が書かれています。

これらは両端のノード間の**結合重み (connection weight)** を表しています。

入力層のノードが持つ値は、結合重みと掛け合わされ、出力層のノードに伝わります。出力層の 1 つのノードには、複数のノードから計算結果が伝わってくるので、これらを全部足し合わせます。

具体的には、以下のような計算をします。

$$u_{11} = w_{11}h_{01} + w_{12}h_{02} + w_{13}h_{03} + b_1 \quad (18.1)$$

$$u_{12} = w_{21}h_{01} + w_{22}h_{02} + w_{23}h_{03} + b_2 \quad (18.2)$$

この式は、入力ベクトル $\mathbf{h}_0$ 、重み行列 $\mathbf{W}_{10}$ 、バイアスベクトル $\mathbf{b}_1$ 、および出力 $\mathbf{u}_1$ を

$$\mathbf{h}_0 = \begin{bmatrix} h_{01} \\ h_{02} \\ h_{03} \end{bmatrix} \quad (18.3)$$

$$\mathbf{W}_{10} = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad (18.4)$$

$$\mathbf{b}_1 = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} \quad (18.5)$$

$$\mathbf{u}_1 = \begin{bmatrix} u_{11} \\ u_{12} \end{bmatrix} \quad (18.6)$$

と定義すれば、

$$\mathbf{u}_1 = \mathbf{W}_{10}\mathbf{h}_0 + \mathbf{b}_1$$

と書くことができ、本節の冒頭に示した式と一致していることが分かります。

以上から、全結合型ニューラルネットワークでは、層と層の間で線形変換が行われていることが分かりました。

ここで、出力層のノードを改めて見てみましょう。

ノードは真ん中に線が引かれて左右に分割されており、先程計算した出力値 u_{11}, u_{12} はその左側の半円の中に書かれています。

ニューラルネットワークの隠れ層では、**一つ前の層に線形変換を適用した結果を受け取り、そこへさらに非線形変換を適用したものを出力します。**

その結果を、この右側の半円の中に書き入れます。

次に、その非線形変換とは何なのか、またなぜ必要なのか説明していきます。

18.3.3 非線形変換

前節で紹介した線形変換のみでは、下図左のような**入出力間が線形な関係性**はよく近似することができるとしても、下図右のような**入出力間が非線形な関係**になっている場合には、観測データをうまく近似することができません。

例えば、線形変換のみで下図右の白い丸で表される観測データから x と y の関係を近似した場合、点線のような直線が得られたとします。

これでは、一部のデータについてはあまりよく当てはまっていないのが分かります。

しかし、もし xy 平面上で下図右の実線のような曲線を表現することができれば、より良く近似できるかもしれません。

このようなケースに対応できるよう、ニューラルネットワークでは各層において、線形変換に続いて非線形変換を施し、層を積み重ねて作られるニューラルネットワーク全体としても非線形性を持つことができるようにしています。

そして、この非線形変換を行う関数のことを、ニューラルネットワークの文脈では**活性化関数 (activation function)**と呼びます。

線形変換の節で用いた例に戻って、説明を進めます。

線形変換を行った結果 u_{11}, u_{12} のそれぞれに活性化関数 a を用いて非線形変換を行い、その結果を h_{11}, h_{12} とおきます。つまり、

$$h_{11} = a(u_{11}) \quad (18.7)$$

$$h_{12} = a(u_{12}) \quad (18.8)$$

です。

これらは**活性化値 (activation)**と呼ばれ、もしもう一つ層が続いているならば、次の層に渡される入力値となります。

ロジスティックシグモイド関数

活性化関数の具体例としては、下図に示す **ロジスティックシグモイド関数 (logistic sigmoid function)** が従来、よく用いられてきました。

以下、**シグモイド関数**と呼びます。

しかし近年、層の数が多いニューラルネットワークではシグモイド関数は活性化関数としてほとんど用いられません。

その理由の一つは、シグモイド関数を活性化関数に採用すると、**勾配消失 (vanishing gradient)** という現象によって学習が進行しなくなる問題が発生しやすくなるためです。

この問題の詳細は本章の最後で紹介します。

ReLU

この問題を回避するために、最近では**正規化線形関数 (ReLU: rectified linear unit)** がよく用いられています。

これは、以下のような関数です。

$\max(0, u)$ は、0 と u を比べて**大きな方を返す**関数です。

すなわち、**ReLU は入力値が負の場合には出力は 0 で一定であり、正の場合には入力をそのまま出力する**という関数です。

シグモイド関数では、入力が 0 から離れた値をとると、どんどん曲線の傾き (勾配) が小さくなっていき、平らになっていくことが一つ前の図からも見て取れます。

それに対し、ReLU 関数は入力値が正であれば、いくら大きくなっていても、傾き (勾配) は一定です。これが後ほど紹介する勾配消失という問題に有効に働きます。

18.3.4 数値を見ながら計算の流れを確認

それでは、上の図のような回帰問題を解く 3 層の全結合型ニューラルネットワークを考えて、**入力を与えられてから出力が得られるまでの一連の計算を、実際に数値を手計算しながら確認してみましょう。**

入力として、

$$\mathbf{x} = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$$

が与えられたとします。

このとき、出力 y はどのように計算されるのでしょうか。

まず、重み行列 $\mathbf{W}_{10}$ とバイアスペクトル $\mathbf{b}_1$ が、図より、以下のような値になっています。

$$\mathbf{W}_{10} = \begin{bmatrix} 3 & 1 & 2 \\ -2 & -3 & -1 \end{bmatrix} \quad (18.9)$$

$$\mathbf{b}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (18.10)$$

これを用いて、まずは入力に対し線形変換を行います。

$$\mathbf{u}_1 = \mathbf{W}_{10}\mathbf{x} + \mathbf{b}_1 = \begin{bmatrix} 3 & 1 & 2 \\ -2 & -3 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (18.11)$$

$$= \begin{bmatrix} 11 \\ -14 \end{bmatrix} \quad (18.12)$$

次に、非線形変換を施します。

今回は、活性化関数として ReLU 関数を採用しましょう。

活性化関数は、ベクトルを受け取ったとしても基本的には要素ごとに適用されます。

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{u}_1) \quad (18.13)$$

$$= \begin{bmatrix} \max(0, u_{11}) \\ \max(0, u_{12}) \end{bmatrix} \quad (18.14)$$

$$= \begin{bmatrix} \max(0, 11) \\ \max(0, -14) \end{bmatrix} \quad (18.15)$$

$$= \begin{bmatrix} 11 \\ 0 \end{bmatrix} \quad (18.16)$$

同様に、出力層の y の値までを計算します。2 層目と 3 層目の間の重み行列 $\mathbf{W}_{21}$ は、出力側の次元が 1 なので、

$$\mathbf{W}_{21} = \begin{bmatrix} 3 & 2 \end{bmatrix}$$

という 1×2 行列です。バイアスベクトル $\mathbf{b}_2$ は、図から

$$\mathbf{b}_2 = \begin{bmatrix} 0 \end{bmatrix}$$

であることが分かります。

これらを用いて隠れ層の出力値 $\mathbf{h}_1$ に対して線形変換を行うと、

$$y = \mathbf{W}_{21}\mathbf{h}_1 + \mathbf{b}_2 = 3 \times 11 + 2 \times 0 + 0 \times 1 = 33$$

が求まりました。

ここでは、ニューラルネットワークを用いて回帰問題を解く場合について説明を行うため、最終出力層に活性化関数を用いていません（注釈 3）。

ここまでで、ニューラルネットワークの順伝播計算の中身について詳しく見てきました。

このような計算過程は**推論 (inference)** と呼ばれ、一般的には**訓練が完了したモデルを用いて新しいデータに対して予測を行う**ことを指します。

今回の例では、すでに $\mathbf{W}_{10}$ や $\mathbf{W}_{21}$ といったパラメータの値は与えられており、それを使って入力に対してニューラルネットワークがどのような値を出力するのかを計算しました。

次節では、この $\mathbf{W}_{10}$ や $\mathbf{W}_{21}$ といったパラメータを決定するニューラルネットワークの**訓練**について解説します。

18.4 ニューラルネットワークの訓練

重回帰分析では、目的関数をモデルのパラメータで微分して出てきた式を $= 0$ とおいて、**実際の数値を使うことなく変数のまま ($\mathbf{x}$ や $\mathbf{t}$ と置いたまま)、解 (最適なパラメータ) を求めることができました。**

このように、変数のままで解を求めることを**解析的に解く**と言い、その答えのことを**解析解 (analytical solution)** と呼びます。

しかし、ニューラルネットワークで表現されるような複雑な関数の場合、最適解を解析的に解くことはほとんどの場合困難です。そのため、別の方法を考える必要があります。

解析的に解く方法に対し、計算機を使って繰り返し数値計算を行って解を求めることを**数值的に解く**といい、求まった解は**数値解 (numerical solution)** と呼ばれます。

ニューラルネットワークでは、基本的に数値的な手法によって最適なパラメータを求めます。

そして、これから紹介する代表的な数値的解法では、求めたいパラメータの**初期値 (initial value)** を事前に決めておく必要があります。

なぜなら、この初期値から少しずつ値を更新していき、望ましい解に近づけていくためです。

このような手法は、**最適化手法 (optimization method)** と呼ばれるものの一種です。

1. モデルを設計し
2. 目的関数を定め
3. 目的関数を最適化するパラメータを求める

という手順は、[単回帰分析](#)と[重回帰分析](#)の章でも、繰り返し出てきました。

そして、**この手順は機械学習の他の多くの手法でもよく登場します。**

本章で扱うニューラルネットワークでも、この手順に従います。

今回は、モデルとしては前節で説明に用いた 3 層のニューラルネットワークを用います。

このため、ステップ 1 はすでに完了しています。

次節からは、ステップ 2 以降を説明していきます。

18.4.1 目的関数

ニューラルネットワークの訓練には、微分可能でさえあれば解きたいタスクに合わせて様々な目的関数を利用することができます。

ここでは、

- **回帰問題**でよく用いられる**平均二乗誤差 (mean squared error)**
- **分類問題**でよく用いられる**交差エントロピー (cross entropy)**

という代表的な 2 つの目的関数を紹介します。

平均二乗誤差

平均二乗誤差 (mean squared error) は、**回帰問題を解きたい場合によく用いられる目的関数です。**

重回帰分析の解説中に紹介した二乗和誤差と似ていますが、各データ点における誤差の総和をとるだけでなく、それをデータ数で割って、誤差の平均値を計算している点が異なります。式で表すと、以下のようになります。

$$L = \frac{1}{N} \sum_{n=1}^N (t_n - y_n)^2$$

ここで、 N はサンプルサイズ、 y_n は n 個目のデータに対するニューラルネットワークの出力値、 t_n は n 個目のデータに対する望ましい正解の値です。

ここで、下の図のような例を考えてみましょう。

今、ある入力 x を与えたときのこのニューラルネットワークの出力 y は、33 でした。ここで、もしこの入

力に対応する目標値が $t = 20$ だとすると、今全体のサンプルサイズが 1 だとして、平均二乗誤差は

$$L = \frac{1}{1}(20 - 33)^2 = 169$$

と計算できます。

交差エントロピー

交差エントロピー (cross entropy) は、**分類問題を解きたい際によく用いられる目的関数です。**

例として、 K クラスの分類問題を考えてみましょう。

ある入力 x が与えられたとき、ニューラルネットワークの出力層に K 個のノードがあり、それぞれがこの入力が k 番目のクラスに属する確率

$$y_k = p(y = k|x)$$

を表しているとします。

これは、入力 x が与えられたという**条件のもとで**、予測クラスを意味する y が k であるような確率、を表す**条件付き確率**です。

ここで、 x が所属するクラスの正解が、

$$\mathbf{t} = [t_1 \quad t_2 \quad \dots \quad t_K]^T$$

というベクトルで与えられているとします。

ただし、このベクトルは t_k ($k = 1, 2, \dots, K$) のいずれか 1 つだけが 1 であり、それ以外は 0 であるようなベクトルであるとして。

これを**ワンホットベクトル (1-hot vector)** と呼びます。

そして、この 1 つだけ値が 1 となっている要素は、その要素のインデックスに対応したクラスが正解であることを意味します。例えば、 $t_3 = 1$ であれば 3 つ目のクラス (3 というインデックスに対応するクラス) が正解であるということになります。

以上を用いて、交差エントロピーは以下のように定義されます。

$$-\sum_{k=1}^K t_k \log y_k$$

これは、 t_k が $k = 1, \dots, K$ のうち正解クラスである一つの k の値でだけ 1 となるので、正解クラスであるような k での $\log y_k$ を取り出して -1 を掛けているのと同じです。

また、 N 個すべてのサンプルを考慮すると、交差エントロピーは、

$$L = -\sum_{n=1}^N \sum_{k=1}^K t_{n,k} \log y_{n,k}$$

となります。

18.4.2 目的関数の最適化

目的関数の値を最小にするようなパラメータの値を求めることで、**ニューラルネットワークを訓練します。**

本節では、最適化アルゴリズムの一つである**勾配降下法 (gradient descent)** を紹介します。

勾配降下法

図中の点線は、パラメータ w を変化させた際の目的関数 L の値を表しています。

この図では簡単のために二次関数で表します。実際のニューラルネットワークの目的関数は、多次元で、かつもっと複雑な形をしていることがほとんどです。

さて、この目的関数が最小値を与えるような w は、どのようにして求められるのでしょうか。

前節で説明したように、ニューラルネットワークのパラメータの初期値を決める必要があります。一般的に**ニューラルネットワークのパラメータは乱数で初期化されます**。

ここでは、例として $w = 4$ という初期化が行われたと考えてみましょう。

そうすると、 $w = 4$ における L の接線の傾き $\frac{\partial L}{\partial w}$ が求まります。この**接線の傾きのことを勾配 (gradient)** と一般的に呼びます。

さて、ここでは仮に $w = 4$ における $\frac{\partial L}{\partial w}$ が 5 であったとしましょう。

このことを

$$\left. \frac{\partial L}{\partial w} \right|_{w=4} = 5$$

と書きます。

この 5 という値は $w = 4$ における $L(w)$ という関数の勾配を表しています。

勾配とは、 w を増加させた際に L が増加する方向を意味しています。今は L の値を小さくしたいわけですから、この**傾きの逆方向へ w を変化させる**、すなわち w から $\frac{\partial L}{\partial w}$ を引けば**良い** ことになります。

これが**ニューラルネットワークのパラメータを目的関数の勾配を用いて更新していく際の基本的な考え方**です。

このときの w の一度の更新量の幅を調整するために、勾配に**学習率 (learning rate)** と呼ばれる値を乗じるのが一般的です。学習率を η とおくと、 w から $\eta \frac{\partial L}{\partial w}$ を引くことで、勾配そのものの値を η にした量だけ w を更新することになります。更新後の w は、

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

となります。

ここで、 $\leftarrow$ は、右側の値で左側の値を置き換える、つまり**更新すること**を意味しています。

例えば、学習率を $\eta = 0.5$ に設定して、更新後の w を実際に求めてみましょう。

w の更新量は**学習率 × 勾配**で決まるので、

$$0.5 \times 5 = 2.5$$

となります。

現在 $w = 4$ なので、**この値を引いて**

$$w \leftarrow 4 - 2.5$$

と更新した後は、 $w = 1.5$ です。

1 度目の更新を行って、 w が $w = 1.5$ の位置に移動しました。

そこで、再度この点においても勾配を求めてみます。

今度は勾配が -2 になっていたとしましょう。

すると**学習率 × 勾配**は

$$0.5 \times (-2) = -1$$

となります。
これを再び用いて、

$$w \leftarrow 1.5 - (-1)$$

と 2 度目の更新を行うと、今度は $w = 2.5$ の位置にきます。

このようにして 2 回更新したあとは、以下の図の位置にパラメータが移動しています。

徐々に L が最小値をとるときの w の値に近づいていることが見て取れます。

こうして、**学習率 $\times$ 勾配**を更新量としてパラメータを変化させていくと、パラメータ w を求めたい L を最小にする w に徐々に近づけていくことができます。

このような勾配を用いた目的関数の最適化手法を**勾配降下法 (gradient descent)**と呼びます。

ニューラルネットワークは、基本的に**微分可能な関数のみをつなげ合わせてできている**(注釈 4)ため、ニューラルネットワーク全体が表す関数も微分可能であり、訓練データセットを用いて勾配降下法によって目的関数を小さくする(局所)最適パラメータを求めることができます。

18.4.3 ミニバッチ学習

通常ニューラルネットワークを勾配降下法で最適化する場合は、データを一つ一つ用いてパラメータを更新するのではなく、**いくつかのデータをまとめて入力し、それぞれの勾配を計算したあと、その勾配の平均値を用いてパラメータの更新を行う方法がよく行われます。**

これを**ミニバッチ学習**と呼びます。

ミニバッチ学習では、以下の手順で訓練を行います。

1. 訓練データセットから一様ランダムに N_b (> 0) 個のデータを抽出する
2. その N_b 個のデータをまとめてニューラルネットワークに入力し、それぞれのデータに対する目的関数の値を計算する
3. N_b 個の目的関数の値の平均をとる
4. この平均の値に対する各パラメータの勾配を求める
5. 求めた勾配を使ってパラメータを更新する

そしてこれを、異なる N_b 個のデータの組み合わせに対して繰り返し行います。

ここで、一度のパラメータ更新に用いられるサンプルの数 N_b を**バッチサイズ (batch size)**と呼びます。

結果的にはデータセットに含まれる全てのデータを使用していきますが、1 度の更新に用いるデータは N_b 個ずつであることに注意してください。

データセットから N_b 個ずつデータを取ってきて使うというミニバッチ学習の様子を模式的に表した図を上 に示します。この図では、データセットをそれぞれが N_b 個ずつデータを含んでいる 10 個のグループに分割 しています。そのあと、それぞれのグループごとにパラメータ更新を逐次的に行います。

ミニバッチ学習と異なり、データセット全体を一度に用いて一回の更新を行う、すなわちグループへ分割し ない方法を**バッチ学習**と呼びます。

勾配降下法では、データを用いてパラメータを 1 度更新するまでのことを **1 イテレーション (iteration)** と呼び、バッチ学習では訓練データセットのデータ全体を 1 度の更新に一気に用いるため、**1 エポック = 1 イテレーション**となります。

一方、例えばデータセットを 10 個のグループに分割してグループごとに更新を行うミニバッチ学習の場合 は、**1 エポック = 10 イテレーション**となります。

このようなミニバッチ学習を用いた勾配降下法は特に、**確率的勾配降下法 (stochastic gradient descent; SGD)**と呼ばれます。

現在多くのニューラルネットワークの最適化手法はこの SGD をベースとした手法となっています。

SGD を用いると、全体の計算時間が劇的に少なくできるだけでなく、下図のように目的関数が複数の谷を持っていたとしても、適当な条件のもとで ”ほとんど確実に” **局所最適解** (注釈 5) に収束することが知られています。

18.4.4 パラメータ更新量の算出

それでは今、下図のような 3 層の全結合型ニューラルネットワークを考え、3 次元の入力ベクトルから、1 次元の値を出力し、正解の値を予測する回帰問題を題材に、パラメータを更新する式の導入を行っていきましょう。

まず、1 層目と 2 層目の間の線形変換が $\mathbf{w}_1, \mathbf{b}_1$ 、2 層目と 3 層目の間の線形変換が $\mathbf{w}_2, \mathbf{b}_2$ というパラメータによって表されているとします。

また、これらをまとめて Θ と表すことにします。つまり、 $\Theta = \{\mathbf{w}_1, \mathbf{b}_1, \mathbf{w}_2, \mathbf{b}_2\}$ です (注釈 6)。

入力 $\mathbf{x} \in \mathbb{R}^3$ 、出力を y とし、目標値を t とおきます。目的関数には以下の平均二乗誤差関数を用いることにします。

$$L = \frac{1}{N} \sum_{n=1}^N (t_n - y_n)^2$$

ただし、 N はデータ数です。

今回は $N = 1$ なので、目的関数は簡単に

$$L = (t - y)^2$$

と書けます。

それでは、この目的関数を各パラメータで微分して、それぞれのパラメータについて更新量を算出してみましょう。

まず、順伝播を計算します。

ニューラルネットワーク全体を一つの関数として見て f と書くことにすると、出力 y は

$$\begin{aligned} y &= f(\mathbf{x}; \Theta) \\ &= \mathbf{w}_2 a_1(\mathbf{w}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 \end{aligned}$$

と書くことができます。ここで、 a_1 は 1 層目のノードで行われる非線形変換（に使用される活性化関数）を意味しています。今回は、**シグモイド関数**をこの活性化関数に用いることにします。

以下、簡単のために、入力に対して行われた線形変換の結果を $\mathbf{u}_1$ とし、中間層の値、すなわち $\mathbf{u}_1$ に活性化関数を適用した結果を $\mathbf{h}_1$ と書きます。

すると、これらの関係は以下のように整理することができます。

$$\begin{aligned} \text{(入力の線形変換)} \quad \mathbf{u}_1 &= \mathbf{w}_1 \mathbf{x} + \mathbf{b}_1 \\ \text{(1 層目のノードでの非線形変換)} \quad \mathbf{h}_1 &= a_1(\mathbf{u}_1) \\ \text{(1 層目の出力の線形変換)} \quad y &= \mathbf{w}_2 \mathbf{h}_1 + \mathbf{b}_2 \end{aligned}$$

パラメータ $\mathbf{w}_2$ の更新量

それではまず、出力層に近い方のパラメータ、 $\mathbf{w}_2$ についての L の勾配を求めてみましょう。

これは、合成関数の偏微分なので、**連鎖律 (chain rule)** を用いて以下のように展開できます。

$$\frac{\partial L}{\partial \mathbf{w}_2} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial \mathbf{w}_2}$$

2 つの偏微分の積が現れました。

これらはそれぞれ、

$$\frac{\partial L}{\partial y} = -2(t - y)$$

$$\frac{\partial y}{\partial \mathbf{w}_2} = \mathbf{h}_1$$

と求まります。

それでは、実際に NumPy を使ってこの勾配の値を計算してみましょう。

ここでは簡単のために、バイアスベクトルはすべて 0 で初期化されているとします。

```
1 import numpy as np
2
3 # 入力
4 x = np.array([2, 3, 1])
5
6 # 正解
7 t = np.array([20])
```

まず、NumPy モジュールを読み込んでから、入力の配列を定義します。

ここでは、上図と同じになるように 2, 3, 1 の 3 つの値を持つ 3 次元ベクトルを定義しています。

また、正解として図と同じように 20 を与えることにしました。

次に、パラメータを定義します。

```
1 # 層間のパラメータ 0-1
2 w1 = np.array([[3, 1, 2], [-2, -3, -1]])
3 b1 = np.array([0, 0])
4
5 # 層間のパラメータ 2-3
6 w2 = np.array([[3, 2]])
7 b2 = np.array([0])
```

ここでは、以下の 4 つのパラメータを定義しました。

1 層目と 2 層目の間の線形変換のパラメータ

$\mathbf{w}_1 \in \mathbb{R}^{2 \times 3}$: 3 次元ベクトルを 2 次元ベクトルに変換する行列

$\mathbf{b}_1 \in \mathbb{R}^2$: 2 次元バイアスベクトル

2 層目と 3 層目の間の線形変換のパラメータ

$\mathbf{w}_2 \in \mathbb{R}^{1 \times 2}$: 2 次元ベクトルを 1 次元ベクトルに変換する行列

$\mathbf{b}_2 \in \mathbb{R}^1$: 1 次元バイアスベクトル

それでは、各層の計算を実際に実行してみましょう。

```
1 # 中間層の計算
2 u1 = w1.dot(x) + b1
3 h1 = 1. / (1 + np.exp(-u1))
4
5 # 出力の計算
6 y = w2.dot(h1) + b2
7
8 print(y)
```

```
[2.99995156]
```


出力は 2.99995156 と求まりました。

つまり、 $f([2, 3, 1]^T) = 2.99995156$ ということになります。

次に、上で導出した

$$\frac{\partial L}{\partial \mathbf{w}_2} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial \mathbf{w}_2}$$

の右辺の 2 つの偏微分をそれぞれ計算してみましょう。

```
1 # dL / dy
2 dLdy = -2 * (t - y)
3
4 # dy / dw_2
5 dydw2 = h1
```

これらを掛け合わせれば、求めたかったパラメータ $\mathbf{w}_2$ についての勾配を得ることができます。

```
1 # dL / dw_2: 求めたい勾配
2 dLdw2 = dLdy * dydw2
3
4 print(dLdw2)
```

```
[-3.39995290e+01 -2.82720335e-05]
```

勾配が求まりました。

これが $\frac{\partial L}{\partial \mathbf{w}_2}$ の値です (注釈 7)。

これを学習率でスケールさせたものを使えば、パラメータ $\mathbf{w}_2$ を更新することができます。

更新式は、具体的には以下ようになります。

$$\mathbf{w}_2 \leftarrow \mathbf{w}_2 - \eta \frac{\partial L}{\partial \mathbf{w}_2}$$

学習率について

学習率が大きすぎると、繰り返しパラメータ更新を行っていく中で目的関数の値が振動したり、発散したりしてしまいます。

逆に小さすぎると、収束に時間がかかってしまいます。

そのため、この学習率を適切に決定することがニューラルネットワークの学習においては非常に重要となります。

多くの場合、学習がきちんと進むもっとも大きな値を経験的に探すということが行われます。

シンプルな画像認識のタスクなどでは大抵、0.1 から 0.01 程度の値が最初に試される場合が比較的多く見られます。

パラメータ $\mathbf{w}_1$ の更新量

次に、 $\mathbf{w}_1$ の更新量も求めてみましょう。

そのためには、 $\mathbf{w}_1$ で目的関数 L を偏微分した値が必要です。

これは以下のように計算できます。

$$\begin{aligned}
 \frac{\partial L}{\partial \mathbf{w}_1} &= \frac{\partial L}{\partial y} \frac{\partial y}{\partial \mathbf{w}_1} \\
 &= \frac{\partial L}{\partial y} \frac{\partial y}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{w}_1} \\
 &= \frac{\partial L}{\partial y} \frac{\partial y}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{u}_1} \frac{\partial \mathbf{u}_1}{\partial \mathbf{w}_1}
 \end{aligned}$$

4 つの偏微分の積になりました。

ここで、活性化関数の勾配

$$\frac{\partial \mathbf{h}_1}{\partial \mathbf{u}_1}$$

が登場しています。

今回は活性化関数 a_1 にシグモイド関数を用いているので、

$$\mathbf{h}_1 = a_1(\mathbf{u}_1) = \frac{1}{1 + \exp(-\mathbf{u}_1)}$$

を $\mathbf{u}_1$ で微分します。

$$\begin{aligned}
 \frac{\partial a_1(\mathbf{u}_1)}{\partial \mathbf{u}_1} &= -\frac{-(\exp(-\mathbf{u}_1))}{(1 + \exp(-\mathbf{u}_1))^2} \\
 &= \frac{1}{1 + \exp(-\mathbf{u}_1)} \cdot \frac{\exp(-\mathbf{u}_1)}{1 + \exp(-\mathbf{u}_1)} \\
 &= \frac{1}{1 + \exp(-\mathbf{u}_1)} \cdot \frac{1 + \exp(-\mathbf{u}_1) - 1}{1 + \exp(-\mathbf{u}_1)} \\
 &= \frac{1}{1 + \exp(-\mathbf{u}_1)} \left(1 - \frac{1}{1 + \exp(-\mathbf{u}_1)} \right) \\
 &= a_1(\mathbf{u}_1) (1 - a_1(\mathbf{u}_1)) \\
 &= \mathbf{h}_1 (1 - \mathbf{h}_1)
 \end{aligned}$$

このように、**シグモイド関数の勾配は、シグモイド関数の出力値を使って簡単に計算することができます。**

これで、上の 4 つの偏微分のうち、初めの 1 つと、3 つ目はすでに求めました。

残りの 2 つは、それぞれ、

$$\begin{aligned}
 \frac{\partial y}{\partial \mathbf{h}_1} &= \mathbf{w}_2 \\
 \frac{\partial \mathbf{u}_1}{\partial \mathbf{w}_1} &= \mathbf{x}
 \end{aligned}$$

と計算できます。

では、さっそく実際に NumPy を用いて計算を実行してみましょう。

```

1  # d y / d h1
2  dydh1 = w2
3
4  # d h1 / d u1
5  dh1du1 = h1 * (1 - h1)
6
7  # d u_1 / d w1
8  du1dw1 = x
9
10 # 上から du1 / dw1 の直前までを一旦計算

```

```

11 dLdu1 = dLdy * dydh1 * dh1du1
12
13 # は du1dw1 (3,) というので、shapeg_u1w1[Noneとして](1, 3)に変形
14 du1dw1 = du1dw1[None]
15
16 # dL / dw_1: 求めたい勾配
17 dLdw1 = dLdu1.T.dot(du1dw1)
18
19 print(dLdw1)

```

```

[[-3.40704286e-03 -5.11056429e-03 -1.70352143e-03]
 [-1.13088040e-04 -1.69632060e-04 -5.65440200e-05]]

```

これが $\frac{\partial L}{\partial \mathbf{w}_1}$ の値です (注釈 7)。

これを用いて、 $\mathbf{w}_2$ と同様に以下のような更新式でパラメータ $\mathbf{w}_1$ の更新をすることができます。

$$\mathbf{w}_1 \leftarrow \mathbf{w}_1 - \eta \frac{\partial L}{\partial \mathbf{w}_1}$$

これで $\mathbf{w}_1$ の更新をすることができました。 $\mathbf{b}_1$ 、 $\mathbf{b}_2$ についても同様の流れで更新することができます。

18.4.5 誤差逆伝播法 (バックプロパゲーション)

ここまでで、各パラメータについての目的関数の導関数を手計算により導出して実際に勾配の数値計算を行うということを体験しました。

では、もっと層数の多いニューラルネットワークの場合は、どうなるでしょうか。

同様に手計算によって全てのパラメータの勾配を求めることも不可能ではないかもしれませんが、層数が多くなれば膨大な時間がかかるでしょう。しかし、**ニューラルネットワークの微分可能な関数を順に適用するという性質を用いると、コンピュータによって自動的に勾配を与える関数を導き出すことが可能です。**

まず、合成関数の偏微分は、**連鎖律によって複数の偏微分の積の形に変形できる**ことを思い出しましょう。

下図は、ここまでの説明で用いていた 3 層の全結合型ニューラルネットワークの出力を得るための計算 (順伝播) と、その値を使って目的関数の値を計算する過程を青い矢印で、そして**前節で手計算によって行った各パラメータによる目的関数の偏微分を計算する過程を赤い矢印で表現した動画**となっています。

Image asset not embedded: 誤差逆伝播法 (backpropagation) の計算過程
 figures/source/23-basics-of-neural-networks/13.backpropagation.gif

まず、目的関数の出力を $l = L(y, t)$ とします。

この図の丸いノードは変数を表し、四角いノードは関数を表しています。

今、一つの巨大な合成関数として見たニューラルネットワーク全体を f と表し、その中で各層間の線形変換に用いられる関数を f_1, f_2 、中間層での非線形変換を a_1 と表します。

今、上図の青い矢印で表されるように、新しい入力 $\mathbf{x}$ がニューラルネットワークに与えられ、それが順々に出力側に伝わっていき、最終的に目的関数の値 l まで計算が終わったとします。

すると次は、目的関数の出力の値を小さくするような各パラメータの更新量を求めたいということになりますが、このために**必要な目的関数の勾配は、各パラメータの丸いノードより先の部分 (出力側) にある関数の勾配だけで計算できる**ことが分かります。

具体的には、それらを全て掛け合わせたものになっています。

つまり、上図の赤い矢印で表されるように、**出力側から入力側に向かって、順伝播とは逆向きに、各関数における入力についての勾配を求めて、掛け合わせていけば、パラメータについての目的関数の勾配が計算でき**

るわけです。

このように、微分の連鎖律の仕組みを用いて、ニューラルネットワークを構成する関数を持つパラメータについての目的関数の勾配を、**順伝播で通った経路を逆向きにたどるようにして**途中の関数の勾配の掛け算によって求めるアルゴリズムを **誤差逆伝播法 (backpropagation)** と呼びます。

18.4.6 勾配消失

活性化関数について初めに触れた際、シグモイド関数には勾配消失という現象が起きやすくなるという問題があり、現在はあまり使われていないと説明をしました。

その理由についてもう少し詳しく見ていきましょう。

上で既に計算した、シグモイド関数の導関数を思い出してみます。

$$a(u) = \frac{1}{1 + \exp(-u)}$$

$$a'(u) = a(u)(1 - a(u))$$

さて、この導関数の値を入力変数に関してプロットしてみると、下記のようになります。

この図の上 2 つは、導関数を構成する 2 つの部分、 $f(u)$ および $1 - f(u)$ の値を別々にプロットしたもので、中央下の図は全体の導関数の値となります。

中央下の導関数の形から明らかなように、入力が原点から遠くなるにつれ勾配の値がどんどん小さくなり、0 に漸近していくことが分かります。

各パラメータの更新量を求めるには、前節で説明したように、**そのパラメータよりも先のすべての関数の勾配を掛け合わせる**必要がありました。

このとき、活性化関数にシグモイド関数を用いていると、勾配は必ず**最大でも 0.25** という値にしかありません。

つまり、シグモイド関数がニューラルネットワーク中に現れるたびに、目的関数の勾配は、**多くとも 0.25 倍**されてしまいます。

これは、層数が増えていけばいくほど、この最大でも 0.25 にしかならないような値が、繰り返し掛け合わされることになってしまい、入力に近い層に流れていく勾配はどんどん 0 に近づいていってしまいます。

具体例を見てみましょう。

今回は 3 層のニューラルネットワークを用いて説明を行っていましたが、4 層の場合を考えてみます。

すると、一番入力に近い線形変換のパラメータの勾配は、多くとも目的関数の勾配を $0.25 \times 0.25 = 0.0625$ 倍したもののということになります。

層数が増えるたびに、指数的に勾配が小さくなるということがよく分かります。

ディープラーニングでは、4 層よりもさらに多くの層を積み重ねたニューラルネットワークが用いられます。

そうすると、活性化関数としてシグモイド関数を使用した場合、**目的関数の勾配が入力に近い関数を持つパラメータへほぼ全く伝わらなくなってしまいます。**

あまりにも小さな勾配しか伝わってこなくなると、パラメータの更新量がほとんど 0 になるため、どんなに目的関数が大きな値になっていても、入力層に近い関数を持つパラメータは変化しなくなります。

つまり初期化時からほとんど値が変わらなくなるということになり、学習が行われていないという状態になるわけです。

これを**勾配消失**と呼び、長らく深い（十数層を超える）ニューラルネットワークの学習が困難であった一つの要因でした。

この解決策として、ReLU 関数が提案され、多層のニューラルネットワークに対する学習も勾配消失を回避しながら行うことができるようになっていきます。

18.5 注釈

18.5.1 注釈 1

通常、数学では線形変換とは $\mathbf{W}$ を掛ける操作のことを指し、 $\mathbf{b}$ を足す操作は含まれません。 $\mathbf{b}$ の加算を含む操作は厳密には「アファイン変換（もしくは アフィン変換）」と呼ばれるものです。しかし、ディープラーニングの文脈ではこの変換も線形変換と呼ばれることが多いです。

18.5.2 注釈 2

ここで、入力層のノードが h_{01}, h_{02}, h_{03} という文字で表されていますが、この h は**隠れ層**を意味する英語の **hidden layer** の頭文字である h から来ています。

入力層は隠れ層ではありませんが、同じ文字を使うことで表記を一般化し、簡便にしています。

18.5.3 注釈 3

また、分類問題を解きたい場合は、クラス数と同じだけのノードを出力層に用意しておき、各ノードがあるクラスに入力が属する確率を表すようにします。

このため、**全出力ノードの値の合計が 1 になるよう正規化**します。

これには、**要素ごと**に適用される活性化関数ではなく、**層ごと**に活性値を計算する別の関数を用いる必要があります。

そのような目的に使用される代表的な関数には、ソフトマックス関数があります。

18.5.4 注釈 4

厳密には目的関数に微分不可能な点が存在する可能性はあります。例えば ReLU は $x = 0$ で微分不可能なため、ReLU を含んだニューラルネットワークには微分不可能な点が存在することになります。このような場合、**劣微分 (subderivative, subdifferential)** という考え方を導入し、全ての点で勾配が決定できるようにすることなどが行われます。

18.5.5 注釈 5

この局所解が大域的最適解と一致する条件については、現在活発に研究が行われています。参考：["A Convergence Theory for Deep Learning via Over-Parameterization"](#)

18.5.6 注釈 6

$\{\}$ という記号は、集合を表し、中に要素を並べて書きます。

18.5.7 注釈 7

ここで、試しに Chainer を使って同じ勾配の計算を行い、NumPy を使ったコードと結果が一致するかどうかチェックしてみましょう。

```
1 import chainer
2 import chainer.functions as F
3 import chainer.links as L
```

```

4
5 # 層間のパラメータ1-2
6 w1 = np.array([[3, 1, 2], [-2, -3, -1]], dtype=np.float32)
7 b1 = np.array([0, 0], dtype=np.float32)
8
9 l1 = L.Linear(2, initialW=w1, initial_bias=b1)
10
11 # 層間のパラメータ2-3
12 w2 = np.array([[3, 2]], dtype=np.float32)
13 b2 = np.array([0], dtype=np.float32)
14
15 l2 = L.Linear(1, initialW=w2, initial_bias=b2)
16
17 # 入力
18 x = np.array([[2, 3, 1]], dtype=np.float32)
19
20 # 出力
21 y = l2(F.sigmoid(l1(x)))
22 print(y) # => should be [2.99995156]
23
24 # 正解
25 t = np.array([[20.]], dtype=np.float32)
26
27 # ロス
28 loss = F.mean_squared_error(y, t)
29
30 # dLdw_2
31 print(chainer.grad([loss], [l2.W])) # => should be [-3.39995290e+01
    -2.82720335e-05]
32
33 # dLdw_1
34 print(chainer.grad([loss], [l1.W])) # => should be [[-3.40704286e-03
    -5.11056429e-03 -1.70352143e-03]
35
    #
    [-1.13088040e-04
    -1.69632060e-04
    -5.65440200e-05]]

```

```

variable([[2.9999516]])
[variable([[-3.3999527e+01, -2.8371891e-05]])]
[variable([[-3.4045703e-03, -5.1068552e-03, -1.7022851e-03],
    [-1.1348747e-04, -1.7023120e-04, -5.6743735e-05]])]

```

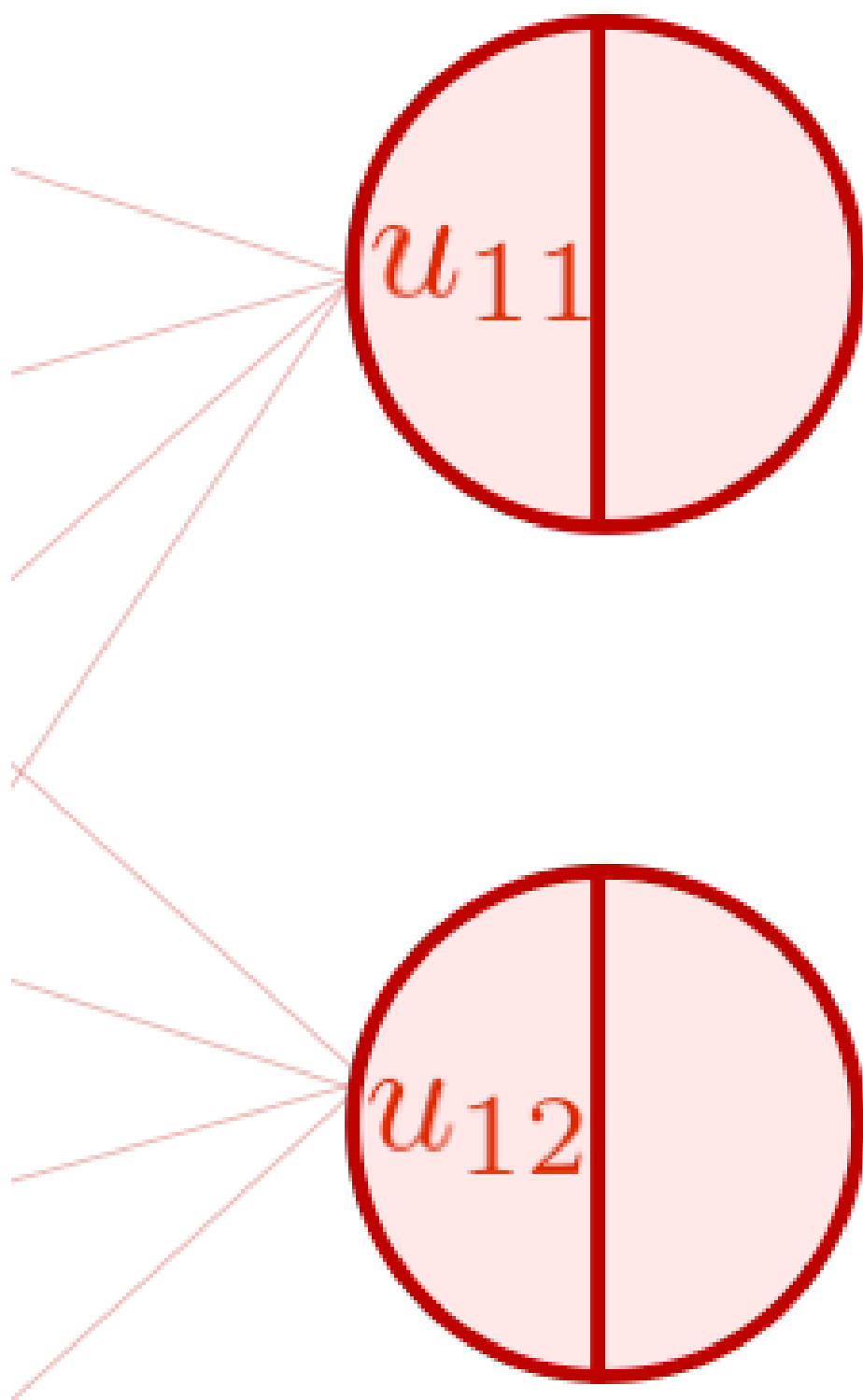


図 18.6 線形変換後の結果

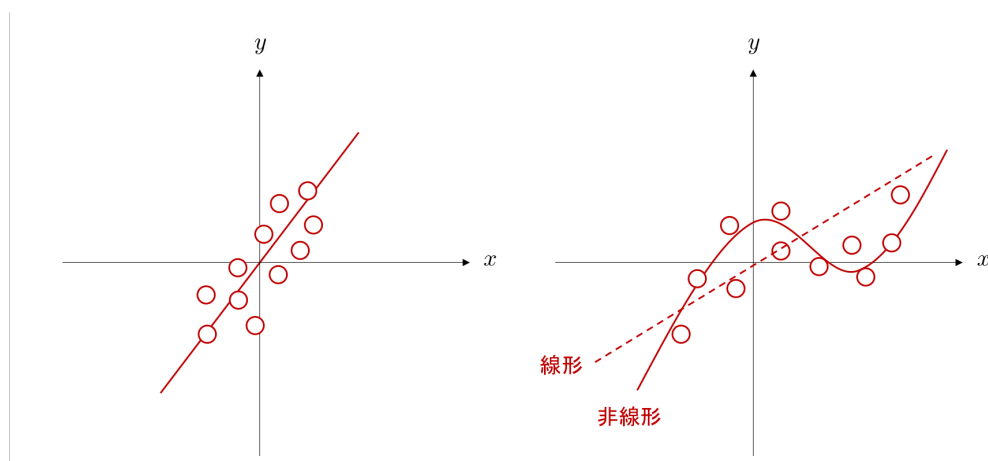


図 18.7 入力と出力の関係

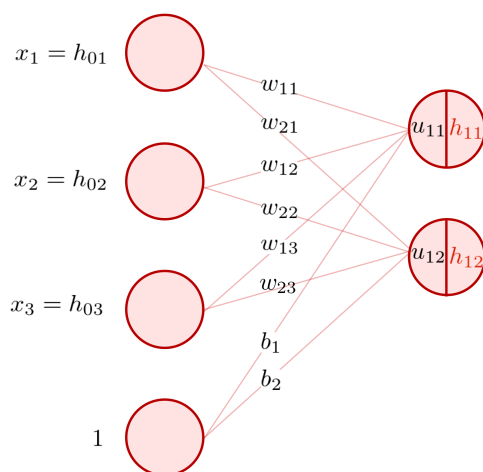


図 18.8 非線形変換

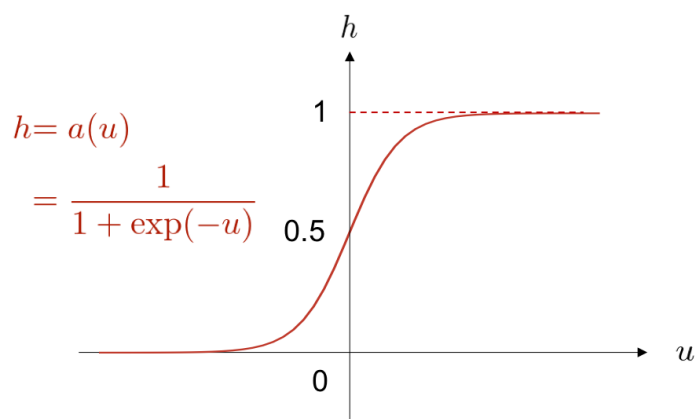


図 18.9 シグモイド関数

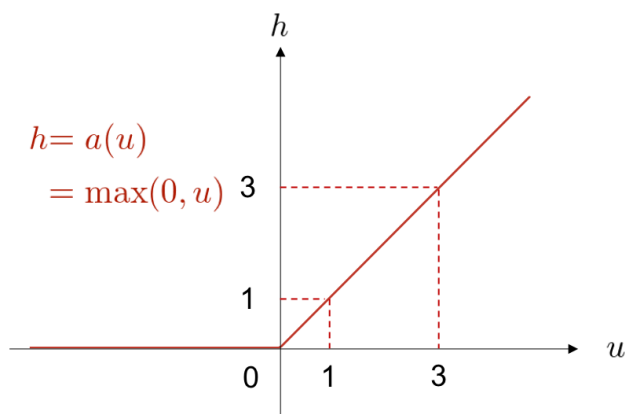


図 18.10 ReLU 関数

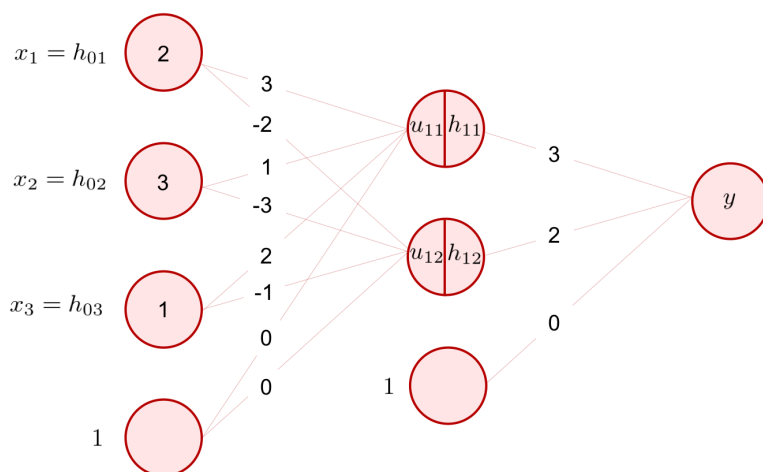


図 18.11 計算の流れ

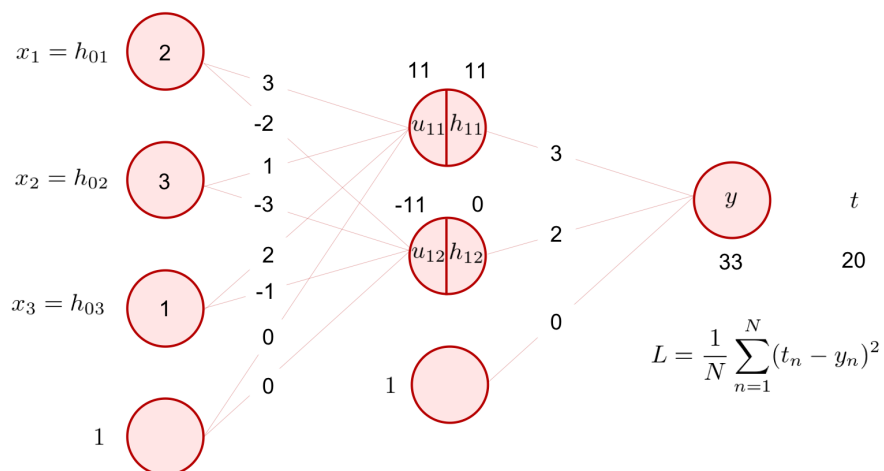


図 18.12 平均二乗誤差

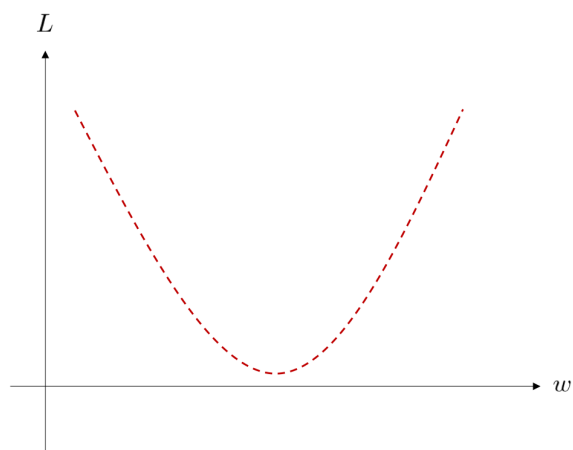


図 18.13 パラメータと目的関数の関係 (イメージ)

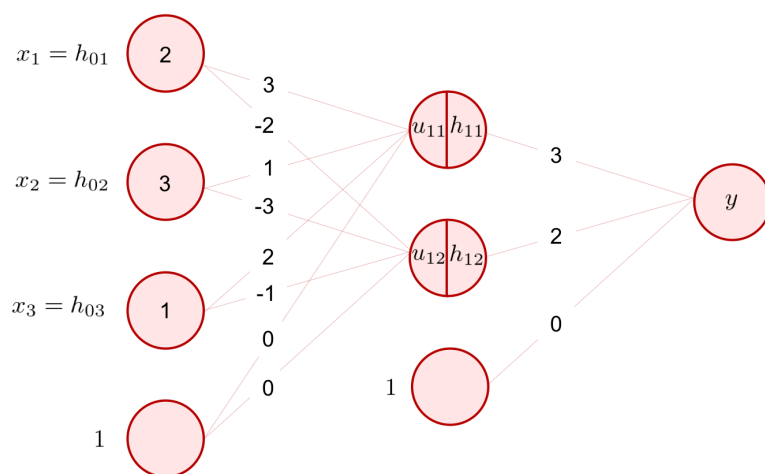


図 18.14 目的関数の接線の傾き

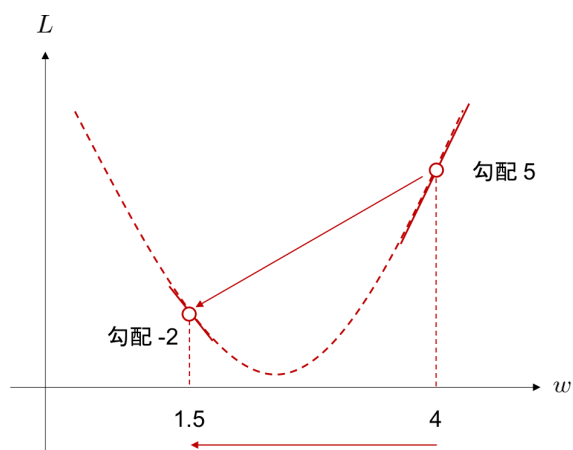


図 18.15 パラメータの更新

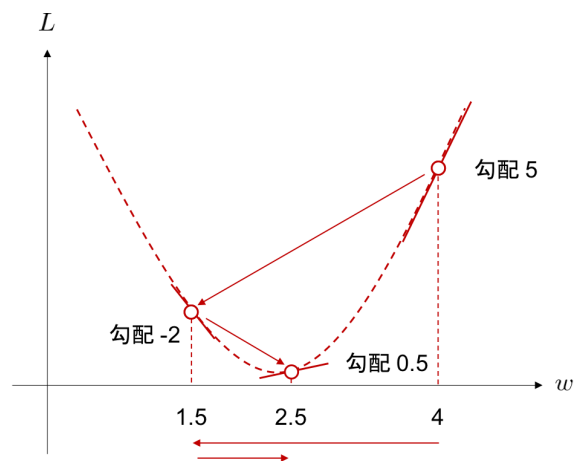


図 18.16 2 度のパラメータの更新

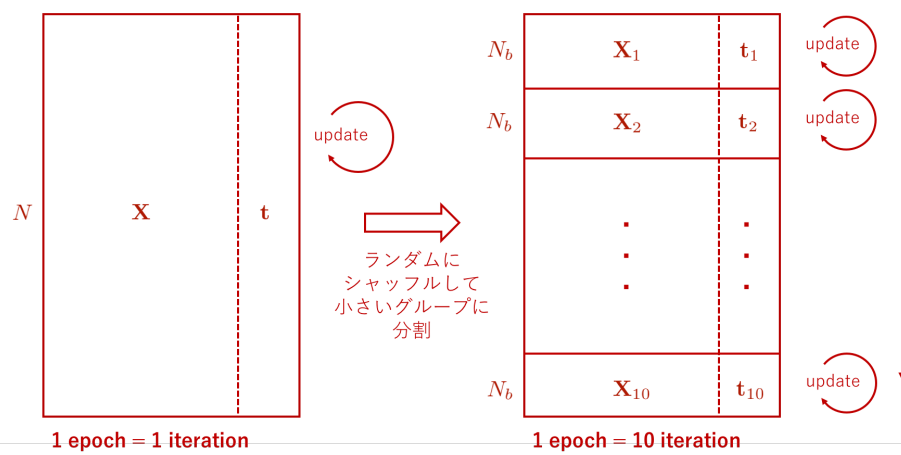


図 18.17 ミニバッチ学習

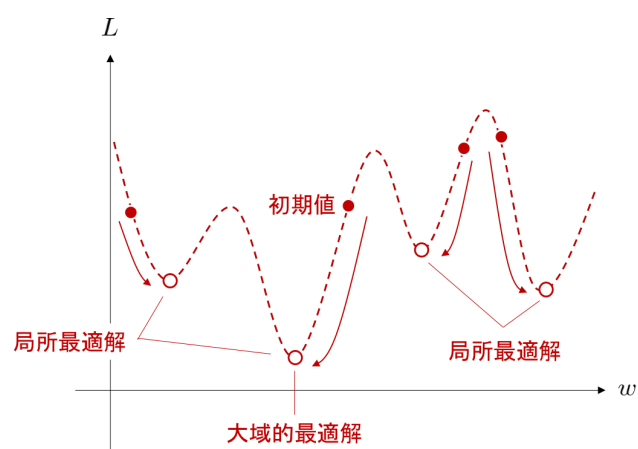


図 18.18 局所最適解と大域最適解

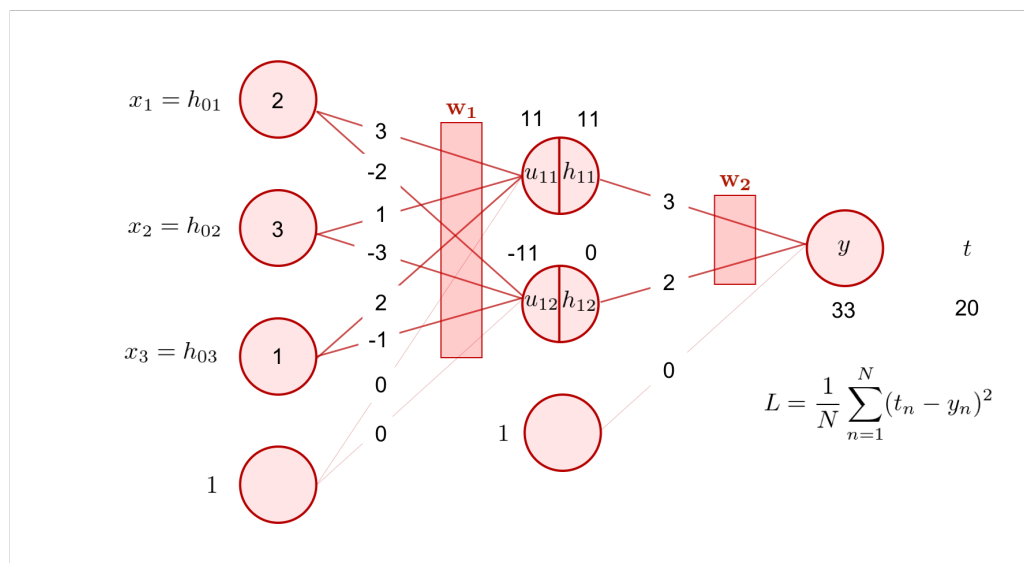


図 18.19 パラメータ更新の例

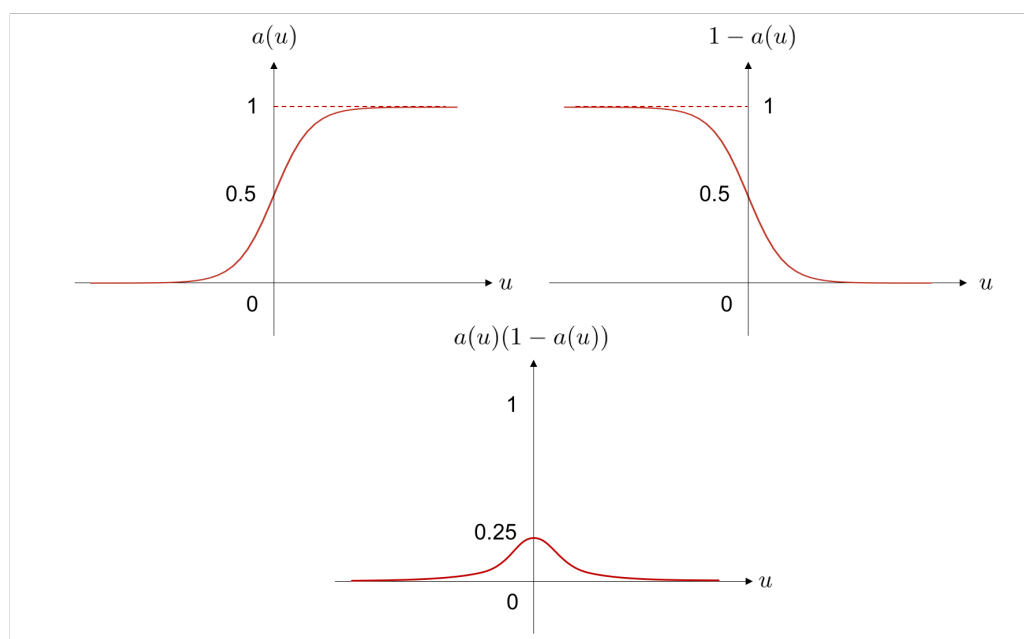


図 18.20 シグモイド関数の導関数